

stv_v2

Generated by Doxygen 1.9.1

1 STV2 - StraTegic Verifier version 2	1
1.1 Usage	1
1.2 Tests	1
1.3 Performance estimation	2
1.4 Specification	2
1.5 Examples and templates	3
1.6 Misc	3
1.7 Web interface	3
1.8 Credits	3
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	11
4.1 File List	11
5 Class Documentation	13
5.1 Action Struct Reference	13
5.1.1 Detailed Description	14
5.1.2 Member Data Documentation	14
5.1.2.1 actionName	14
5.1.2.2 hash	14
5.1.2.3 states	14
5.2 ActionTemplate Class Reference	14
5.2.1 Detailed Description	15
5.2.2 Constructor & Destructor Documentation	15
5.2.2.1 ActionTemplate()	15
5.2.3 Member Data Documentation	15
5.2.3.1 actionName	15
5.2.3.2 hash	16
5.2.3.3 states	16
5.3 Agent Class Reference	16
5.3.1 Detailed Description	17
5.3.2 Constructor & Destructor Documentation	17
5.3.2.1 Agent()	17
5.3.3 Member Function Documentation	17
5.3.3.1 includesState()	17
5.3.4 Member Data Documentation	18
5.3.4.1 id	18
5.3.4.2 initState	18
5.3.4.3 localStates	18

5.3.4.4 localTransitions	18
5.3.4.5 name	19
5.3.4.6 vars	19
5.4 AgentTemplate Class Reference	19
5.4.1 Detailed Description	20
5.4.2 Constructor & Destructor Documentation	20
5.4.2.1 AgentTemplate()	20
5.4.3 Member Function Documentation	20
5.4.3.1 addInitial()	20
5.4.3.2 addLocal()	21
5.4.3.3 addPersistent()	21
5.4.3.4 addTransition()	22
5.4.3.5 generateAgent()	23
5.4.3.6 setIdent()	23
5.4.3.7 setInitState()	24
5.4.4 Friends And Related Function Documentation	24
5.4.4.1 DotGraph	24
5.5 Assignment Class Reference	25
5.5.1 Detailed Description	25
5.5.2 Constructor & Destructor Documentation	26
5.5.2.1 Assignment()	26
5.5.3 Member Function Documentation	26
5.5.3.1 assign()	26
5.5.4 Member Data Documentation	26
5.5.4.1 ident	26
5.5.4.2 value	27
5.6 Cfg Struct Reference	27
5.6.1 Member Data Documentation	28
5.6.1.1 add_epsilon_transitions	28
5.6.1.2 counterexample	28
5.6.1.3 dotdir	28
5.6.1.4 fixpoint	29
5.6.1.5 fname	29
5.6.1.6 formula	29
5.6.1.7 formula_from_parameter	29
5.6.1.8 model_id	29
5.6.1.9 natural_strategy	29
5.6.1.10 output_dot_files	29
5.6.1.11 output_global_model	30
5.6.1.12 output_local_models	30
5.6.1.13 probability	30
5.6.1.14 reduce	30

5.6.1.15 reduce_all	30
5.6.1.16 reduce_args	30
5.6.1.17 strategy_file_path	30
5.6.1.18 stv_mode	31
5.6.1.19 verify_strategy	31
5.7 Condition Struct Reference	31
5.7.1 Detailed Description	31
5.7.2 Member Data Documentation	32
5.7.2.1 comparedValue	32
5.7.2.2 conditionOperator	32
5.7.2.3 var	32
5.8 DecisionEntry Struct Reference	32
5.8.1 Member Function Documentation	33
5.8.1.1 toString()	33
5.8.2 Member Data Documentation	33
5.8.2.1 activeProbabilityEntryType	34
5.8.2.2 decision	34
5.8.2.3 depth	34
5.8.2.4 globalState	34
5.8.2.5 globalTransitionControlled	34
5.8.2.6 newStatus	34
5.8.2.7 previousProbabilityAnswerValues	35
5.8.2.8 previousProbabilityEntryType	35
5.8.2.9 prevStatus	35
5.8.2.10 stateProbabilityPrevious	35
5.8.2.11 strategy	35
5.8.2.12 type	35
5.9 DotGraph Class Reference	36
5.9.1 Constructor & Destructor Documentation	37
5.9.1.1 DotGraph() [1/4]	37
5.9.1.2 DotGraph() [2/4]	37
5.9.1.3 DotGraph() [3/4]	37
5.9.1.4 DotGraph() [4/4]	38
5.9.2 Member Function Documentation	38
5.9.2.1 addEdge()	38
5.9.2.2 addNode()	38
5.9.2.3 saveToFile()	38
5.9.3 Member Data Documentation	39
5.9.3.1 edges	39
5.9.3.2 graphBase	39
5.9.3.3 graphName	39
5.9.3.4 nodes	39

5.9.3.5 styleString	40
5.10 EpistemicClass Struct Reference	40
5.10.1 Detailed Description	41
5.10.2 Member Data Documentation	41
5.10.2.1 fixedCoalitionTransition	41
5.10.2.2 globalStates	41
5.10.2.3 hash	41
5.11 ExprAdd Class Reference	42
5.11.1 Detailed Description	43
5.11.2 Constructor & Destructor Documentation	43
5.11.2.1 ExprAdd()	43
5.11.3 Member Function Documentation	43
5.11.3.1 eval() [1/2]	43
5.11.3.2 eval() [2/2]	44
5.12 ExprAnd Class Reference	45
5.12.1 Detailed Description	46
5.12.2 Constructor & Destructor Documentation	46
5.12.2.1 ExprAnd()	46
5.12.3 Member Function Documentation	47
5.12.3.1 eval() [1/2]	47
5.12.3.2 eval() [2/2]	47
5.13 ExprConst Class Reference	47
5.13.1 Detailed Description	49
5.13.2 Constructor & Destructor Documentation	49
5.13.2.1 ExprConst()	49
5.13.3 Member Function Documentation	49
5.13.3.1 eval() [1/2]	49
5.13.3.2 eval() [2/2]	49
5.14 ExprDiv Class Reference	50
5.14.1 Detailed Description	51
5.14.2 Constructor & Destructor Documentation	51
5.14.2.1 ExprDiv()	51
5.14.3 Member Function Documentation	52
5.14.3.1 eval() [1/2]	52
5.14.3.2 eval() [2/2]	52
5.15 ExprEq Class Reference	52
5.15.1 Detailed Description	54
5.15.2 Constructor & Destructor Documentation	54
5.15.2.1 ExprEq()	54
5.15.3 Member Function Documentation	54
5.15.3.1 eval() [1/2]	54
5.15.3.2 eval() [2/2]	54

5.16 ExprGe Class Reference	55
5.16.1 Detailed Description	56
5.16.2 Constructor & Destructor Documentation	56
5.16.2.1 ExprGe()	56
5.16.3 Member Function Documentation	57
5.16.3.1 eval() [1/2]	57
5.16.3.2 eval() [2/2]	57
5.17 ExprGt Class Reference	57
5.17.1 Detailed Description	59
5.17.2 Constructor & Destructor Documentation	59
5.17.2.1 ExprGt()	59
5.17.3 Member Function Documentation	59
5.17.3.1 eval() [1/2]	59
5.17.3.2 eval() [2/2]	59
5.18 ExprHart Class Reference	60
5.18.1 Detailed Description	61
5.18.2 Constructor & Destructor Documentation	61
5.18.2.1 ExprHart()	61
5.18.3 Member Function Documentation	62
5.18.3.1 eval() [1/2]	62
5.18.3.2 eval() [2/2]	62
5.19 ExprIdent Class Reference	63
5.19.1 Detailed Description	64
5.19.2 Constructor & Destructor Documentation	64
5.19.2.1 ExprIdent()	64
5.19.3 Member Function Documentation	64
5.19.3.1 eval() [1/2]	64
5.19.3.2 eval() [2/2]	65
5.20 ExprKnow Class Reference	65
5.20.1 Detailed Description	67
5.20.2 Constructor & Destructor Documentation	67
5.20.2.1 ExprKnow()	67
5.20.3 Member Function Documentation	67
5.20.3.1 eval() [1/2]	67
5.20.3.2 eval() [2/2]	67
5.21 ExprLe Class Reference	68
5.21.1 Detailed Description	69
5.21.2 Constructor & Destructor Documentation	69
5.21.2.1 ExprLe()	69
5.21.3 Member Function Documentation	70
5.21.3.1 eval() [1/2]	70
5.21.3.2 eval() [2/2]	70

5.22 ExprLt Class Reference	70
5.22.1 Detailed Description	72
5.22.2 Constructor & Destructor Documentation	72
5.22.2.1 ExprLt()	72
5.22.3 Member Function Documentation	72
5.22.3.1 eval() [1/2]	72
5.22.3.2 eval() [2/2]	72
5.23 ExprMul Class Reference	73
5.23.1 Detailed Description	74
5.23.2 Constructor & Destructor Documentation	74
5.23.2.1 ExprMul()	74
5.23.3 Member Function Documentation	75
5.23.3.1 eval() [1/2]	75
5.23.3.2 eval() [2/2]	75
5.24 ExprNe Class Reference	75
5.24.1 Detailed Description	77
5.24.2 Constructor & Destructor Documentation	77
5.24.2.1 ExprNe()	77
5.24.3 Member Function Documentation	77
5.24.3.1 eval() [1/2]	77
5.24.3.2 eval() [2/2]	77
5.25 ExprNode Class Reference	78
5.25.1 Detailed Description	79
5.25.2 Member Function Documentation	79
5.25.2.1 eval() [1/2]	79
5.25.2.2 eval() [2/2]	79
5.26 ExprNot Class Reference	79
5.26.1 Detailed Description	81
5.26.2 Constructor & Destructor Documentation	81
5.26.2.1 ExprNot()	81
5.26.3 Member Function Documentation	81
5.26.3.1 eval() [1/2]	81
5.26.3.2 eval() [2/2]	81
5.27 ExprOr Class Reference	82
5.27.1 Detailed Description	83
5.27.2 Constructor & Destructor Documentation	83
5.27.2.1 ExprOr()	83
5.27.3 Member Function Documentation	84
5.27.3.1 eval() [1/2]	84
5.27.3.2 eval() [2/2]	84
5.28 ExprRem Class Reference	84
5.28.1 Detailed Description	86

5.28.2 Constructor & Destructor Documentation	86
5.28.2.1 ExprRem()	86
5.28.3 Member Function Documentation	86
5.28.3.1 eval() [1/2]	86
5.28.3.2 eval() [2/2]	86
5.29 ExprSub Class Reference	87
5.29.1 Detailed Description	88
5.29.2 Constructor & Destructor Documentation	88
5.29.2.1 ExprSub()	88
5.29.3 Member Function Documentation	89
5.29.3.1 eval() [1/2]	89
5.29.3.2 eval() [2/2]	89
5.30 Formula Struct Reference	89
5.30.1 Detailed Description	90
5.30.2 Member Data Documentation	90
5.30.2.1 coalition	90
5.30.2.2 isCTL	90
5.30.2.3 isF	91
5.30.2.4 p	91
5.30.2.5 probability	91
5.30.2.6 probabilitySign	91
5.31 FormulaTemplate Struct Reference	91
5.31.1 Detailed Description	92
5.31.2 Member Data Documentation	92
5.31.2.1 coalition	92
5.31.2.2 formula	92
5.31.2.3 isF	92
5.31.2.4 probability	92
5.31.2.5 probabilitySign	92
5.32 GlobalModel Struct Reference	93
5.32.1 Detailed Description	93
5.32.2 Member Data Documentation	93
5.32.2.1 agents	93
5.32.2.2 epistemicClasses	94
5.32.2.3 epistemicClassesKnowledge	94
5.32.2.4 formula	94
5.32.2.5 globalStates	94
5.32.2.6 initState	94
5.33 GlobalModelGenerator Class Reference	95
5.33.1 Detailed Description	97
5.33.2 Constructor & Destructor Documentation	97
5.33.2.1 GlobalModelGenerator()	97

5.33.2.2 $\sim$ GlobalModelGenerator()	97
5.33.3 Member Function Documentation	98
5.33.3.1 checkLocalStates()	98
5.33.3.2 computeEpistemicClassHash()	98
5.33.3.3 computeGlobalStateHash()	98
5.33.3.4 createProbabilityStrategy()	99
5.33.3.5 expandAllStates()	99
5.33.3.6 expandAndReduceAllStates()	99
5.33.3.7 expandState()	99
5.33.3.8 expandStateAndReturn()	100
5.33.3.9 findGlobalStateInEpistemicClass()	100
5.33.3.10 findOrCreateEpistemicClass()	100
5.33.3.11 findOrCreateEpistemicClassForKnowledge()	101
5.33.3.12 generateGlobalTransitions()	101
5.33.3.13 generateInitState()	101
5.33.3.14 generateNextMDP()	102
5.33.3.15 generateStateFromLocalStates()	102
5.33.3.16 getActionNameFromStateInStrategy()	102
5.33.3.17 getAgentInstanceByName()	103
5.33.3.18 getCoalitionActionSignature()	103
5.33.3.19 getCoalitionIdentifier()	103
5.33.3.20 getCurrentGlobalModel()	104
5.33.3.21 getFormula()	104
5.33.3.22 getFormulaCorectness()	104
5.33.3.23 getFormulaSize()	104
5.33.3.24 getNextPath()	105
5.33.3.25 initModel()	105
5.33.3.26 initStrategy()	105
5.33.3.27 markFormulaAsIncorrect()	105
5.33.4 Member Data Documentation	106
5.33.4.1 actionCounts	106
5.33.4.2 addedStates	106
5.33.4.3 agentIndex	106
5.33.4.4 candidateStateDepths	106
5.33.4.5 choiceIndices	106
5.33.4.6 coalitionTransitions	106
5.33.4.7 correctModel	107
5.33.4.8 currentStrategy	107
5.33.4.9 formula	107
5.33.4.10 globalModel	107
5.33.4.11 globalModelCandidates	107
5.33.4.12 localModels	107

5.33.4.13 opponentsTransitions	108
5.33.4.14 stateDepths	108
5.33.4.15 stateHashMapCache	108
5.33.4.16 strategiesExhausted	108
5.33.4.17 strategyCollection	108
5.33.4.18 strategyGenerationInit	108
5.34 GlobalState Struct Reference	109
5.34.1 Detailed Description	110
5.34.2 Constructor & Destructor Documentation	110
5.34.2.1 GlobalState()	110
5.34.3 Member Function Documentation	110
5.34.3.1 getGlobalStateEnvironment()	110
5.34.3.2 toString()	110
5.34.4 Member Data Documentation	111
5.34.4.1 epistemicClasses	111
5.34.4.2 epistemicClassesAllAgents	111
5.34.4.3 globalTransitions	111
5.34.4.4 goodState	111
5.34.4.5 hash	111
5.34.4.6 isExpanded	111
5.34.4.7 localStatesProjection	112
5.34.4.8 preimage	112
5.34.4.9 probability	112
5.34.4.10 probabilityLoop	112
5.34.4.11 probabilityResult	112
5.34.4.12 stateVerifResult	112
5.34.4.13 verificationStatus	113
5.35 GlobalModelGenerator::GlobalStateTupleHash Struct Reference	113
5.35.1 Member Function Documentation	113
5.35.1.1 operator>()	113
5.36 GlobalTransition Struct Reference	114
5.36.1 Detailed Description	114
5.36.2 Constructor & Destructor Documentation	115
5.36.2.1 GlobalTransition()	115
5.36.3 Member Function Documentation	115
5.36.3.1 getProbability()	115
5.36.3.2 joinLocalTransitionNames()	115
5.36.4 Member Data Documentation	115
5.36.4.1 from	115
5.36.4.2 id	116
5.36.4.3 isInvalidDecision	116
5.36.4.4 localTransitions	116

5.36.4.5 next_id	116
5.36.4.6 to	116
5.37 HistoryDbg Class Reference	117
5.37.1 Detailed Description	118
5.37.2 Constructor & Destructor Documentation	118
5.37.2.1 HistoryDbg()	118
5.37.2.2 ~HistoryDbg()	118
5.37.3 Member Function Documentation	118
5.37.3.1 addEntry()	118
5.37.3.2 cloneEntry()	118
5.37.3.3 markEntry()	119
5.37.3.4 print()	119
5.37.4 Member Data Documentation	119
5.37.4.1 entries	119
5.38 HistoryEntry Struct Reference	120
5.38.1 Detailed Description	121
5.38.2 Member Function Documentation	121
5.38.2.1 toString()	121
5.38.3 Member Data Documentation	121
5.38.3.1 decision	121
5.38.3.2 depth	121
5.38.3.3 globalState	121
5.38.3.4 globalTransitionControlled	122
5.38.3.5 newStatus	122
5.38.3.6 next	122
5.38.3.7 prev	122
5.38.3.8 prevStatus	122
5.38.3.9 strategy	122
5.38.3.10 type	123
5.39 LocalModels Struct Reference	123
5.39.1 Detailed Description	123
5.39.2 Member Data Documentation	124
5.39.2.1 agents	124
5.40 LocalState Class Reference	124
5.40.1 Detailed Description	125
5.40.2 Member Function Documentation	125
5.40.2.1 compare()	125
5.40.2.2 toString()	125
5.40.3 Member Data Documentation	126
5.40.3.1 agent	126
5.40.3.2 environment	126
5.40.3.3 epistemicGlobalStates	126

5.40.3.4 id	126
5.40.3.5 localTransitions	127
5.40.3.6 name	127
5.41 LocalStateTemplate Class Reference	127
5.41.1 Detailed Description	128
5.41.2 Member Data Documentation	128
5.41.2.1 name	128
5.41.2.2 transitions	128
5.42 LocalTransition Struct Reference	128
5.42.1 Detailed Description	129
5.42.2 Member Data Documentation	129
5.42.2.1 agent	129
5.42.2.2 conditions	129
5.42.2.3 from	130
5.42.2.4 id	130
5.42.2.5 isShared	130
5.42.2.6 localName	130
5.42.2.7 name	130
5.42.2.8 probability	130
5.42.2.9 sharedCount	131
5.42.2.10 to	131
5.43 ModelParser Class Reference	131
5.43.1 Detailed Description	132
5.43.2 Constructor & Destructor Documentation	132
5.43.2.1 ModelParser()	132
5.43.2.2 ~ModelParser()	132
5.43.3 Member Function Documentation	132
5.43.3.1 parse()	132
5.43.3.2 parseAndOverwriteFormula()	133
5.44 ProbabilityEntry Class Reference	133
5.44.1 Detailed Description	134
5.44.2 Constructor & Destructor Documentation	134
5.44.2.1 ProbabilityEntry() [1/2]	134
5.44.2.2 ProbabilityEntry() [2/2]	134
5.44.2.3 ~ProbabilityEntry()	134
5.44.3 Member Function Documentation	134
5.44.3.1 changeMinProbabilityFalse()	135
5.44.3.2 changeMinProbabilityTrue()	135
5.44.3.3 changeSumProbabilityFalse()	135
5.44.3.4 changeSumProbabilityTrue()	135
5.44.3.5 changeVerificationMode()	135
5.44.3.6 getMinProbability()	135

5.44.3.7 getSumProbability()	135
5.44.3.8 getVerificationMode()	136
5.44.3.9 returnProbability()	136
5.44.3.10 setMinProbability()	136
5.44.3.11 setSumProbability()	136
5.45 ProbabilityTrueFalse Struct Reference	136
5.45.1 Detailed Description	137
5.45.2 Member Data Documentation	137
5.45.2.1 probabilityFalse	137
5.45.2.2 probabilityTrue	137
5.46 ProbAdd Class Reference	137
5.46.1 Detailed Description	138
5.46.2 Constructor & Destructor Documentation	138
5.46.2.1 ProbAdd()	138
5.46.3 Member Function Documentation	139
5.46.3.1 eval() [1/2]	139
5.46.3.2 eval() [2/2]	139
5.47 ProbConst Class Reference	139
5.47.1 Detailed Description	141
5.47.2 Constructor & Destructor Documentation	141
5.47.2.1 ProbConst()	141
5.47.3 Member Function Documentation	141
5.47.3.1 eval() [1/2]	141
5.47.3.2 eval() [2/2]	141
5.48 ProbDiv Class Reference	142
5.48.1 Detailed Description	143
5.48.2 Constructor & Destructor Documentation	143
5.48.2.1 ProbDiv()	143
5.48.3 Member Function Documentation	144
5.48.3.1 eval() [1/2]	144
5.48.3.2 eval() [2/2]	144
5.49 ProbMul Class Reference	144
5.49.1 Detailed Description	146
5.49.2 Constructor & Destructor Documentation	146
5.49.2.1 ProbMul()	146
5.49.3 Member Function Documentation	146
5.49.3.1 eval() [1/2]	146
5.49.3.2 eval() [2/2]	146
5.50 ProbNode Class Reference	147
5.50.1 Detailed Description	148
5.50.2 Member Function Documentation	148
5.50.2.1 eval() [1/2]	148

5.50.2.2 eval() [2/2]	148
5.51 ProbSub Class Reference	149
5.51.1 Detailed Description	150
5.51.2 Constructor & Destructor Documentation	150
5.51.2.1 ProbSub()	150
5.51.3 Member Function Documentation	150
5.51.3.1 eval() [1/2]	150
5.51.3.2 eval() [2/2]	150
5.52 Result Struct Reference	151
5.52.1 Member Data Documentation	152
5.52.1.1 probabilityResult	152
5.52.1.2 verificationResult	152
5.53 StateVerificationInfo Struct Reference	152
5.53.1 Detailed Description	153
5.53.2 Member Data Documentation	153
5.53.2.1 controlled	153
5.53.2.2 controlledProbabilisticTransitionsLeftToProcess	153
5.53.2.3 controlledTransitionsLeftToProcess	153
5.53.2.4 depth	154
5.53.2.5 fromState	154
5.53.2.6 globalState	154
5.53.2.7 gotResponseFromOtherState	154
5.53.2.8 gotThroughPreselectedTransition	154
5.53.2.9 hasControlledProbabilistic	154
5.53.2.10 hasUncontrolledProbabilistic	154
5.53.2.11 hasValidControlledTransition	154
5.53.2.12 hasValidUncontrolledTransition	155
5.53.2.13 isControlledByCoalition	155
5.53.2.14 processed	155
5.53.2.15 uncontrolled	155
5.53.2.16 uncontrolledProbabilisticTransitionsLeftToProcess	155
5.53.2.17 uncontrolledTransitionsLeftToProcess	155
5.53.2.18 verifResult	155
5.54 StrategyBitsComparator Struct Reference	156
5.54.1 Detailed Description	156
5.54.2 Member Function Documentation	156
5.54.2.1 operator()	156
5.55 StrategyCollection Class Reference	157
5.55.1 Detailed Description	157
5.55.2 Constructor & Destructor Documentation	157
5.55.2.1 StrategyCollection() [1/2]	157
5.55.2.2 ~StrategyCollection()	158

5.55.2.3 StrategyCollection() [2/2]	158
5.55.3 Member Function Documentation	158
5.55.3.1 addAction()	158
5.55.3.2 addStrategy()	158
5.55.3.3 clear()	158
5.55.3.4 getStrategy()	158
5.56 StrategyCollectionTemplate Class Reference	159
5.56.1 Constructor & Destructor Documentation	159
5.56.1.1 StrategyCollectionTemplate()	159
5.56.2 Member Function Documentation	159
5.56.2.1 addAction()	159
5.56.3 Member Data Documentation	160
5.56.3.1 selectedStrategy	160
5.57 StrategyEntry Struct Reference	160
5.57.1 Member Data Documentation	161
5.57.1.1 actionName	161
5.57.1.2 globalValues	161
5.57.1.3 type	161
5.58 StrategyParser Class Reference	161
5.58.1 Detailed Description	162
5.58.2 Constructor & Destructor Documentation	162
5.58.2.1 StrategyParser()	162
5.58.2.2 ~StrategyParser()	162
5.58.3 Member Function Documentation	162
5.58.3.1 parse()	162
5.59 STRATSTYPE Union Reference	163
5.59.1 Member Data Documentation	163
5.59.1.1 expr	164
5.59.1.2 floatno	164
5.59.1.3 ident	164
5.59.1.4 identList	164
5.59.1.5 val	164
5.60 TransitionTemplate Class Reference	164
5.60.1 Detailed Description	165
5.60.2 Constructor & Destructor Documentation	165
5.60.2.1 TransitionTemplate()	165
5.60.3 Member Data Documentation	166
5.60.3.1 assignments	166
5.60.3.2 condition	166
5.60.3.3 endState	166
5.60.3.4 matchName	166
5.60.3.5 patternName	167

5.60.3.6 probability	167
5.60.3.7 shared	167
5.60.3.8 startState	167
5.61 Var Struct Reference	167
5.61.1 Detailed Description	168
5.61.2 Member Data Documentation	168
5.61.2.1 agent	168
5.61.2.2 initialValue	168
5.61.2.3 name	168
5.61.2.4 persistent	169
5.62 Verification Class Reference	169
5.62.1 Detailed Description	171
5.62.2 Constructor & Destructor Documentation	171
5.62.2.1 Verification()	171
5.62.2.2 $\sim$ Verification()	172
5.62.3 Member Function Documentation	172
5.62.3.1 addHistoryContext()	172
5.62.3.2 addHistoryDecision()	172
5.62.3.3 addHistoryMarkDecisionAsInvalid()	173
5.62.3.4 addHistoryStateStatus()	173
5.62.3.5 addHistoryUncontrolledDecision()	173
5.62.3.6 areGlobalStatesInTheSameEpistemicClass()	174
5.62.3.7 checkUncontrolledSet()	174
5.62.3.8 equivalentGlobalTransitions()	175
5.62.3.9 fixpointVerify()	175
5.62.3.10 getEpistemicClassForGlobalState()	175
5.62.3.11 getNaturalStrategy()	176
5.62.3.12 getReducedStrategy()	176
5.62.3.13 getStrategyComplexity()	176
5.62.3.14 globalStateToValueBits()	176
5.62.3.15 historyDecisionsERR()	177
5.62.3.16 increaseProbability()	177
5.62.3.17 isAgentInCoalition()	177
5.62.3.18 isGlobalTransitionControlledByCoalition()	178
5.62.3.19 lowerProbability()	178
5.62.3.20 maxFixpointVerify()	178
5.62.3.21 minFixpointVerify()	178
5.62.3.22 newHistoryMarkDecisionAsInvalid()	179
5.62.3.23 printCurrentHistory()	179
5.62.3.24 reduceStrategy()	179
5.62.3.25 restoreHistory()	180
5.62.3.26 revertLastDecision()	180

5.62.3.27 undoHistoryUntil()	181
5.62.3.28 undoLastHistoryEntry()	181
5.62.3.29 verify()	181
5.62.3.30 verifyGlobalState()	181
5.62.3.31 verifyLocalStates()	182
5.62.3.32 verifyMDP()	182
5.62.3.33 verifyStrategy()	182
5.62.3.34 verifyTransitionSets()	183
5.62.4 Member Data Documentation	183
5.62.4.1 generator	183
5.62.4.2 historyEnd	183
5.62.4.3 historyStart	183
5.62.4.4 historyToRestore	184
5.62.4.5 mode	184
5.62.4.6 naturalStrategy	184
5.62.4.7 reductionComplexityBefore	184
5.62.4.8 revertToGlobalState	184
5.62.4.9 strategyVariableLimit	184
5.62.4.10 variableNames	185
5.63 VerificationIterative Class Reference	185
5.63.1 Detailed Description	187
5.63.2 Constructor & Destructor Documentation	187
5.63.2.1 VerificationIterative()	187
5.63.2.2 ~VerificationIterative()	187
5.63.3 Member Function Documentation	188
5.63.3.1 addHistoryAnswerProbability()	188
5.63.3.2 addHistoryContext()	188
5.63.3.3 addHistoryDecision()	188
5.63.3.4 addHistoryMarkDecisionAsInvalid()	189
5.63.3.5 addHistoryProbability()	189
5.63.3.6 addHistoryStateStatus()	189
5.63.3.7 areGlobalStatesInTheSameEpistemicClass()	190
5.63.3.8 checkIfProbabilistic()	190
5.63.3.9 dissolveLoopedProbabilityTransition()	190
5.63.3.10 equivalentGlobalTransitions()	191
5.63.3.11 findLoopedProbabilityTransitions()	191
5.63.3.12 getEpistemicClassForGlobalState()	191
5.63.3.13 getNaturalStrategy()	192
5.63.3.14 getReducedStrategy()	192
5.63.3.15 getStrategyComplexity()	192
5.63.3.16 globalStateToValueBits()	192
5.63.3.17 isAgentInCoalition()	193

5.63.3.18 isGlobalTransitionControlledByCoalition()	193
5.63.3.19 reduceStrategy()	193
5.63.3.20 revertToLastDecision()	194
5.63.3.21 testForAndFixBadAgents()	194
5.63.3.22 undoLastHistoryEntry()	195
5.63.3.23 verify()	195
5.63.3.24 verifyLocalStates()	195
5.63.4 Member Data Documentation	195
5.63.4.1 generator	195
5.63.4.2 history	196
5.63.4.3 mode	196
5.63.4.4 naturalStrategy	196
5.63.4.5 reductionComplexityBefore	196
5.63.4.6 revertToGlobalState	196
5.63.4.7 statesToProcess	196
5.63.4.8 strategyVariableLimit	197
5.63.4.9 variableNames	197
5.64 yy_buffer_state Struct Reference	197
5.64.1 Member Data Documentation	198
5.64.1.1 yy_at_bol	198
5.64.1.2 yy_bs_column	198
5.64.1.3 yy_bs_lineno	198
5.64.1.4 yy_buf_pos	198
5.64.1.5 yy_buf_size	198
5.64.1.6 yy_buffer_status	199
5.64.1.7 yy_ch_buf	199
5.64.1.8 yy_fill_buffer	199
5.64.1.9 yy_input_file	199
5.64.1.10 yy_is_interactive	199
5.64.1.11 yy_is_our_buffer	199
5.64.1.12 yy_n_chars	199
5.65 yy_trans_info Struct Reference	200
5.65.1 Member Data Documentation	200
5.65.1.1 yy_nxt	200
5.65.1.2 yy_verify	200
5.66 yyallocc Union Reference	201
5.66.1 Member Data Documentation	201
5.66.1.1 yyss_alloc	201
5.66.1.2 yyvs_alloc	201
5.67 YYSTYPE Union Reference	202
5.67.1 Member Data Documentation	202
5.67.1.1 agent	202

5.67.1.2 assign	202
5.67.1.3 assignSet	203
5.67.1.4 condList	203
5.67.1.5 expr	203
5.67.1.6 floatno	203
5.67.1.7 ident	203
5.67.1.8 identSet	203
5.67.1.9 model	203
5.67.1.10 prob	203
5.67.1.11 trans	204
5.67.1.12 val	204
6 File Documentation	205
6.1 Agent.cpp File Reference	205
6.1.1 Detailed Description	205
6.2 Agent.hpp File Reference	205
6.2.1 Detailed Description	206
6.3 Common.hpp File Reference	206
6.3.1 Detailed Description	207
6.4 ConditionOperator.hpp File Reference	207
6.4.1 Enumeration Type Documentation	208
6.4.1.1 ConditionOperator	208
6.5 Constants.hpp File Reference	208
6.5.1 Macro Definition Documentation	208
6.5.1.1 DEBUG_ON	208
6.5.1.2 VERBOSE	209
6.6 DotGraph.cpp File Reference	209
6.6.1 Detailed Description	209
6.7 DotGraph.hpp File Reference	209
6.7.1 Detailed Description	210
6.7.2 Enumeration Type Documentation	210
6.7.2.1 DotGraphBase	210
6.8 EpistemicClass.hpp File Reference	211
6.8.1 Detailed Description	211
6.9 expressions.cc File Reference	211
6.9.1 Detailed Description	212
6.10 expressions.hpp File Reference	212
6.10.1 Detailed Description	213
6.10.2 Typedef Documentation	214
6.10.2.1 Environment	214
6.11 GlobalModel.hpp File Reference	214
6.11.1 Detailed Description	214

6.12 GlobalModelGenerator.cpp File Reference	215
6.12.1 Detailed Description	215
6.12.2 Variable Documentation	215
6.12.2.1 config	215
6.13 GlobalModelGenerator.hpp File Reference	216
6.13.1 Detailed Description	216
6.14 GlobalState.cpp File Reference	217
6.14.1 Detailed Description	217
6.15 GlobalState.hpp File Reference	217
6.15.1 Detailed Description	218
6.15.2 Variable Documentation	218
6.15.2.1 config	218
6.16 GlobalStateVerificationStatus.hpp File Reference	218
6.16.1 Enumeration Type Documentation	218
6.16.1.1 GlobalStateVerificationStatus	218
6.17 GlobalTransition.cpp File Reference	219
6.17.1 Detailed Description	219
6.18 GlobalTransition.hpp File Reference	219
6.18.1 Detailed Description	220
6.19 LocalState.cpp File Reference	220
6.19.1 Detailed Description	220
6.20 LocalState.hpp File Reference	221
6.20.1 Detailed Description	221
6.21 LocalTransition.hpp File Reference	221
6.21.1 Detailed Description	222
6.22 main.cpp File Reference	222
6.22.1 Function Documentation	223
6.22.1.1 main()	223
6.22.2 Variable Documentation	223
6.22.2.1 config	223
6.22.2.2 formulaDescription	223
6.22.2.3 modelDescription	223
6.23 ModelParser.cc File Reference	224
6.23.1 Detailed Description	224
6.23.2 Function Documentation	224
6.23.2.1 set_input_string()	224
6.23.2.2 yyparse()	225
6.23.2.3 yyrestart()	225
6.23.3 Variable Documentation	225
6.23.3.1 formulaDescription	225
6.23.3.2 modelDescription	225
6.24 ModelParser.hpp File Reference	225

6.25 nodes.cc File Reference	226
6.25.1 Detailed Description	226
6.26 nodes.hpp File Reference	227
6.26.1 Detailed Description	227
6.27 parser.c File Reference	228
6.27.1 Macro Definition Documentation	231
6.27.1.1 YY_	231
6.27.1.2 YY_ACCESSING_SYMBOL	231
6.27.1.3 YY_ASSERT	231
6.27.1.4 YY_ATTRIBUTE_PURE	231
6.27.1.5 YY_ATTRIBUTE_UNUSED	231
6.27.1.6 YY_CAST	232
6.27.1.7 YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN	232
6.27.1.8 YY_IGNORE_MAYBE_UNINITIALIZED_END	232
6.27.1.9 YY_IGNORE_USELESS_CAST_BEGIN	232
6.27.1.10 YY_IGNORE_USELESS_CAST_END	232
6.27.1.11 YY_INITIAL_VALUE	232
6.27.1.12 YY_NULLPTR	232
6.27.1.13 YY_REDUCE_PRINT	233
6.27.1.14 YY_REINTERPRET_CAST	233
6.27.1.15 YY_STACK_PRINT	233
6.27.1.16 YY_SYMBOL_PRINT	233
6.27.1.17 YY_USE	233
6.27.1.18 YYABORT	233
6.27.1.19 YYACCEPT	234
6.27.1.20 YYBACKUP	234
6.27.1.21 YYBISON	234
6.27.1.22 YYBISON_VERSION	234
6.27.1.23 yyclearin	234
6.27.1.24 YYCOPY	235
6.27.1.25 YYCOPY_NEEDED	235
6.27.1.26 YYDPRINTF	235
6.27.1.27 YYERRCODE	235
6.27.1.28 yyerrok	235
6.27.1.29 YYERROR	235
6.27.1.30 YYFINAL	236
6.27.1.31 YYFREE	236
6.27.1.32 YYINITDEPTH	236
6.27.1.33 YYLAST	236
6.27.1.34 YYMALLOC	236
6.27.1.35 YYMAXDEPTH	236
6.27.1.36 YYMAXUTOK	236

6.27.1.37 YNNNTS	236
6.27.1.38 YYNOMEM	237
6.27.1.39 YYNRULES	237
6.27.1.40 YYNSTATES	237
6.27.1.41 YYNTOKENS	237
6.27.1.42 YYPACT_NINF	237
6.27.1.43 yypact_value_is_default	237
6.27.1.44 YYPOPSTACK	237
6.27.1.45 YYPTRDIFF_MAXIMUM	238
6.27.1.46 YYPTRDIFF_T	238
6.27.1.47 YYPULL	238
6.27.1.48 YYPURE	238
6.27.1.49 YYPUSH	238
6.27.1.50 YYRECOVERING	238
6.27.1.51 YYSIZE_MAXIMUM	238
6.27.1.52 YYSIZE_T	239
6.27.1.53 YYSIZEOF	239
6.27.1.54 YYSKELETON_NAME	239
6.27.1.55 YYSTACK_ALLOC	239
6.27.1.56 YYSTACK_ALLOC_MAXIMUM	239
6.27.1.57 YYSTACK_BYTES	239
6.27.1.58 YYSTACK_FREE	239
6.27.1.59 YYSTACK_GAP_MAXIMUM	240
6.27.1.60 YYSTACK_RELOCATE	240
6.27.1.61 YYTABLE_NINF	240
6.27.1.62 yytable_value_is_error	240
6.27.1.63 YYTRANSLATE	240
6.27.2 Typedef Documentation	240
6.27.2.1 YY_BUFFER_STATE	241
6.27.2.2 yy_state_fast_t	241
6.27.2.3 yy_state_t	241
6.27.2.4 yysymbol_kind_t	241
6.27.2.5 yytype_int16	241
6.27.2.6 yytype_int8	241
6.27.2.7 yytype_uint16	241
6.27.2.8 yytype_uint8	241
6.27.3 Enumeration Type Documentation	241
6.27.3.1 anonymous enum	241
6.27.3.2 yysymbol_kind_t	242
6.27.4 Function Documentation	244
6.27.4.1 free()	244
6.27.4.2 main2()	244

6.27.4.3 malloc()	244
6.27.4.4 set_input_string()	244
6.27.4.5 yy_scan_string()	244
6.27.4.6 yyerror()	245
6.27.4.7 yylex()	245
6.27.4.8 yyparse()	245
6.27.4.9 yyrestart()	245
6.27.5 Variable Documentation	245
6.27.5.1 formulaDescription	245
6.27.5.2 modelDescription	245
6.27.5.3 yychar	245
6.27.5.4 yylval	246
6.27.5.5 yynerrs	246
6.27.5.6 yytext	246
6.28 parser.h File Reference	246
6.28.1 Macro Definition Documentation	247
6.28.1.1 YYDEBUG	247
6.28.1.2 YYSTYPE_IS_DECLARED	247
6.28.1.3 YYSTYPE_IS_TRIVIAL	247
6.28.1.4 YYTOKENTYPE	248
6.28.2 Typedef Documentation	248
6.28.2.1 YYSTYPE	248
6.28.2.2 yytoken_kind_t	248
6.28.3 Enumeration Type Documentation	248
6.28.3.1 yytokentype	248
6.28.4 Function Documentation	249
6.28.4.1 yyparse()	249
6.28.5 Variable Documentation	249
6.28.5.1 yylval	249
6.29 README.md File Reference	249
6.30 scanner.c File Reference	249
6.30.1 Macro Definition Documentation	252
6.30.1.1 BEGIN	252
6.30.1.2 ECHO	253
6.30.1.3 EOB_ACT_CONTINUE_SCAN	253
6.30.1.4 EOB_ACT_END_OF_FILE	253
6.30.1.5 EOB_ACT_LAST_MATCH	253
6.30.1.6 FLEX_BETA	253
6.30.1.7 FLEX_SCANNER	253
6.30.1.8 FLEXINT_H	253
6.30.1.9 INITIAL	253
6.30.1.10 INT16_MAX	254

6.30.1.11 INT16_MIN	254
6.30.1.12 INT32_MAX	254
6.30.1.13 INT32_MIN	254
6.30.1.14 INT8_MAX	254
6.30.1.15 INT8_MIN	254
6.30.1.16 REJECT	254
6.30.1.17 SIZE_MAX	254
6.30.1.18 UINT16_MAX	255
6.30.1.19 UINT32_MAX	255
6.30.1.20 UINT8_MAX	255
6.30.1.21 unput	255
6.30.1.22 YY_AT_BOL	255
6.30.1.23 YY_BREAK	255
6.30.1.24 YY_BUF_SIZE	255
6.30.1.25 YY_BUFFER_EOF_PENDING	256
6.30.1.26 YY_BUFFER_NEW	256
6.30.1.27 YY_BUFFER_NORMAL	256
6.30.1.28 YY_CURRENT_BUFFER	256
6.30.1.29 YY_CURRENT_BUFFER_LVALUE	256
6.30.1.30 YY_DECL	256
6.30.1.31 YY_DECL_IS_OURS	256
6.30.1.32 YY_DO_BEFORE_ACTION	257
6.30.1.33 YY_END_OF_BUFFER	257
6.30.1.34 YY_END_OF_BUFFER_CHAR	257
6.30.1.35 YY_EXIT_FAILURE	257
6.30.1.36 YY_EXTRA_TYPE	257
6.30.1.37 YY_FATAL_ERROR	257
6.30.1.38 YY_FLEX_MAJOR_VERSION	257
6.30.1.39 YY_FLEX_MINOR_VERSION	258
6.30.1.40 YY_FLEX_SUBMINOR_VERSION	258
6.30.1.41 YY_FLUSH_BUFFER	258
6.30.1.42 YY_INPUT	258
6.30.1.43 YY_INT_ALIGNED	258
6.30.1.44 YY_LESS_LINENO	259
6.30.1.45 YY_LINENO_REWIND_TO	259
6.30.1.46 YY_MORE_ADJ	259
6.30.1.47 yy_new_buffer	259
6.30.1.48 YY_NEW_FILE	259
6.30.1.49 YY_NULL	259
6.30.1.50 YY_NUM_RULES	259
6.30.1.51 YY_READ_BUF_SIZE	260
6.30.1.52 YY_RESTORE YY_MORE_OFFSET	260

6.30.1.53 YY_RULE_SETUP	260
6.30.1.54 YY_SC_TO_UI	260
6.30.1.55 yy_set_bol	260
6.30.1.56 yy_set_interactive	260
6.30.1.57 YY_SKIP_YYWRAP	261
6.30.1.58 YY_START	261
6.30.1.59 YY_START_STACK_INCR	261
6.30.1.60 YY_STATE_BUF_SIZE	261
6.30.1.61 YY_STATE_EOF	261
6.30.1.62 YY_STRUCT_YY_BUFFER_STATE	261
6.30.1.63 YY_TYPEDEF_YY_BUFFER_STATE	261
6.30.1.64 YY_TYPEDEF_YY_SIZE_T	262
6.30.1.65 YY_USER_ACTION	262
6.30.1.66 yyconst	262
6.30.1.67 yyless [1/2]	262
6.30.1.68 yyless [2/2]	262
6.30.1.69 yymore	263
6.30.1.70 yynoreturn	263
6.30.1.71 YYSTATE	263
6.30.1.72 YYTABLES_NAME	263
6.30.1.73 yyterminate	263
6.30.1.74 yytext_ptr	263
6.30.1.75 yywrap	263
6.30.2 Typedef Documentation	263
6.30.2.1 flex_int16_t	264
6.30.2.2 flex_int32_t	264
6.30.2.3 flex_int8_t	264
6.30.2.4 flex_uint16_t	264
6.30.2.5 flex_uint32_t	264
6.30.2.6 flex_uint8_t	264
6.30.2.7 YY_BUFFER_STATE	264
6.30.2.8 YY_CHAR	264
6.30.2.9 yy_size_t	265
6.30.2.10 yy_state_type	265
6.30.3 Function Documentation	265
6.30.3.1 if()	265
6.30.3.2 yy_create_buffer()	265
6.30.3.3 yy_delete_buffer()	265
6.30.3.4 yy_flush_buffer()	265
6.30.3.5 yy_scan_buffer()	266
6.30.3.6 yy_scan_bytes()	266
6.30.3.7 yy_scan_string()	266

6.30.3.8	yy_switch_to_buffer()	266
6.30.3.9	yyalloc()	266
6.30.3.10	yyfree()	266
6.30.3.11	yyget_debug()	266
6.30.3.12	yyget_extra()	267
6.30.3.13	yyget_in()	267
6.30.3.14	yyget_leng()	267
6.30.3.15	yyget_lineno()	267
6.30.3.16	yyget_out()	267
6.30.3.17	yyget_text()	267
6.30.3.18	yylex()	267
6.30.3.19	yylex_destroy()	268
6.30.3.20	yypop_buffer_state()	268
6.30.3.21	yypush_buffer_state()	268
6.30.3.22	yyrealloc()	268
6.30.3.23	yyrestart()	268
6.30.3.24	yyset_debug()	268
6.30.3.25	yyset_extra()	268
6.30.3.26	yyset_in()	269
6.30.3.27	yyset_lineno()	269
6.30.3.28	yyset_out()	269
6.30.4	Variable Documentation	269
6.30.4.1	yy_act	269
6.30.4.2	yy_bp	269
6.30.4.3	yy_cp	269
6.30.4.4	YY_DECL	270
6.30.4.5	yy_flex_debug	270
6.30.4.6	yyin	270
6.30.4.7	yyleng	270
6.30.4.8	yylineno	270
6.30.4.9	yyout	270
6.30.4.10	yytext	270
6.31	scanner.l File Reference	271
6.32	strategyNodes.cc File Reference	271
6.33	strategyNodes.hpp File Reference	271
6.33.1	Detailed Description	272
6.34	strategyParser.c File Reference	272
6.34.1	Macro Definition Documentation	275
6.34.1.1	YY_	275
6.34.1.2	YY_ACCESSING_SYMBOL	275
6.34.1.3	YY_ASSERT	275
6.34.1.4	YY_ATTRIBUTE_PURE	276

6.34.1.5 YY_ATTRIBUTE_UNUSED	276
6.34.1.6 YY_CAST	276
6.34.1.7 YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN	276
6.34.1.8 YY_IGNORE_MAYBE_UNINITIALIZED_END	276
6.34.1.9 YY_IGNORE_USELESS_CAST_BEGIN	276
6.34.1.10 YY_IGNORE_USELESS_CAST_END	276
6.34.1.11 YY_INITIAL_VALUE	277
6.34.1.12 YY_NULLPTR	277
6.34.1.13 YY_REDUCE_PRINT	277
6.34.1.14 YY_REINTERPRET_CAST	277
6.34.1.15 YY_STACK_PRINT	277
6.34.1.16 YY_SYMBOL_PRINT	277
6.34.1.17 YY_USE	278
6.34.1.18 YYABORT	278
6.34.1.19 YYACCEPT	278
6.34.1.20 YYBACKUP	278
6.34.1.21 YYBISON	278
6.34.1.22 YYBISON_VERSION	278
6.34.1.23 yychar	279
6.34.1.24 yyclearin	279
6.34.1.25 YYCOPY	279
6.34.1.26 YYCOPY_NEEDED	279
6.34.1.27 yydebug	279
6.34.1.28 YYDPRINTF	279
6.34.1.29 YYERRCODE	280
6.34.1.30 yyerrok	280
6.34.1.31 yyerror	280
6.34.1.32 YYERROR	280
6.34.1.33 YYFINAL	280
6.34.1.34 YYFREE	280
6.34.1.35 YYINITDEPTH	280
6.34.1.36 YYLAST	280
6.34.1.37 yylex	281
6.34.1.38 yylval	281
6.34.1.39 YYMALLOC	281
6.34.1.40 YYMAXDEPTH	281
6.34.1.41 YYMAXUTOK	281
6.34.1.42 yynerrs	281
6.34.1.43 YYNNTS	281
6.34.1.44 YYNOMEM	281
6.34.1.45 YYNRULES	282
6.34.1.46 YYNSTATES	282

6.34.1.47 YYNTOKENS	282
6.34.1.48 YYPACT_NINF	282
6.34.1.49 yypact_value_is_default	282
6.34.1.50 yyparse	282
6.34.1.51 YYPOPSTACK	282
6.34.1.52 YYPTRDIFF_MAXIMUM	283
6.34.1.53 YYPTRDIFF_T	283
6.34.1.54 YYPULL	283
6.34.1.55 YYPURE	283
6.34.1.56 YYPUSH	283
6.34.1.57 YYRECOVERING	283
6.34.1.58 YYSIZE_MAXIMUM	283
6.34.1.59 YYSIZE_T	284
6.34.1.60 YYSIZEOF	284
6.34.1.61 YYSKELETON_NAME	284
6.34.1.62 YYSTACK_ALLOC	284
6.34.1.63 YYSTACK_ALLOC_MAXIMUM	284
6.34.1.64 YYSTACK_BYTES	284
6.34.1.65 YYSTACK_FREE	284
6.34.1.66 YYSTACK_GAP_MAXIMUM	285
6.34.1.67 YYSTACK_RELOCATE	285
6.34.1.68 YYSTYPE	285
6.34.1.69 YYTABLE_NINF	285
6.34.1.70 yytable_value_is_error	285
6.34.1.71 YYTRANSLATE	285
6.34.2 Typedef Documentation	286
6.34.2.1 YY_BUFFER_STATE	286
6.34.2.2 yy_state_fast_t	286
6.34.2.3 yy_state_t	286
6.34.2.4 yysymbol_kind_t	286
6.34.2.5 yytype_int16	286
6.34.2.6 yytype_int8	286
6.34.2.7 yytype_uint16	286
6.34.2.8 yytype_uint8	286
6.34.3 Enumeration Type Documentation	286
6.34.3.1 anonymous enum	286
6.34.3.2 yysymbol_kind_t	287
6.34.4 Function Documentation	289
6.34.4.1 free()	289
6.34.4.2 malloc()	289
6.34.4.3 set_input_string_strat()	289
6.34.4.4 strat_scan_string()	289

6.34.4.5 straterror()	289
6.34.4.6 stratlex()	290
6.34.4.7 stratrestart()	290
6.34.5 Variable Documentation	290
6.34.5.1 strategyCollectionTemplate	290
6.34.5.2 strattext	290
6.34.5.3 yychar	290
6.34.5.4 yylval	290
6.34.5.5 yynerrs	290
6.35 StrategyParser.cc File Reference	291
6.35.1 Detailed Description	291
6.35.2 Function Documentation	291
6.35.2.1 set_input_string_strat()	291
6.35.2.2 stratparse()	292
6.35.2.3 stratrestart()	292
6.35.3 Variable Documentation	292
6.35.3.1 strategyCollectionTemplate	292
6.36 strategyParser.h File Reference	292
6.36.1 Macro Definition Documentation	293
6.36.1.1 STRATDEBUG	293
6.36.1.2 STRATSTYPE_IS_DECLARED	293
6.36.1.3 STRATSTYPE_IS_TRIVIAL	293
6.36.1.4 STRATTOKENTYPE	294
6.36.2 Typedef Documentation	294
6.36.2.1 STRATSTYPE	294
6.36.2.2 strattoken_kind_t	294
6.36.3 Enumeration Type Documentation	294
6.36.3.1 strattokentype	294
6.36.4 Function Documentation	295
6.36.4.1 stratparse()	295
6.36.5 Variable Documentation	295
6.36.5.1 stratlval	295
6.37 StrategyParser.hpp File Reference	295
6.37.1 Detailed Description	296
6.38 strategyScanner.c File Reference	296
6.38.1 Macro Definition Documentation	300
6.38.1.1 BEGIN	300
6.38.1.2 ECHO	300
6.38.1.3 EOB_ACT_CONTINUE_SCAN	300
6.38.1.4 EOB_ACT_END_OF_FILE	300
6.38.1.5 EOB_ACT_LAST_MATCH	300
6.38.1.6 FLEX_BETA	301

6.38.1.7 FLEX_SCANNER	301
6.38.1.8 FLEXINT_H	301
6.38.1.9 INITIAL	301
6.38.1.10 INT16_MAX	301
6.38.1.11 INT16_MIN	301
6.38.1.12 INT32_MAX	301
6.38.1.13 INT32_MIN	301
6.38.1.14 INT8_MAX	302
6.38.1.15 INT8_MIN	302
6.38.1.16 REJECT	302
6.38.1.17 SIZE_MAX	302
6.38.1.18 strat_create_buffer_ALREADY_DEFINED	302
6.38.1.19 strat_delete_buffer_ALREADY_DEFINED	302
6.38.1.20 strat_flex_debug_ALREADY_DEFINED	302
6.38.1.21 strat_flush_buffer_ALREADY_DEFINED	302
6.38.1.22 strat_init_buffer_ALREADY_DEFINED	303
6.38.1.23 strat_load_buffer_state_ALREADY_DEFINED	303
6.38.1.24 strat_scan_buffer_ALREADY_DEFINED	303
6.38.1.25 strat_scan_bytes_ALREADY_DEFINED	303
6.38.1.26 strat_scan_string_ALREADY_DEFINED	303
6.38.1.27 strat_switch_to_buffer_ALREADY_DEFINED	303
6.38.1.28 stratalloc_ALREADY_DEFINED	303
6.38.1.29 stratensure_buffer_stack_ALREADY_DEFINED	303
6.38.1.30 stratfree_ALREADY_DEFINED	304
6.38.1.31 stratin_ALREADY_DEFINED	304
6.38.1.32 stratleng_ALREADY_DEFINED	304
6.38.1.33 stratlex_ALREADY_DEFINED	304
6.38.1.34 stratlineno_ALREADY_DEFINED	304
6.38.1.35 stratout_ALREADY_DEFINED	304
6.38.1.36 stratpop_buffer_state_ALREADY_DEFINED	304
6.38.1.37 stratpush_buffer_state_ALREADY_DEFINED	304
6.38.1.38 stratrealloc_ALREADY_DEFINED	305
6.38.1.39 stratrestart_ALREADY_DEFINED	305
6.38.1.40 strattext_ALREADY_DEFINED	305
6.38.1.41 stratwrap	305
6.38.1.42 stratwrap_ALREADY_DEFINED	305
6.38.1.43 UINT16_MAX	305
6.38.1.44 UINT32_MAX	305
6.38.1.45 UINT8_MAX	305
6.38.1.46 unput	306
6.38.1.47 YY_AT_BOL	306
6.38.1.48 YY_BREAK	306

6.38.1.49 YY_BUF_SIZE	306
6.38.1.50 YY_BUFFER_EOF_PENDING	306
6.38.1.51 YY_BUFFER_NEW	306
6.38.1.52 YY_BUFFER_NORMAL	306
6.38.1.53 yy_create_buffer	307
6.38.1.54 YY_CURRENT_BUFFER	307
6.38.1.55 YY_CURRENT_BUFFER_LVALUE	307
6.38.1.56 YY_DECL	307
6.38.1.57 YY_DECL_IS_OURS	307
6.38.1.58 yy_delete_buffer	307
6.38.1.59 YY_DO_BEFORE_ACTION	307
6.38.1.60 YY_END_OF_BUFFER	308
6.38.1.61 YY_END_OF_BUFFER_CHAR	308
6.38.1.62 YY_EXIT_FAILURE	308
6.38.1.63 YY_EXTRA_TYPE	308
6.38.1.64 YY_FATAL_ERROR	308
6.38.1.65 yy_flex_debug	308
6.38.1.66 YY_FLEX_MAJOR_VERSION	308
6.38.1.67 YY_FLEX_MINOR_VERSION	309
6.38.1.68 YY_FLEX_SUBMINOR_VERSION	309
6.38.1.69 yy_flush_buffer	309
6.38.1.70 YY_FLUSH_BUFFER	309
6.38.1.71 yy_init_buffer	309
6.38.1.72 YY_INPUT	309
6.38.1.73 YY_INT_ALIGNED	310
6.38.1.74 YY_LESS_LINENO	310
6.38.1.75 YY_LINENO_REWIND_TO	310
6.38.1.76 yy_load_buffer_state	310
6.38.1.77 YY_MORE_ADJ	310
6.38.1.78 yy_new_buffer	310
6.38.1.79 YY_NEW_FILE	310
6.38.1.80 YY_NULL	311
6.38.1.81 YY_NUM_RULES	311
6.38.1.82 YY_READ_BUF_SIZE	311
6.38.1.83 YY_RESTORE YY_MORE_OFFSET	311
6.38.1.84 YY_RULE_SETUP	311
6.38.1.85 YY_SC_TO_UI	311
6.38.1.86 yy_scan_buffer	311
6.38.1.87 yy_scan_bytes	312
6.38.1.88 yy_scan_string	312
6.38.1.89 yy_set_bol	312
6.38.1.90 yy_set_interactive	312

6.38.1.91 YY_SKIP_YYWRAP	312
6.38.1.92 YY_START	313
6.38.1.93 YY_START_STACK_INCR	313
6.38.1.94 YY_STATE_BUF_SIZE	313
6.38.1.95 YY_STATE_EOF	313
6.38.1.96 YY_STRUCT_YY_BUFFER_STATE	313
6.38.1.97 yy_switch_to_buffer	313
6.38.1.98 YY_TYPEDEF_YY_BUFFER_STATE	313
6.38.1.99 YY_TYPEDEF_YY_SIZE_T	314
6.38.1.100 YY_USER_ACTION	314
6.38.1.101 yyalloc	314
6.38.1.102 yyconst	314
6.38.1.103 yyensure_buffer_stack	314
6.38.1.104 yyfree	314
6.38.1.105 yyget_debug	314
6.38.1.106 yyget_extra	314
6.38.1.107 yyget_in	315
6.38.1.108 yyget_leng	315
6.38.1.109 yyget_lineno	315
6.38.1.110 yyget_out	315
6.38.1.111 yyget_text	315
6.38.1.112 yyin	315
6.38.1.113 yyleng	315
6.38.1.114 yyless [1/2]	316
6.38.1.115 yyless [2/2]	316
6.38.1.116 yylex	316
6.38.1.117 yylex_destroy	316
6.38.1.118 yylex_init	316
6.38.1.119 yylex_init_extra	317
6.38.1.120 yylineno	317
6.38.1.121 yymore	317
6.38.1.122 yynoreturn	317
6.38.1.123 yyout	317
6.38.1.124 yypop_buffer_state	317
6.38.1.125 yypush_buffer_state	317
6.38.1.126 yyrealloc	317
6.38.1.127 yyrestart	318
6.38.1.128 yyset_debug	318
6.38.1.129 yyset_extra	318
6.38.1.130 yyset_in	318
6.38.1.131 yyset_lineno	318
6.38.1.132 yyset_out	318

6.38.1.133 YYSTATE	318
6.38.1.134 YYTABLES_NAME	318
6.38.1.135 yyterminate	319
6.38.1.136 yytext	319
6.38.1.137 yytext_ptr	319
6.38.1.138 yywrap	319
6.38.2 Typedef Documentation	319
6.38.2.1 flex_int16_t	319
6.38.2.2 flex_int32_t	319
6.38.2.3 flex_int8_t	319
6.38.2.4 flex_uint16_t	320
6.38.2.5 flex_uint32_t	320
6.38.2.6 flex_uint8_t	320
6.38.2.7 YY_BUFFER_STATE	320
6.38.2.8 YY_CHAR	320
6.38.2.9 yy_size_t	320
6.38.2.10 yy_state_type	320
6.38.3 Function Documentation	320
6.38.3.1 if()	321
6.38.3.2 stratlex()	321
6.38.3.3 yy_create_buffer()	321
6.38.3.4 yy_delete_buffer()	321
6.38.3.5 yy_flush_buffer()	321
6.38.3.6 yy_scan_buffer()	321
6.38.3.7 yy_scan_bytes()	321
6.38.3.8 yy_scan_string()	322
6.38.3.9 yy_switch_to_buffer()	322
6.38.3.10 yyalloc()	322
6.38.3.11 yyfree()	322
6.38.3.12 yypush_buffer_state()	322
6.38.3.13 yyrealloc()	322
6.38.3.14 yyrestart()	322
6.38.3.15 yyset_debug()	323
6.38.3.16 yyset_extra()	323
6.38.3.17 yyset_in()	323
6.38.3.18 yyset_lineno()	323
6.38.3.19 yyset_out()	323
6.38.4 Variable Documentation	323
6.38.4.1 yy_act	323
6.38.4.2 yy_bp	324
6.38.4.3 yy_cp	324
6.38.4.4 YY_DECL	324

6.38.4.5 yy_flex_debug	324
6.38.4.6 yyin	324
6.38.4.7 yyleng	324
6.38.4.8 yylineno	324
6.38.4.9 yyout	325
6.38.4.10 yytext	325
6.39 strategyScanner.l File Reference	325
6.40 test.cpp File Reference	325
6.40.1 Function Documentation	325
6.40.1.1 main()	326
6.40.2 Variable Documentation	326
6.40.2.1 config	326
6.41 Types.hpp File Reference	326
6.41.1 Detailed Description	327
6.41.2 Enumeration Type Documentation	327
6.41.2.1 ProbabilitySign	327
6.41.2.2 VerificationFormulaMode	328
6.42 TypesDependency.hpp File Reference	328
6.42.1 Detailed Description	329
6.42.2 Macro Definition Documentation	329
6.42.2.1 STRATEGY_BITS	329
6.42.3 Enumeration Type Documentation	329
6.42.3.1 HistoryEntryType	329
6.42.3.2 ProbabilityCalculationType	330
6.42.3.3 StrategyEntryType	330
6.42.3.4 VerifResult	330
6.43 Utils.cpp File Reference	331
6.43.1 Detailed Description	332
6.43.2 Function Documentation	332
6.43.2.1 agentToString()	332
6.43.2.2 envToString()	332
6.43.2.3 getContextModel()	332
6.43.2.4 getLocalStatesSCC()	333
6.43.2.5 getMemCap()	333
6.43.2.6 loadConfigFromArgs()	333
6.43.2.7 loadConfigFromFile()	333
6.43.2.8 localModelsToString()	333
6.43.2.9 outputGlobalModel()	334
6.43.2.10 tarjanVisit()	334
6.43.3 Variable Documentation	335
6.43.3.1 config	335
6.44 Utils.hpp File Reference	335

6.44.1 Function Documentation	336
6.44.1.1 agentToString()	336
6.44.1.2 envToString()	336
6.44.1.3 getContextModel()	337
6.44.1.4 getLocalStatesSCC()	337
6.44.1.5 getMemCap()	337
6.44.1.6 loadConfigFromArgs()	337
6.44.1.7 loadConfigFromFile()	338
6.44.1.8 localModelsToString()	338
6.44.1.9 outputGlobalModel()	338
6.45 Verification.cpp File Reference	338
6.45.1 Detailed Description	339
6.45.2 Macro Definition Documentation	339
6.45.2.1 DEPTH_PREFIX	339
6.45.3 Function Documentation	339
6.45.3.1 dbgHistEnt()	339
6.45.3.2 dbgVerifStatus()	340
6.45.3.3 verStatusToStr()	340
6.45.4 Variable Documentation	340
6.45.4.1 config	340
6.46 Verification.hpp File Reference	341
6.46.1 Macro Definition Documentation	342
6.46.1.1 STRATEGY_BITS	342
6.46.2 Enumeration Type Documentation	342
6.46.2.1 TraversalMode	342
6.46.3 Function Documentation	342
6.46.3.1 verStatusToStr()	342
6.47 VerificationIterative.cpp File Reference	343
6.47.1 Detailed Description	343
6.47.2 Macro Definition Documentation	343
6.47.2.1 DEPTH_PREFIX	344
6.47.3 Function Documentation	344
6.47.3.1 verifStatusToStr()	344
6.47.4 Variable Documentation	344
6.47.4.1 config	344
6.48 VerificationIterative.hpp File Reference	344
6.48.1 Enumeration Type Documentation	345
6.48.1.1 GraphTraversalMode	345
6.48.2 Function Documentation	346
6.48.2.1 verifStatusToStr()	346

Chapter 1

STV2 - StraTegic Verifier version 2

1.1 Usage

To run:

```
cd build
make clean
make
./stv
```

or

```
cd build
./build-run
```

Configuration file:

build/config.txt

CLI configuration overwrite:

```
# Input model
./stv --file PATH_TO_MODEL
./stv -f PATH_TO_MODEL
# Mode
./stv -m 0 #
./stv -m 1 # generate GlobalModel
./stv -m 2 # run verification
./stv -m 3 # same as 1 && 2
# Flags
# --OUTPUT_GLOBAL_MODEL      stdout data on global model (after expandAllStates)
# --OUTPUT_LOCAL_MODELS     stdout data on local models ()
# --OUTPUT_DOT_FILES        generate .dot files for agent templates, local and global models
# --ADD_EPSILON_TRANSITIONS generate global models with epsilon transitions
# --OVERWRITE_FORMULA       replace the formula from the model file with a different one
# --COUNTEREXAMPLE          output counterexample path if the formula verification returns an ERR
# --REDUCE                  reduce the amount of states and transitions using a DFS-POR algorithm and
#                           select the first correct transition
# --REDUCE_ALL              reduce the amount of states and transitions using a DFS-POR algorithm and
#                           select all available transitions
# --FIXPOINT                enables fixpoint approximation
# --NATURAL_STRATEGY        generate a natural strategy for the given model
# --STRATEGY_FROM_FILE      set a strategy to the one given in a file
```

1.2 Tests

To run tests:

```
cd build
make clean
make sample_test
./sample_test
```

```
or
cd build
./build-test
```

To run larger tests:

```
cd build
make clean
make sample_test
./sample_test
```

```
or
cd build
./build-big-test
```

You might need to run

```
ulimit -s unlimited
```

beforehand

1.3 Performance estimation

Ubuntu/WSL:

```
# Minimal
> /usr/bin/time -f "%M\t%e" ./stv
# %M - maximum resident set size in KB
# %e - elapsed real time (wall clock) in seconds
# More detailed
> /usr/bin/time -f "time result\ncmd:%C\nreal %es\nuser %Us \nsys %Ss \nmemory:%MKB \ncpu %P" ./stv
# %C command line and arguments
# %e elapsed real time (wall clock) in seconds
# %U user time in seconds
# %S system (kernel) time in seconds
# %M maximum resident set size in KB
# %P percent of CPU this job got
# Full (verbose)
> /usr/bin/time -v ./stv
```

1.4 Specification

The specification language was inspired by ISPL (Interpreted Systems Programming Language) from [MCMAS](#). The detailed syntax for the input format can be derived from `*./src/reader/{parser.y,scanner.l}*`, which intrinsically make up an EBNF grammar.

For the most parts, it is simple enough to get intuition just from looking at example's source code and the program's output.

IMPORTANT NOTES:

1. the (local) action names must be unique;
2. the transition relation (from the global model) should be serial;

1.5 Examples and templates

In `*/examples*` and `*/tests/examples*` there are several ready-to-use MAS specification files together with a proposed property (captured by ATL formula) for verification.

Often, we would want to reason about different (data-)configurations of the same system.

Using the templates we can parameterize the system specification, such that we only need to describe its dynamic behavior.

A template can be fed with a configuration data to generate a concrete instance of a system.

Moreover, their use is independent from the tool: one can choose any templating engine (of the myriads available) or even write a custom one from the scratch.

Here, we utilize the [EJS](#) templating engine. It has a CLI support, which is comes in handy for the tests/benchmarks that involve systems in multiple configurations.

```
# EJS feeds the data (as a list of key:val pairs) to the template file to generate the output:
> npm exec -- ejs TEMPLATE_FILE.ejs -i "{PARAM1:VAL1,PARAM2:VAL2,...}" -o OUTPUT_FILE.txt
# Possible generation query for the "trains":
> npm exec -- ejs Trains.ejs -i '{"N_TRAINS":3,"WITH_FORMULA":1}' -o 3Trains1Controller.txt
# Possible generation query for the "simple voting":
> npm exec -- ejs Simple_voting.ejs -i '{"N_VOTERS":2,"N_CANDIDATES":1,"WITH_FORMULA":0}' -o
    2Voters1Coercer1Candidate.txt
```

1.6 Misc

With `OUTPUT_DOT_FILES` flag the program outputs `*.dot*` files for templates, local and global models where:

- nodes are labelled with its location name (comma-separated for the global state)
- shared transitions are denoted by blue colour

Use Graphviz ([link](#)) to view in other format (eps, pdf, jpeg, etc.):

```
# Analogously for other formats
dot -Tpng lts_of_AGENT.dot > lts_of_AGENT.png
```

For the smaller graphs use `dot2png.sh` script, which converts all `*.dot*` files from a current folder to `*.png*`.

For bigger ones use `svg` format (may be viewed in Inkscape) and `dot2svg.sh`.

1.7 Web interface

The web interface is available at <http://stv.cs-htiew.com/>.

1.8 Credits

Lead developer: Mateusz Kamiński (@Mathisplay)

Project supervisor: Damian Kurpiewski (@blackbat13)

Initial codebase: Witold Pazderski

Additional features: Yan Kim

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Action	13
ActionTemplate	14
Agent	16
AgentTemplate	19
Assignment	25
Cfg	27
Condition	31
DecisionEntry	32
DotGraph	36
EpistemicClass	40
ExprNode	78
ExprAdd	42
ExprAnd	45
ExprConst	47
ExprDiv	50
ExprEq	52
ExprGe	55
ExprGt	57
ExprHart	60
ExprIdent	63
ExprKnow	65
ExprLe	68
ExprLt	70
ExprMul	73
ExprNe	75
ExprNot	79
ExprOr	82
ExprRem	84
ExprSub	87
Formula	89
FormulaTemplate	91
GlobalModel	93
GlobalModelGenerator	95
GlobalState	109
GlobalModelGenerator::GlobalStateTupleHash	113

GlobalTransition	114
HistoryDbg	117
HistoryEntry	120
LocalModels	123
LocalState	124
LocalStateTemplate	127
LocalTransition	128
ModelParser	131
ProbabilityEntry	133
ProbabilityTrueFalse	136
ProbNode	147
ProbAdd	137
ProbConst	139
ProbDiv	142
ProbMul	144
ProbSub	149
Result	151
StateVerificationInfo	152
StrategyBitsComparator	156
StrategyCollection	157
StrategyCollectionTemplate	159
StrategyEntry	160
StrategyParser	161
STATSTYPE	163
TransitionTemplate	164
Var	167
Verification	169
VerificationIterative	185
yy_buffer_state	197
yy_trans_info	200
yyalloc	201
YYSTYPE	202

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Action	Single action template for probabilistic verification	13
ActionTemplate	Represents a selected action	14
Agent	Contains all data for a single Agent , including id, name and all of the agents' variables	16
AgentTemplate	Represents a single agent loaded from the description from a file	19
Assignment	Represents an assingment	25
Cfg		27
Condition	Represents a condition for LocalTransition	31
DecisionEntry		32
DotGraph		36
EpistemicClass	Represents a single epistemic class	40
ExprAdd	Node for addition	42
ExprAnd	Node for AND operator	45
ExprConst	Node for a constant	47
ExprDiv	Node for division	50
ExprEq	Node for "==" operator	52
ExprGe	Node for ">=" operator	55
ExprGt	Node for ">" operator	57
ExprHart	Node for a Hartley operator	60
ExprIdent	Node for an identifier	63

ExprKnow	Node for a knowledge operator	65
ExprLe	Node for "<=" operator	68
ExprLt	Node for "<" operator	70
ExprMul	Node for multiplication	73
ExprNe	Node for "!=" operator	75
ExprNode	Base node for expressions	78
ExprNot	Node for NOT operator	79
ExprOr	Node for OR operator	82
ExprRem	Node for modulo	84
ExprSub	Node for subtraction	87
Formula	Contains the processed verification formula	89
FormulaTemplate	Contains a template for coalition of Agent as string from the formula	91
GlobalModel	Represents a global model, containing agents and a formula	93
GlobalModelGenerator	Stores the local models, formula and a global model	95
GlobalState	Represents a single global state	109
GlobalModelGenerator::GlobalStateTupleHash		113
GlobalTransition	Represents a single global transition	114
HistoryDbg	Stores history and allows displaying it to the console	117
HistoryEntry	Structure used to save model traversal history	120
LocalModels	Represents a single local model, contains all agents and variables	123
LocalState	Represents a single LocalState , containing id, name and internal variables	124
LocalStateTemplate	A template for the local state	127
LocalTransition	Represents a single local transition, containing id, global name, local name, is shared and count of the appearances	128
ModelParser	A parser for converting a text file into a model	131
ProbabilityEntry	Class for handling the probability operations during probability verification	133
ProbabilityTrueFalse	Contains probability of a state being in a true or false state	136
ProbAdd	Node for addition	137
ProbConst	Node for a constant	139
ProbDiv	Node for division	142

ProbMul	
Node for multiplication	144
ProbNode	
Base node for probability calculations	147
ProbSub	
Node for subtraction	149
Result	151
StateVerificationInfo	
Handles all single state verification information during probabilistic verification	152
StrategyBitsComparator	
A comparator for two bitsets containing values for the global states	156
StrategyCollection	
Handles currently selected strategy when the strategy is set	157
StrategyCollectionTemplate	159
StrategyEntry	160
StrategyParser	
A parser for converting a text file into a strategy	161
STRATSTYPE	163
TransitionTemplate	
Represents a meta-transition	164
Var	
Represents a variable in the model, containing name, initial value and persistence	167
Verification	
A class that verifies if the model fulfills the formula. Also can do some operations on decision history	169
VerificationIterative	
A class that verifies if the model fulfills the formula. Also can do some operations on decision history	185
yy_buffer_state	197
yy_trans_info	200
yyalloc	201
YYSTYPE	202

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

Agent.cpp	Class of an agent. Class of an agent	205
Agent.hpp	Class of an agent. Class of an agent	205
Common.hpp	Contains all commonly used classes and structs. Contains all commonly used classes and structs	206
ConditionOperator.hpp		207
Constants.hpp		208
DotGraph.cpp	Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax	209
DotGraph.hpp	Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax	209
EpistemicClass.hpp	Struct of an epistemic class. Struct of an epistemic class	211
expressions.cc	Eval and helper class for expressions. Eval and helper class for expressions	211
expressions.hpp	Eval and helper class for expressions. Eval and helper class for expressions	212
GlobalModel.hpp	Struct of a global model. Struct of a global model	214
GlobalModelGenerator.cpp	Generator of a global model. Class for initializing and generating a global model	215
GlobalModelGenerator.hpp	Generator of a global model. Class for initializing and generating a global model	216
GlobalState.cpp	Struct representing a global state. Struct representing a global state	217
GlobalState.hpp	Struct representing a global state. Struct representing a global state	217
GlobalStateVerificationStatus.hpp		218
GlobalTransition.cpp	Struct representing a global transition. Struct representing a global transition	219
GlobalTransition.hpp	Struct representing a global transition. Struct representing a global transition	219

LocalState.cpp	
Class representing a local state. Class representing a local state	220
LocalState.hpp	
Class representing a local state. Class representing a local state	221
LocalTransition.hpp	
Class representing a local transition. Class representing a local transition	221
main.cpp	222
ModelParser.cc	
A model parser. A parser for converting a text file into a model	224
ModelParser.hpp	225
nodes.cc	
Parser templates. Class for setting up a new objects from a parser	226
nodes.hpp	
Parser templates. Class for setting up a new objects from a parser	227
parser.c	228
parser.h	246
scanner.c	249
scanner.l	271
strategyNodes.cc	271
strategyNodes.hpp	
Strategy parser templates. Class for setting up a new objects from a parser	271
strategyParser.c	272
StrategyParser.cc	
A strategy file parser. A parser for converting a text file into a strategy	291
strategyParser.h	292
StrategyParser.hpp	
Strategy file parser. A parser for converting a text file into a strategy	295
strategyScanner.c	296
strategyScanner.l	325
test.cpp	325
Types.hpp	
Custom data structures. Data structures and classes containing model data	326
TypesDependency.hpp	
Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures	328
Utils.cpp	
Utility functions. A collection of utility functions to use in the project	331
Utils.hpp	335
Verification.cpp	
Class for verification of the formula on a model. Class for verification of the specified formula on a specified model	338
Verification.hpp	341
VerificationIterative.cpp	
Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model	343
VerificationIterative.hpp	344

Chapter 5

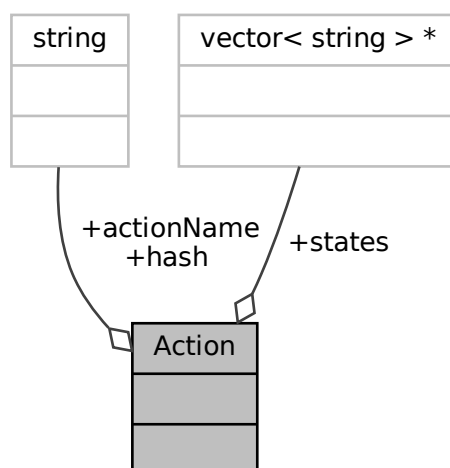
Class Documentation

5.1 Action Struct Reference

Single action template for probabilistic verification.

```
#include <TypesDependency.hpp>
```

Collaboration diagram for Action:



Public Attributes

- vector< string > * [states](#)
- string [hash](#)
- string [actionName](#)

5.1.1 Detailed Description

Single action template for probabilistic verification.

5.1.2 Member Data Documentation

5.1.2.1 actionName

```
string Action::actionName
```

5.1.2.2 hash

```
string Action::hash
```

5.1.2.3 states

```
vector<string>* Action::states
```

The documentation for this struct was generated from the following file:

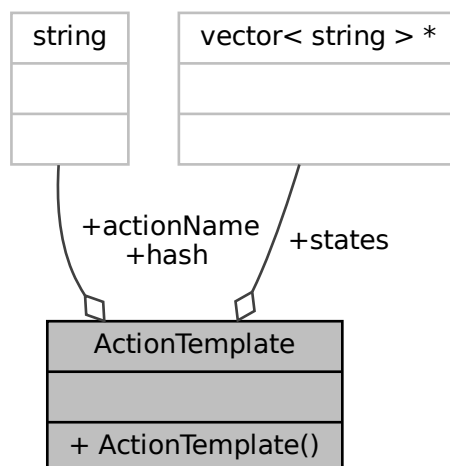
- [TypesDependency.hpp](#)

5.2 ActionTemplate Class Reference

Represents a selected action.

```
#include <strategyNodes.hpp>
```

Collaboration diagram for ActionTemplate:



Public Member Functions

- [ActionTemplate](#) (vector< string > *_states, string _actionName)
Constructor for the [ActionTemplate](#).

Public Attributes

- vector< string > * [states](#)
Vector of states from which the action has to be executed from.
- string [hash](#)
Contains the states in a.
- string [actionName](#)

5.2.1 Detailed Description

Represents a selected action.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 ActionTemplate()

```
ActionTemplate::ActionTemplate (
    vector< string > * _states,
    string _actionName )
```

Constructor for the [ActionTemplate](#).

Constructor for an [ActionTemplate](#).

Parameters

_states	Vector of states from which the action has to be executed from.
_actionName	Action executed from the aforementioned states.

5.2.3 Member Data Documentation

5.2.3.1 actionName

```
string ActionTemplate::actionName
```

5.2.3.2 hash

```
string ActionTemplate::hash
```

Contains the states in a.

5.2.3.3 states

```
vector<string>* ActionTemplate::states
```

Vector of states from which the action has to be executed from.

The documentation for this class was generated from the following files:

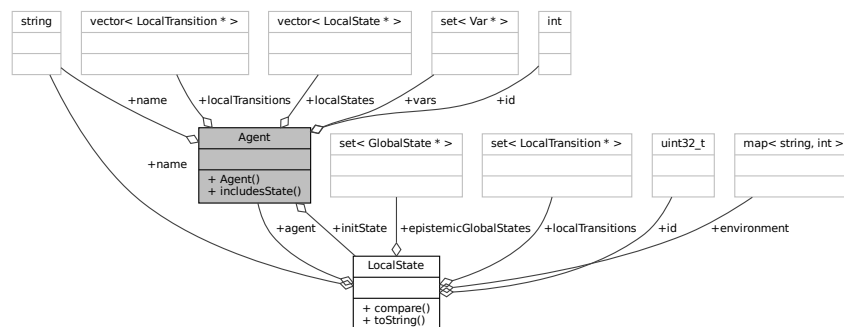
- [strategyNodes.hpp](#)
- [strategyNodes.cc](#)

5.3 Agent Class Reference

Contains all data for a single [Agent](#), including id, name and all of the agents' variables.

```
#include <Agent.hpp>
```

Collaboration diagram for Agent:



Public Member Functions

- [Agent](#) (int _id, string _name)
Constructor for the [Agent](#) class, assigning it an id and name.
- [LocalState](#) * [includesState](#) ([LocalState](#) *state)
Checks if there is an equivalent [LocalState](#) in the model to the one passed as an argment.

Public Attributes

- `int id`
Identifier of the agent.
- `string name`
Name of the agent.
- `set< Var * > vars`
Variable names for the agent.
- `LocalState * initState`
Initial state of the agent.
- `vector< LocalState * > localStates`
Local states for this agent.
- `vector< LocalTransition * > localTransitions`
Local transitions for this agent.

5.3.1 Detailed Description

Contains all data for a single [Agent](#), including id, name and all of the agents' variables.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Agent()

```
Agent::Agent (
    int _id,
    string _name ) [inline]
```

Constructor for the [Agent](#) class, assigning it an id and name.

Parameters

<code>_id</code>	Identifier of the new agent.
<code>_name</code>	Name of the new agent.

5.3.3 Member Function Documentation

5.3.3.1 includesState()

```
LocalState * Agent::includesState (
    LocalState * state )
```

Checks if there is an equivalent [LocalState](#) in the model to the one passed as an argument.

Parameters

<code>state</code>	A pointer to LocalState to be checked.
--------------------	--

Returns

Returns a pointer to an equivalent [LocalState](#) if such exists, otherwise returns NULL.

5.3.4 Member Data Documentation

5.3.4.1 id

```
int Agent::id
```

Identifier of the agent.

5.3.4.2 initState

```
LocalState* Agent::initState
```

Initial state of the agent.

5.3.4.3 localStates

```
vector<LocalState> Agent::localStates
```

Local states for this agent.

5.3.4.4 localTransitions

```
vector<LocalTransition> Agent::localTransitions
```

Local transitions for this agent.

5.3.4.5 name

```
string Agent::name
```

Name of the agent.

5.3.4.6 vars

```
set<Var*> Agent::vars
```

Variable names for the agent.

The documentation for this class was generated from the following files:

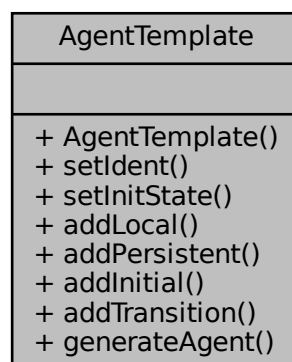
- [Agent.hpp](#)
- [Agent.cpp](#)

5.4 AgentTemplate Class Reference

Represents a single agent loaded from the description from a file.

```
#include <nodes.hpp>
```

Collaboration diagram for AgentTemplate:



Public Member Functions

- [AgentTemplate](#) ()
Constructor for an [AgentTemplate](#).
- virtual [AgentTemplate](#) & [setIdent](#) (string _ident)
Set the identifier of an agent.
- virtual [AgentTemplate](#) & [setInitState](#) (string _startState)
Set the initial state of the agent.
- virtual [AgentTemplate](#) & [addLocal](#) (set< string > *variables)
Adds local variables to an agent.
- virtual [AgentTemplate](#) & [addPersistent](#) (set< string > *variables)
Adds persistent variables to an agent.
- virtual [AgentTemplate](#) & [addInitial](#) (set< [Assignment](#) * > *assigns)
Adds initial assignments.
- virtual [AgentTemplate](#) & [addTransition](#) ([TransitionTemplate](#) *_transition)
Adds a transition to the agent.
- virtual [Agent](#) * [generateAgent](#) (int id)
Generate a new agent for the model.

Friends

- class [DotGraph](#)

5.4.1 Detailed Description

Represents a single agent loaded from the description from a file.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 AgentTemplate()

```
AgentTemplate::AgentTemplate ( )
```

Constructor for an [AgentTemplate](#).

5.4.3 Member Function Documentation

5.4.3.1 addInitial()

```
AgentTemplate & AgentTemplate::addInitial (
    set< Assignment * > * assigns ) [virtual]
```

Adds initial assignments.

Sets initial values of agent's variables.

Parameters

<i>assigns</i>	Assignments to be added.
----------------	--------------------------

Returns

Returns a pointer to self.

Parameters

<i>assigns</i>	Set of variables to assign.
----------------	-----------------------------

Returns

Returns itself.

5.4.3.2 addLocal()

```
AgentTemplate & AgentTemplate::addLocal (
    set< string > * variables ) [virtual]
```

Adds local variables to an agent.

Adds local variables to the agent.

Parameters

<i>variables</i>	Set of variables to be added.
------------------	-------------------------------

Returns

Returns a pointer to self.

Parameters

<i>variables</i>	A pointer to a set of strings with the variables to be added.
------------------	---

Returns

Returns itself.

5.4.3.3 addPersistent()

```
AgentTemplate & AgentTemplate::addPersistent (
    set< string > * variables ) [virtual]
```

Adds persistent variables to an agent.

Adds persistent variables to the agent.

Parameters

<i>variables</i>	Set of variables to be added.
------------------	-------------------------------

Returns

Returns a pointer to self.

Parameters

<i>variables</i>	A pointer to a set of strings with the variables to be added.
------------------	---

Returns

Returns itself.

5.4.3.4 addTransition()

```
AgentTemplate & AgentTemplate::addTransition (
    TransitionTemplate * _transition ) [virtual]
```

Adds a transition to the agent.

Adds a transition for the agent.

Parameters

<i>_transition</i>	Transition to be added.
--------------------	-------------------------

Returns

Returns a pointer to self.

Parameters

<i>_transition</i>	Transition to be added.
--------------------	-------------------------

Returns

Returns itself.

5.4.3.5 generateAgent()

```
Agent * AgentTemplate::generateAgent (
    int id ) [virtual]
```

Generate a new agent for the model.

Generates an agent for the model.

Parameters

<i>id</i>	Identification number defining a new Agent .
-----------	--

Returns

Returns a pointer to a new [Agent](#).

Parameters

<i>id</i>	Identifier of the new Agent .
-----------	---

Returns

Returns a pointer to a newly created [Agent](#).

5.4.3.6 setIdent()

```
AgentTemplate & AgentTemplate::setIdent (
    string _ident ) [virtual]
```

Set the identifier of an agent.

Sets the identifier of an agent.

Parameters

<i>_ident</i>	New agent identifier.
---------------	-----------------------

Returns

Returns a pointer to self.

Parameters

<i>_ident</i>	String with a new identifier.
---------------	-------------------------------

Returns

Returns itself.

5.4.3.7 setInitState()

```
AgentTemplate & AgentTemplate::setInitState (
    string _initState ) [virtual]
```

Set the initial state of the agent.

Sets initial state of an agent.

Parameters

<code>_startState</code>	New inital agent state.
--------------------------	-------------------------

Returns

Returns a pointer to self.

Parameters

<code>_initState</code>	String with a new state.
-------------------------	--------------------------

Returns

Returns itself.

5.4.4 Friends And Related Function Documentation**5.4.4.1 DotGraph**

```
friend class DotGraph [friend]
```

The documentation for this class was generated from the following files:

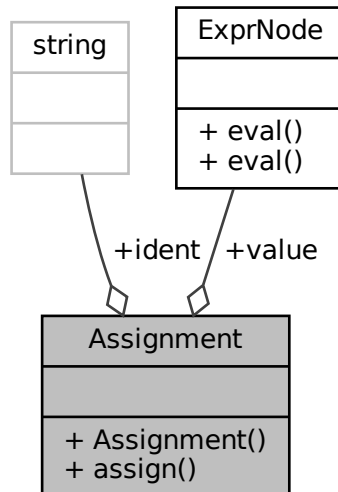
- [nodes.hpp](#)
- [nodes.cc](#)

5.5 Assignment Class Reference

Represents an assingment.

```
#include <nodes.hpp>
```

Collaboration diagram for Assignment:



Public Member Functions

- **Assignment** (string `_ident`, **ExprNode** * `_exp`)
*Constructor for an **Assignment** class.*
- virtual void **assign** (**Environment** &env)
Make an assignment in a given environment.

Public Attributes

- string **ident**
To what we should assign a value.
- **ExprNode** * **value**
A value to be assigned.

5.5.1 Detailed Description

Represents an assingment.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 Assignment()

```
Assignment::Assignment (
    string _ident,
    ExprNode * _exp ) [inline]
```

Constructor for an [Assignment](#) class.

Parameters

<code>_ident</code>	To what we should assign a value.
<code>_exp</code>	A value to be assigned.

5.5.3 Member Function Documentation

5.5.3.1 assign()

```
virtual void Assignment::assign (
    Environment & env ) [inline], [virtual]
```

Make an assignment in a given environment.

Parameters

<code>env</code>	Environment in which to make an assignment.
------------------	---

5.5.4 Member Data Documentation

5.5.4.1 ident

```
string Assignment::ident
```

To what we should assign a value.

5.5.4.2 value

`ExprNode* Assignment::value`

A value to be assigned.

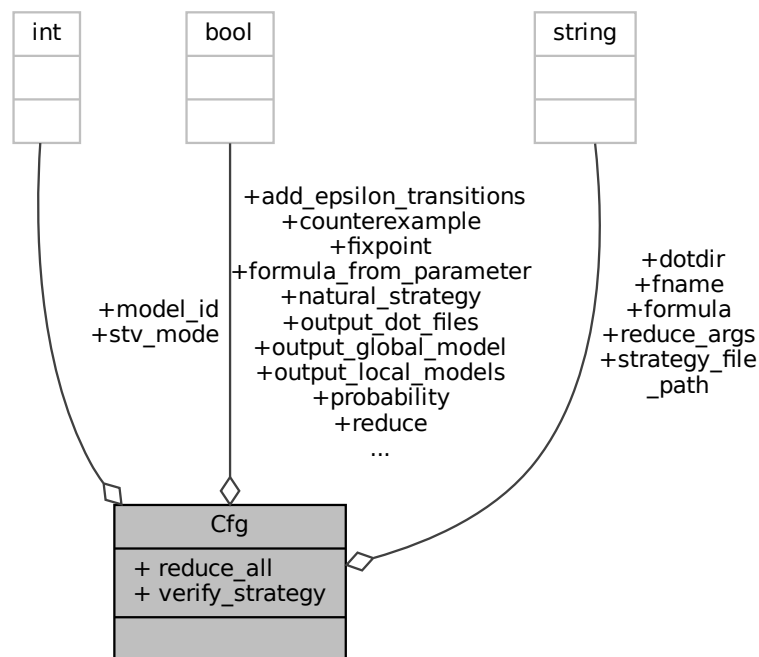
The documentation for this class was generated from the following file:

- [nodes.hpp](#)

5.6 Cfg Struct Reference

```
#include <Common.hpp>
```

Collaboration diagram for Cfg:



Public Attributes

- `std::string` [fname](#)
path to input file with system specification
- `int` [stv_mode](#)
stv_code as sum/combination of (1 - expandAllStates, 2 - verify, 4 - print metadata, 8 - run experiments)
- `bool` [output_local_models](#)
(obsolete) print data on local model

- bool [output_global_model](#)
(obsolete) print data on local model
- bool [output_dot_files](#)
flag for .dot export (by default exports templates and local/global models)
- std::string [dotdir](#)
pathprefix for .dot files export
- int [model_id](#)
- bool [add_epsilon_transitions](#)
add epsilon transitions to the states in the model when it's blocked for some reason
- bool [formula_from_parameter](#)
- std::string [formula](#)
- bool [counterexample](#)
- bool [reduce](#)
- bool [reduce_all](#)
- std::string [reduce_args](#)
- bool [fixpoint](#)
- bool [natural_strategy](#)
- bool [probability](#) = false
- bool [verify_strategy](#)
- std::string [strategy_file_path](#)

5.6.1 Member Data Documentation

5.6.1.1 [add_epsilon_transitions](#)

```
bool Cfg::add_epsilon_transitions
```

add epsilon transitions to the states in the model when it's blocked for some reason

5.6.1.2 [counterexample](#)

```
bool Cfg::counterexample
```

5.6.1.3 [dotdir](#)

```
std::string Cfg::dotdir
```

pathprefix for .dot files export

5.6.1.4 fixpoint

```
bool Cfg::fixpoint
```

5.6.1.5 fname

```
std::string Cfg::fname
```

path to input file with system specification

5.6.1.6 formula

```
std::string Cfg::formula
```

5.6.1.7 formula_from_parameter

```
bool Cfg::formula_from_parameter
```

5.6.1.8 model_id

```
int Cfg::model_id
```

5.6.1.9 natural_strategy

```
bool Cfg::natural_strategy
```

5.6.1.10 output_dot_files

```
bool Cfg::output_dot_files
```

flag for .dot export (by default exports templates and local/global models)

5.6.1.11 output_global_model

```
bool Cfg::output_global_model
```

(obsolete) print data on local model

5.6.1.12 output_local_models

```
bool Cfg::output_local_models
```

(obsolete) print data on local model

5.6.1.13 probability

```
bool Cfg::probability = false
```

5.6.1.14 reduce

```
bool Cfg::reduce
```

5.6.1.15 reduce_all

```
bool Cfg::reduce_all
```

5.6.1.16 reduce_args

```
std::string Cfg::reduce_args
```

5.6.1.17 strategy_file_path

```
std::string Cfg::strategy_file_path
```

5.6.1.18 stv_mode

```
int Cfg::stv_mode
```

stv_code as sum/combination of (1 - expandAllStates, 2 - verify, 4 - print metadata, 8 - run experiments)

5.6.1.19 verify_strategy

```
bool Cfg::verify_strategy
```

The documentation for this struct was generated from the following file:

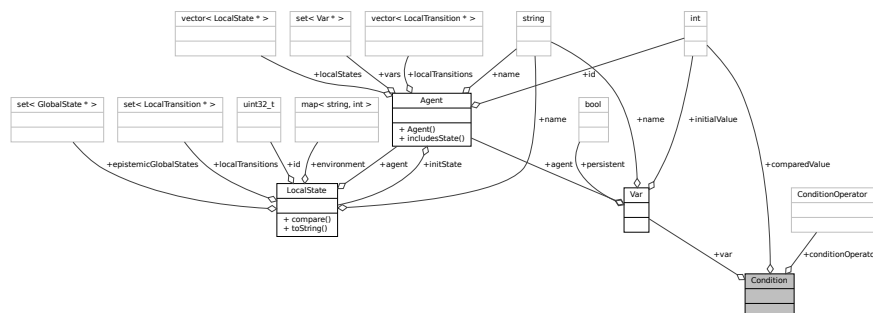
- [Common.hpp](#)

5.7 Condition Struct Reference

Represents a condition for [LocalTransition](#).

```
#include <Types.hpp>
```

Collaboration diagram for Condition:



Public Attributes

- [Var * var](#)
Pointer to a variable.
- [ConditionOperator conditionOperator](#)
Conditional operator for the variable.
- int [comparedValue](#)
Condition value to be met.

5.7.1 Detailed Description

Represents a condition for [LocalTransition](#).

Public Member Functions

- string [toString](#) ()

Public Attributes

- [HistoryEntryType](#) type = [HistoryEntryType::CONTEXT](#)
Type of the history record.
- [GlobalState](#) * [globalState](#) = nullptr
Saved global state.
- [GlobalTransition](#) * [decision](#) = nullptr
Selected transition.
- bool [globalTransitionControlled](#) = false
Is the transition controlled by an agent in coalition.
- [GlobalStateVerificationStatus](#) prevStatus = [GlobalStateVerificationStatus::UNVERIFIED](#)
Previous model verification state.
- [GlobalStateVerificationStatus](#) newStatus = [GlobalStateVerificationStatus::UNVERIFIED](#)
Next model verification state.
- int [depth](#) = 0
Iteration depth.
- [StrategyEntry](#) strategy = [StrategyEntry\(\)](#)
Holds currently processed strategy for the current state.
- float [stateProbabilityPrevious](#) = 0.0
Holds previous [GlobalState](#) probability.
- [ProbabilityCalculationType](#) previousProbabilityEntryType = [ProbabilityCalculationType::NONE_PROBABILITY](#)
Holds previous probability answer calculation method.
- [ProbabilityCalculationType](#) activeProbabilityEntryType = [ProbabilityCalculationType::SUM_PROBABILITY](#)
Holds currently used probability calculation method, used to rollback correct probability value.
- [ProbabilityTrueFalse](#) previousProbabilityAnswerValues
Holds previous true/false probability.

5.8.1 Member Function Documentation

5.8.1.1 toString()

```
string DecisionEntry::toString ( ) [inline]
```

5.8.2 Member Data Documentation

5.8.2.1 activeProbabilityEntryType

```
ProbabilityCalculationType DecisionEntry::activeProbabilityEntryType = ProbabilityCalculationType::SUM_PROBABILITY
```

Holds currently used probability calculation method, used to rollback correct probability value.

5.8.2.2 decision

```
GlobalTransition* DecisionEntry::decision = nullptr
```

Selected transition.

5.8.2.3 depth

```
int DecisionEntry::depth = 0
```

Iteration depth.

5.8.2.4 globalState

```
GlobalState* DecisionEntry::globalState = nullptr
```

Saved global state.

5.8.2.5 globalTransitionControlled

```
bool DecisionEntry::globalTransitionControlled = false
```

Is the transition controlled by an agent in coalition.

5.8.2.6 newStatus

```
GlobalStateVerificationStatus DecisionEntry::newStatus = GlobalStateVerificationStatus::UNVERIFIED
```

Next model verification state.

5.8.2.7 previousProbabilityAnswerValues

`ProbabilityTrueFalse` `DecisionEntry::previousProbabilityAnswerValues`

Holds previous true/false probability.

5.8.2.8 previousProbabilityEntryType

`ProbabilityCalculationType` `DecisionEntry::previousProbabilityEntryType` = `ProbabilityCalculationType::NONE_PROB`

Holds previous probability answer calculation method.

5.8.2.9 prevStatus

`GlobalStateVerificationStatus` `DecisionEntry::prevStatus` = `GlobalStateVerificationStatus::UNVERIFIED`

Previous model verification state.

5.8.2.10 stateProbabilityPrevious

`float` `DecisionEntry::stateProbabilityPrevious` = 0.0

Holds previous `GlobalState` probability.

5.8.2.11 strategy

`StrategyEntry` `DecisionEntry::strategy` = `StrategyEntry()`

Holds currently processed strategy for the current state.

5.8.2.12 type

`HistoryEntryType` `DecisionEntry::type` = `HistoryEntryType::CONTEXT`

Type of the history record.

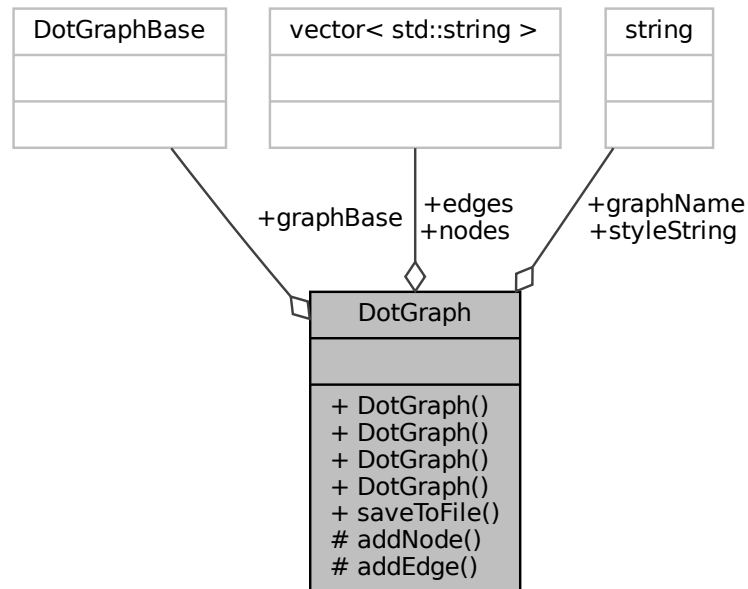
The documentation for this struct was generated from the following file:

- [VerificationIterative.hpp](#)

5.9 DotGraph Class Reference

```
#include <DotGraph.hpp>
```

Collaboration diagram for DotGraph:



Public Member Functions

- [DotGraph \(\)](#)
empty graph
- [DotGraph \(GlobalModel *const gm, bool extended=false, bool correct=false\)](#)
parses the transitions/states templates into nodes/edges
- [DotGraph \(Agent *const ag, bool extended=false\)](#)
parses the local transitions/states templates into nodes/edges
- [DotGraph \(AgentTemplate *const at\)](#)
parses the edge/state templates into nodes/edges
- void [saveToFile](#) (std::string pathprefix, std::string nameprefix, std::string basename="")
creates a .dot file

Public Attributes

- std::vector< std::string > [nodes](#)
- std::vector< std::string > [edges](#)
- std::string [graphName](#)
- [DotGraphBase](#) [graphBase](#)

Static Public Attributes

- static string [styleString](#)
(temporary hard-coded) graphviz visual configuration

Protected Member Functions

- void [addNode](#) (std::string id, std::string name)
creates a node string in graphviz syntax
- void [addEdge](#) (std::string src, std::string trg, std::string label)
creates an edge string in graphviz syntax

5.9.1 Constructor & Destructor Documentation

5.9.1.1 DotGraph() [1/4]

```
DotGraph::DotGraph ( )
```

empty graph

5.9.1.2 DotGraph() [2/4]

```
DotGraph::DotGraph (
    GlobalModel *const gm,
    bool extended = false,
    bool correct = false )
```

parses the transitions/states templates into nodes/edges

5.9.1.3 DotGraph() [3/4]

```
DotGraph::DotGraph (
    Agent *const ag,
    bool extended = false )
```

parses the local transitions/states templates into nodes/edges

5.9.1.4 DotGraph() [4/4]

```
DotGraph::DotGraph (
    AgentTemplate *const at )
```

parses the edge/state templates into nodes/edges

5.9.2 Member Function Documentation

5.9.2.1 addEdge()

```
void DotGraph::addEdge (
    std::string src,
    std::string trg,
    std::string label ) [protected]
```

creates an edge string in graphviz syntax

Parameters

<i>src</i>	id of a source node
<i>trg</i>	id of a target node
<i>label</i>	edge label and, possibly, extra attributes

5.9.2.2 addNode()

```
void DotGraph::addNode (
    std::string id,
    std::string name ) [protected]
```

creates a node string in graphviz syntax

Parameters

<i>id</i>	unique internal node identifier
<i>name</i>	displayed node label/name

5.9.2.3 saveToFile()

```
void DotGraph::saveToFile (
    std::string pathprefix,
```

```
std::string nameprefix,  
std::string basename = "" )
```

creates a .dot file

Parameters

<i>basename</i>	name of file (parent graph name if blank)
-----------------	---

5.9.3 Member Data Documentation

5.9.3.1 edges

```
std::vector<std::string> DotGraph::edges
```

5.9.3.2 graphBase

```
DotGraphBase DotGraph::graphBase
```

5.9.3.3 graphName

```
std::string DotGraph::graphName
```

5.9.3.4 nodes

```
std::vector<std::string> DotGraph::nodes
```


5.10.1 Detailed Description

Represents a single epistemic class.

5.10.2 Member Data Documentation

5.10.2.1 fixedCoalitionTransition

```
GlobalTransition* EpistemicClass::fixedCoalitionTransition
```

Transition that was already selected in this epistemic class. Model has to choose this transition if it is already set.

5.10.2.2 globalStates

```
map<string, GlobalState*> EpistemicClass::globalStates
```

Map of [GlobalState](#) hashes to according [GlobalState](#) pointers bound to this epistemic class.

5.10.2.3 hash

```
string EpistemicClass::hash
```

Hash of that epistemic class.

The documentation for this struct was generated from the following file:

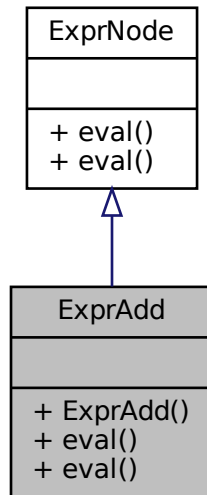
- [EpistemicClass.hpp](#)

5.11 ExprAdd Class Reference

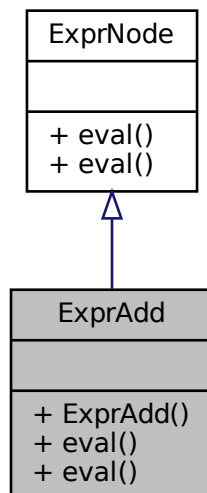
Node for addition.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprAdd:



Collaboration diagram for ExprAdd:



Public Member Functions

- [ExprAdd](#) ([ExprNode](#) * _larg, [ExprNode](#) * _rarg)
Addition expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.11.1 Detailed Description

Node for addition.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 ExprAdd()

```
ExprAdd::ExprAdd (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Addition expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.11.3 Member Function Documentation

5.11.3.1 eval() [1/2]

```
int ExprAdd::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.11.3.2 eval() [2/2]

```
int ExprAdd::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

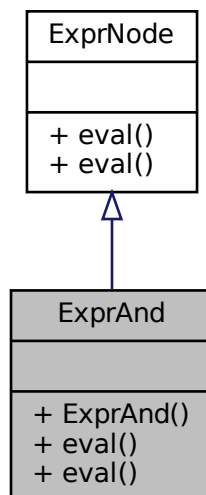
- [expressions.hpp](#)
- [expressions.cc](#)

5.12 ExprAnd Class Reference

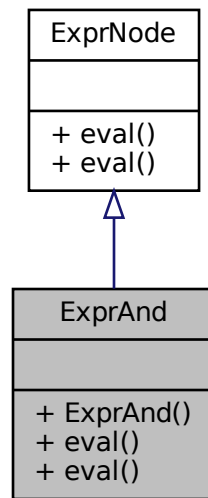
Node for AND operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprAnd:



Collaboration diagram for ExprAnd:



Public Member Functions

- [ExprAnd](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Logic AND expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.12.1 Detailed Description

Node for AND operator.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 ExprAnd()

```
ExprAnd::ExprAnd (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Logic AND expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.12.3 Member Function Documentation

5.12.3.1 `eval()` [1/2]

```
int ExprAnd::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.12.3.2 `eval()` [2/2]

```
int ExprAnd::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

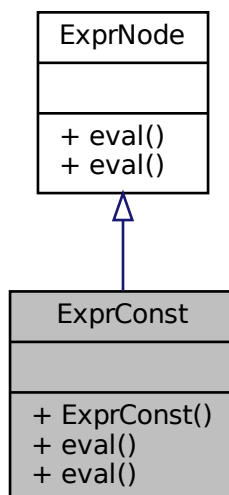
- [expressions.hpp](#)
- [expressions.cc](#)

5.13 ExprConst Class Reference

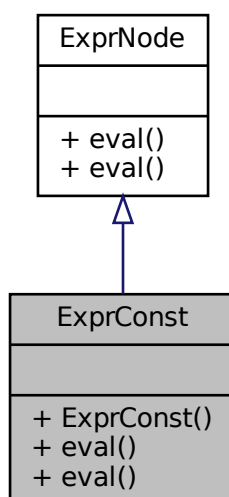
Node for a constant.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprConst:



Collaboration diagram for ExprConst:



Public Member Functions

- [ExprConst](#) (int _val)

Constant expression constructor.

- virtual int `eval` ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)

Calculates the expression value.

- virtual int `eval` ([Environment](#) &env)

5.13.1 Detailed Description

Node for a constant.

5.13.2 Constructor & Destructor Documentation

5.13.2.1 ExprConst()

```
ExprConst::ExprConst (
    int _val ) [inline]
```

Constant expression constructor.

Parameters

<code>_val</code>	ExprConst value.
-------------------	----------------------------------

5.13.3 Member Function Documentation

5.13.3.1 eval() [1/2]

```
int ExprConst::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.13.3.2 eval() [2/2]

```
int ExprConst::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

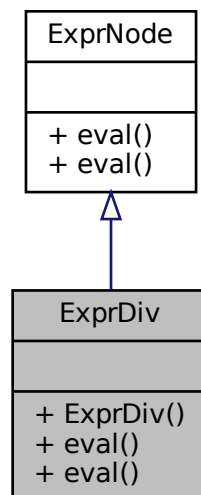
- [expressions.hpp](#)
- [expressions.cc](#)

5.14 ExprDiv Class Reference

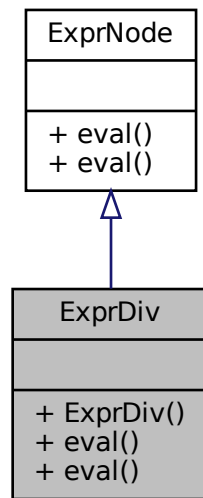
Node for division.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprDiv:



Collaboration diagram for ExprDiv:



Public Member Functions

- [ExprDiv](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Division expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.14.1 Detailed Description

Node for division.

5.14.2 Constructor & Destructor Documentation

5.14.2.1 ExprDiv()

```
ExprDiv::ExprDiv (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Division expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.14.3 Member Function Documentation

5.14.3.1 `eval()` [1/2]

```
int ExprDiv::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.14.3.2 `eval()` [2/2]

```
int ExprDiv::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

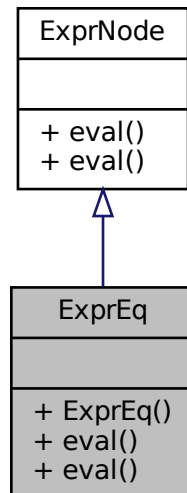
- [expressions.hpp](#)
- [expressions.cc](#)

5.15 ExprEq Class Reference

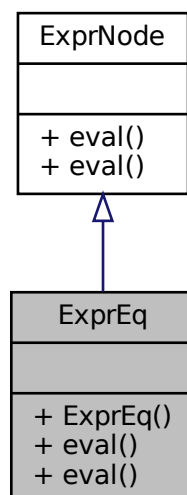
Node for "==" operator.


```
#include <expressions.hpp>
```

Inheritance diagram for ExprEq:



Collaboration diagram for ExprEq:



Public Member Functions

- [ExprEq](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)

Equals expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.15.1 Detailed Description

Node for "==" operator.

5.15.2 Constructor & Destructor Documentation

5.15.2.1 ExprEq()

```
ExprEq::ExprEq (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Equals expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.15.3 Member Function Documentation

5.15.3.1 eval() [1/2]

```
int ExprEq::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.15.3.2 eval() [2/2]

```
int ExprEq::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

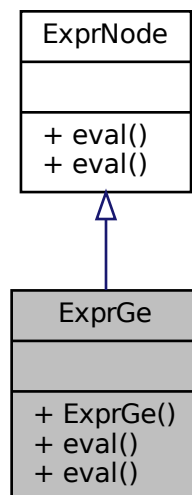
- [expressions.hpp](#)
- [expressions.cc](#)

5.16 ExprGe Class Reference

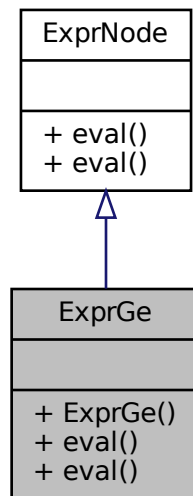
Node for ">=" operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprGe:



Collaboration diagram for ExprGe:



Public Member Functions

- [ExprGe](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Greater or equal expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.16.1 Detailed Description

Node for ">=" operator.

5.16.2 Constructor & Destructor Documentation

5.16.2.1 ExprGe()

```
ExprGe::ExprGe (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Greater or equal expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.16.3 Member Function Documentation

5.16.3.1 `eval()` [1/2]

```
int ExprGe::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.16.3.2 `eval()` [2/2]

```
int ExprGe::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

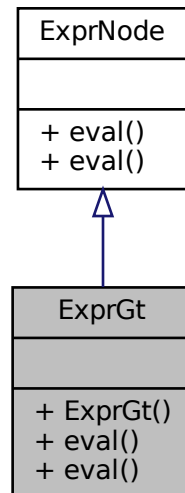
- [expressions.hpp](#)
- [expressions.cc](#)

5.17 ExprGt Class Reference

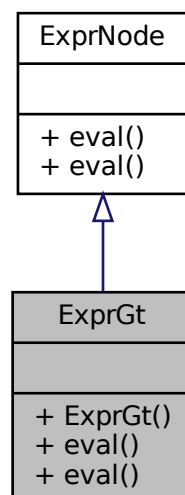
Node for ">" operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprGt:



Collaboration diagram for ExprGt:



Public Member Functions

- [ExprGt](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)

Greater than expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.17.1 Detailed Description

Node for ">" operator.

5.17.2 Constructor & Destructor Documentation

5.17.2.1 ExprGt()

```
ExprGt::ExprGt (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Greater than expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.17.3 Member Function Documentation

5.17.3.1 eval() [1/2]

```
int ExprGt::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.17.3.2 eval() [2/2]

```
int ExprGt::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

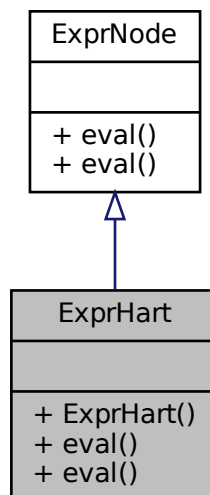
- [expressions.hpp](#)
- [expressions.cc](#)

5.18 ExprHart Class Reference

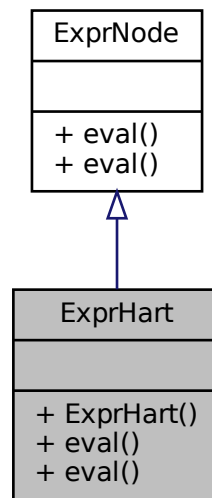
Node for a Hartley operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprHart:



Collaboration diagram for ExprHart:



Public Member Functions

- [ExprHart](#) (string `_agentName`, bool `_le`, int `_val`, vector< [ExprNode](#) * > `*_arg`)
Constant expression constructor.
- virtual int `eval` ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int `eval` ([Environment](#) &env)

5.18.1 Detailed Description

Node for a Hartley operator.

5.18.2 Constructor & Destructor Documentation

5.18.2.1 ExprHart()

```
ExprHart::ExprHart (
    string _agentName,
    bool _le,
    int _val,
    vector< ExprNode * > *_arg ) [inline]
```

Constant expression constructor.

Parameters

<i>_agentName</i>	Agent name for Hartley operator.
<i>_le</i>	Flag for if Hartley coefficient should be less equal or greater equal.
<i>_val</i>	Number to compare the result to.
<i>_arg</i>	Expression to verify with the given knowledge.

5.18.3 Member Function Documentation

5.18.3.1 `eval()` [1/2]

```
int ExprHart::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.18.3.2 `eval()` [2/2]

```
int ExprHart::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

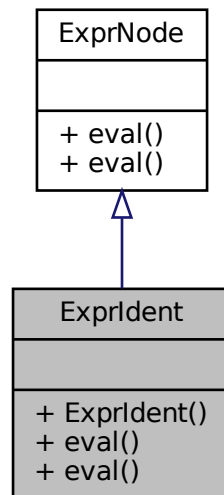
- [expressions.hpp](#)
- [expressions.cc](#)

5.19 ExprIdent Class Reference

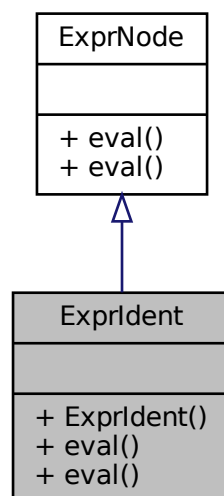
Node for an identifier.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprIdent:



Collaboration diagram for ExprIdent:



Public Member Functions

- [ExprIdent](#) (string _ident)
Identifier expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.19.1 Detailed Description

Node for an identifier.

5.19.2 Constructor & Destructor Documentation

5.19.2.1 ExprIdent()

```
ExprIdent::ExprIdent (
    string _ident ) [inline]
```

Identifier expression constructor.

Parameters

<code>_ident</code>	ExprIdent value.
---------------------	----------------------------------

5.19.3 Member Function Documentation

5.19.3.1 eval() [1/2]

```
int ExprIdent::eval (
    Environment & env ) [virtual]
```

Parameters

<code>env</code>	
------------------	--

Returns

Implements [ExprNode](#).

5.19.3.2 eval() [2/2]

```
int ExprIdent::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

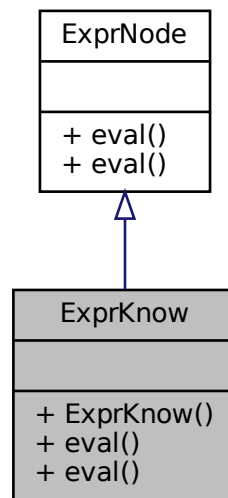
- [expressions.hpp](#)
- [expressions.cc](#)

5.20 ExprKnow Class Reference

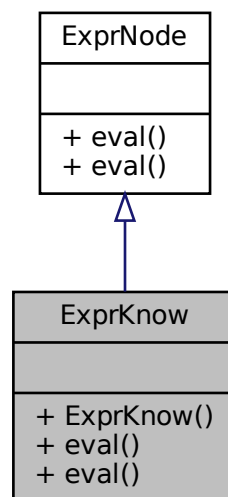
Node for a knowledge operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprKnow:



Collaboration diagram for ExprKnow:



Public Member Functions

- [ExprKnow](#) (string _agentName, [ExprNode](#) *_arg)
Constant expression constructor.

- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.20.1 Detailed Description

Node for a knowledge operator.

5.20.2 Constructor & Destructor Documentation

5.20.2.1 ExprKnow()

```
ExprKnow::ExprKnow (
    string _agentName,
    ExprNode * _arg ) [inline]
```

Constant expression constructor.

Parameters

<code>_agentName</code>	Agent name for knowledge operator.
<code>_arg</code>	Expression to verify with the given knowledge.

5.20.3 Member Function Documentation

5.20.3.1 eval() [1/2]

```
int ExprKnow::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.20.3.2 eval() [2/2]

```
int ExprKnow::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

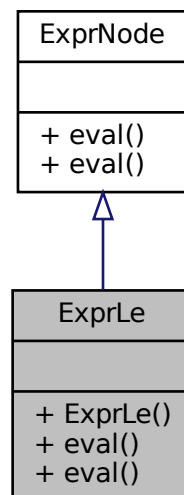
- [expressions.hpp](#)
- [expressions.cc](#)

5.21 ExprLe Class Reference

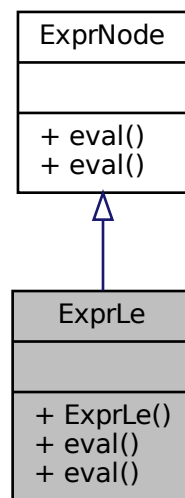
Node for "<=" operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprLe:



Collaboration diagram for ExprLe:



Public Member Functions

- [ExprLe](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Less or equal expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.21.1 Detailed Description

Node for "<=" operator.

5.21.2 Constructor & Destructor Documentation

5.21.2.1 ExprLe()

```
ExprLe::ExprLe (
    ExprNode *_larg,
    ExprNode *_rarg ) [inline]
```

Less or equal expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.21.3 Member Function Documentation

5.21.3.1 `eval()` [1/2]

```
int ExprLe::eval (  
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.21.3.2 `eval()` [2/2]

```
int ExprLe::eval (  
    Environment & env,  
    GlobalModelGenerator * generator,  
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

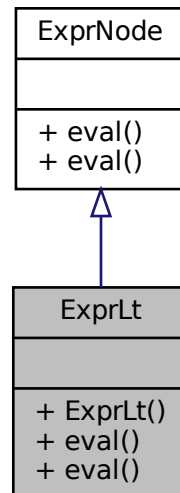
- [expressions.hpp](#)
- [expressions.cc](#)

5.22 ExprLt Class Reference

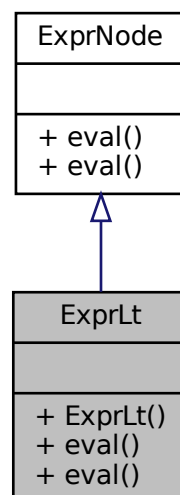
Node for "<" operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprLt:



Collaboration diagram for ExprLt:



Public Member Functions

- [ExprLt](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)

Less than expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.22.1 Detailed Description

Node for "<" operator.

5.22.2 Constructor & Destructor Documentation

5.22.2.1 ExprLt()

```
ExprLt::ExprLt (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Less than expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.22.3 Member Function Documentation

5.22.3.1 eval() [1/2]

```
int ExprLt::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.22.3.2 eval() [2/2]

```
int ExprLt::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

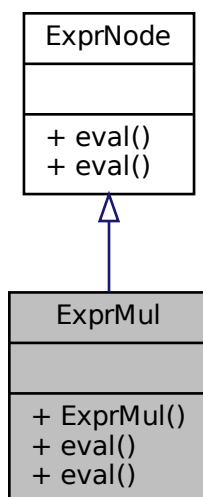
- [expressions.hpp](#)
- [expressions.cc](#)

5.23 ExprMul Class Reference

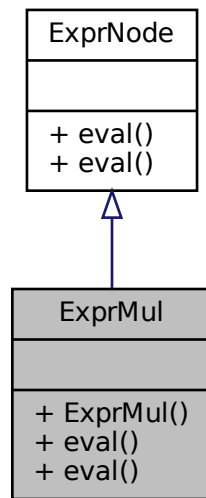
Node for multiplication.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprMul:



Collaboration diagram for ExprMul:



Public Member Functions

- [ExprMul](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Multiplication expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.23.1 Detailed Description

Node for multiplication.

5.23.2 Constructor & Destructor Documentation

5.23.2.1 ExprMul()

```
ExprMul::ExprMul (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Multiplication expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.23.3 Member Function Documentation

5.23.3.1 `eval()` [1/2]

```
int ExprMul::eval (  
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.23.3.2 `eval()` [2/2]

```
int ExprMul::eval (  
    Environment & env,  
    GlobalModelGenerator * generator,  
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

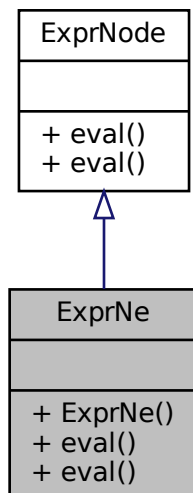
- [expressions.hpp](#)
- [expressions.cc](#)

5.24 ExprNe Class Reference

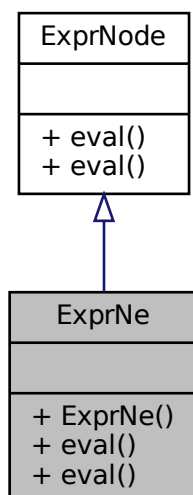
Node for "!=" operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprNe:



Collaboration diagram for ExprNe:



Public Member Functions

- [ExprNe](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)

Not equals expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.24.1 Detailed Description

Node for "!=" operator.

5.24.2 Constructor & Destructor Documentation

5.24.2.1 ExprNe()

```
ExprNe::ExprNe (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Not equals expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.24.3 Member Function Documentation

5.24.3.1 eval() [1/2]

```
int ExprNe::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.24.3.2 eval() [2/2]

```
int ExprNe::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

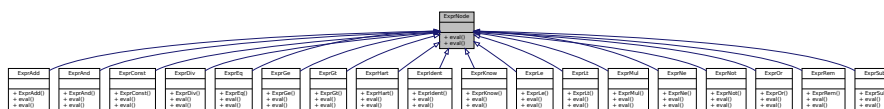
- [expressions.hpp](#)
- [expressions.cc](#)

5.25 ExprNode Class Reference

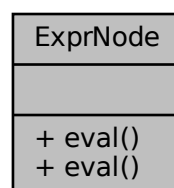
Base node for expressions.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprNode:



Collaboration diagram for ExprNode:



Public Member Functions

- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)=0
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)=0

5.25.1 Detailed Description

Base node for expressions.

5.25.2 Member Function Documentation

5.25.2.1 eval() [1/2]

```
virtual int ExprNode::eval (
    Environment & env ) [pure virtual]
```

Implemented in [ExprHart](#), [ExprKnow](#), [ExprGe](#), [ExprGt](#), [ExprLe](#), [ExprLt](#), [ExprNe](#), [ExprEq](#), [ExprNot](#), [ExprOr](#), [ExprAnd](#), [ExprRem](#), [ExprDiv](#), [ExprMul](#), [ExprSub](#), [ExprAdd](#), [ExprIdent](#), and [ExprConst](#).

5.25.2.2 eval() [2/2]

```
virtual int ExprNode::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [pure virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implemented in [ExprHart](#), [ExprKnow](#), [ExprGe](#), [ExprGt](#), [ExprLe](#), [ExprLt](#), [ExprNe](#), [ExprEq](#), [ExprNot](#), [ExprOr](#), [ExprAnd](#), [ExprRem](#), [ExprDiv](#), [ExprMul](#), [ExprSub](#), [ExprAdd](#), [ExprIdent](#), and [ExprConst](#).

The documentation for this class was generated from the following file:

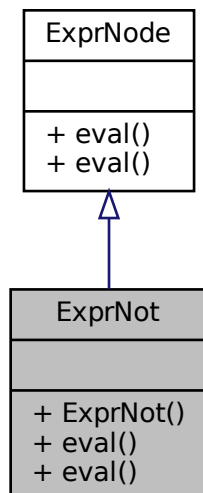
- [expressions.hpp](#)

5.26 ExprNot Class Reference

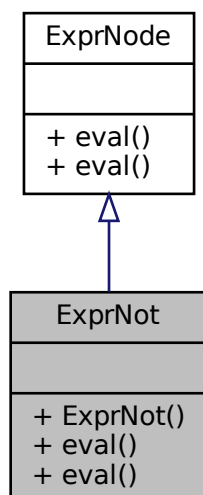
Node for NOT operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprNot:



Collaboration diagram for ExprNot:



Public Member Functions

- [ExprNot](#) ([ExprNode](#) *_arg)

Logic NOT expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.26.1 Detailed Description

Node for NOT operator.

5.26.2 Constructor & Destructor Documentation

5.26.2.1 ExprNot()

```
ExprNot::ExprNot (
    ExprNode * _arg ) [inline]
```

Logic NOT expression constructor.

Parameters

<code>_arg</code>	Calculates the expression value.
-------------------	----------------------------------

5.26.3 Member Function Documentation

5.26.3.1 eval() [1/2]

```
int ExprNot::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.26.3.2 eval() [2/2]

```
int ExprNot::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

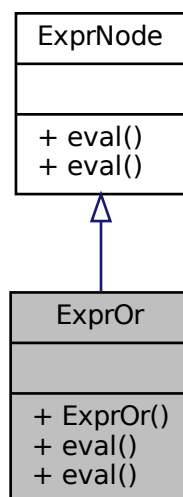
- [expressions.hpp](#)
- [expressions.cc](#)

5.27 ExprOr Class Reference

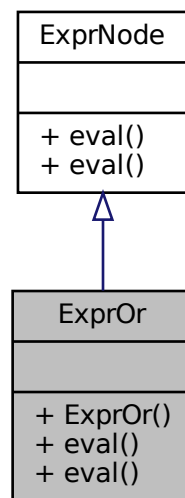
Node for OR operator.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprOr:



Collaboration diagram for ExprOr:



Public Member Functions

- [ExprOr](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Logic OR expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.27.1 Detailed Description

Node for OR operator.

5.27.2 Constructor & Destructor Documentation

5.27.2.1 ExprOr()

```
ExprOr::ExprOr (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Logic OR expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.27.3 Member Function Documentation

5.27.3.1 `eval()` [1/2]

```
int ExprOr::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.27.3.2 `eval()` [2/2]

```
int ExprOr::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

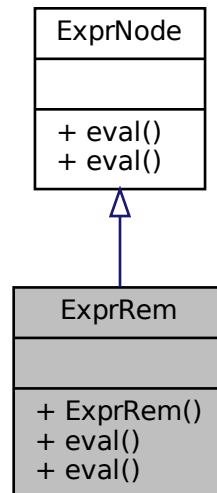
- [expressions.hpp](#)
- [expressions.cc](#)

5.28 ExprRem Class Reference

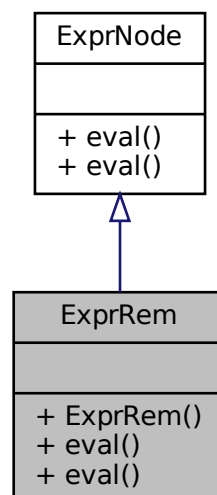
Node for modulo.


```
#include <expressions.hpp>
```

Inheritance diagram for ExprRem:



Collaboration diagram for ExprRem:



Public Member Functions

- [ExprRem](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)

Modulo expression constructor.

- virtual int `eval` (`Environment` &env, `GlobalModelGenerator` *generator, `GlobalState` *globalState)

Calculates the expression value.

- virtual int `eval` (`Environment` &env)

5.28.1 Detailed Description

Node for modulo.

5.28.2 Constructor & Destructor Documentation

5.28.2.1 ExprRem()

```
ExprRem::ExprRem (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Modulo expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.28.3 Member Function Documentation

5.28.3.1 eval() [1/2]

```
int ExprRem::eval (
    Environment & env ) [virtual]
```

Implements `ExprNode`.

5.28.3.2 eval() [2/2]

```
int ExprRem::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

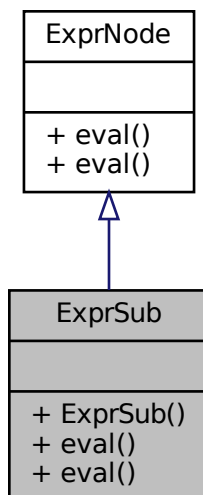
- [expressions.hpp](#)
- [expressions.cc](#)

5.29 ExprSub Class Reference

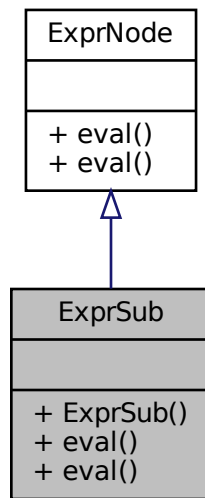
Node for subtraction.

```
#include <expressions.hpp>
```

Inheritance diagram for ExprSub:



Collaboration diagram for ExprSub:



Public Member Functions

- [ExprSub](#) ([ExprNode](#) *_larg, [ExprNode](#) *_rarg)
Subtraction expression constructor.
- virtual int [eval](#) ([Environment](#) &env, [GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual int [eval](#) ([Environment](#) &env)

5.29.1 Detailed Description

Node for subtraction.

5.29.2 Constructor & Destructor Documentation

5.29.2.1 ExprSub()

```
ExprSub::ExprSub (
    ExprNode * _larg,
    ExprNode * _rarg ) [inline]
```

Subtraction expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.29.3 Member Function Documentation

5.29.3.1 `eval()` [1/2]

```
int ExprSub::eval (
    Environment & env ) [virtual]
```

Implements [ExprNode](#).

5.29.3.2 `eval()` [2/2]

```
int ExprSub::eval (
    Environment & env,
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns an integer.

Implements [ExprNode](#).

The documentation for this class was generated from the following files:

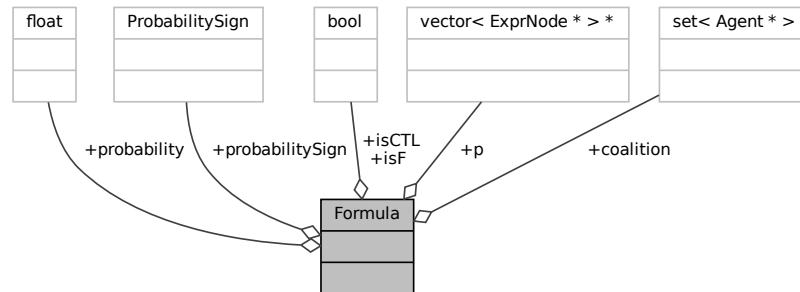
- [expressions.hpp](#)
- [expressions.cc](#)

5.30 Formula Struct Reference

Contains the processed verification formula.

```
#include <Types.hpp>
```

Collaboration diagram for Formula:



Public Attributes

- set< [Agent](#) * > [coalition](#)
Coalition of [Agent](#) from the formula.
- vector< [ExprNode](#) * > * [p](#)
- bool [isF](#)
- bool [isCTL](#)
- float [probability](#) = 1
- [ProbabilitySign](#) [probabilitySign](#) = [ProbabilitySign::NONE](#)

5.30.1 Detailed Description

Contains the processed verification formula.

5.30.2 Member Data Documentation

5.30.2.1 coalition

```
set<Agent*> Formula::coalition
```

Coalition of [Agent](#) from the formula.

5.30.2.2 isCTL

```
bool Formula::isCTL
```

5.30.2.3 isF

```
bool Formula::isF
```

5.30.2.4 p

```
vector<ExprNode*>* Formula::p
```

5.30.2.5 probability

```
float Formula::probability = 1
```

5.30.2.6 probabilitySign

```
ProbabilitySign Formula::probabilitySign = ProbabilitySign::NONE
```

The documentation for this struct was generated from the following file:

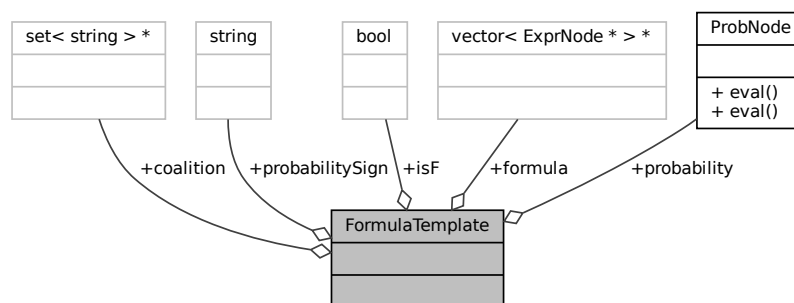
- [Types.hpp](#)

5.31 FormulaTemplate Struct Reference

Contains a template for coalition of [Agent](#) as string from the formula.

```
#include <Types.hpp>
```

Collaboration diagram for FormulaTemplate:



Public Attributes

- `set< string > * coalition`
- `vector< ExprNode * > * formula`
- `bool isF`
- `ProbNode * probability`
- `string probabilitySign = ""`

5.31.1 Detailed Description

Contains a template for coalition of [Agent](#) as string from the formula.

5.31.2 Member Data Documentation

5.31.2.1 coalition

```
set<string>* FormulaTemplate::coalition
```

5.31.2.2 formula

```
vector<ExprNode*>* FormulaTemplate::formula
```

5.31.2.3 isF

```
bool FormulaTemplate::isF
```

5.31.2.4 probability

```
ProbNode* FormulaTemplate::probability
```

5.31.2.5 probabilitySign

```
string FormulaTemplate::probabilitySign = ""
```

The documentation for this struct was generated from the following file:

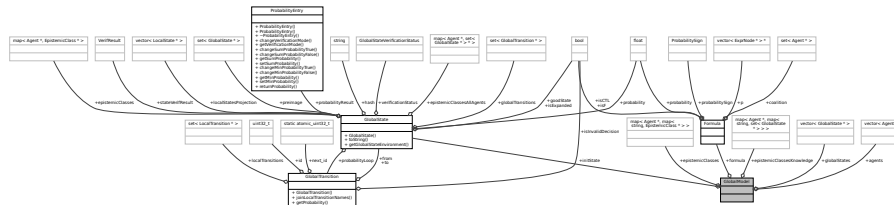
- [Types.hpp](#)

5.32 GlobalModel Struct Reference

Represents a global model, containing agents and a formula.

```
#include <GlobalModel.hpp>
```

Collaboration diagram for GlobalModel:



Public Attributes

- `vector< Agent * > agents`
Pointers to all agents in a model.
- `Formula * formula`
A pointer to a [Formula](#).
- `GlobalState * initState`
Pointer to the initial state of the model.
- `vector< GlobalState * > globalStates`
Every [GlobalState](#) in the model.
- `map< Agent *, map< string, EpistemicClass * > > epistemicClasses`
Map of [Agent](#) pointers to a map of [EpistemicClass](#) for graph traversal.
- `map< Agent *, map< string, set< GlobalState * > > > epistemicClassesKnowledge`
Map of [Agent](#) pointers to a map of [EpistemicClass](#) for knowledge checks.

5.32.1 Detailed Description

Represents a global model, containing agents and a formula.

5.32.2 Member Data Documentation

5.32.2.1 agents

```
vector<Agent*> GlobalModel::agents
```

Pointers to all agents in a model.

5.32.2.2 epistemicClasses

```
map<Agent*, map<string, EpistemicClass*> > GlobalModel::epistemicClasses
```

Map of [Agent](#) pointers to a map of [EpistemicClass](#) for graph traversal.

5.32.2.3 epistemicClassesKnowledge

```
map<Agent*, map<string, set<GlobalState*> > > GlobalModel::epistemicClassesKnowledge
```

Map of [Agent](#) pointers to a map of [EpistemicClass](#) for knowledge checks.

5.32.2.4 formula

```
Formula* GlobalModel::formula
```

A pointer to a [Formula](#).

5.32.2.5 globalStates

```
vector<GlobalState*> GlobalModel::globalStates
```

Every [GlobalState](#) in the model.

5.32.2.6 initState

```
GlobalState* GlobalModel::initState
```

Pointer to the initial state of the model.

The documentation for this struct was generated from the following file:

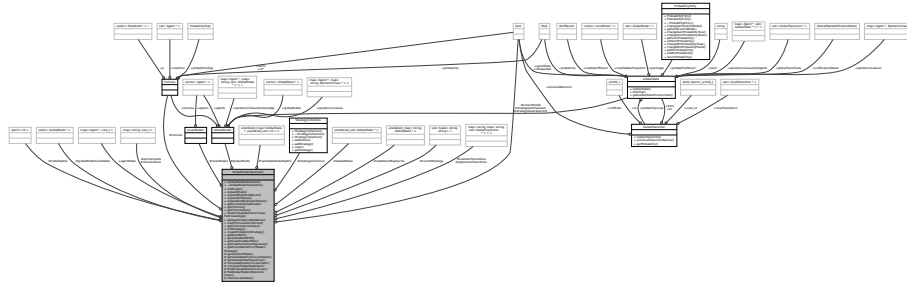
- [GlobalModel.hpp](#)

5.33 GlobalModelGenerator Class Reference

Stores the local models, formula and a global model.

```
#include <GlobalModelGenerator.hpp>
```

Collaboration diagram for GlobalModelGenerator:



Classes

- struct [GlobalStateTupleHash](#)

Public Member Functions

- [GlobalModelGenerator \(\)](#)
Constructor for [GlobalModelGenerator](#) class.
- [~GlobalModelGenerator \(\)](#)
Destructor for [GlobalModelGenerator](#) class.
- [GlobalState * initModel \(LocalModels *localModels, Formula *formula\)](#)
Initializes a global model from local models and a formula.
- [void expandState \(GlobalState *state\)](#)
Goes through all [GlobalTransition](#) in a given [GlobalState](#) and creates new [GlobalStates](#) connected to the given one.
- [vector< GlobalState * > expandStateAndReturn \(GlobalState *state, bool returnAnyway=false\)](#)
[GlobalModelGenerator::expandState](#) that also additionally returns a vector of newly created states.
- [void expandAllStates \(bool additionalProbSplit=false\)](#)
Expands the states starting from the initial [GlobalState](#) and continues until there are no more states to expand.
- [void expandAndReduceAllStates \(\)](#)
Expands and reduces the states starting from the initial [GlobalState](#) using a DFS-POR algorithm and continues until there are no more states to expand.
- [GlobalModel * getCurrentGlobalModel \(\)](#)
Get for a [GlobalModel](#) used in initialization.
- [Formula * getFormula \(\)](#)
Get for the [Formula](#) used in initialization.
- [int getFormulaSize \(\)](#)
Get size of the [Formula](#) used in initialization.
- [set< GlobalState * > * findOrCreateEpistemicClassForKnowledge \(vector< LocalState * > *localStates, GlobalState *globalState, Agent *agent\)](#)
Checks if a vector of [LocalState](#) is already an epistemic class for a given [Agent](#), if not, creates a new one.
- [Agent * getAgentInstanceByName \(string agentName\)](#)
Get a pointer to an agent by using its name.

- void `markFormulaAsIncorrect` ()
Sets formula to an incorrectly written state.
- bool `getFormulaCorectness` ()
Gets formula status.
- void `initStrategy` (StrategyCollection *strat)
Initiates the strategy with a given StrategyCollection.
- set< set< tuple< string, string > > > `createProbabilityStrategy` (LocalModels *localModels)
Prepares the strategy generating structures for the probabilistic verification.
- set< tuple< string, string > > * `getNextPath` ()
Retrieves the next strategy one at a time (iteratively). Systematically tries all combinations of action choices at each epistemic class.
- MDP `generateNextMDP` (bool makeOpponentGoMax=false)
- string `getCoalitionIdentifier` (vector< LocalState * > *localStates)
Returns a concatenated epistemic class ID of all coalition members in given LocalStates.
- string `getCoalitionActionSignature` (GlobalTransition *transition, char sep=';')
Returns coalition-only action signature, ordered by agentIndex and using localName when available.
- string `getActionNameFromStateInStrategy` (GlobalState *state)
Gives next action's name from a given state.

Public Attributes

- map< Agent *, size_t > `agentIndex`
auxiliary variable mapping Agent pointer to its index (replace size_t with if needed later)

Protected Member Functions

- GlobalState * `generateInitState` ()
Generates initial state of the model from GlobalModel in memory.
- GlobalState * `generateStateFromLocalStates` (vector< LocalState * > *localStates, set< LocalTransition * > *viaLocalTransitions, GlobalState *prevGlobalState)
Creates a new GlobalState using some of the internally known model data and given local states, transitions that were used to get there and the previous global state.
- void `generateGlobalTransitions` (GlobalState *fromGlobalState, set< LocalTransition * > localTransitions, map< Agent *, vector< LocalTransition * > > transitionsByAgent)
Adds all shared global transitions to a GlobalState.
- string `computeEpistemicClassHash` (vector< LocalState * > *localStates, Agent *agent)
Creates a hash from a set of LocalState and an Agent.
- string `computeGlobalStateHash` (vector< LocalState * > *localStates)
Creates a hash from a set of LocalState.
- EpistemicClass * `findOrCreateEpistemicClass` (vector< LocalState * > *localStates, Agent *agent)
Checks if a vector of LocalState is already an epistemic class for a given Agent, if not, creates a new one.
- GlobalState * `findGlobalStateInEpistemicClass` (vector< LocalState * > *localStates, EpistemicClass *epistemicClass)
Gets a GlobalState from an EpistemicClass if it exists in that episcemic class.
- bool `checkLocalStates` (vector< LocalState * > *localStates, GlobalState *globalState)

Protected Attributes

- [LocalModels](#) * [localModels](#)
LocalModels used in *initModel*.
- [Formula](#) * [formula](#)
Formula used in *initModel*.
- [GlobalModel](#) * [globalModel](#)
GlobalModel created in *initModel*.
- bool [correctModel](#)
Flag holding info if model is actually correct.
- unordered_map< [GlobalState](#) *, unordered_set< int > > [candidateStateDepths](#)
Lookup map containing global states and the depths that the global state is contained in. Used for reductions.
- stack< [GlobalState](#) * > [globalModelCandidates](#)
Candidate states to be used in reductions. Used for reductions.
- stack< int > [stateDepths](#)
Saved depths of states added to statesToExpand. Used for reductions.
- unordered_set< [GlobalState](#) * > [addedStates](#)
States that were added to a stack of states. Used for reductions.
- [StrategyCollection](#) * [strategyCollection](#)
Strategy to check during verification (if any)
- set< tuple< string, string > > [currentStrategy](#)
- map< string, size_t > [choiceIndices](#)
- map< string, size_t > [actionCounts](#)
- bool [strategyGenerationInit](#) = false
- bool [strategiesExhausted](#) = false
- unordered_map< string, [GlobalState](#) * > [stateHashMapCache](#)
- map< string, map< string, set< [GlobalTransition](#) * > > > [coalitionTransitions](#)
- map< string, map< string, set< [GlobalTransition](#) * > > > [opponentsTransitions](#)

5.33.1 Detailed Description

Stores the local models, formula and a global model.

5.33.2 Constructor & Destructor Documentation

5.33.2.1 GlobalModelGenerator()

```
GlobalModelGenerator::GlobalModelGenerator ( )
```

Constructor for [GlobalModelGenerator](#) class.

5.33.2.2 ~GlobalModelGenerator()

```
GlobalModelGenerator::~~GlobalModelGenerator ( )
```

Destructor for [GlobalModelGenerator](#) class.

5.33.3 Member Function Documentation

5.33.3.1 checkLocalStates()

```
bool GlobalModelGenerator::checkLocalStates (
    vector< LocalState * > * localStates,
    GlobalState * globalState ) [protected]
```

5.33.3.2 computeEpistemicClassHash()

```
string GlobalModelGenerator::computeEpistemicClassHash (
    vector< LocalState * > * localStates,
    Agent * agent ) [protected]
```

Creates a hash from a set of [LocalState](#) and an [Agent](#).

Parameters

<i>localStates</i>	Pointer to a vector of pointers of LocalState and pointer to and Agent to turn into a hash.
--------------------	---

Returns

Returns a string with a hash.

5.33.3.3 computeGlobalStateHash()

```
string GlobalModelGenerator::computeGlobalStateHash (
    vector< LocalState * > * localStates ) [protected]
```

Creates a hash from a set of [LocalState](#).

Parameters

<i>localStates</i>	Pointer to a vector of pointers of LocalState to turn into a hash.
--------------------	--

Returns

Returns a string with a hash.

5.33.3.4 createProbabilityStrategy()

```
set< set< tuple< string, string > > > GlobalModelGenerator::createProbabilityStrategy (
    LocalModels * localModels )
```

Prepares the strategy generating structures for the probabilistic verification.

Parameters

<i>localModels</i>	LocalModels to generate the coalition transition buckets from.
--------------------	--

5.33.3.5 expandAllStates()

```
void GlobalModelGenerator::expandAllStates (
    bool additionalProbSplit = false )
```

Expands the states starting from the initial [GlobalState](#) and continues until there are no more states to expand.

5.33.3.6 expandAndReduceAllStates()

```
void GlobalModelGenerator::expandAndReduceAllStates ( )
```

Expands and reduces the states starting from the initial [GlobalState](#) using a DFS-POR algorithm and continues until there are no more states to expand.

Parameters

<i>depth</i>	Current depth of the recursive generation of all states.
--------------	--

5.33.3.7 expandState()

```
void GlobalModelGenerator::expandState (
    GlobalState * state )
```

Goes through all [GlobalTransition](#) in a given [GlobalState](#) and creates new GlobalStates connected to the given one.

Parameters

<i>state</i>	A state from which the expansion should start.
--------------	--

5.33.3.8 expandStateAndReturn()

```
vector< GlobalState * > GlobalModelGenerator::expandStateAndReturn (
    GlobalState * state,
    bool returnAnyway = false )
```

[GlobalModelGenerator::expandState](#) that also additionally returns a vector of newly created states.

Parameters

<i>state</i>	A state from which the expansion should start.
--------------	--

5.33.3.9 findGlobalStateInEpistemicClass()

```
GlobalState * GlobalModelGenerator::findGlobalStateInEpistemicClass (
    vector< LocalState * > * localStates,
    EpistemicClass * epistemicClass ) [protected]
```

Gets a [GlobalState](#) from an [EpistemicClass](#) if it exists in that episcemic class.

Parameters

<i>localStates</i>	Pointer to a vector of pointers to LocalState , from which will be generated a global state hash.
<i>epistemicClass</i>	Epistemic class in which to check if a GlobalState exists.

Returns

Returns a pointer to a [GlobalState](#) if it exists in given epistemic class, otherwise returns nullptr.

5.33.3.10 findOrCreateEpistemicClass()

```
EpistemicClass * GlobalModelGenerator::findOrCreateEpistemicClass (
    vector< LocalState * > * localStates,
    Agent * agent ) [protected]
```

Checks if a vector of [LocalState](#) is already an epistemic class for a given [Agent](#), if not, creates a new one.

Parameters

<i>localStates</i>	Local states from agent.
<i>agent</i>	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing [EpistemicClass](#).

5.33.3.11 findOrCreateEpistemicClassForKnowledge()

```
set< GlobalState * > * GlobalModelGenerator::findOrCreateEpistemicClassForKnowledge (
    vector< LocalState * > * localStates,
    GlobalState * global,
    Agent * agent )
```

Checks if a vector of [LocalState](#) is already an epistemic class for a given [Agent](#), if not, creates a new one.

Parameters

<i>localStates</i>	Local states from agent.
<i>agent</i>	Agent for which to check the existence of an epistemic class.

Returns

A pointer to a new or existing [EpistemicClass](#).

5.33.3.12 generateGlobalTransitions()

```
void GlobalModelGenerator::generateGlobalTransitions (
    GlobalState * fromGlobalState,
    set< LocalTransition * > localTransitions,
    map< Agent *, vector< LocalTransition * >> transitionsByAgent ) [protected]
```

Adds all shared global transitions to a [GlobalState](#).

Parameters

<i>fromGlobalState</i>	Global state to add transitions to.
<i>localTransitions</i>	Initially empty, available local transitions by each agent from transitionsByAgent.
<i>transitionsByAgent</i>	Mapped transitions to an agent, only with transitions available for the agent at this moment.

5.33.3.13 generateInitState()

```
GlobalState * GlobalModelGenerator::generateInitState ( ) [protected]
```

Generates initial state of the model from [GlobalModel](#) in memory.

Returns

Returns a pointer to an initial [GlobalState](#).

5.33.3.14 generateNextMDP()

```
MDP GlobalModelGenerator::generateNextMDP (
    bool makeOpponentGoMax = false )
```

5.33.3.15 generateStateFromLocalStates()

```
GlobalState * GlobalModelGenerator::generateStateFromLocalStates (
    vector< LocalState * > * localStates,
    set< LocalTransition * > * viaLocalTransitions,
    GlobalState * prevGlobalState ) [protected]
```

Creates a new [GlobalState](#) using some of the internally known model data and given local states, transitions that were used to get there and the previous global state.

Parameters

<i>localStates</i>	LocalStates from which the new GlobalState will be built.
<i>viaLocalTransitions</i>	Pointer to a set of pointers to LocalTransition from which the changes in variables, as a result of traversing through the transition, will be made in a new GlobalState .
<i>prevGlobalState</i>	Pointer to GlobalState from which all persistent variables will be copied over from to the new GlobalState .

Returns

Returns a pointer to a new or already existing in the same epistemic class [GlobalModel](#).

5.33.3.16 getActionNameFromStateInStrategy()

```
string GlobalModelGenerator::getActionNameFromStateInStrategy (
    GlobalState * state )
```

Gives next action's name from a given state.

Parameters

<i>state</i>	GlobalState from which an action name would be retrieved if a set strategy is present.
--------------	--

Returns

String containing the action name ending on a semicolon.

5.33.3.17 getAgentInstanceByName()

```
Agent * GlobalModelGenerator::getAgentInstanceByName (
    string agentName )
```

Get a pointer to an agent by using its name.

Parameters

<i>agentName</i>	Name of an Agent that we want to get its instance.
------------------	--

Returns

Pointer to an [Agent](#).

5.33.3.18 getCoalitionActionSignature()

```
string GlobalModelGenerator::getCoalitionActionSignature (
    GlobalTransition * transition,
    char sep = ';' )
```

Returns coalition-only action signature, ordered by agentIndex and using localName when available.

5.33.3.19 getCoalitionIdentifier()

```
string GlobalModelGenerator::getCoalitionIdentifier (
    vector< LocalState * > * localStates )
```

Returns a concatenated epistemic class ID of all coalition members in given LocalStates.

Parameters

<i>localStates</i>	Vector of LocalStates to extract coalition IDs from.
--------------------	--

Returns

[LocalState](#) IDs for coalition members separated with semicolons, ending on a semicolon.

5.33.3.20 `getCurrentGlobalModel()`

```
GlobalModel * GlobalModelGenerator::getCurrentGlobalModel ( )
```

Get for a [GlobalModel](#) used in initialization.

Returns

Returns a pointer to a global model.

5.33.3.21 `getFormula()`

```
Formula * GlobalModelGenerator::getFormula ( )
```

Get for the [Formula](#) used in initialization.

Returns

Returns a pointer to the formula structure.

5.33.3.22 `getFormulaCorectness()`

```
bool GlobalModelGenerator::getFormulaCorectness ( )
```

Gets formula status.

Returns

True if formula is written correctly, false otherwise.

5.33.3.23 `getFormulaSize()`

```
int GlobalModelGenerator::getFormulaSize ( )
```

Get size of the [Formula](#) used in initialization.

Returns

Returns the formula size.

5.33.3.24 getNextPath()

```
set< tuple< string, string > > * GlobalModelGenerator::getNextPath ( )
```

Retrieves the next strategy one at a time (iteratively). Systematically tries all combinations of action choices at each epistemic class.

Returns

Pointer to next strategy, or nullptr if all exhausted.

5.33.3.25 initModel()

```
GlobalState * GlobalModelGenerator::initModel (
    LocalModels * localModels,
    Formula * formula )
```

Initializes a global model from local models and a formula.

Parameters

<i>localModels</i>	Pointer to LocalModels that will construct a global model.
<i>formula</i>	Pointer to a Formula to include into the model.

Returns

Returns a pointer to initial state of the global model.

5.33.3.26 initStrategy()

```
void GlobalModelGenerator::initStrategy (
    StrategyCollection * strat )
```

Initiates the strategy with a given [StrategyCollection](#).

Parameters

<i>strat</i>	StrategyCollection to initialize the strategy.
--------------	--

5.33.3.27 markFormulaAsIncorrect()

```
void GlobalModelGenerator::markFormulaAsIncorrect ( )
```

Sets formula to an incorrectly written state.

5.33.4 Member Data Documentation

5.33.4.1 actionCounts

```
map<string, size_t> GlobalModelGenerator::actionCounts [protected]
```

5.33.4.2 addedStates

```
unordered_set<GlobalState*> GlobalModelGenerator::addedStates [protected]
```

States that were added to a stack of states. Used for reductions.

5.33.4.3 agentIndex

```
map<Agent*, size_t> GlobalModelGenerator::agentIndex
```

auxiliary variable mapping [Agent](#) pointer to its index (replace size_t with if needed later)

5.33.4.4 candidateStateDepths

```
unordered_map<GlobalState*, unordered_set<int> > GlobalModelGenerator::candidateStateDepths  
[protected]
```

Lookup map containing global states and the depths that the global state is contained in. Used for reductions.

5.33.4.5 choiceIndices

```
map<string, size_t> GlobalModelGenerator::choiceIndices [protected]
```

5.33.4.6 coalitionTransitions

```
map<string, map<string, set<GlobalTransition*> > > GlobalModelGenerator::coalitionTransitions  
[protected]
```

5.33.4.7 correctModel

```
bool GlobalModelGenerator::correctModel [protected]
```

Flag holding info if model is actually correct.

5.33.4.8 currentStrategy

```
set<tuple<string, string> > GlobalModelGenerator::currentStrategy [protected]
```

5.33.4.9 formula

```
Formula* GlobalModelGenerator::formula [protected]
```

[Formula](#) used in initModel.

5.33.4.10 globalModel

```
GlobalModel* GlobalModelGenerator::globalModel [protected]
```

[GlobalModel](#) created in initModel.

5.33.4.11 globalModelCandidates

```
stack<GlobalState*> GlobalModelGenerator::globalModelCandidates [protected]
```

Candidate states to be used in reductions. Used for reductions.

5.33.4.12 localModels

```
LocalModels* GlobalModelGenerator::localModels [protected]
```

[LocalModels](#) used in initModel.

5.33.4.13 opponentsTransitions

```
map<string, map<string, set<GlobalTransition*> > > GlobalModelGenerator::opponentsTransitions  
[protected]
```

5.33.4.14 stateDepths

```
stack<int> GlobalModelGenerator::stateDepths [protected]
```

Saved depths of states added to statesToExpand. Used for reductions.

5.33.4.15 stateHashMapCache

```
unordered_map<string, GlobalState*> GlobalModelGenerator::stateHashMapCache [protected]
```

5.33.4.16 strategiesExhausted

```
bool GlobalModelGenerator::strategiesExhausted = false [protected]
```

5.33.4.17 strategyCollection

```
StrategyCollection* GlobalModelGenerator::strategyCollection [protected]
```

Strategy to check during verification (if any)

5.33.4.18 strategyGenerationInit

```
bool GlobalModelGenerator::strategyGenerationInit = false [protected]
```

The documentation for this class was generated from the following files:

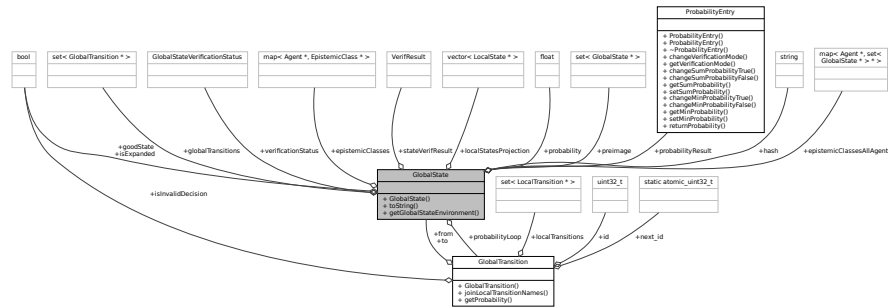
- [GlobalModelGenerator.hpp](#)
- [GlobalModelGenerator.cpp](#)

5.34 GlobalState Struct Reference

Represents a single global state.

```
#include <GlobalState.hpp>
```

Collaboration diagram for GlobalState:



Public Member Functions

- [GlobalState](#) ()
- `std::string` [toString](#) (string indent="")
Debug information on the given [GlobalState](#).
- `map< string, int >` [getGlobalStateEnvironment](#) ()
Get for the environment of a given global state.

Public Attributes

- `string` [hash](#)
Hash of the global state used in quick checks if the states are in the same epistemic class.
- `map< Agent *, EpistemicClass * >` [epistemicClasses](#)
Map of agents and the epistemic classes that belongs to the respective agent.
- `bool` [isExpanded](#)
If false, the state can be still expanded, potentially creating new states, otherwise the expansion of the state already occurred and is not necessary.
- `GlobalStateVerificationStatus verificationStatus = GLOBAL_STATE_VERIFICATION_STATUS::UNVERIFIED`
Current verification status of this state.
- `float` [probability](#) = `config.probability` ? 0.0 : 1.0
Collective probability of the state.
- `bool` [goodState](#) = false
- `set< GlobalTransition * >` [globalTransitions](#)
Every [GlobalTransition](#) in the model.
- `set< GlobalState * >` [preimage](#)
Preimage for the global state, i.e. states from which there is a transition to this state.
- `vector< LocalState * >` [localStatesProjection](#)
Local states of each agent that define this global state.
- `map< Agent *, set< GlobalState * > * >` [epistemicClassesAllAgents](#)
States in the same epistemic class as the current one, for KBC.
- `ProbabilityEntry` [probabilityResult](#)
- `GlobalTransition * probabilityLoop = nullptr`
Used to resolve self loops during probability calculation.
- `VerifResult stateVerifResult = VerifResult::NOT_VERIFIED`

5.34.1 Detailed Description

Represents a single global state.

5.34.2 Constructor & Destructor Documentation

5.34.2.1 GlobalState()

```
GlobalState::GlobalState ( )
```

5.34.3 Member Function Documentation

5.34.3.1 getGlobalStateEnvironment()

```
map< string, int > GlobalState::getGlobalStateEnvironment ( )
```

Get for the environment of a given global state.

Returns

A map of variable names and their values for the current global state.

5.34.3.2 toString()

```
std::string GlobalState::toString (
    string indent = "" )
```

Debug information on the given [GlobalState](#).

Parameters

<i>indent</i>	- optional indentation string
---------------	-------------------------------

Returns

[GlobalState](#) data

[GlobalState](#) data

5.34.4 Member Data Documentation

5.34.4.1 epistemicClasses

```
map<Agent*, EpistemicClass*> GlobalState::epistemicClasses
```

Map of agents and the epistemic classes that belongs to the respective agent.

5.34.4.2 epistemicClassesAllAgents

```
map<Agent*, set<GlobalState*>*> GlobalState::epistemicClassesAllAgents
```

States in the same epistemic class as the current one, for KBC.

5.34.4.3 globalTransitions

```
set<GlobalTransition*> GlobalState::globalTransitions
```

Every [GlobalTransition](#) in the model.

5.34.4.4 goodState

```
bool GlobalState::goodState = false
```

5.34.4.5 hash

```
string GlobalState::hash
```

Hash of the global state used in quick checks if the states are in the same epistemic class.

5.34.4.6 isExpanded

```
bool GlobalState::isExpanded
```

If false, the state can be still expanded, potentially creating new states, otherwise the expansion of the state already occurred and is not necessary.

5.34.4.7 localStatesProjection

```
vector<LocalState*> GlobalState::localStatesProjection
```

Local states of each agent that define this global state.

5.34.4.8 preimage

```
set<GlobalState*> GlobalState::preimage
```

Preimage for the global state, i.e. states from which there is a transition to this state.

5.34.4.9 probability

```
float GlobalState::probability = config.probability ? 0.0 : 1.0
```

Collective probability of the state.

5.34.4.10 probabilityLoop

```
GlobalTransition* GlobalState::probabilityLoop = nullptr
```

Used to resolve self loops during probability calculation.

5.34.4.11 probabilityResult

```
ProbabilityEntry GlobalState::probabilityResult
```

5.34.4.12 stateVerifResult

```
VerifResult GlobalState::stateVerifResult = VerifResult::NOT_VERIFIED
```

5.34.4.13 verificationStatus

`GlobalStateVerificationStatus` `GlobalState::verificationStatus` = `GLOBAL_STATE_VERIFICATION_STATUS::UNVERIFIED`

Current verivation status of this state.

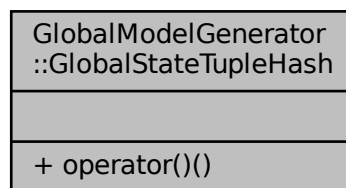
The documentation for this struct was generated from the following files:

- [GlobalState.hpp](#)
- [GlobalState.cpp](#)

5.35 GlobalModelGenerator::GlobalStateTupleHash Struct Reference

```
#include <GlobalModelGenerator.hpp>
```

Collaboration diagram for GlobalModelGenerator::GlobalStateTupleHash:



Public Member Functions

- `size_t operator()` (`const tuple< GlobalState *, int > &t`) `const`

5.35.1 Member Function Documentation

5.35.1.1 operator()

```
size_t GlobalModelGenerator::GlobalStateTupleHash::operator() (
    const tuple< GlobalState *, int > & t ) const [inline]
```

The documentation for this struct was generated from the following file:

- [GlobalModelGenerator.hpp](#)

5.36 GlobalTransition Struct Reference

Represents a single global transition.

```
#include <GlobalTransition.hpp>
```

Collaboration diagram for GlobalTransition:



Public Member Functions

- [GlobalTransition](#) ()
Constructor for [GlobalTransition](#).
- string [joinLocalTransitionNames](#) (char sep=';')
Concatenates local transition names into a string.
- float [getProbability](#) ()

Public Attributes

- uint32_t [id](#)
Identifier of the transition.
- bool [isInvalidDecision](#)
Marks if the transition is invalid, true if there is no point in traversing that transition, otherwise false.
- [GlobalState](#) * [from](#)
Binding to a [GlobalState](#) from which this transition goes from.
- [GlobalState](#) * [to](#)
Binding to a [GlobalState](#) from which this transition goes to.
- set< [LocalTransition](#) * > [localTransitions](#)
Local transitions that define this global transition. A single transition or more in case of shared transitions.

Static Public Attributes

- static atomic_uint32_t [next_id](#)

5.36.1 Detailed Description

Represents a single global transition.

5.36.2 Constructor & Destructor Documentation

5.36.2.1 GlobalTransition()

```
GlobalTransition::GlobalTransition ( )
```

Constructor for [GlobalTransition](#).

5.36.3 Member Function Documentation

5.36.3.1 getProbability()

```
float GlobalTransition::getProbability ( )
```

5.36.3.2 joinLocalTransitionNames()

```
string GlobalTransition::joinLocalTransitionNames (
    char sep = ';' )
```

Concatenates local transition names into a string.

Parameters

<i>sep</i>	Separator used for separating the transition names.
------------	---

Returns

String of transition names joined with the given separator and ending on the separator.

5.36.4 Member Data Documentation

5.36.4.1 from

```
GlobalState* GlobalTransition::from
```

Binding to a [GlobalState](#) from which this transition goes from.

5.36.4.2 id

```
uint32_t GlobalTransition::id
```

Identifier of the transition.

5.36.4.3 isInvalidDecision

```
bool GlobalTransition::isInvalidDecision
```

Marks if the transition is invalid, true if there is no point in traversing that transition, otherwise false.

5.36.4.4 localTransitions

```
set<LocalTransition*> GlobalTransition::localTransitions
```

Local transitions that define this global transition. A single transition or more in case of shared transitions.

5.36.4.5 next_id

```
atomic_uint32_t GlobalTransition::next_id [static]
```

5.36.4.6 to

```
GlobalState* GlobalTransition::to
```

Binding to a [GlobalState](#) from which this transition goes to.

The documentation for this struct was generated from the following files:

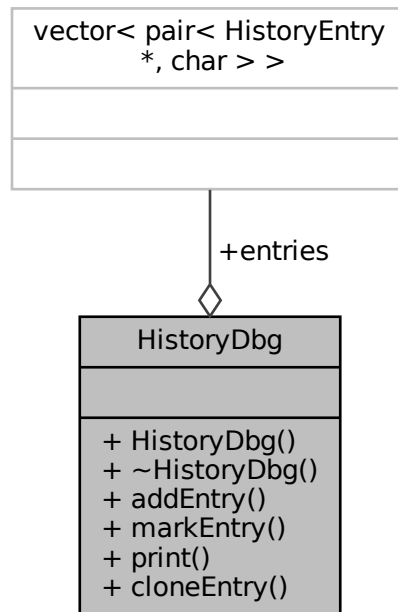
- [GlobalTransition.hpp](#)
- [GlobalTransition.cpp](#)

5.37 HistoryDbg Class Reference

Stores history and allows displaying it to the console.

```
#include <Verification.hpp>
```

Collaboration diagram for HistoryDbg:



Public Member Functions

- [HistoryDbg \(\)](#)
A constructor for [HistoryDbg](#).
- [~HistoryDbg \(\)](#)
A destructor for [HistoryDbg](#).
- void [addEntry](#) ([HistoryEntry](#) *entry)
Adds a [HistoryEntry](#) to the debug history.
- void [markEntry](#) ([HistoryEntry](#) *entry, char chr)
Marks an entry in the debug history with a char.
- void [print](#) (string prefix)
Prints every entry from the algorithm's path.
- [HistoryEntry](#) * [cloneEntry](#) ([HistoryEntry](#) *entry)
Checks if the [HistoryEntry](#) pointer exists in the debug history.

Public Attributes

- vector< pair< [HistoryEntry](#) *, char > > [entries](#)
A pair of history entries and a char marking history type.

5.37.1 Detailed Description

Stores history and allows displaying it to the console.

5.37.2 Constructor & Destructor Documentation

5.37.2.1 HistoryDbg()

```
HistoryDbg::HistoryDbg ( )
```

A constructor for [HistoryDbg](#).

5.37.2.2 ~HistoryDbg()

```
HistoryDbg::~~HistoryDbg ( )
```

A destructor for [HistoryDbg](#).

5.37.3 Member Function Documentation

5.37.3.1 addEntry()

```
void HistoryDbg::addEntry (
    HistoryEntry * entry )
```

Adds a [HistoryEntry](#) to the debug history.

Parameters

<i>entry</i>	A pointer to the HistoryEntry that will be added to the history.
--------------	--

5.37.3.2 cloneEntry()

```
HistoryEntry * HistoryDbg::cloneEntry (
    HistoryEntry * entry )
```

Checks if the [HistoryEntry](#) pointer exists in the debug history.

Parameters

<i>entry</i>	A pointer to a HistoryEntry to be checked.
--------------	--

Returns

Identity function if the entry is in history, otherwise returns nullptr.

5.37.3.3 markEntry()

```
void HistoryDbg::markEntry (
    HistoryEntry * entry,
    char chr )
```

Marks an entry in the debug history with a char.

Parameters

<i>entry</i>	A pointer to a HistoryEntry that is supposed to be marked.
<i>chr</i>	A char that will be made into a pair with a HistoryEntry .

5.37.3.4 print()

```
void HistoryDbg::print (
    string prefix )
```

Prints every entry from the algorithm's path.

Parameters

<i>prefix</i>	A prefix string to append to the front of every entry.
---------------	--

5.37.4 Member Data Documentation

5.37.4.1 entries

```
vector<pair<HistoryEntry*, char> > HistoryDbg::entries
```

A pair of history entries and a char marking history type.

The documentation for this class was generated from the following files:

5.38.1 Detailed Description

Structure used to save model traversal history.

5.38.2 Member Function Documentation

5.38.2.1 toString()

```
string HistoryEntry::toString ( ) [inline]
```

Converts [HistoryEntry](#) to string.

Returns

A string with the description of this history record.

5.38.3 Member Data Documentation

5.38.3.1 decision

```
GlobalTransition* HistoryEntry::decision
```

Selected transition.

5.38.3.2 depth

```
int HistoryEntry::depth
```

Recursion depth.

5.38.3.3 globalState

```
GlobalState* HistoryEntry::globalState
```

Saved global state.

5.38.3.4 globalTransitionControlled

```
bool HistoryEntry::globalTransitionControlled
```

Is the transition controlled by an agent in coalition.

5.38.3.5 newStatus

```
GlobalStateVerificationStatus HistoryEntry::newStatus
```

Next model verification state.

5.38.3.6 next

```
HistoryEntry* HistoryEntry::next
```

Pointer to the next [HistoryEntry](#).

5.38.3.7 prev

```
HistoryEntry* HistoryEntry::prev
```

Pointer to the previous [HistoryEntry](#).

5.38.3.8 prevStatus

```
GlobalStateVerificationStatus HistoryEntry::prevStatus
```

Previous model verification state.

5.38.3.9 strategy

```
StrategyEntry HistoryEntry::strategy
```

Holds currently processed strategy for the current state.

5.38.3.10 type

`HistoryEntryType` `HistoryEntry::type`

Type of the history record.

The documentation for this struct was generated from the following file:

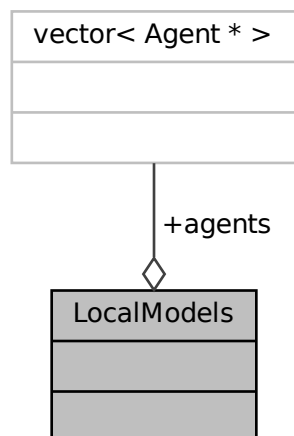
- [Verification.hpp](#)

5.39 LocalModels Struct Reference

Represents a single local model, contains all agents and variables.

```
#include <Types.hpp>
```

Collaboration diagram for LocalModels:



Public Attributes

- `vector< Agent * > agents`
A vector of agents for the current model.

5.39.1 Detailed Description

Represents a single local model, contains all agents and variables.

5.39.2 Member Data Documentation

5.39.2.1 agents

```
vector<Agent*> LocalModels::agents
```

A vector of agents for the current model.

The documentation for this struct was generated from the following file:

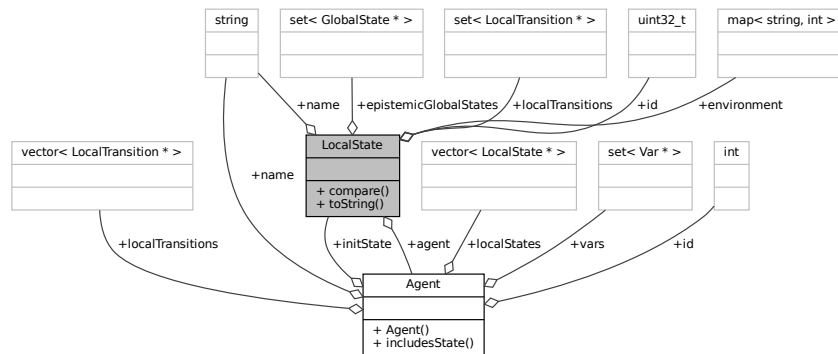
- [Types.hpp](#)

5.40 LocalState Class Reference

Represents a single [LocalState](#), containing id, name and internal variables.

```
#include <LocalState.hpp>
```

Collaboration diagram for LocalState:



Public Member Functions

- `bool compare (LocalState *state)`
Function comparing two states.
- `string toString (string indent="")`
Debug information on the given [LocalState](#).

Public Attributes

- `uint32_t id`
State identifier.
- `string name`
State name.
- `map< string, int > environment`
Local variables as a name and their current values.
- `Agent * agent`
Binding to an [Agent](#).
- `set< LocalTransition * > localTransitions`
Binding to the set of [LocalTransition](#).
- `set< GlobalState * > epistemicGlobalStates`
Binding to the set of [GlobalState](#) that this [LocalState](#) belongs to.

5.40.1 Detailed Description

Represents a single [LocalState](#), containing id, name and internal variables.

5.40.2 Member Function Documentation

5.40.2.1 `compare()`

```
bool LocalState::compare (
    LocalState * state )
```

Function comparing two states.

Parameters

<code>state</code>	A pointer to LocalState to which this state should be compared to.
--------------------	--

Returns

Returns true if the current [LocalState](#) is the same as the passed one, otherwise false.

5.40.2.2 `toString()`

```
string LocalState::toString (
    string indent = "" )
```

Debug information on the given [LocalState](#).

Parameters

<i>indent</i>	- optional indentation string
---------------	-------------------------------

Returns

[LocalState](#) data

[LocalState](#) data

5.40.3 Member Data Documentation

5.40.3.1 agent

[Agent](#)* `LocalState::agent`

Binding to an [Agent](#).

5.40.3.2 environment

`map<string, int> LocalState::environment`

Local variables as a name and their current values.

5.40.3.3 epistemicGlobalStates

`set<GlobalState> LocalState::epistemicGlobalStates`

Binding to the set of [GlobalState](#) that this [LocalState](#) belongs to.

5.40.3.4 id

`uint32_t LocalState::id`

State identifier.

5.40.3.5 localTransitions

```
set<LocalTransition*> LocalState::localTransitions
```

Binding to the set of [LocalTransition](#).

5.40.3.6 name

```
string LocalState::name
```

State name.

The documentation for this class was generated from the following files:

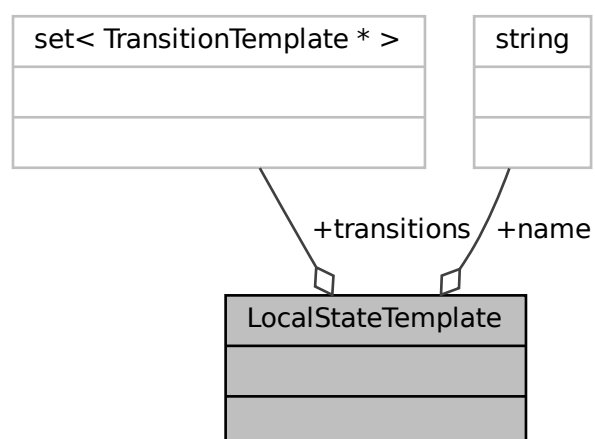
- [LocalState.hpp](#)
- [LocalState.cpp](#)

5.41 LocalStateTemplate Class Reference

A template for the local state.

```
#include <nodes.hpp>
```

Collaboration diagram for LocalStateTemplate:



Public Attributes

- string [name](#)
Name of the local state.
- set< [TransitionTemplate](#) * > [transitions](#)
Local transitions going out from this state.

5.41.1 Detailed Description

A template for the local state.

5.41.2 Member Data Documentation

5.41.2.1 name

```
string LocalStateTemplate::name
```

Name of the local state.

5.41.2.2 transitions

```
set<TransitionTemplate*> LocalStateTemplate::transitions
```

Local transitions going out from this state.

The documentation for this class was generated from the following file:

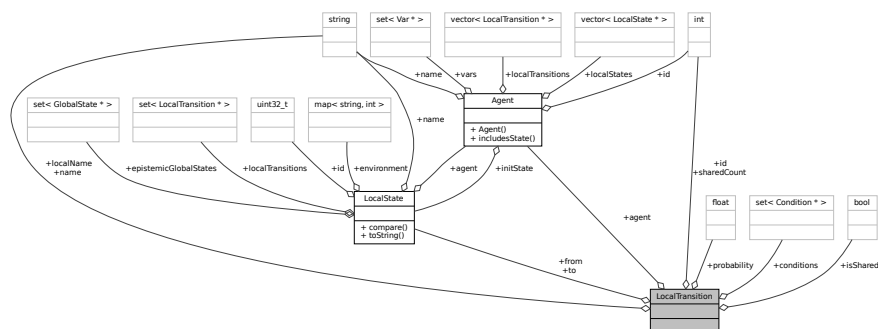
- [nodes.hpp](#)

5.42 LocalTransition Struct Reference

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

```
#include <LocalTransition.hpp>
```

Collaboration diagram for LocalTransition:



Public Attributes

- int [id](#)
Identifier of the transition.
- string [name](#)
Name of the transition (global).
- string [localName](#)
Name of the transition (local).
- bool [isShared](#)
Is the transition appearing somewhere else, true if yes, false if no.
- int [sharedCount](#)
Count of recurring appearances of this transition.
- set< [Condition](#) * > [conditions](#)
Conditions that have to be fulfilled for the transition to be available.
- float [probability](#)
Used for probability formula verification. The probability of this [LocalTransition](#) executing.
- [Agent](#) * [agent](#)
Binding to an [Agent](#).
- [LocalState](#) * [from](#)
Binding to a [LocalState](#) from which this transition goes from.
- [LocalState](#) * [to](#)
Binding to a [LocalState](#) from which this transition goes to.

5.42.1 Detailed Description

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

5.42.2 Member Data Documentation

5.42.2.1 agent

```
Agent* LocalTransition::agent
```

Binding to an [Agent](#).

5.42.2.2 conditions

```
set<Condition*> LocalTransition::conditions
```

Conditions that have to be fulfilled for the transition to be available.

5.42.2.3 from

```
LocalState* LocalTransition::from
```

Binding to a [LocalState](#) from which this transition goes from.

5.42.2.4 id

```
int LocalTransition::id
```

Identifier of the transition.

5.42.2.5 isShared

```
bool LocalTransition::isShared
```

Is the transition appearing somewhere else, true if yes, false if no.

5.42.2.6 localName

```
string LocalTransition::localName
```

Name of the transition (local).

5.42.2.7 name

```
string LocalTransition::name
```

Name of the transition (global).

5.42.2.8 probability

```
float LocalTransition::probability
```

Used for probability formula verification. The probability of this [LocalTransition](#) executing.

5.42.2.9 sharedCount

```
int LocalTransition::sharedCount
```

Count of recurring appearances of this transition.

5.42.2.10 to

```
LocalState* LocalTransition::to
```

Binding to a [LocalState](#) from which this transition goes to.

The documentation for this struct was generated from the following file:

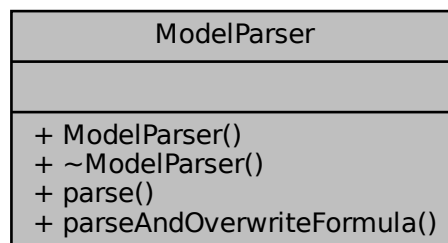
- [LocalTransition.hpp](#)

5.43 ModelParser Class Reference

A parser for converting a text file into a model.

```
#include <ModelParser.hpp>
```

Collaboration diagram for ModelParser:



Public Member Functions

- [ModelParser](#) ()
ModelParser constructor.
- [~ModelParser](#) ()
ModelParser destructor.
- tuple< [LocalModels](#), [Formula](#) > [parse](#) (string fileName)
Parses a file with given name into a usable model.
- tuple< [LocalModels](#), [Formula](#) > [parseAndOverwriteFormula](#) (string fileName, string s)
Parses a text file in the given file path and internally replaces the verification formula with the given one.

5.43.1 Detailed Description

A parser for converting a text file into a model.

5.43.2 Constructor & Destructor Documentation

5.43.2.1 ModelParser()

```
ModelParser::ModelParser ( )
```

[ModelParser](#) constructor.

5.43.2.2 ~ModelParser()

```
ModelParser::~~ModelParser ( )
```

[ModelParser](#) destructor.

5.43.3 Member Function Documentation

5.43.3.1 parse()

```
tuple< LocalModels, Formula > ModelParser::parse (
    string fileName )
```

Parses a file with given name into a usable model.

Parameters

<i>fileName</i>	Name of the file to be converted into a model.
-----------------	--

Returns

Pointer to a model created from a given file.

5.43.3.2 parseAndOverwriteFormula()

```
tuple< LocalModels, Formula > ModelParser::parseAndOverwriteFormula (
    string fileName,
    string s )
```

Parses a text file in the given file path and internally replaces the verification formula with the given one.

Parameters

<i>fileName</i>	Path to the text file.
<i>s</i>	New formula.

Returns

Returns a new model generation structure with a replaced formula.

The documentation for this class was generated from the following files:

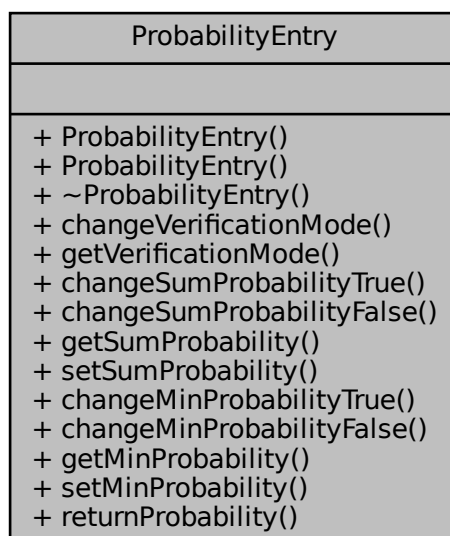
- [ModelParser.hpp](#)
- [ModelParser.cc](#)

5.44 ProbabilityEntry Class Reference

Class for handling the probability operations during probability verification.

```
#include <TypesDependency.hpp>
```

Collaboration diagram for ProbabilityEntry:



Public Member Functions

- [ProbabilityEntry](#) ()
- [ProbabilityEntry](#) ([ProbabilityCalculationType](#) mode)
- [~ProbabilityEntry](#) ()
- void [changeVerificationMode](#) ([ProbabilityCalculationType](#) mode)
- [ProbabilityCalculationType](#) [getVerificationMode](#) ()
- void [changeSumProbabilityTrue](#) (float changeProbabilityBy)
- void [changeSumProbabilityFalse](#) (float changeProbabilityBy)
- [ProbabilityTrueFalse](#) [getSumProbability](#) ()
- void [setSumProbability](#) ([ProbabilityTrueFalse](#) newSumProbability)
- void [changeMinProbabilityTrue](#) (float newProbability)
- void [changeMinProbabilityFalse](#) (float newProbability)
- [ProbabilityTrueFalse](#) [getMinProbability](#) ()
- void [setMinProbability](#) ([ProbabilityTrueFalse](#) newMinProbability)
- [ProbabilityTrueFalse](#) [returnProbability](#) ()

5.44.1 Detailed Description

Class for handling the probability operations during probability verification.

5.44.2 Constructor & Destructor Documentation

5.44.2.1 [ProbabilityEntry](#)() [1/2]

```
ProbabilityEntry::ProbabilityEntry ( ) [inline]
```

5.44.2.2 [ProbabilityEntry](#)() [2/2]

```
ProbabilityEntry::ProbabilityEntry (  
    ProbabilityCalculationType mode ) [inline]
```

5.44.2.3 [~ProbabilityEntry](#)()

```
ProbabilityEntry::~~ProbabilityEntry ( ) [inline]
```

5.44.3 Member Function Documentation

5.44.3.1 changeMinProbabilityFalse()

```
void ProbabilityEntry::changeMinProbabilityFalse (
    float newProbability ) [inline]
```

5.44.3.2 changeMinProbabilityTrue()

```
void ProbabilityEntry::changeMinProbabilityTrue (
    float newProbability ) [inline]
```

5.44.3.3 changeSumProbabilityFalse()

```
void ProbabilityEntry::changeSumProbabilityFalse (
    float changeProbabilityBy ) [inline]
```

5.44.3.4 changeSumProbabilityTrue()

```
void ProbabilityEntry::changeSumProbabilityTrue (
    float changeProbabilityBy ) [inline]
```

5.44.3.5 changeVerificationMode()

```
void ProbabilityEntry::changeVerificationMode (
    ProbabilityCalculationType mode ) [inline]
```

5.44.3.6 getMinProbability()

```
ProbabilityTrueFalse ProbabilityEntry::getMinProbability ( ) [inline]
```

5.44.3.7 getSumProbability()

```
ProbabilityTrueFalse ProbabilityEntry::getSumProbability ( ) [inline]
```

5.44.3.8 getVerificationMode()

```
ProbabilityCalculationType ProbabilityEntry::getVerificationMode ( ) [inline]
```

5.44.3.9 returnProbability()

```
ProbabilityTrueFalse ProbabilityEntry::returnProbability ( ) [inline]
```

5.44.3.10 setMinProbability()

```
void ProbabilityEntry::setMinProbability (
    ProbabilityTrueFalse newMinProbability ) [inline]
```

5.44.3.11 setSumProbability()

```
void ProbabilityEntry::setSumProbability (
    ProbabilityTrueFalse newSumProbability ) [inline]
```

The documentation for this class was generated from the following file:

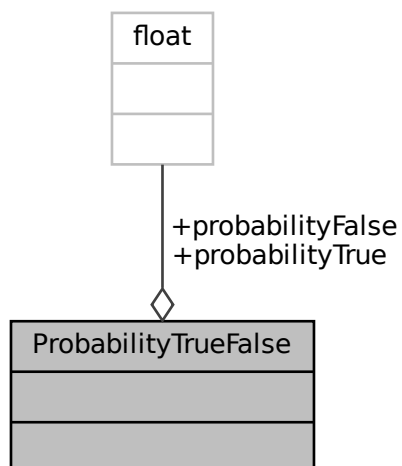
- [TypesDependency.hpp](#)

5.45 ProbabilityTrueFalse Struct Reference

Contains probability of a state being in a true or false state.

```
#include <TypesDependency.hpp>
```

Collaboration diagram for ProbabilityTrueFalse:



Public Attributes

- float [probabilityTrue](#) = 0.0
- float [probabilityFalse](#) = 0.0

5.45.1 Detailed Description

Contains probability of a state being in a true or false state.

5.45.2 Member Data Documentation

5.45.2.1 [probabilityFalse](#)

```
float ProbabilityTrueFalse::probabilityFalse = 0.0
```

5.45.2.2 [probabilityTrue](#)

```
float ProbabilityTrueFalse::probabilityTrue = 0.0
```

The documentation for this struct was generated from the following file:

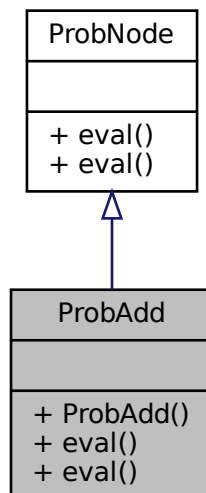
- [TypesDependency.hpp](#)

5.46 ProbAdd Class Reference

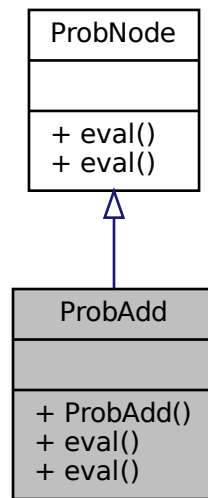
Node for addition.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbAdd:



Collaboration diagram for ProbAdd:



Public Member Functions

- [ProbAdd](#) ([ProbNode](#) *_larg, [ProbNode](#) *_rarg)
Addition expression constructor.
- virtual float [eval](#) ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual float [eval](#) ()

5.46.1 Detailed Description

Node for addition.

5.46.2 Constructor & Destructor Documentation

5.46.2.1 ProbAdd()

```

ProbAdd::ProbAdd (
    ProbNode * _larg,
    ProbNode * _rarg ) [inline]
  
```

Addition expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.46.3 Member Function Documentation

5.46.3.1 `eval()` [1/2]

```
float ProbAdd::eval ( ) [virtual]
```

Implements [ProbNode](#).

5.46.3.2 `eval()` [2/2]

```
float ProbAdd::eval (
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns a float.

Implements [ProbNode](#).

The documentation for this class was generated from the following files:

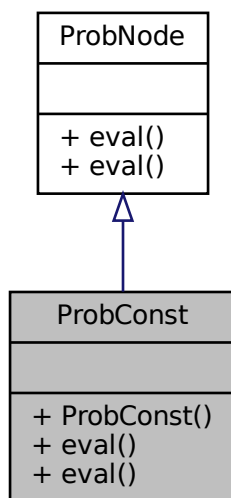
- [expressions.hpp](#)
- [expressions.cc](#)

5.47 ProbConst Class Reference

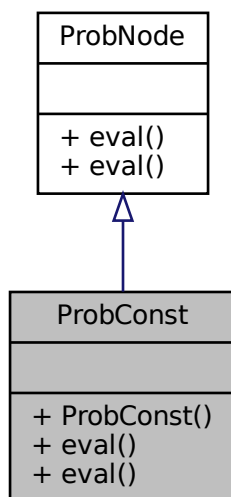
Node for a constant.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbConst:



Collaboration diagram for ProbConst:



Public Member Functions

- [ProbConst](#) (float _val)
Constant expression constructor.

- virtual float [eval](#) ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual float [eval](#) ()

5.47.1 Detailed Description

Node for a constant.

5.47.2 Constructor & Destructor Documentation

5.47.2.1 ProbConst()

```
ProbConst::ProbConst (  
    float _val ) [inline]
```

Constant expression constructor.

Parameters

<code>_val</code>	ProbConst value.
-------------------	----------------------------------

5.47.3 Member Function Documentation

5.47.3.1 [eval\(\)](#) [1/2]

```
float ProbConst::eval ( ) [virtual]
```

Implements [ProbNode](#).

5.47.3.2 [eval\(\)](#) [2/2]

```
float ProbConst::eval (  
    GlobalModelGenerator * generator,  
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns a float.

Implements [ProbNode](#).

The documentation for this class was generated from the following files:

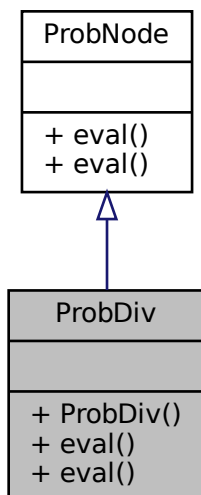
- [expressions.hpp](#)
- [expressions.cc](#)

5.48 ProbDiv Class Reference

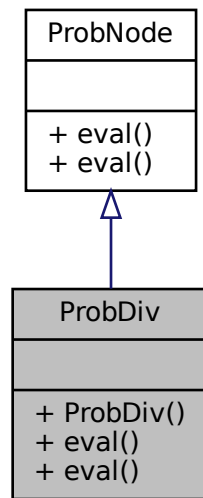
Node for division.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbDiv:



Collaboration diagram for ProbDiv:



Public Member Functions

- [ProbDiv](#) ([ProbNode](#) *_larg, [ProbNode](#) *_rarg)
Division expression constructor.
- virtual float [eval](#) ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual float [eval](#) ()

5.48.1 Detailed Description

Node for division.

5.48.2 Constructor & Destructor Documentation

5.48.2.1 ProbDiv()

```

ProbDiv::ProbDiv (
    ProbNode * _larg,
    ProbNode * _rarg ) [inline]
  
```

Division expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.48.3 Member Function Documentation

5.48.3.1 `eval()` [1/2]

```
float ProbDiv::eval ( ) [virtual]
```

Implements [ProbNode](#).

5.48.3.2 `eval()` [2/2]

```
float ProbDiv::eval (
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<code>env</code>	Environment values.
------------------	---------------------

Returns

Returns a float.

Implements [ProbNode](#).

The documentation for this class was generated from the following files:

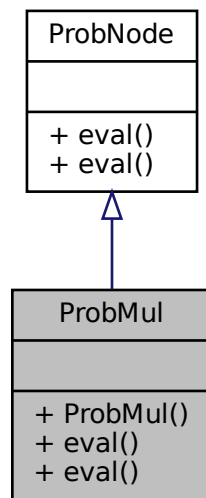
- [expressions.hpp](#)
- [expressions.cc](#)

5.49 ProbMul Class Reference

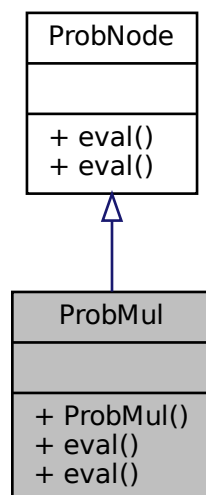
Node for multiplication.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbMul:



Collaboration diagram for ProbMul:



Public Member Functions

- [ProbMul](#) ([ProbNode](#) *_larg, [ProbNode](#) *_rarg)
Multiplication expression constructor.

- virtual float `eval` ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual float `eval` ()

5.49.1 Detailed Description

Node for multiplication.

5.49.2 Constructor & Destructor Documentation

5.49.2.1 ProbMul()

```
ProbMul::ProbMul (
    ProbNode * _larg,
    ProbNode * _rarg ) [inline]
```

Multiplication expression constructor.

Parameters

<code>_larg</code>	Left argument of the expression.
<code>_rarg</code>	Right argument of the expression.

5.49.3 Member Function Documentation

5.49.3.1 `eval()` [1/2]

```
float ProbMul::eval ( ) [virtual]
```

Implements [ProbNode](#).

5.49.3.2 `eval()` [2/2]

```
float ProbMul::eval (
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns a float.

Implements [ProbNode](#).

The documentation for this class was generated from the following files:

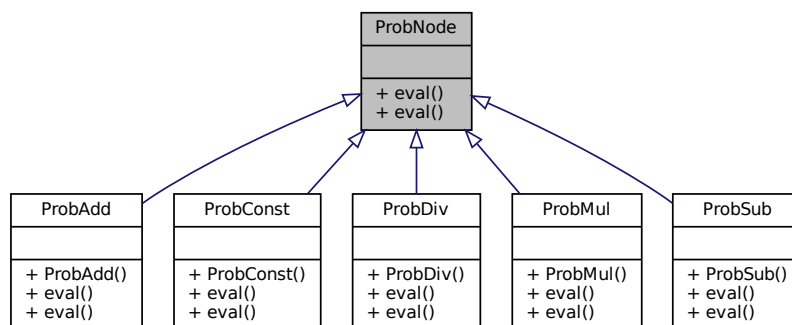
- [expressions.hpp](#)
- [expressions.cc](#)

5.50 ProbNode Class Reference

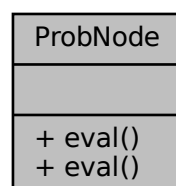
Base node for probability calculations.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbNode:



Collaboration diagram for ProbNode:



Public Member Functions

- virtual float [eval](#) ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)=0
Calculates the expression value.
- virtual float [eval](#) ()=0

5.50.1 Detailed Description

Base node for probability calculations.

5.50.2 Member Function Documentation

5.50.2.1 [eval\(\)](#) [1/2]

```
virtual float ProbNode::eval ( ) [pure virtual]
```

Implemented in [ProbDiv](#), [ProbMul](#), [ProbSub](#), [ProbAdd](#), and [ProbConst](#).

5.50.2.2 [eval\(\)](#) [2/2]

```
virtual float ProbNode::eval (  
    GlobalModelGenerator * generator,  
    GlobalState * globalState ) [pure virtual]
```

Calculates the expression value.

Returns

Returns a float.

Implemented in [ProbDiv](#), [ProbMul](#), [ProbSub](#), [ProbAdd](#), and [ProbConst](#).

The documentation for this class was generated from the following file:

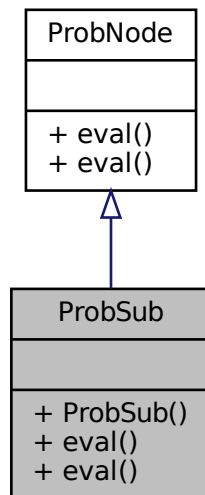
- [expressions.hpp](#)

5.51 ProbSub Class Reference

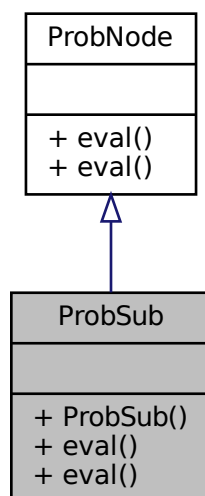
Node for subtraction.

```
#include <expressions.hpp>
```

Inheritance diagram for ProbSub:



Collaboration diagram for ProbSub:



Public Member Functions

- [ProbSub](#) ([ProbNode](#) * _larg, [ProbNode](#) * _rarg)
Subtraction expression constructor.
- virtual float [eval](#) ([GlobalModelGenerator](#) *generator, [GlobalState](#) *globalState)
Calculates the expression value.
- virtual float [eval](#) ()

5.51.1 Detailed Description

Node for subtraction.

5.51.2 Constructor & Destructor Documentation

5.51.2.1 ProbSub()

```
ProbSub::ProbSub (
    ProbNode * _larg,
    ProbNode * _rarg ) [inline]
```

Subtraction expression constructor.

Parameters

_larg	Left argument of the expression.
_rarg	Right argument of the expression.

5.51.3 Member Function Documentation

5.51.3.1 eval() [1/2]

```
float ProbSub::eval ( ) [virtual]
```

Implements [ProbNode](#).

5.51.3.2 eval() [2/2]

```
float ProbSub::eval (
    GlobalModelGenerator * generator,
    GlobalState * globalState ) [virtual]
```

Calculates the expression value.

Parameters

<i>env</i>	Environment values.
------------	---------------------

Returns

Returns a float.

Implements [ProbNode](#).

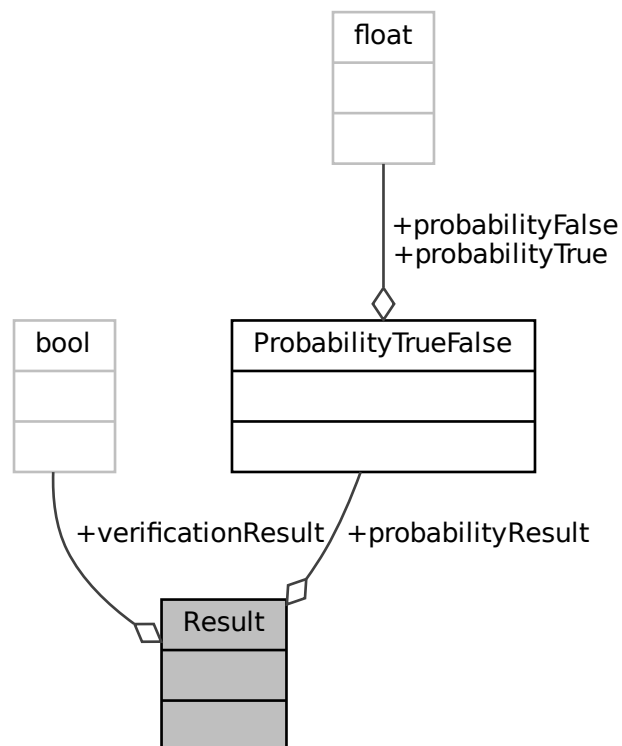
The documentation for this class was generated from the following files:

- [expressions.hpp](#)
- [expressions.cc](#)

5.52 Result Struct Reference

```
#include <Types.hpp>
```

Collaboration diagram for Result:



Public Attributes

- `GlobalState * globalState = nullptr`
- `StateVerificationInfo * fromState = nullptr`
- `queue< GlobalTransition * > controlledTransitionsLeftToProcess`
- `queue< GlobalTransition * > uncontrolledTransitionsLeftToProcess`
- `map< string, set< GlobalTransition * > > controlledProbabilisticTransitionsLeftToProcess`
- `map< string, set< GlobalTransition * > > uncontrolledProbabilisticTransitionsLeftToProcess`
- `int depth = 0`
- `bool processed = false`
- `VerifResult verifResult = VerifResult::NOT_VERIFIED`
- `bool controlled = false`
- `bool uncontrolled = false`
- `bool hasControlledProbabilistic = false`
- `bool hasUncontrolledProbabilistic = false`
- `bool hasValidControlledTransition = false`
- `bool hasValidUncontrolledTransition = true`
- `bool isControlledByCoalition = false`
- `bool gotThroughPreselectedTransition = false`
- `bool gotResponseFromOtherState = false`

5.53.1 Detailed Description

Handles all single state verification information during probabilistic verification.

5.53.2 Member Data Documentation

5.53.2.1 controlled

```
bool StateVerificationInfo::controlled = false
```

5.53.2.2 controlledProbabilisticTransitionsLeftToProcess

```
map<string, set<GlobalTransition*> > StateVerificationInfo::controlledProbabilisticTransitions←→
LeftToProcess
```

5.53.2.3 controlledTransitionsLeftToProcess

```
queue<GlobalTransition*> StateVerificationInfo::controlledTransitionsLeftToProcess
```

5.53.2.4 depth

```
int StateVerificationInfo::depth = 0
```

5.53.2.5 fromState

```
StateVerificationInfo* StateVerificationInfo::fromState = nullptr
```

5.53.2.6 globalState

```
GlobalState* StateVerificationInfo::globalState = nullptr
```

5.53.2.7 gotResponseFromOtherState

```
bool StateVerificationInfo::gotResponseFromOtherState = false
```

5.53.2.8 gotThroughPreselectedTransition

```
bool StateVerificationInfo::gotThroughPreselectedTransition = false
```

5.53.2.9 hasControlledProbabilistic

```
bool StateVerificationInfo::hasControlledProbabilistic = false
```

5.53.2.10 hasUncontrolledProbabilistic

```
bool StateVerificationInfo::hasUncontrolledProbabilistic = false
```

5.53.2.11 hasValidControlledTransition

```
bool StateVerificationInfo::hasValidControlledTransition = false
```

5.53.2.12 hasValidUncontrolledTransition

```
bool StateVerificationInfo::hasValidUncontrolledTransition = true
```

5.53.2.13 isControlledByCoalition

```
bool StateVerificationInfo::isControlledByCoalition = false
```

5.53.2.14 processed

```
bool StateVerificationInfo::processed = false
```

5.53.2.15 uncontrolled

```
bool StateVerificationInfo::uncontrolled = false
```

5.53.2.16 uncontrolledProbabilisticTransitionsLeftToProcess

```
map<string, set<GlobalTransition*> > StateVerificationInfo::uncontrolledProbabilisticTransitionsLeftToProcess
```

5.53.2.17 uncontrolledTransitionsLeftToProcess

```
queue<GlobalTransition*> StateVerificationInfo::uncontrolledTransitionsLeftToProcess
```

5.53.2.18 verifResult

```
VerifResult StateVerificationInfo::verifResult = VerifResult::NOT_VERIFIED
```

The documentation for this struct was generated from the following file:

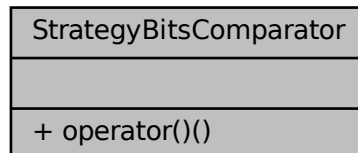
- [TypesDependency.hpp](#)

5.54 StrategyBitsComparator Struct Reference

A comparator for two bitsets containing values for the global states.

```
#include <TypesDependency.hpp>
```

Collaboration diagram for StrategyBitsComparator:



Public Member Functions

- bool [operator\(\)](#) (const bitset< [STRATEGY_BITS](#) > &b1, const bitset< [STRATEGY_BITS](#) > &b2) const

5.54.1 Detailed Description

A comparator for two bitsets containing values for the global states.

5.54.2 Member Function Documentation

5.54.2.1 operator()

```
bool StrategyBitsComparator::operator() (
    const bitset< STRATEGY\_BITS > & b1,
    const bitset< STRATEGY\_BITS > & b2 ) const [inline]
```

The documentation for this struct was generated from the following file:

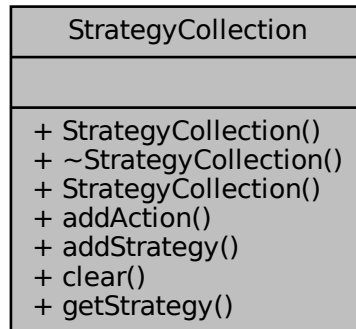
- [TypesDependency.hpp](#)

5.55 StrategyCollection Class Reference

Handles currently selected strategy when the strategy is set.

```
#include <TypesDependency.hpp>
```

Collaboration diagram for StrategyCollection:



Public Member Functions

- [StrategyCollection](#) ()
- [~StrategyCollection](#) ()
- [StrategyCollection](#) (string hash, [Action](#) action)
- void [addAction](#) ([Action](#) action)
- void [addStrategy](#) ([StrategyCollection](#) strategy)
- void [clear](#) ()
- map< string, [Action](#) > [getStrategy](#) ()

5.55.1 Detailed Description

Handles currently selected strategy when the strategy is set.

5.55.2 Constructor & Destructor Documentation

5.55.2.1 StrategyCollection() [1/2]

```
StrategyCollection::StrategyCollection ( ) [inline]
```

5.55.2.2 ~StrategyCollection()

```
StrategyCollection::~~StrategyCollection ( ) [inline]
```

5.55.2.3 StrategyCollection() [2/2]

```
StrategyCollection::StrategyCollection (
    string hash,
    Action action ) [inline]
```

5.55.3 Member Function Documentation

5.55.3.1 addAction()

```
void StrategyCollection::addAction (
    Action action ) [inline]
```

5.55.3.2 addStrategy()

```
void StrategyCollection::addStrategy (
    StrategyCollection strategy ) [inline]
```

5.55.3.3 clear()

```
void StrategyCollection::clear ( ) [inline]
```

5.55.3.4 getStrategy()

```
map<string, Action> StrategyCollection::getStrategy ( ) [inline]
```

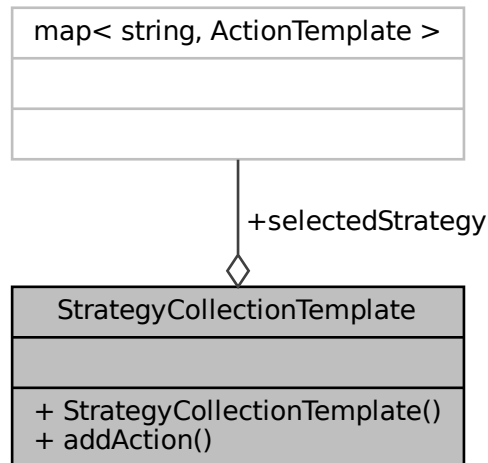
The documentation for this class was generated from the following file:

- [TypesDependency.hpp](#)

5.56 StrategyCollectionTemplate Class Reference

```
#include <strategyNodes.hpp>
```

Collaboration diagram for StrategyCollectionTemplate:



Public Member Functions

- [StrategyCollectionTemplate](#) ()
- void [addAction](#) (vector< string > *_states, string _actionName)

Public Attributes

- map< string, [ActionTemplate](#) > [selectedStrategy](#)

5.56.1 Constructor & Destructor Documentation

5.56.1.1 StrategyCollectionTemplate()

```
StrategyCollectionTemplate::StrategyCollectionTemplate ( )
```

5.56.2 Member Function Documentation

5.56.2.1 addAction()

```
void StrategyCollectionTemplate::addAction (
    vector< string > *_states,
    string _actionName )
```

Parameters

<i>newAction</i>	
------------------	--

5.56.3 Member Data Documentation

5.56.3.1 selectedStrategy

```
map<string, ActionTemplate> StrategyCollectionTemplate::selectedStrategy
```

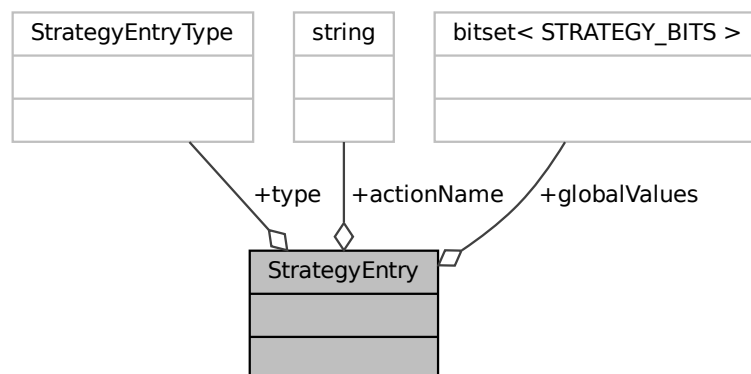
The documentation for this class was generated from the following files:

- [strategyNodes.hpp](#)
- [strategyNodes.cc](#)

5.57 StrategyEntry Struct Reference

```
#include <TypesDependency.hpp>
```

Collaboration diagram for StrategyEntry:



Public Attributes

- [StrategyEntryType](#) type = NOT_MODIFIED
- [bitset< STRATEGY_BITS >](#) globalValues = 0
- string * [actionName](#)

5.57.1 Member Data Documentation

5.57.1.1 actionName

```
string* StrategyEntry::actionName
```

5.57.1.2 globalValues

```
bitset<STRATEGY_BITS> StrategyEntry::globalValues = 0
```

5.57.1.3 type

```
StrategyEntryType StrategyEntry::type = NOT_MODIFIED
```

The documentation for this struct was generated from the following file:

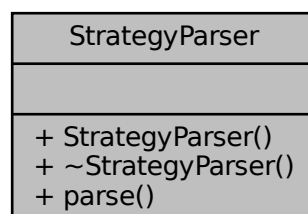
- [TypesDependency.hpp](#)

5.58 StrategyParser Class Reference

A parser for converting a text file into a strategy.

```
#include <StrategyParser.hpp>
```

Collaboration diagram for StrategyParser:



Public Member Functions

- [StrategyParser](#) ()
StrategyParser constructor.
- [~StrategyParser](#) ()
ModelParser destructor.
- [StrategyCollection](#) * [parse](#) (string fileName)
Parses a file with given name into a usable strategy.

5.58.1 Detailed Description

A parser for converting a text file into a strategy.

5.58.2 Constructor & Destructor Documentation

5.58.2.1 StrategyParser()

```
StrategyParser::StrategyParser ( )
```

[StrategyParser](#) constructor.

5.58.2.2 ~StrategyParser()

```
StrategyParser::~~StrategyParser ( )
```

[ModelParser](#) destructor.

5.58.3 Member Function Documentation

5.58.3.1 parse()

```
StrategyCollection * StrategyParser::parse (
    string fileName )
```

Parses a file with given name into a usable strategy.

Parameters

<i>fileName</i>	Name of the file to be converted into a strategy.
-----------------	---

Returns

Pointer to a model created from a given file.

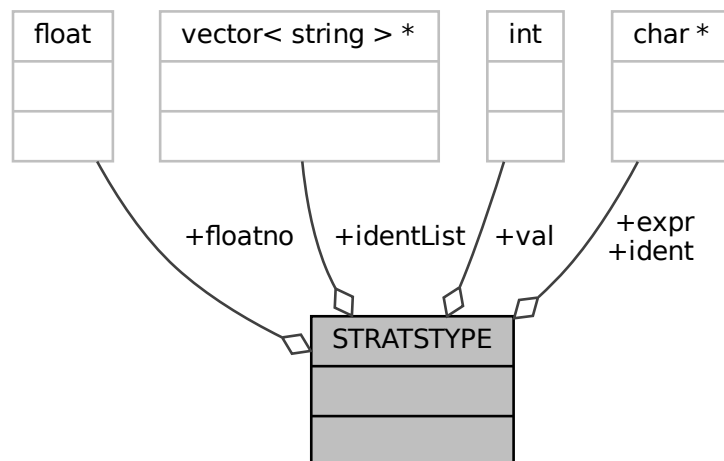
The documentation for this class was generated from the following files:

- [StrategyParser.hpp](#)
- [StrategyParser.cc](#)

5.59 STRATSTYPE Union Reference

```
#include <strategyParser.h>
```

Collaboration diagram for STRATSTYPE:

**Public Attributes**

- char * [expr](#)
- int [val](#)
- float [floatno](#)
- char * [ident](#)
- vector< string > * [identList](#)

5.59.1 Member Data Documentation

5.59.1.1 expr

```
char* STRATSTYPE::expr
```

5.59.1.2 floatno

```
float STRATSTYPE::floatno
```

5.59.1.3 ident

```
char* STRATSTYPE::ident
```

5.59.1.4 identList

```
vector<string>* STRATSTYPE::identList
```

5.59.1.5 val

```
int STRATSTYPE::val
```

The documentation for this union was generated from the following file:

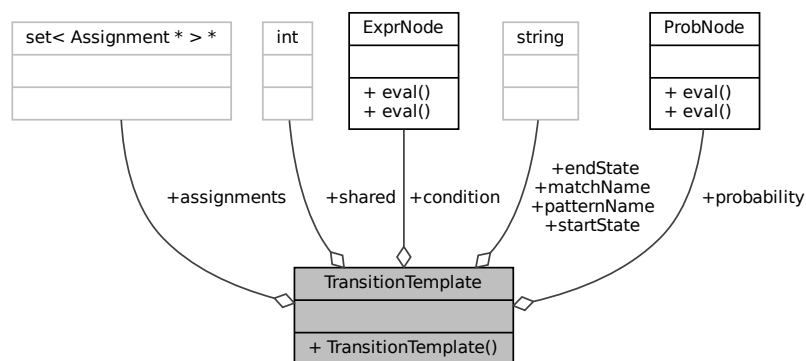
- [strategyParser.h](#)

5.60 TransitionTemplate Class Reference

Represents a meta-transition.

```
#include <nodes.hpp>
```

Collaboration diagram for TransitionTemplate:



Public Member Functions

- [TransitionTemplate](#) (int _shared, string _patternName, string _matchName, string _startState, string _endState, [ExprNode](#) *_cond, set< [Assignment](#) * > *_assign, [ProbNode](#) *_prob)
TransitionTemplate constructor.

Public Attributes

- int [shared](#)
Needed amount of needed agents. -1 if not shared.
- string [patternName](#)
Name of the pattern.
- string [matchName](#)
Global name for shared transitions.
- string [startState](#)
Start state name.
- string [endState](#)
End state name.
- [ExprNode](#) * [condition](#)
Condition expression that has do be fulfilled in that transition.
- set< [Assignment](#) * > * [assignments](#)
Set of assignments.
- [ProbNode](#) * [probability](#)
Probability of executing the action.

5.60.1 Detailed Description

Represents a meta-transition.

5.60.2 Constructor & Destructor Documentation

5.60.2.1 TransitionTemplate()

```
TransitionTemplate::TransitionTemplate (
    int _shared,
    string _patternName,
    string _matchName,
    string _startState,
    string _endState,
    ExprNode * _cond,
    set< Assignment * > * _assign,
    ProbNode * _prob ) [inline]
```

[TransitionTemplate](#) constructor.

Parameters

<i>_shared</i>	Needed amount of needed agents. -1 if not shared.
<i>_patternName</i>	Name of the pattern.
<i>_matchName</i>	Global name for shared transitions.
<i>_startState</i>	Start state name.
<i>_endState</i>	End state name.
<i>_cond</i>	Condition expression that has do be fulfilled in that transition.
<i>_assign</i>	Set of assignments.
<i>_prob</i>	Probability of executing the action.

5.60.3 Member Data Documentation

5.60.3.1 assignments

```
set<Assignment*>* TransitionTemplate::assignments
```

Set of assignments.

5.60.3.2 condition

```
ExprNode* TransitionTemplate::condition
```

[Condition](#) expression that has do be fulfilled in that transition.

5.60.3.3 endState

```
string TransitionTemplate::endState
```

End state name.

5.60.3.4 matchName

```
string TransitionTemplate::matchName
```

Global name for shared transitions.

5.60.3.5 patternName

```
string TransitionTemplate::patternName
```

Name of the pattern.

5.60.3.6 probability

```
ProbNode* TransitionTemplate::probability
```

Probability of executing the action.

5.60.3.7 shared

```
int TransitionTemplate::shared
```

Needed amount of needed agents. -1 if not shared.

5.60.3.8 startState

```
string TransitionTemplate::startState
```

Start state name.

The documentation for this class was generated from the following file:

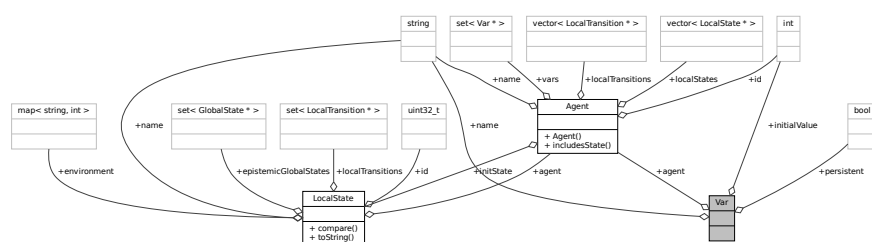
- [nodes.hpp](#)

5.61 Var Struct Reference

Represents a variable in the model, containing name, initial value and persistence.

```
#include <Types.hpp>
```

Collaboration diagram for Var:



Public Attributes

- string `name`
Variable name.
- int `initialValue`
Initial value of the variable.
- bool `persistent`
True if variable is persistent, i.e. it should appear in all states in the model, false otherwise.
- `Agent * agent`
Reference to an agent, to which this variable belongs to.

5.61.1 Detailed Description

Represents a variable in the model, containing name, initial value and persistence.

5.61.2 Member Data Documentation

5.61.2.1 `agent`

```
Agent* Var::agent
```

Reference to an agent, to which this variable belongs to.

5.61.2.2 `initialValue`

```
int Var::initialValue
```

Initial value of the variable.

5.61.2.3 `name`

```
string Var::name
```

Variable name.

5.61.2.4 persistent

```
bool Var::persistent
```

True if variable is persistent, i.e. it should appear in all states in the model, false otherwise.

The documentation for this struct was generated from the following file:

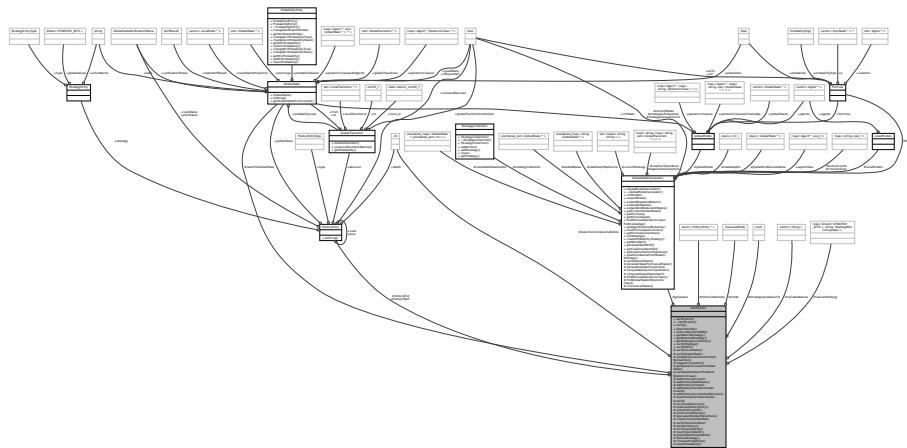
- [Types.hpp](#)

5.62 Verification Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

```
#include <Verification.hpp>
```

Collaboration diagram for Verification:



Public Member Functions

- [Verification](#) ([GlobalModelGenerator](#) *generator)
Constructor for [Verification](#).
- [~Verification](#) ()
Destructor for [Verification](#).
- bool [verify](#) ()
Starts the process of formula verification on a model.
- bool [fixpointVerify](#) ()
- void [historyDecisionsERR](#) ()
Prints out the first ERR path.
- map< bitset< [STRATEGY_BITS](#) >, string, [StrategyBitsComparator](#) > [getNaturalStrategy](#) ()
Get natural strategy map.
- vector< tuple< vector< tuple< bool, string > >, string > > [getReducedStrategy](#) ()
Takes natural strategy from memory and reduces it using [reduceStrategy\(\)](#).
- int [getStrategyComplexity](#) ()
Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- [Result](#) [verifyStrategy](#) ()
Verifies a strategy with the previously given models and formula.
- [Result](#) [verifyMDP](#) ()

Protected Member Functions

- bool [verifyLocalStates](#) (vector< [LocalState](#) * > *localStates, [GlobalState](#) *globalState)
Verifies a set of [LocalState](#) that a [GlobalState](#) is composed of with a hardcoded formula.
- bool [verifyGlobalState](#) ([GlobalState](#) *globalState, int depth)
Recursively verifies [GlobalState](#).
- bool [isGlobalTransitionControlledByCoalition](#) ([GlobalTransition](#) *globalTransition)
Checks if any of the [LocalTransition](#) in a given [GlobalTransition](#) has an [Agent](#) in a coalition in the formula.
- bool [isAgentInCoalition](#) ([Agent](#) *agent)
Checks if the [Agent](#) is in a coalition based on the formula in a [GlobalModelGenerator](#).
- [EpistemicClass](#) * [getEpistemicClassForGlobalState](#) ([GlobalState](#) *globalState)
Gets the [EpistemicClass](#) for the agent in passed [GlobalState](#), i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool [areGlobalStatesInTheSameEpistemicClass](#) ([GlobalState](#) *globalState1, [GlobalState](#) *globalState2)
Compares two [GlobalState](#) and checks if their [EpistemicClass](#) is the same.
- void [addHistoryDecision](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [DECISION](#) and puts it on top of the stack of the decision history.
- void [addHistoryStateStatus](#) ([GlobalState](#) *globalState, [GlobalStateVerificationStatus](#) prevStatus, [GlobalStateVerificationStatus](#) newStatus)
Creates a [HistoryEntry](#) of the type [STATE_STATUS](#) and puts it to the top of the decision history.
- bool [addHistoryContext](#) ([GlobalState](#) *globalState, int depth, [GlobalTransition](#) *decision, bool global↔
TransitionControlled)
Creates a [HistoryEntry](#) of the type [CONTEXT](#) and puts it to the top of the decision history.
- void [addHistoryMarkDecisionAsInvalid](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [MARK_DECISION_AS_INVALID](#) and puts it to the top of the decision history.
- void [addHistoryUncontrolledDecision](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [UNCONTROLLED_DECISION](#) and puts it on top of the stack of the decision history.
- [HistoryEntry](#) * [newHistoryMarkDecisionAsInvalid](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [MARK_DECISION_AS_INVALID](#) and returns it.
- bool [revertLastDecision](#) (int depth)
Reverts [GlobalState](#) and history to the previous decision state.
- void [undoLastHistoryEntry](#) (bool freeMemory)
Removes the top entry of the history stack.
- void [undoHistoryUntil](#) ([HistoryEntry](#) *historyEntry, bool inclusive, int depth)
Rolls back the history entries up to the certain [HistoryEntry](#).
- void [printCurrentHistory](#) (int depth)
Prints current history to the console.
- bool [equivalentGlobalTransitions](#) ([GlobalTransition](#) *globalTransition1, [GlobalTransition](#) *globalTransition2)
Checks if two global transitions are made up of the same local transitions.
- bool [checkUncontrolledSet](#) (set< [GlobalTransition](#) * > uncontrolledGlobalTransitions, [GlobalState](#) *global↔
State, int depth, bool hasOmittedTransitions, bool mixed=false)
Verifies if each transition from a given state yields a correct result.
- bool [verifyTransitionSets](#) (set< [GlobalTransition](#) * > controlledGlobalTransitions, set< [GlobalTransition](#) *
> uncontrolledGlobalTransitions, [GlobalState](#) *globalState, int depth, bool hasOmittedTransitions, bool is↔
FMode, bool mixed=false)
Checks if given transition sets are able to fulfill the formula for its given epistemic class.
- bool [restoreHistory](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *globalTransition, int depth, bool controlled)
Restores the decisions made for a given global state and transition in current recursion depth.
- bool [minFixpointVerify](#) ()
- bool [maxFixpointVerify](#) ()
- bitset< [STRATEGY_BITS](#) > [globalStateToValueBits](#) ([GlobalState](#) *globalState)

Changes globalState variables to a bitset used in natural strategy synthesis.

- `vector< tuple< vector< tuple< bool, string > >, string > > reduceStrategy (vector< tuple< vector< tuple< bool, string > >, string > > strategyEntries, short lockedColumn=0, bool upperHalf=false)`

Natural strategy reduction algorithm.

- `void increaseProbability (GlobalState *currentState, GlobalTransition *decision)`

Increases next GlobalState probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.

- `void lowerProbability (GlobalState *currentStateFrom, GlobalState *currentStateTo, set< LocalTransition * > decision)`

Decreases previous GlobalState probability reached from the decision by subtracting from it a multiplied currentState probability with the decision probability.

Protected Attributes

- `TraversalMode mode`

Current mode of model traversal.

- `GlobalState * revertToGlobalState`

Global state to which revert will rollback to.

- `stack< HistoryEntry * > historyToRestore`

A history of decisions to be rolled back.

- `GlobalModelGenerator * generator`

Holds current model and formula.

- `HistoryEntry * historyStart`

Pointer to the start of model traversal history.

- `HistoryEntry * historyEnd`

Pointer to the end of model traversal history.

- `map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > naturalStrategy`

A table of actions paired with globalState internal variables state for natural strategy construction.

- `short strategyVariableLimit`

Max strategy variables found.

- `vector< string > variableNames`

Easily readable variable names for natural strategy generation.

- `int reductionComplexityBefore`

Natural strategy complexity before reduction.

5.62.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.62.2 Constructor & Destructor Documentation

5.62.2.1 Verification()

```
Verification::Verification (
    GlobalModelGenerator * generator )
```

Constructor for [Verification](#).

Parameters

<i>generator</i>	Pointer to GlobalModelGenerator
------------------	---

5.62.2.2 ~Verification()

`Verification::~~Verification ( )`

Destructor for [Verification](#).

5.62.3 Member Function Documentation**5.62.3.1 addHistoryContext()**

```
bool Verification::addHistoryContext (
    GlobalState * globalState,
    int depth,
    GlobalTransition * decision,
    bool globalTransitionControlled ) [protected]
```

Creates a [HistoryEntry](#) of the type CONTEXT and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>depth</i>	Depth of the recursion of the validation algorithm.
<i>decision</i>	Pointer to a transition GlobalTransition selected by the algorithm.
<i>globalTransitionControlled</i>	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.62.3.2 addHistoryDecision()

```
void Verification::addHistoryDecision (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```

Creates a [HistoryEntry](#) of the type DECISION and puts it on top of the stack of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.62.3.3 addHistoryMarkDecisionAsInvalid()

```
void Verification::addHistoryMarkDecisionAsInvalid (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```

Creates a [HistoryEntry](#) of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a transition GlobalTransition selected by the algorithm.

5.62.3.4 addHistoryStateStatus()

```
void Verification::addHistoryStateStatus (
    GlobalState * globalState,
    GlobalStateVerificationStatus prevStatus,
    GlobalStateVerificationStatus newStatus ) [protected]
```

Creates a [HistoryEntry](#) of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>prevStatus</i>	Previous GlobalStateVerificationStatus to be logged.
<i>newStatus</i>	New GlobalStateVerificationStatus to be logged.

5.62.3.5 addHistoryUncontrolledDecision()

```
void Verification::addHistoryUncontrolledDecision (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```

Creates a [HistoryEntry](#) of the type UNCONTROLLED_DECISION and puts it on top of the stack of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.62.3.6 areGlobalStatesInTheSameEpistemicClass()

```
bool Verification::areGlobalStatesInTheSameEpistemicClass (
    GlobalState * globalState1,
    GlobalState * globalState2 ) [protected]
```

Compares two [GlobalState](#) and checks if their [EpistemicClass](#) is the same.

Parameters

<i>globalState1</i>	Pointer to the first GlobalState .
<i>globalState2</i>	Pointer to the second GlobalState .

Returns

Returns true if the [EpistemicClass](#) is the same for both of the [GlobalState](#). Returns false if they are different or at least one of them has no [EpistemicClass](#).

5.62.3.7 checkUncontrolledSet()

```
bool Verification::checkUncontrolledSet (
    set< GlobalTransition * > uncontrolledGlobalTransitions,
    GlobalState * globalState,
    int depth,
    bool hasOmittedTransitions,
    bool mixed = false ) [protected]
```

Verifies if each transition from a given state yields a correct result.

Parameters

<i>uncontrolledGlobalTransitions</i>	A set of global transitions to be checked.
<i>globalState</i>	Currently processed global state.
<i>depth</i>	Current recursion depth.
<i>hasOmittedTransitions</i>	Flag with the information about skipped unneeded transitions.

Returns

Returns true if every transition yields a correct result, false otherwise.

5.62.3.8 equivalentGlobalTransitions()

```
bool Verification::equivalentGlobalTransitions (
    GlobalTransition * globalTransition1,
    GlobalTransition * globalTransition2 ) [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

<i>globalTransition1</i>	First global transition to compare.
<i>globalTransition2</i>	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.62.3.9 fixpointVerify()

```
bool Verification::fixpointVerify ( )
```

5.62.3.10 getEpistemicClassForGlobalState()

```
EpistemicClass * Verification::getEpistemicClassForGlobalState (
    GlobalState * globalState ) [protected]
```

Gets the [EpistemicClass](#) for the agent in passed [GlobalState](#), i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
--------------------	--

Returns

Pointer to the [EpistemicClass](#) that a coalition of agents from the formula belong to. If there is no such [EpistemicClass](#), returns false.

5.62.3.11 `getNaturalStrategy()`

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > Verification::getNaturalStrategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.62.3.12 `getReducedStrategy()`

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::getReducedStrategy ( )
```

Takes natural strategy from memory and reduces it using `reduceStrategy()`.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.62.3.13 `getStrategyComplexity()`

```
int Verification::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.62.3.14 `globalStateToValueBits()`

```
bitset< STRATEGY_BITS > Verification::globalStateToValueBits (
    GlobalState * globalState ) [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

<i>globalState</i>	State that we want to extract variable values from.
--------------------	---

Returns

Bitset of environment variable values.

5.62.3.15 historyDecisionsERR()

```
void Verification::historyDecisionsERR ( )
```

Prints out the first ERR path.

5.62.3.16 increaseProbability()

```
void Verification::increaseProbability (
    GlobalState * currentState,
    GlobalTransition * decision ) [protected]
```

Increases next [GlobalState](#) probability reached through the decision by adding to it a multiplied currentState probability with the decision probability.

Parameters

<i>currentState</i>	From which GlobalState the action was executed.
<i>decision</i>	GlobalTransition containing all transitions in the executed action and the next reachable GlobalState .

5.62.3.17 isAgentInCoalition()

```
bool Verification::isAgentInCoalition (
    Agent * agent ) [protected]
```

Checks if the [Agent](#) is in a coalition based on the formula in a [GlobalModelGenerator](#).

Parameters

<i>agent</i>	Pointer to an Agent that is to be checked.
--------------	--

Returns

Returns true if the [Agent](#) is in a coalition, otherwise returns false.

5.62.3.18 isGlobalTransitionControlledByCoalition()

```
bool Verification::isGlobalTransitionControlledByCoalition (
    GlobalTransition * globalTransition ) [protected]
```

Checks if any of the [LocalTransition](#) in a given [GlobalTransition](#) has an [Agent](#) in a coalition in the formula.

Parameters

<i>globalTransition</i>	Pointer to a GlobalTransition in a model.
-------------------------	---

Returns

Returns true if the [Agent](#) is in coalition in the formula, otherwise returns false.

5.62.3.19 lowerProbability()

```
void Verification::lowerProbability (
    GlobalState * currentStateFrom,
    GlobalState * currentStateTo,
    set< LocalTransition * > decision ) [protected]
```

Decreases previous [GlobalState](#) probability reached from the decision by subtracting from it a multiplied current↔ State probability with the decision probability.

Parameters

<i>currentStateFrom</i>	From which GlobalState the action was executed.
<i>currentStateTo</i>	To which global state the action was executed.
<i>decision</i>	Set of LocalTransition containing all transitions in the executed action.

5.62.3.20 maxFixpointVerify()

```
bool Verification::maxFixpointVerify ( ) [protected]
```

5.62.3.21 minFixpointVerify()

```
bool Verification::minFixpointVerify ( ) [protected]
```

5.62.3.22 newHistoryMarkDecisionAsInvalid()

```
HistoryEntry * Verification::newHistoryMarkDecisionAsInvalid (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```

Creates a [HistoryEntry](#) of the type MARK_DECISION_AS_INVALID and returns it.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a transition GlobalTransition selected by the algorithm.

Returns

Returns pointer to a new [HistoryEntry](#).

5.62.3.23 printCurrentHistory()

```
void Verification::printCurrentHistory (
    int depth ) [protected]
```

Prints current history to the console.

Parameters

<i>depth</i>	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.
--------------	--

5.62.3.24 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > Verification::reduceStrategy (
    vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries,
    short lockedColumn = 0,
    bool upperHalf = false ) [protected]
```

Natural strategy reduction algorithm.

Parameters

<i>strategyEntries</i>	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
<i>lockedColumn</i>	Index of the furthest leftmost locked column.
<i>upperHalf</i>	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.62.3.25 restoreHistory()

```
bool Verification::restoreHistory (
    GlobalState * globalState,
    GlobalTransition * globalTransition,
    int depth,
    bool controlled ) [protected]
```

Restores the decisions made for a given global state and transition in current recursion depth.

Parameters

<i>globalState</i>	Currently processed global state.
<i>globalTransition</i>	Previously selected global transition from the given state to mark as invalid.
<i>depth</i>	Current recursion depth.
<i>controlled</i>	Flag with the information about current type of transition. True if controlled, false if uncontrolled.

Returns

Returns true if current top of the history entires matches with

5.62.3.26 revertLastDecision()

```
bool Verification::revertLastDecision (
    int depth ) [protected]
```

Reverts [GlobalState](#) and history to the previous decision state.

Parameters

<i>depth</i>	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.
--------------	--

Returns

Returns true if rollback is successful, otherwise returns false.

5.62.3.27 undoHistoryUntil()

```
void Verification::undoHistoryUntil (
    HistoryEntry * historyEntry,
    bool inclusive,
    int depth ) [protected]
```

Rolls back the history entries up to the certain [HistoryEntry](#).

Parameters

<i>historyEntry</i>	Pointer to a HistoryEntry that the history has to be rolled back to.
<i>inclusive</i>	True if the rollback has to remove the specified entry too.
<i>depth</i>	Integer that will be multiplied by 4 and appended as a prefix to the optional debug log.

5.62.3.28 undoLastHistoryEntry()

```
void Verification::undoLastHistoryEntry (
    bool freeMemory ) [protected]
```

Removes the top entry of the history stack.

Parameters

<i>freeMemory</i>	True if the entry has to be removed from memory.
-------------------	--

5.62.3.29 verify()

```
bool Verification::verify ( )
```

Starts the process of formula verification on a model.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.62.3.30 verifyGlobalState()

```
bool Verification::verifyGlobalState (
    GlobalState * globalState,
    int depth ) [protected]
```

Recursively verifies [GlobalState](#).

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>depth</i>	Current depth of the recursion.

Returns

Returns true if the verification is PENDING or VERIFIED_OK, otherwise returns false.

5.62.3.31 verifyLocalStates()

```
bool Verification::verifyLocalStates (
    vector< LocalState * > * localStates,
    GlobalState * globalState ) [protected]
```

Verifies a set of [LocalState](#) that a [GlobalState](#) is composed of with a hardcoded formula.

Parameters

<i>localStates</i>	A pointer to a set of pointers to LocalState .
--------------------	--

Returns

Returns true if there is a [LocalState](#) with a specific set of values, fulfilling the criteria, otherwise returns false.

5.62.3.32 verifyMDP()

```
Result Verification::verifyMDP ( )
```

5.62.3.33 verifyStrategy()

```
Result Verification::verifyStrategy ( )
```

Verifies a strategy with the previously given models and formula.

Returns

[Result](#) containing data of the verification result.

5.62.3.34 verifyTransitionSets()

```
bool Verification::verifyTransitionSets (
    set< GlobalTransition * > controlledGlobalTransitions,
    set< GlobalTransition * > uncontrolledGlobalTransitions,
    GlobalState * globalState,
    int depth,
    bool hasOmittedTransitions,
    bool isFMode,
    bool mixedTransitions = false ) [protected]
```

Checks if given transition sets are able to fulfill the formula for its given epistemic class.

Parameters

<i>controlledGlobalTransitions</i>	Set of controlled transitions in the current global state.
<i>uncontrolledGlobalTransitions</i>	Set of uncontrolled transitions in the current global state.
<i>globalState</i>	Currently processed global state.
<i>depth</i>	Current recursion depth.
<i>hasOmittedTransitions</i>	Flag with the information about skipped unneeded transitions.

Returns

True if there is a correct choice for an agent to take, false otherwise.

5.62.4 Member Data Documentation

5.62.4.1 generator

```
GlobalModelGenerator* Verification::generator [protected]
```

Holds current model and formula.

5.62.4.2 historyEnd

```
HistoryEntry* Verification::historyEnd [protected]
```

Pointer to the end of model traversal history.

5.62.4.3 historyStart

```
HistoryEntry* Verification::historyStart [protected]
```

Pointer to the start of model traversal history.

5.62.4.4 historyToRestore

```
stack<HistoryEntry*> Verification::historyToRestore [protected]
```

A history of decisions to be rolled back.

5.62.4.5 mode

```
TraversalMode Verification::mode [protected]
```

Current mode of model traversal.

5.62.4.6 naturalStrategy

```
map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> Verification::naturalStrategy  
[protected]
```

A table of actions paired with globalState internal variables state for natural strategy construction.

5.62.4.7 reductionComplexityBefore

```
int Verification::reductionComplexityBefore [protected]
```

Natural strategy complexity before reduction.

5.62.4.8 revertToGlobalState

```
GlobalState* Verification::revertToGlobalState [protected]
```

Global state to which revert will rollback to.

5.62.4.9 strategyVariableLimit

```
short Verification::strategyVariableLimit [protected]
```

Max strategy variables found.

5.62.4.10 variableNames

```
vector<string> Verification::variableNames [protected]
```

Easily readable variable names for natural strategy generation.

The documentation for this class was generated from the following files:

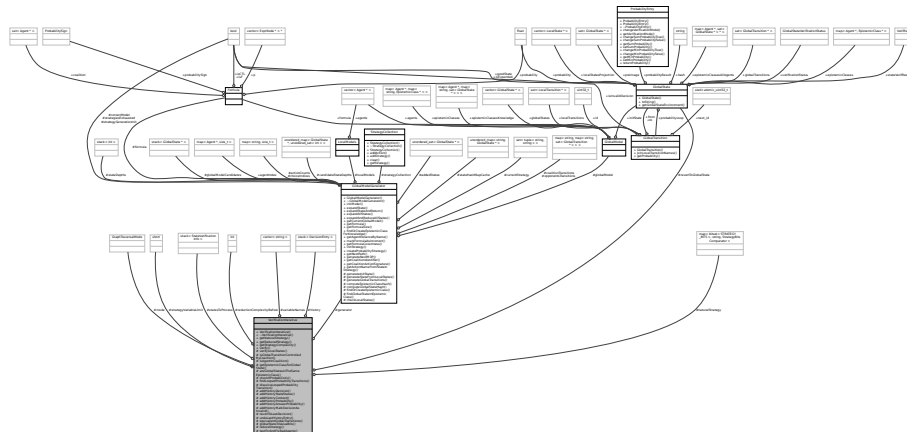
- [Verification.hpp](#)
- [Verification.cpp](#)

5.63 VerificationIterative Class Reference

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

```
#include <VerificationIterative.hpp>
```

Collaboration diagram for VerificationIterative:



Public Member Functions

- [VerificationIterative](#) ([GlobalModelGenerator](#) *generator)
Constructor for [Verification](#).
- [~VerificationIterative](#) ()
Destructor for [Verification](#).
- [map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator >](#) [getNaturalStrategy](#) ()
Get natural strategy map.
- [vector< tuple< vector< tuple< bool, string > >, string > >](#) [getReducedStrategy](#) ()
Takes natural strategy from memory and reduces it using [reduceStrategy\(\)](#).
- [int](#) [getStrategyComplexity](#) ()
Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.
- [Result](#) [verify](#) ()
Verifies a strategy with the previously given models and formula.

Protected Member Functions

- bool [verifyLocalStates](#) (vector< [LocalState](#) * > *localStates, [GlobalState](#) *globalState)
Verifies a set of [LocalState](#) that a [GlobalState](#) is composed of with a hardcoded formula.
- bool [isGlobalTransitionControlledByCoalition](#) ([GlobalTransition](#) *globalTransition)
Checks if any of the [LocalTransition](#) in a given [GlobalTransition](#) has an [Agent](#) in a coalition in the formula.
- bool [isAgentInCoalition](#) ([Agent](#) *agent)
Checks if the [Agent](#) is in a coalition based on the formula in a [GlobalModelGenerator](#).
- [EpistemicClass](#) * [getEpistemicClassForGlobalState](#) ([GlobalState](#) *globalState)
Gets the [EpistemicClass](#) for the agent in passed [GlobalState](#), i.e. transitions from indistinguishable state from certain other states for an agent to other states.
- bool [areGlobalStatesInTheSameEpistemicClass](#) ([GlobalState](#) *globalState1, [GlobalState](#) *globalState2)
Compares two [GlobalState](#) and checks if their [EpistemicClass](#) is the same.
- string [checkIfProbabilistic](#) ([GlobalTransition](#) *transitionToCheck)
Checks if a given transition is a probabilistic one.
- void [findLoopedProbabilityTransitions](#) (map< string, set< [GlobalTransition](#) * >> *transitionSets)
Looks for the probability transitions with loops and dissolves them.
- void [dissolveLoopedProbabilityTransition](#) ([GlobalTransition](#) *transitionToDissolve, set< [GlobalTransition](#) * > *probabilityTransitions)
Marks a loop as a transition to dissolve.
- void [addHistoryDecision](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [DECISION](#) and puts it on top of the stack of the decision history.
- void [addHistoryStateStatus](#) ([GlobalState](#) *globalState, [GlobalStateVerificationStatus](#) prevStatus, [GlobalStateVerificationStatus](#) newStatus)
Creates a [HistoryEntry](#) of the type [STATE_STATUS](#) and puts it to the top of the decision history.
- bool [addHistoryContext](#) ([GlobalState](#) *globalState, int depth, [GlobalTransition](#) *decision, bool global↔
TransitionControlled)
Creates a [HistoryEntry](#) of the type [CONTEXT](#) and puts it to the top of the decision history.
- void [addHistoryProbability](#) ([GlobalTransition](#) *decision)
- void [addHistoryAnswerProbability](#) ([StateVerificationInfo](#) *globalState, [ProbabilityCalculationType](#) current↔
Mode, [ProbabilityTrueFalse](#) newProbabilityValues)
Adds a history entry for the change of probabilistic answer in a particular state.
- void [addHistoryMarkDecisionAsInvalid](#) ([GlobalState](#) *globalState, [GlobalTransition](#) *decision)
Creates a [HistoryEntry](#) of the type [MARK_DECISION_AS_INVALID](#) and puts it to the top of the decision history.
- bool [revertToLastDecision](#) ()
Reverts the taken actions to the point of the last decision.
- void [undoLastHistoryEntry](#) ()
Removes the top entry of the history stack.
- bool [equivalentGlobalTransitions](#) ([GlobalTransition](#) *globalTransition1, [GlobalTransition](#) *globalTransition2)
Checks if two global transitions are made up of the same local transitions.
- bitset< [STRATEGY_BITS](#) > [globalStateToValueBits](#) ([GlobalState](#) *globalState)
Changes globalState variables to a bitset used in natural strategy synthesis.
- vector< tuple< vector< tuple< bool, string > >, string > > [reduceStrategy](#) (vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries, short lockedColumn=0, bool upperHalf=false)
Natural strategy reduction algorithm.
- bool [testForAndFixBadAgents](#) ([StateVerificationInfo](#) *stateVerificationInfo)
Fixes potentially blocking agent actions.

Protected Attributes

- [GraphTraversalMode mode](#) = [GraphTraversalMode::NORMAL_TRAVERSAL](#)
Current mode of model traversal.
- [GlobalState * revertToGlobalState](#)
Global state to which revert will rollback to.
- [GlobalModelGenerator * generator](#)
Holds current model and formula.
- `stack< DecisionEntry > history`
Stack of decisions made.
- `map< bitset< STRATEGY\_BITS >, string, StrategyBitsComparator > naturalStrategy`
A table of actions paired with globalState internal variables state for natural strategy construction.
- `short strategyVariableLimit`
Max strategy variables found.
- `vector< string > variableNames`
Easily readable variable names for natural strategy generation.
- `int reductionComplexityBefore`
Natural strategy complexity before reduction.
- `stack< StateVerificationInfo > statesToProcess`

5.63.1 Detailed Description

A class that verifies if the model fulfills the formula. Also can do some operations on decision history.

5.63.2 Constructor & Destructor Documentation

5.63.2.1 VerificationIterative()

```
VerificationIterative::VerificationIterative (
    GlobalModelGenerator * generator )
```

Constructor for [Verification](#).

Parameters

<i>generator</i>	Pointer to GlobalModelGenerator
------------------	---

5.63.2.2 ~VerificationIterative()

```
VerificationIterative::~~VerificationIterative ( )
```

Destructor for [Verification](#).

5.63.3 Member Function Documentation

5.63.3.1 addHistoryAnswerProbability()

```
void VerificationIterative::addHistoryAnswerProbability (
    StateVerificationInfo * stateVerifInfo,
    ProbabilityCalculationType currentMode,
    ProbabilityTrueFalse newProbabilityValues ) [protected]
```

Adds a history entry for the change of probabilistic answer in a particular state.

Parameters

<i>globalState</i>	
<i>currentMode</i>	SUM_PROBABILITY if controlled, MIN_PROBABILITY if uncontrolled.
<i>newProbabilityValues</i>	

5.63.3.2 addHistoryContext()

```
bool VerificationIterative::addHistoryContext (
    GlobalState * globalState,
    int depth,
    GlobalTransition * decision,
    bool globalTransitionControlled ) [protected]
```

Creates a [HistoryEntry](#) of the type CONTEXT and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>depth</i>	Depth of the recursion of the validation algorithm.
<i>decision</i>	Pointer to a transition GlobalTransition selected by the algorithm.
<i>globalTransitionControlled</i>	True if the GlobalTransition is in the set of global transitions controlled by a coalition and it is not a fixed global transition.

Returns

Returns true if the natural strategy is good for now.

5.63.3.3 addHistoryDecision()

```
void VerificationIterative::addHistoryDecision (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```


Creates a [HistoryEntry](#) of the type DECISION and puts it on top of the stack of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a GlobalTransition that is to be recorded in the decision history.

5.63.3.4 addHistoryMarkDecisionAsInvalid()

```
void VerificationIterative::addHistoryMarkDecisionAsInvalid (
    GlobalState * globalState,
    GlobalTransition * decision ) [protected]
```

Creates a [HistoryEntry](#) of the type MARK_DECISION_AS_INVALID and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>decision</i>	Pointer to a transition GlobalTransition selected by the algorithm.

5.63.3.5 addHistoryProbability()

```
void VerificationIterative::addHistoryProbability (
    GlobalTransition * decision ) [protected]
```

5.63.3.6 addHistoryStateStatus()

```
void VerificationIterative::addHistoryStateStatus (
    GlobalState * globalState,
    GlobalStateVerificationStatus prevStatus,
    GlobalStateVerificationStatus newStatus ) [protected]
```

Creates a [HistoryEntry](#) of the type STATE_STATUS and puts it to the top of the decision history.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
<i>prevStatus</i>	Previous GlobalStateVerificationStatus to be logged.
<i>newStatus</i>	New GlobalStateVerificationStatus to be logged.

5.63.3.7 areGlobalStatesInTheSameEpistemicClass()

```
bool VerificationIterative::areGlobalStatesInTheSameEpistemicClass (
    GlobalState * globalState1,
    GlobalState * globalState2 ) [protected]
```

Compares two [GlobalState](#) and checks if their [EpistemicClass](#) is the same.

Parameters

<i>globalState1</i>	Pointer to the first GlobalState .
<i>globalState2</i>	Pointer to the second GlobalState .

Returns

Returns true if the [EpistemicClass](#) is the same for both of the [GlobalState](#). Returns false if they are different or at least one of them has no [EpistemicClass](#).

5.63.3.8 checkIfProbabilistic()

```
string VerificationIterative::checkIfProbabilistic (
    GlobalTransition * transitionToCheck ) [protected]
```

Checks if a given transition is a probabilistic one.

Parameters

<i>transitionToCheck</i>	Transition to be checked if possesses probability != 1.
--------------------------	---

Returns

Returns concatenated names of local transitions separated with semicolons if true, returns empty string otherwise.

5.63.3.9 dissolveLoopedProbabilityTransition()

```
void VerificationIterative::dissolveLoopedProbabilityTransition (
    GlobalTransition * transitionToDissolve,
    set< GlobalTransition * > * probabilityTransitions ) [protected]
```

Marks a loop as a transition to dissolve.

Parameters

<i>transitionToDissolve</i>	GlobalTransition to ignore.
<i>probabilityTransitions</i>	Set of GlobalTransitions to remove the transition from (unused)

5.63.3.10 equivalentGlobalTransitions()

```
bool VerificationIterative::equivalentGlobalTransitions (
    GlobalTransition * globalTransition1,
    GlobalTransition * globalTransition2 ) [protected]
```

Checks if two global transitions are made up of the same local transitions.

Parameters

<i>globalTransition1</i>	First global transition to compare.
<i>globalTransition2</i>	Second global transition to compare.

Returns

True if the two global transitions have the same local transitions, false otherwise.

5.63.3.11 findLoopedProbabilityTransitions()

```
void VerificationIterative::findLoopedProbabilityTransitions (
    map< string, set< GlobalTransition * >> * transitionSets ) [protected]
```

Looks for the probability transitions with loops and dissolves them.

Parameters

<i>transitionSets</i>	Set of GlobalTransitions to check for the loops.
-----------------------	--

5.63.3.12 getEpistemicClassForGlobalState()

```
EpistemicClass * VerificationIterative::getEpistemicClassForGlobalState (
    GlobalState * globalState ) [protected]
```

Gets the [EpistemicClass](#) for the agent in passed [GlobalState](#), i.e. transitions from indistinguishable state from certain other states for an agent to other states.

Parameters

<i>globalState</i>	Pointer to a GlobalState of the model.
--------------------	--

Returns

Pointer to the [EpistemicClass](#) that a coalition of agents from the formula belong to. If there is no such [EpistemicClass](#), returns false.

5.63.3.13 getNaturalStrategy()

```
map< bitset< STRATEGY_BITS >, string, StrategyBitsComparator > VerificationIterative::get↵
NaturalStrategy ( )
```

Get natural strategy map.

Returns

Map of variable values and action names.

5.63.3.14 getReducedStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::get↵
ReducedStrategy ( )
```

Takes natural strategy from memory and reduces it using [reduceStrategy\(\)](#).

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.63.3.15 getStrategyComplexity()

```
int VerificationIterative::getStrategyComplexity ( )
```

Natural strategy complexity before reduction. Each variable, negation, and, or are treated as value 1.

Returns

Total sum of the natural strategy complexity before reduction.

5.63.3.16 globalStateToValueBits()

```
bitset< STRATEGY_BITS > VerificationIterative::globalStateToValueBits (
    GlobalState * globalState ) [protected]
```

Changes globalState variables to a bitset used in natural strategy synthesis.

Parameters

<i>globalState</i>	State that we want to extract variable values from.
--------------------	---

Returns

Bitset of environment variable values.

5.63.3.17 isAgentInCoalition()

```
bool VerificationIterative::isAgentInCoalition (
    Agent * agent ) [protected]
```

Checks if the [Agent](#) is in a coalition based on the formula in a [GlobalModelGenerator](#).

Parameters

<i>agent</i>	Pointer to an Agent that is to be checked.
--------------	--

Returns

Returns true if the [Agent](#) is in a coalition, otherwise returns false.

5.63.3.18 isGlobalTransitionControlledByCoalition()

```
bool VerificationIterative::isGlobalTransitionControlledByCoalition (
    GlobalTransition * globalTransition ) [protected]
```

Checks if any of the [LocalTransition](#) in a given [GlobalTransition](#) has an [Agent](#) in a coalition in the formula.

Parameters

<i>globalTransition</i>	Pointer to a GlobalTransition in a model.
-------------------------	---

Returns

Returns true if the [Agent](#) is in coalition in the formula, otherwise returns false.

5.63.3.19 reduceStrategy()

```
vector< tuple< vector< tuple< bool, string > >, string > > VerificationIterative::reduce↵
Strategy (
```

```
vector< tuple< vector< tuple< bool, string >>, string >> strategyEntries,
short lockedColumn = 0,
bool upperHalf = false ) [protected]
```

Natural strategy reduction algorithm.

Parameters

<i>strategyEntries</i>	Vector of tuples <vector of tuples <variable value, variable name>, action name> containing not reduced natural strategy.
<i>lockedColumn</i>	Index of the furthest leftmost locked column.
<i>upperHalf</i>	Indicates if the first columns of the processed strategy entries can't be shuffled. True if they can't be shuffled, false otherwise.

Returns

Vector of tuples <vector of tuples <variable value, variable name>, action name>.

5.63.3.20 revertToLastDecision()

```
bool VerificationIterative::revertToLastDecision ( ) [protected]
```

Reverts the taken actions to the point of the last decision.

Returns

True if reverted correctly, false otherwise.

5.63.3.21 testForAndFixBadAgents()

```
bool VerificationIterative::testForAndFixBadAgents (
    StateVerificationInfo * stateVerificationInfo ) [protected]
```

Fixes potentially blocking agent actions.

Parameters

<i>stateVerificationInfo</i>	Information for the state verification.
------------------------------	---

Returns

True if changed transitions into uncontrolled, false otherwise.

5.63.3.22 undoLastHistoryEntry()

```
void VerificationIterative::undoLastHistoryEntry ( ) [protected]
```

Removes the top entry of the history stack.

5.63.3.23 verify()

```
Result VerificationIterative::verify ( )
```

Verifies a strategy with the previously given models and formula.

Returns

[Result](#) containing data of the verification result.

5.63.3.24 verifyLocalStates()

```
bool VerificationIterative::verifyLocalStates (
    vector< LocalState * > * localStates,
    GlobalState * globalState ) [protected]
```

Verifies a set of [LocalState](#) that a [GlobalState](#) is composed of with a hardcoded formula.

Parameters

<i>localStates</i>	A pointer to a set of pointers to LocalState .
--------------------	--

Returns

Returns true if there is a [LocalState](#) with a specific set of values, fulfilling the criteria, otherwise returns false.

5.63.4 Member Data Documentation**5.63.4.1 generator**

```
GlobalModelGenerator* VerificationIterative::generator [protected]
```

Holds current model and formula.

5.63.4.2 history

```
stack<DecisionEntry> VerificationIterative::history [protected]
```

Stack of decisions made.

5.63.4.3 mode

```
GraphTraversalMode VerificationIterative::mode = GraphTraversalMode::NORMAL_TRAVERSAL [protected]
```

Current mode of model traversal.

5.63.4.4 naturalStrategy

```
map<bitset<STRATEGY_BITS>, string, StrategyBitsComparator> VerificationIterative::natural↔  
Strategy [protected]
```

A table of actions paired with globalState internal variables state for natural strategy construction.

5.63.4.5 reductionComplexityBefore

```
int VerificationIterative::reductionComplexityBefore [protected]
```

Natural strategy complexity before reduction.

5.63.4.6 revertToGlobalState

```
GlobalState* VerificationIterative::revertToGlobalState [protected]
```

Global state to which revert will rollback to.

5.63.4.7 statesToProcess

```
stack<StateVerificationInfo> VerificationIterative::statesToProcess [protected]
```


5.63.4.8 strategyVariableLimit

```
short VerificationIterative::strategyVariableLimit [protected]
```

Max strategy variables found.

5.63.4.9 variableNames

```
vector<string> VerificationIterative::variableNames [protected]
```

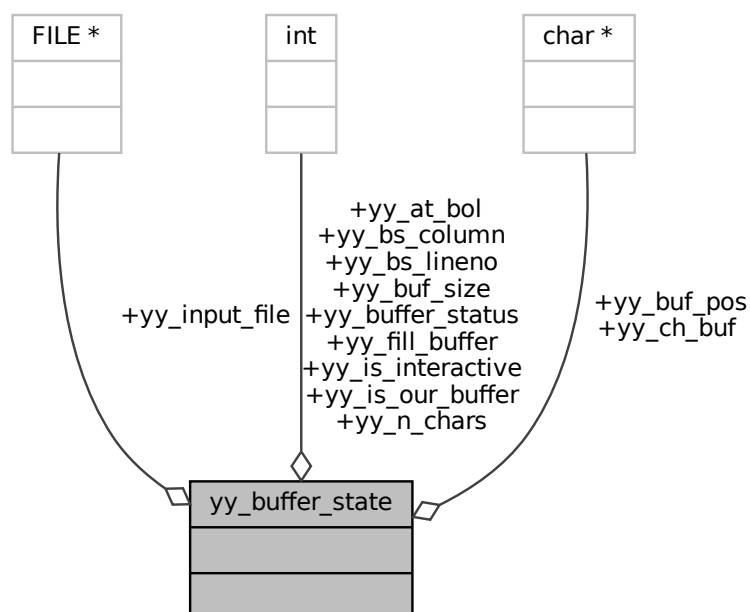
Easily readable variable names for natural strategy generation.

The documentation for this class was generated from the following files:

- [VerificationIterative.hpp](#)
- [VerificationIterative.cpp](#)

5.64 yy_buffer_state Struct Reference

Collaboration diagram for yy_buffer_state:



Public Attributes

- FILE * [yy_input_file](#)
- char * [yy_ch_buf](#)
- char * [yy_buf_pos](#)
- int [yy_buf_size](#)
- int [yy_n_chars](#)
- int [yy_is_our_buffer](#)
- int [yy_is_interactive](#)
- int [yy_at_bol](#)
- int [yy_bs_lineno](#)
- int [yy_bs_column](#)
- int [yy_fill_buffer](#)
- int [yy_buffer_status](#)

5.64.1 Member Data Documentation

5.64.1.1 [yy_at_bol](#)

```
int yy_buffer_state::yy_at_bol
```

5.64.1.2 [yy_bs_column](#)

```
int yy_buffer_state::yy_bs_column
```

The column count.

5.64.1.3 [yy_bs_lineno](#)

```
int yy_buffer_state::yy_bs_lineno
```

The line count.

5.64.1.4 [yy_buf_pos](#)

```
char * yy_buffer_state::yy_buf_pos
```

5.64.1.5 [yy_buf_size](#)

```
int yy_buffer_state::yy_buf_size
```

5.64.1.6 yy_buffer_status

```
int yy_buffer_state::yy_buffer_status
```

5.64.1.7 yy_ch_buf

```
char * yy_buffer_state::yy_ch_buf
```

5.64.1.8 yy_fill_buffer

```
int yy_buffer_state::yy_fill_buffer
```

5.64.1.9 yy_input_file

```
FILE * yy_buffer_state::yy_input_file
```

5.64.1.10 yy_is_interactive

```
int yy_buffer_state::yy_is_interactive
```

5.64.1.11 yy_is_our_buffer

```
int yy_buffer_state::yy_is_our_buffer
```

5.64.1.12 yy_n_chars

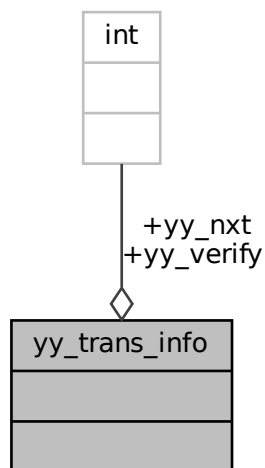
```
int yy_buffer_state::yy_n_chars
```

The documentation for this struct was generated from the following files:

- [scanner.c](#)
- [strategyScanner.c](#)

5.65 yy_trans_info Struct Reference

Collaboration diagram for yy_trans_info:



Public Attributes

- [flex_int32_t yy_verify](#)
- [flex_int32_t yy_nxt](#)

5.65.1 Member Data Documentation

5.65.1.1 yy_nxt

```
flex_int32_t yy_trans_info::yy_nxt
```

5.65.1.2 yy_verify

```
flex_int32_t yy_trans_info::yy_verify
```

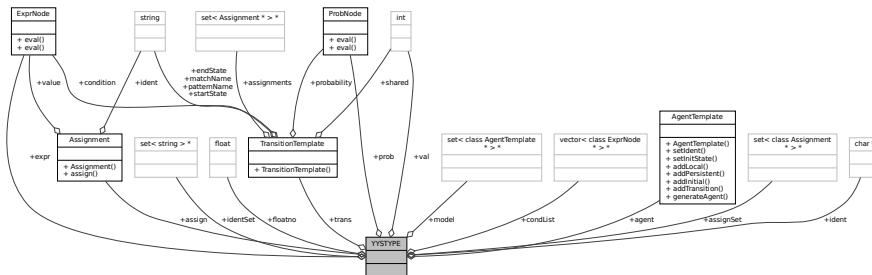
The documentation for this struct was generated from the following files:

- [scanner.c](#)
- [strategyScanner.c](#)

5.67 YYSTYPE Union Reference

```
#include <parser.h>
```

Collaboration diagram for YYSTYPE:



Public Attributes

- set< class [AgentTemplate](#) * > * [model](#)
- class [ExprNode](#) * [expr](#)
- class [ProbNode](#) * [prob](#)
- class [Assignment](#) * [assign](#)
- class [TransitionTemplate](#) * [trans](#)
- class [AgentTemplate](#) * [agent](#)
- set< class [Assignment](#) * > * [assignSet](#)
- char * [ident](#)
- set< string > * [identSet](#)
- int [val](#)
- float [floatno](#)
- vector< class [ExprNode](#) * > * [condList](#)

5.67.1 Member Data Documentation

5.67.1.1 agent

```
class AgentTemplate* YYSTYPE::agent
```

5.67.1.2 assign

```
class Assignment* YYSTYPE::assign
```

5.67.1.3 assignSet

```
set<class Assignment*>* YYSTYPE::assignSet
```

5.67.1.4 condList

```
vector<class ExprNode*>* YYSTYPE::condList
```

5.67.1.5 expr

```
class ExprNode* YYSTYPE::expr
```

5.67.1.6 floatno

```
float YYSTYPE::floatno
```

5.67.1.7 ident

```
char* YYSTYPE::ident
```

5.67.1.8 identSet

```
set<string>* YYSTYPE::identSet
```

5.67.1.9 model

```
set<class AgentTemplate*>* YYSTYPE::model
```

5.67.1.10 prob

```
class ProbNode* YYSTYPE::prob
```

5.67.1.11 trans

```
class TransitionTemplate* YYSTYPE::trans
```

5.67.1.12 val

```
int YYSTYPE::val
```

The documentation for this union was generated from the following file:

- [parser.h](#)

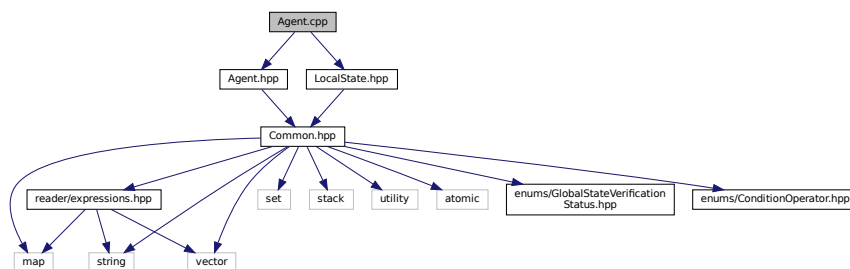
Chapter 6

File Documentation

6.1 Agent.cpp File Reference

Class of an agent. Class of an agent.

```
#include "Agent.hpp"
#include "LocalState.hpp"
Include dependency graph for Agent.cpp:
```



6.1.1 Detailed Description

Class of an agent. Class of an agent.

6.2 Agent.hpp File Reference

Class of an agent. Class of an agent.

6.4.1 Enumeration Type Documentation

6.4.1.1 ConditionOperator

enum `ConditionOperator`

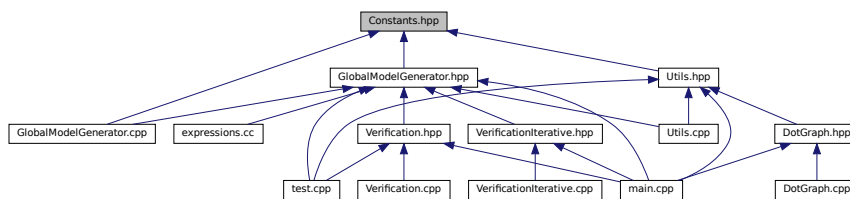
Conditional operator for the variable.

Enumerator

Equals	Variable should be equal to the value.
NotEquals	Variable should be not equal to the value.

6.5 Constants.hpp File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define VERBOSE 0`
- `#define DEBUG_ON 0`

6.5.1 Macro Definition Documentation

6.5.1.1 DEBUG_ON

```
#define DEBUG_ON 0
```

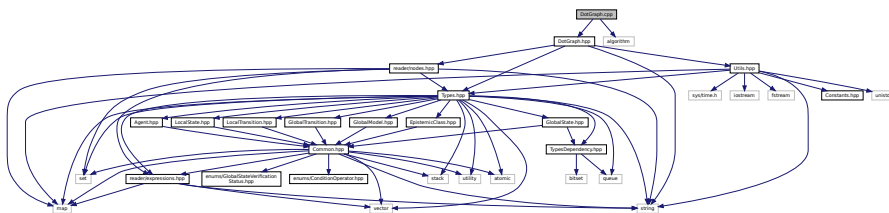
6.5.1.2 VERBOSE

```
#define VERBOSE 0
```

6.6 DotGraph.cpp File Reference

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "DotGraph.hpp"
#include <algorithm>
Include dependency graph for DotGraph.cpp:
```



6.6.1 Detailed Description

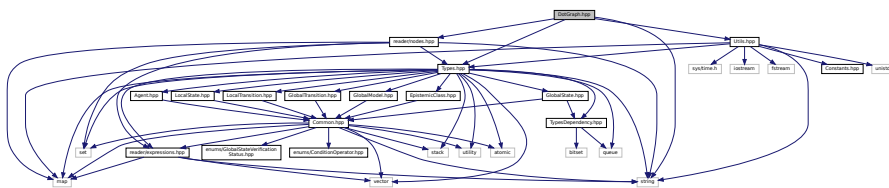
Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.7 DotGraph.hpp File Reference

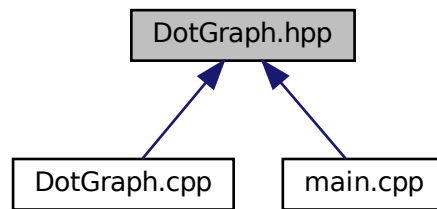
Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

```
#include "Types.hpp"
#include "Utils.hpp"
#include "reader/nodes.hpp"
#include <string>
```

Include dependency graph for DotGraph.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [DotGraph](#)

Enumerations

- enum [DotGraphBase](#) { [LOCAL_MODEL](#) , [GLOBAL_MODEL](#) , [AGENT_TEMPLATE](#) }

6.7.1 Detailed Description

Class for drawing graphs of input models. Class used for making graph files of input models created by parsing the data and adding graphviz syntax.

6.7.2 Enumeration Type Documentation

6.7.2.1 DotGraphBase

enum [DotGraphBase](#)

Enumerator

LOCAL_MODEL	(unfolded) local model = TS with states and transitions
GLOBAL_MODEL	(unfolded) global model = TS with states and transitions
AGENT_TEMPLATE	(folded/compact) agent graph = sets of locations and labelled edges

6.9.1 Detailed Description

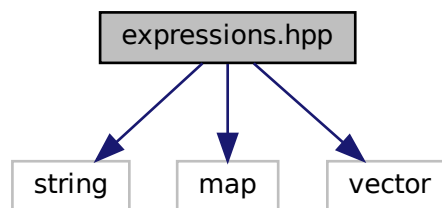
Eval and helper class for expressions. Eval and helper class for expressions.

6.10 expressions.hpp File Reference

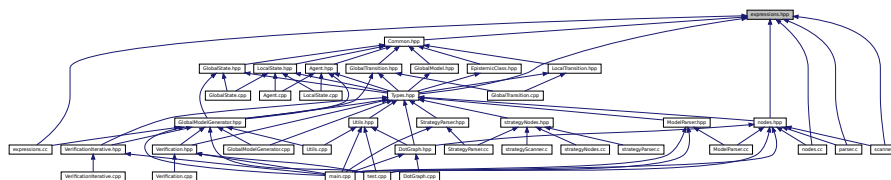
Eval and helper class for expressions. Eval and helper class for expressions.

```
#include <string>
#include <map>
#include <vector>
```

Include dependency graph for expressions.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [ExprNode](#)
Base node for expressions.
- class [ExprConst](#)
Node for a constant.
- class [ExprIdent](#)
Node for an identifier.
- class [ExprAdd](#)
Node for addition.
- class [ExprSub](#)
Node for subtraction.
- class [ExprMul](#)

- Node for multiplication.*
- class [ExprDiv](#)
 - Node for division.*
- class [ExprRem](#)
 - Node for modulo.*
- class [ExprAnd](#)
 - Node for AND operator.*
- class [ExprOr](#)
 - Node for OR operator.*
- class [ExprNot](#)
 - Node for NOT operator.*
- class [ExprEq](#)
 - Node for "==" operator.*
- class [ExprNe](#)
 - Node for "!=" operator.*
- class [ExprLt](#)
 - Node for "<" operator.*
- class [ExprLe](#)
 - Node for "<=" operator.*
- class [ExprGt](#)
 - Node for ">" operator.*
- class [ExprGe](#)
 - Node for ">=" operator.*
- class [ExprKnow](#)
 - Node for a knowledge operator.*
- class [ExprHart](#)
 - Node for a Hartley operator.*
- class [ProbNode](#)
 - Base node for probability calculations.*
- class [ProbConst](#)
 - Node for a constant.*
- class [ProbAdd](#)
 - Node for addition.*
- class [ProbSub](#)
 - Node for subtraction.*
- class [ProbMul](#)
 - Node for multiplication.*
- class [ProbDiv](#)
 - Node for division.*

Typedefs

- typedef map< string, int > [Environment](#)
 - Variable names with their values.*

6.10.1 Detailed Description

Eval and helper class for expressions. Eval and helper class for expressions.

6.10.2 Typedef Documentation

6.10.2.1 Environment

```
typedef map<string, int> Environment
```

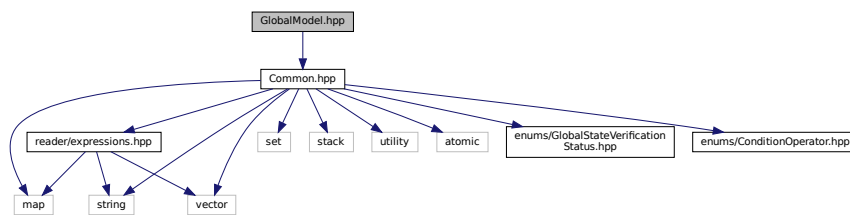
Variable names with their values.

6.11 GlobalModel.hpp File Reference

Struct of a global model. Struct of a global model.

```
#include "Common.hpp"
```

Include dependency graph for GlobalModel.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [GlobalModel](#)
Represents a global model, containing agents and a formula.

6.11.1 Detailed Description

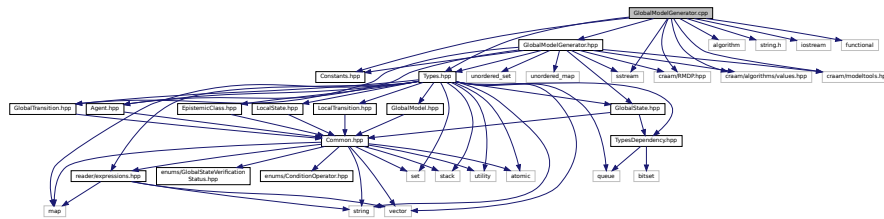
Struct of a global model. Struct of a global model.

6.12 GlobalModelGenerator.cpp File Reference

Generator of a global model. Class for initializing and generating a global model.

```
#include "GlobalModelGenerator.hpp"
#include "Types.hpp"
#include "Constants.hpp"
#include <algorithm>
#include <string.h>
#include <iostream>
#include <sstream>
#include <functional>
#include "craam/RMDP.hpp"
#include "craam/algorithms/values.hpp"
#include "craam/modeltools.hpp"
```

Include dependency graph for GlobalModelGenerator.cpp:



Variables

- [Cfg config](#)

6.12.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

6.12.2 Variable Documentation

6.12.2.1 config

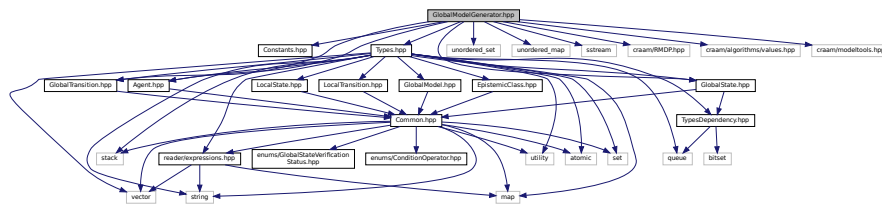
[Cfg config](#) [extern]

6.13 GlobalModelGenerator.hpp File Reference

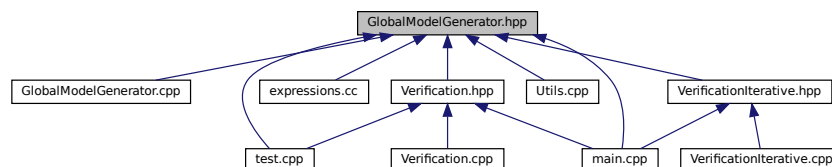
Generator of a global model. Class for initializing and generating a global model.

```
#include "Constants.hpp"
#include "GlobalState.hpp"
#include "GlobalTransition.hpp"
#include "Agent.hpp"
#include "Types.hpp"
#include <unordered_set>
#include <unordered_map>
#include <sstream>
#include "craam/RMDP.hpp"
#include "craam/algorithms/values.hpp"
#include "craam/modeltools.hpp"
```

Include dependency graph for GlobalModelGenerator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GlobalModelGenerator](#)
Stores the local models, formula and a global model.
- struct [GlobalModelGenerator::GlobalStateTupleHash](#)

6.13.1 Detailed Description

Generator of a global model. Class for initializing and generating a global model.

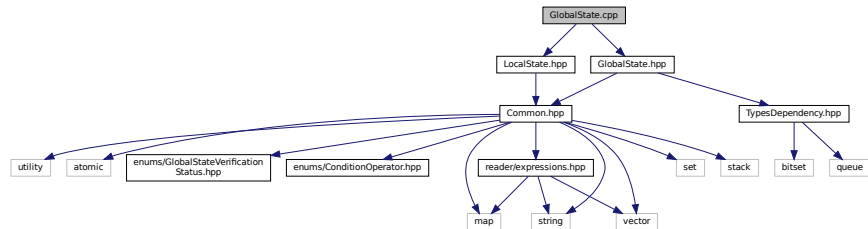
6.14 GlobalState.cpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "GlobalState.hpp"
```

```
#include "LocalState.hpp"
```

Include dependency graph for GlobalState.cpp:



6.14.1 Detailed Description

Struct representing a global state. Struct representing a global state.

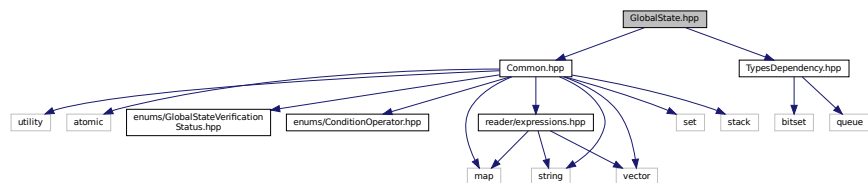
6.15 GlobalState.hpp File Reference

Struct representing a global state. Struct representing a global state.

```
#include "Common.hpp"
```

```
#include "TypesDependency.hpp"
```

Include dependency graph for GlobalState.hpp:



This graph shows which files directly or indirectly include this file:



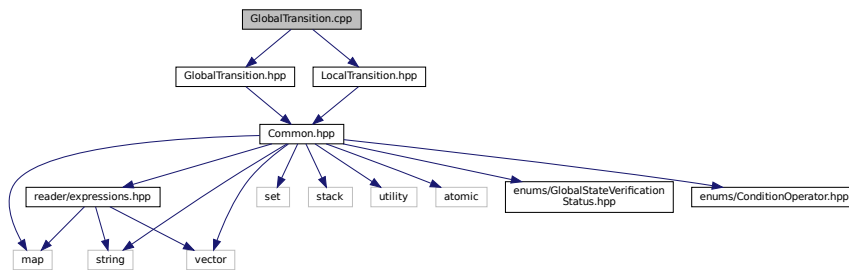
Enumerator

UNVERIFIED	State is not verified.
PENDING	Entered the state but it is not verified as correct or incorrect yet.
VERIFIED_OK	The state has been verified and is correct.
VERIFIED_ERR	

6.17 GlobalTransition.cpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "GlobalTransition.hpp"
#include "LocalTransition.hpp"
Include dependency graph for GlobalTransition.cpp:
```



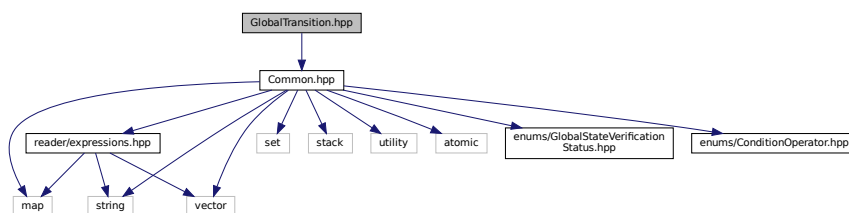
6.17.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

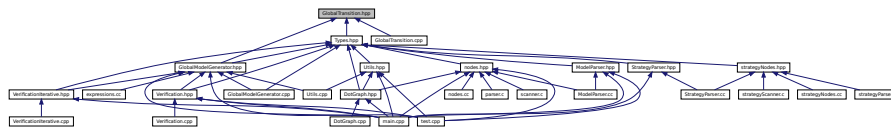
6.18 GlobalTransition.hpp File Reference

Struct representing a global transition. Struct representing a global transition.

```
#include "Common.hpp"
Include dependency graph for GlobalTransition.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- struct [GlobalTransition](#)
Represents a single global transition.

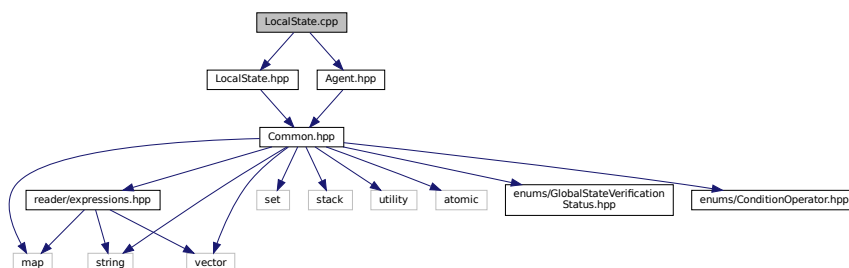
6.18.1 Detailed Description

Struct representing a global transition. Struct representing a global transition.

6.19 LocalState.cpp File Reference

Class representing a local state. Class representing a local state.

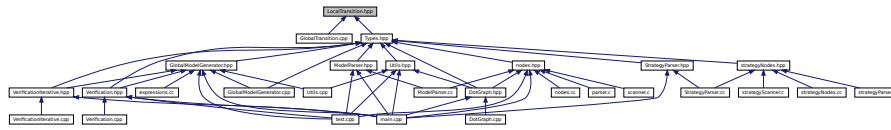
```
#include "LocalState.hpp"
#include "Agent.hpp"
Include dependency graph for LocalState.cpp:
```



6.19.1 Detailed Description

Class representing a local state. Class representing a local state.

This graph shows which files directly or indirectly include this file:



Classes

- struct [LocalTransition](#)

Represents a single local transition, containing id, global name, local name, is shared and count of the appearances.

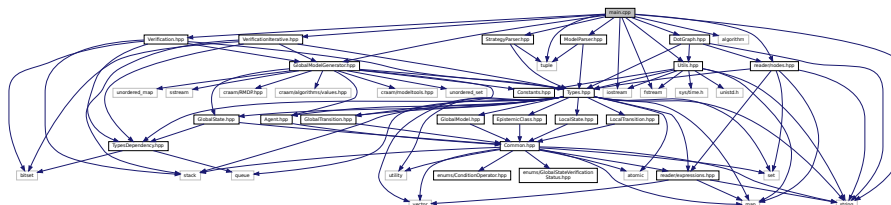
6.21.1 Detailed Description

Class representing a local transition. Class representing a local transition.

6.22 main.cpp File Reference

```
#include "GlobalModelGenerator.hpp"
#include "Verification.hpp"
#include "VerificationIterative.hpp"
#include "ModelParser.hpp"
#include "StrategyParser.hpp"
#include "Utils.hpp"
#include <iostream>
#include <fstream>
#include <string>
#include <tuple>
#include <algorithm>
#include "reader/nodes.hpp"
#include "DotGraph.hpp"
```

Include dependency graph for main.cpp:



Functions

- int [main](#) (int argc, char *argv[])

Variables

- [Cfg config](#)
- `set< AgentTemplate * > * modelDescription`
- `FormulaTemplate formulaDescription`

6.22.1 Function Documentation

6.22.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

6.22.2 Variable Documentation

6.22.2.1 config

[Cfg](#) config

6.22.2.2 formulaDescription

[FormulaTemplate](#) formulaDescription [extern]

6.22.2.3 modelDescription

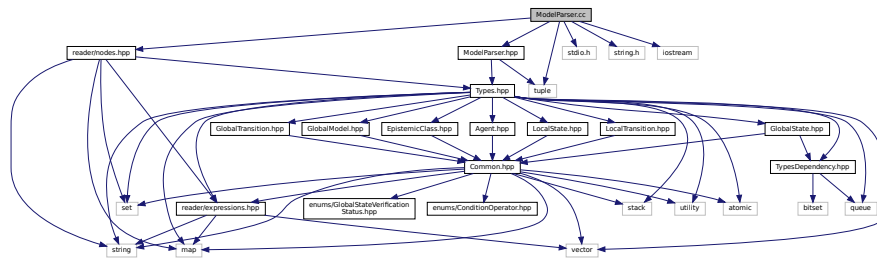
`set<AgentTemplate*>* modelDescription` [extern]

6.23 ModelParser.cc File Reference

A model parser. A parser for converting a text file into a model.

```
#include "ModelParser.hpp"
#include "reader/nodes.hpp"
#include <stdio.h>
#include <string.h>
#include <tuple>
#include <iostream>
```

Include dependency graph for ModelParser.cc:



Functions

- int [yparse](#) ()
- void [yyrestart](#) (FILE *)
- void [set_input_string](#) (const char *in)

Variables

- set< [AgentTemplate](#) * > * [modelDescription](#)
- [FormulaTemplate](#) [formulaDescription](#)

6.23.1 Detailed Description

A model parser. A parser for converting a text file into a model.

6.23.2 Function Documentation

6.23.2.1 set_input_string()

```
void set_input_string (
    const char * in )
```

6.23.2.2 yyparse()

```
int yyparse ( )
```

6.23.2.3 yyrestart()

```
void yyrestart (
    FILE * )
```

6.23.3 Variable Documentation

6.23.3.1 formulaDescription

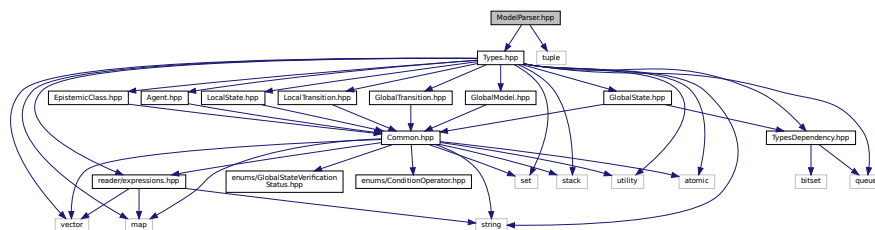
```
FormulaTemplate formulaDescription
```

6.23.3.2 modelDescription

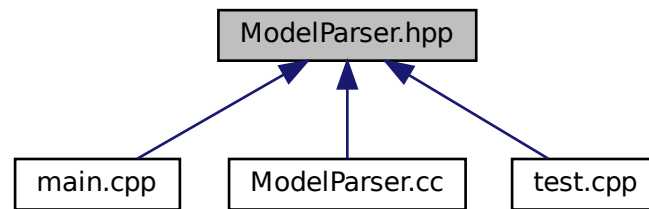
```
set<AgentTemplate*>* modelDescription
```

6.24 ModelParser.hpp File Reference

```
#include "Types.hpp"
#include <tuple>
Include dependency graph for ModelParser.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [ModelParser](#)

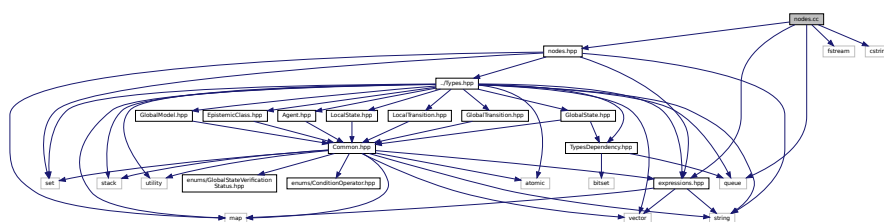
A parser for converting a text file into a model.

6.25 nodes.cc File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include "expressions.hpp"
#include "nodes.hpp"
#include <queue>
#include <fstream>
#include <cstring>
```

Include dependency graph for nodes.cc:



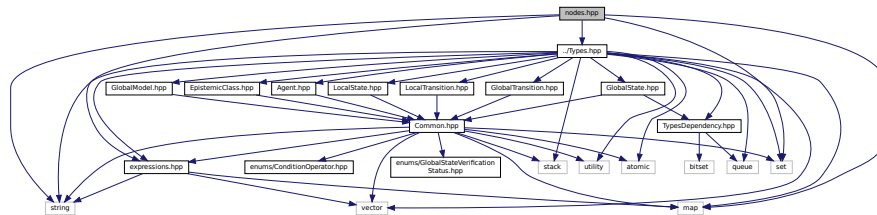
6.25.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

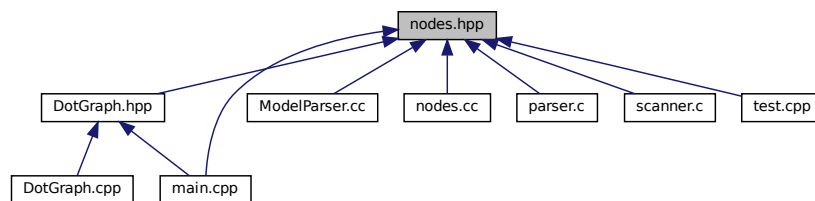
6.26 nodes.hpp File Reference

Parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "expressions.hpp"
#include "../Types.hpp"
Include dependency graph for nodes.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Assignment](#)
Represents an assingment.
- class [TransitionTemplate](#)
Represents a meta-transition.
- class [LocalStateTemplate](#)
A template for the local state.
- class [AgentTemplate](#)
Represents a single agent loaded from the description from a file.

6.26.1 Detailed Description

Parser templates. Class for setting up a new objects from a parser.

- `#define YYSTACK_ALLOC YYMALLOC`
- `#define YYSTACK_FREE YYFREE`
- `#define YYSTACK_ALLOC_MAXIMUM YYSIZE_MAXIMUM`
- `#define YYMALLOC malloc`
- `#define YYFREE free`
- `#define YYSTACK_GAP_MAXIMUM (YYSIZEOF (union yyalloc) - 1)`
- `#define YYSTACK_BYTES(N)`
- `#define YYCOPY_NEEDED 1`
- `#define YYSTACK_RELOCATE(Stack_alloc, Stack)`
- `#define YYCOPY(Dst, Src, Count)`
- `#define YYFINAL 3`
- `#define YYLAST 294`
- `#define YYNTOKENS 39`
- `#define YYNNTS 29`
- `#define YYNRULES 92`
- `#define YYNSTATES 217`
- `#define YYMAXUTOK 281`
- `#define YYTRANSLATE(YYX)`
- `#define YY_ACCESSING_SYMBOL(State) YY_CAST (yysymbol_kind_t, yystos[State])`
- `#define YYPACT_NINF (-81)`
- `#define yypact_value_is_default(Yyn) ((Yyn) == YYPACT_NINF)`
- `#define YYTABLE_NINF (-90)`
- `#define yytable_value_is_error(Yyn) 0`
- `#define yyerrok (yyerrstatus = 0)`
- `#define yyclearin (yychar = YYEMPTY)`
- `#define YYACCEPT goto yyacceptlab`
- `#define YYABORT goto yyabortlab`
- `#define YYERROR goto yyerrorlab`
- `#define YYNOMEM goto yyexhaustedlab`
- `#define YYRECOVERING() (!yyerrstatus)`
- `#define YYBACKUP(Token, Value)`
- `#define YYERRCODE YYUNDEF`
- `#define YYDPRINTF(Args) ((void) 0)`
- `#define YY_SYMBOL_PRINT(Title, Kind, Value, Location)`
- `#define YY_STACK_PRINT(Bottom, Top)`
- `#define YY_REDUCE_PRINT(Rule)`
- `#define YYINITDEPTH 200`
- `#define YYMAXDEPTH 10000`
- `#define YYPOPSTACK(N) (yyvsp -= (N), yyssp -= (N))`

Typedefs

- `typedef struct yy_buffer_state * YY_BUFFER_STATE`
- `typedef enum yysymbol_kind_t yysymbol_kind_t`
- `typedef signed char yytype_int8`
- `typedef short yytype_int16`
- `typedef unsigned char yytype_uint8`
- `typedef unsigned short yytype_uint16`
- `typedef yytype_uint8 yy_state_t`
- `typedef int yy_state_fast_t`

Enumerations

- enum `yysymbol_kind_t` {
`YYSMBOL_YYEMPTY` = -2, `YYSMBOL_YYEOF` = 0, `YYSMBOL_YYerror` = 1, `YYSMBOL_YYUNDEF` = 2,
`YYSMBOL_T_AGENT` = 3, `YYSMBOL_T_INIT` = 4, `YYSMBOL_T_LOCAL` = 5, `YYSMBOL_T_INITIAL` = 6,
`YYSMBOL_T_FORMULA` = 7, `YYSMBOL_T_PERSISTENT` = 8, `YYSMBOL_T_TO` = 9,
`YYSMBOL_T_ASSIGN` = 10,
`YYSMBOL_T_OR` = 11, `YYSMBOL_T_AND` = 12, `YYSMBOL_T_EQ` = 13, `YYSMBOL_T_NE` = 14,
`YYSMBOL_T_GT` = 15, `YYSMBOL_T_GE` = 16, `YYSMBOL_T_LT` = 17, `YYSMBOL_T_LE` = 18,
`YYSMBOL_T_NUM` = 19, `YYSMBOL_T_SHARED` = 20, `YYSMBOL_T_FLOAT` = 21, `YYSMBOL_T_IDENT` = 22,
`YYSMBOL_T_KNOW` = 23, `YYSMBOL_T_HARTLEY` = 24, `YYSMBOL_T_PROB` = 25,
`YYSMBOL_T_REST` = 26,
`YYSMBOL_27_` = 27, `YYSMBOL_28_` = 28, `YYSMBOL_29_` = 29, `YYSMBOL_30_` = 30,
`YYSMBOL_31_` = 31, `YYSMBOL_32_` = 32, `YYSMBOL_33_` = 33, `YYSMBOL_34_` = 34,
`YYSMBOL_35_` = 35, `YYSMBOL_36_` = 36, `YYSMBOL_37_` = 37, `YYSMBOL_38_` = 38,
`YYSMBOL_YYACCEPT` = 39, `YYSMBOL_model` = 40, `YYSMBOL_spec` = 41, `YYSMBOL_query` = 42,
`YYSMBOL_formula` = 43, `YYSMBOL_probability_equality` = 44, `YYSMBOL_cond_list` = 45,
`YYSMBOL_coalition` = 46,
`YYSMBOL_agent` = 47, `YYSMBOL_description` = 48, `YYSMBOL_local` = 49, `YYSMBOL_persistent` = 50,
`YYSMBOL_ident_list` = 51, `YYSMBOL_initial` = 52, `YYSMBOL_assign_list` = 53, `YYSMBOL_assignment` = 54,
`YYSMBOL_transition` = 55, `YYSMBOL_shared` = 56, `YYSMBOL_assignments` = 57, `YYSMBOL_num_exp` = 58,
`YYSMBOL_num_mul` = 59, `YYSMBOL_num_elem` = 60, `YYSMBOL_prob_exp` = 61, `YYSMBOL_prob_mul` = 62,
`YYSMBOL_prob_elem` = 63, `YYSMBOL_condition` = 64, `YYSMBOL_cond` = 65, `YYSMBOL_cond_and` = 66,
`YYSMBOL_cond_elem` = 67, `YYSMBOL_YYEMPTY` = -2, `YYSMBOL_YYEOF` = 0, `YYSMBOL_YYerror` = 1,
`YYSMBOL_YYUNDEF` = 2, `YYSMBOL_T_TO` = 3, `YYSMBOL_T_ASSIGN` = 4, `YYSMBOL_T_OR` = 5,
`YYSMBOL_T_AND` = 6, `YYSMBOL_T_EQ` = 7, `YYSMBOL_T_NE` = 8, `YYSMBOL_T_GT` = 9,
`YYSMBOL_T_GE` = 10, `YYSMBOL_T_LT` = 11, `YYSMBOL_T_LE` = 12, `YYSMBOL_T_NUM` = 13,
`YYSMBOL_T_SHARED` = 14, `YYSMBOL_T_FLOAT` = 15, `YYSMBOL_T_IDENT` = 16,
`YYSMBOL_17_` = 17,
`YYSMBOL_YYACCEPT` = 18, `YYSMBOL_strat` = 19, `YYSMBOL_ident_list` = 20 }
- enum { `YYENOMEM` = -2 }

Functions

- int `yylex` ()
- void `yyrestart` (FILE *)
- void `yyerror` (const char *)
- `YY_BUFFER_STATE` `yy_scan_string` (const char *str)
- void * `malloc` (YYSIZE_T)
- void `free` (void *)
- int `yyparse` (void)
- void `set_input_string` (const char *in)
- int `main2` (int argc, char *argv[])

Variables

- char * [yytext](#)
- set< class [AgentTemplate](#) * > * [modelDescription](#)
- [FormulaTemplate](#) [formulaDescription](#)
- int [yychar](#)
- [YYSTYPE](#) [yylval](#)
- int [yynerrs](#)

6.27.1 Macro Definition Documentation

6.27.1.1 YY_

```
#define YY_(  
    Msgid ) Msgid
```

6.27.1.2 YY_ACCESSING_SYMBOL

```
#define YY_ACCESSING_SYMBOL(  
    State ) YY\_CAST (yysymbol\_kind\_t, yystos[State])
```

Accessing symbol of state STATE.

6.27.1.3 YY_ASSERT

```
#define YY_ASSERT(  
    E ) ((void) (0 && (E)))
```

6.27.1.4 YY_ATTRIBUTE_PURE

```
#define YY_ATTRIBUTE_PURE
```

6.27.1.5 YY_ATTRIBUTE_UNUSED

```
#define YY_ATTRIBUTE_UNUSED
```

6.27.1.6 YY_CAST

```
#define YY_CAST(  
    Type,  
    Val ) ((Type) (Val))
```

6.27.1.7 YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN

```
#define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN
```

6.27.1.8 YY_IGNORE_MAYBE_UNINITIALIZED_END

```
#define YY_IGNORE_MAYBE_UNINITIALIZED_END
```

6.27.1.9 YY_IGNORE_USELESS_CAST_BEGIN

```
#define YY_IGNORE_USELESS_CAST_BEGIN
```

6.27.1.10 YY_IGNORE_USELESS_CAST_END

```
#define YY_IGNORE_USELESS_CAST_END
```

6.27.1.11 YY_INITIAL_VALUE

```
#define YY_INITIAL_VALUE(  
    Value ) Value
```

6.27.1.12 YY_NULLPTR

```
#define YY_NULLPTR ((void*)0)
```

6.27.1.13 YY_REDUCE_PRINT

```
#define YY_REDUCE_PRINT(  
    Rule )
```

6.27.1.14 YY_REINTERPRET_CAST

```
#define YY_REINTERPRET_CAST(  
    Type,  
    Val ) ((Type) (Val))
```

6.27.1.15 YY_STACK_PRINT

```
#define YY_STACK_PRINT(  
    Bottom,  
    Top )
```

6.27.1.16 YY_SYMBOL_PRINT

```
#define YY_SYMBOL_PRINT(  
    Title,  
    Kind,  
    Value,  
    Location )
```

6.27.1.17 YY_USE

```
#define YY_USE(  
    E ) ((void) (E))
```

6.27.1.18 YYABORT

```
#define YYABORT goto yyabortlab
```

6.27.1.19 YYACCEPT

```
#define YYACCEPT goto yyacceptlab
```

6.27.1.20 YYBACKUP

```
#define YYBACKUP(  
    Token,  
    Value )
```

Value:

```
do  
  if (yychar == YYEMPTY)  
  {  
    yychar = (Token);  
    yyval = (Value);  
    YYPOPSTACK (yylen);  
    yystate = *yyssp;  
    goto yybackup;  
  }  
  else  
  {  
    yyerror (YY_("syntax error: cannot back up")); \  
    YYERROR;  
  }  
while (0)
```

6.27.1.21 YYBISON

```
#define YYBISON 30802
```

6.27.1.22 YYBISON_VERSION

```
#define YYBISON_VERSION "3.8.2"
```

6.27.1.23 yyclearin

```
#define yyclearin (yychar = YYEMPTY)
```

6.27.1.24 YYCOPY

```
#define YYCOPY(  
    Dst,  
    Src,  
    Count )
```

Value:

```
do  
{  
    YYPTRDIFF_T yyi;  
    for (yyi = 0; yyi < (Count); yyi++)  
        (Dst)[yyi] = (Src)[yyi];  
}  
while (0)
```

```
\\  
\\  
\\  
\\  
\\
```

6.27.1.25 YYCOPY_NEEDED

```
#define YYCOPY_NEEDED 1
```

6.27.1.26 YYDPRINTF

```
#define YYDPRINTF(  
    Args ) ((void) 0)
```

6.27.1.27 YYERRCODE

```
#define YYERRCODE YYUNDEF
```

6.27.1.28 yyerrok

```
#define yyerrok (yyerrstatus = 0)
```

6.27.1.29 YYERROR

```
#define YYERROR goto yyerrorlab
```

6.27.1.30 YYFINAL

```
#define YYFINAL 3
```

6.27.1.31 YYFREE

```
#define YYFREE free
```

6.27.1.32 YYINITDEPTH

```
#define YYINITDEPTH 200
```

6.27.1.33 YYLAST

```
#define YYLAST 294
```

6.27.1.34 YYMALLOC

```
#define YYMALLOC malloc
```

6.27.1.35 YYMAXDEPTH

```
#define YYMAXDEPTH 10000
```

6.27.1.36 YYMAXUTOK

```
#define YYMAXUTOK 281
```

6.27.1.37 YYNNTS

```
#define YYNNTS 29
```


6.27.1.38 YYNOMEM

```
#define YYNOMEM goto yyexhaustedlab
```

6.27.1.39 YYNRULES

```
#define YYNRULES 92
```

6.27.1.40 YYNSTATES

```
#define YYNSTATES 217
```

6.27.1.41 YYNTOKENS

```
#define YYNTOKENS 39
```

6.27.1.42 YYPACT_NINF

```
#define YYPACT_NINF (-81)
```

6.27.1.43 yypact_value_is_default

```
#define yypact_value_is_default(  
    Yyn )    ((Yyn) == YYPACT\_NINF)
```

6.27.1.44 YYPOPSTACK

```
#define YYPOPSTACK(  
    N ) (yyvsp -= (N), yyssp -= (N))
```

6.27.1.45 YPTRDIFF_MAXIMUM

```
#define YPTRDIFF_MAXIMUM LONG_MAX
```

6.27.1.46 YPTRDIFF_T

```
#define YPTRDIFF_T long
```

6.27.1.47 YYPULL

```
#define YYPULL 1
```

6.27.1.48 YYPURE

```
#define YYPURE 0
```

6.27.1.49 YYPUSH

```
#define YYPUSH 0
```

6.27.1.50 YYRECOVERING

```
#define YYRECOVERING( ) (!yyerrstatus)
```

6.27.1.51 YYSIZE_MAXIMUM

```
#define YYSIZE_MAXIMUM
```

Value:

```
YY_CAST (YPTRDIFF_T,  
         (YPTRDIFF_MAXIMUM < YY_CAST (YYSIZE_T, -1)  
         ? YPTRDIFF_MAXIMUM  
         : YY_CAST (YYSIZE_T, -1)))
```

6.27.1.52 YYSIZE_T

```
#define YYSIZE_T unsigned
```

6.27.1.53 YYSIZEOF

```
#define YYSIZEOF(  
    X ) YY_CAST (YYPTRDIFF_T, sizeof (X))
```

6.27.1.54 YYSKELETON_NAME

```
#define YYSKELETON_NAME "yacc.c"
```

6.27.1.55 YYSTACK_ALLOC

```
#define YYSTACK_ALLOC YYMALLOC
```

6.27.1.56 YYSTACK_ALLOC_MAXIMUM

```
#define YYSTACK_ALLOC_MAXIMUM YYSIZE_MAXIMUM
```

6.27.1.57 YYSTACK_BYTES

```
#define YYSTACK_BYTES(  
    N )
```

Value:

```
( (N) * (YYSIZEOF (yy_state_t) + YYSIZEOF (YYSTYPE)) \  
  + YYSTACK_GAP_MAXIMUM)
```

6.27.1.58 YYSTACK_FREE

```
#define YYSTACK_FREE YYFREE
```

6.27.1.59 YYSTYPE_GAP_MAXIMUM

```
#define YYSTYPE_GAP_MAXIMUM (YYSIZEOF (union yyallocl) - 1)
```

6.27.1.60 YYSTACK_RELOCATE

```
#define YYSTACK_RELOCATE(  
    Stack_alloc,  
    Stack )
```

Value:

```
do  
{  
    YPTRDIFF_T yynewbytes;  
    YYCOPY (&yyptr->Stack_alloc, Stack, yysize);  
    Stack = &yyptr->Stack_alloc;  
    yynewbytes = yystacksize * YYSIZEOF (*Stack) + YYSTACK_GAP_MAXIMUM; \  
    yyptr += yynewbytes / YYSIZEOF (*yyptr);  
}  
while (0)
```

6.27.1.61 YYTABLE_NINF

```
#define YYTABLE_NINF (-90)
```

6.27.1.62 yytable_value_is_error

```
#define yytable_value_is_error(  
    Yyn ) 0
```

6.27.1.63 YYTRANSLATE

```
#define YYTRANSLATE(  
    YYX )
```

Value:

```
(0 <= (YYX) && (YYX) <= YYMAXUTOK \  
? YY_CAST (yy_symbol_kind_t, yytranslate[YYX]) \  
: YYSYMBOL_YYUNDEF)
```

6.27.2 Typedef Documentation

6.27.2.1 YY_BUFFER_STATE

```
typedef struct yy_buffer_state* YY_BUFFER_STATE
```

6.27.2.2 yy_state_fast_t

```
typedef int yy_state_fast_t
```

6.27.2.3 yy_state_t

```
typedef yytype_uint8 yy_state_t
```

6.27.2.4 yysymbol_kind_t

```
typedef enum yysymbol_kind_t yysymbol_kind_t
```

6.27.2.5 yytype_int16

```
typedef short yytype_int16
```

6.27.2.6 yytype_int8

```
typedef signed char yytype_int8
```

6.27.2.7 yytype_uint16

```
typedef unsigned short yytype_uint16
```

6.27.2.8 yytype_uint8

```
typedef unsigned char yytype_uint8
```

6.27.3 Enumeration Type Documentation

6.27.3.1 anonymous enum

```
anonymous enum
```

Enumerator

YYENOMEM	
----------	--

6.27.3.2 yysymbol_kind_t

```
enum yysymbol_kind_t
```

Enumerator

YYSYMBOL_YEMPTY	
YYSYMBOL_YEOF	
YYSYMBOL_YError	
YYSYMBOL_YUNDEF	
YYSYMBOL_T_AGENT	
YYSYMBOL_T_INIT	
YYSYMBOL_T_LOCAL	
YYSYMBOL_T_INITIAL	
YYSYMBOL_T_FORMULA	
YYSYMBOL_T_PERSISTENT	
YYSYMBOL_T_TO	
YYSYMBOL_T_ASSIGN	
YYSYMBOL_T_OR	
YYSYMBOL_T_AND	
YYSYMBOL_T_EQ	
YYSYMBOL_T_NE	
YYSYMBOL_T_GT	
YYSYMBOL_T_GE	
YYSYMBOL_T_LT	
YYSYMBOL_T_LE	
YYSYMBOL_T_NUM	
YYSYMBOL_T_SHARED	
YYSYMBOL_T_FLOAT	
YYSYMBOL_T_IDENT	
YYSYMBOL_T_KNOW	
YYSYMBOL_T_HARTLEY	
YYSYMBOL_T_PROB	
YYSYMBOL_T_REST	
YYSYMBOL_27_	
YYSYMBOL_28_	
YYSYMBOL_29_	
YYSYMBOL_30_	
YYSYMBOL_31_	
YYSYMBOL_32_	
YYSYMBOL_33_	
YYSYMBOL_34_	
YYSYMBOL_35_	
YYSYMBOL_36_	

Enumerator

YYSYMBOL_37_	
YYSYMBOL_38_	
YYSYMBOL_YYACCEPT	
YYSYMBOL_model	
YYSYMBOL_spec	
YYSYMBOL_query	
YYSYMBOL_formula	
YYSYMBOL_probability_equality	
YYSYMBOL_cond_list	
YYSYMBOL_coalition	
YYSYMBOL_agent	
YYSYMBOL_description	
YYSYMBOL_local	
YYSYMBOL_persistent	
YYSYMBOL_ident_list	
YYSYMBOL_initial	
YYSYMBOL_assign_list	
YYSYMBOL_assignment	
YYSYMBOL_transition	
YYSYMBOL_shared	
YYSYMBOL_assignments	
YYSYMBOL_num_exp	
YYSYMBOL_num_mul	
YYSYMBOL_num_elem	
YYSYMBOL_prob_exp	
YYSYMBOL_prob_mul	
YYSYMBOL_prob_elem	
YYSYMBOL_condition	
YYSYMBOL_cond	
YYSYMBOL_cond_and	
YYSYMBOL_cond_elem	
YYSYMBOL_YYEMPTY	
YYSYMBOL_YYEOF	
YYSYMBOL_YYerror	
YYSYMBOL_YYUNDEF	
YYSYMBOL_T_TO	
YYSYMBOL_T_ASSIGN	
YYSYMBOL_T_OR	
YYSYMBOL_T_AND	
YYSYMBOL_T_EQ	
YYSYMBOL_T_NE	
YYSYMBOL_T_GT	
YYSYMBOL_T_GE	
YYSYMBOL_T_LT	
YYSYMBOL_T_LE	
YYSYMBOL_T_NUM	
YYSYMBOL_T_SHARED	
YYSYMBOL_T_FLOAT	

Enumerator

YYSMBOL_T_IDENT	
YYSMBOL_17_	
YYSMBOL_YYACCEPT	
YYSMBOL_strat	
YYSMBOL_ident_list	

6.27.4 Function Documentation

6.27.4.1 free()

```
void free (
    void * )
```

6.27.4.2 main2()

```
int main2 (
    int argc,
    char * argv[] )
```

6.27.4.3 malloc()

```
void* malloc (
    YYSIZE_T )
```

6.27.4.4 set_input_string()

```
void set_input_string (
    const char * in )
```

6.27.4.5 yy_scan_string()

```
YY_BUFFER_STATE yy_scan_string (
    const char * str )
```


6.27.4.6 yyerror()

```
void yyerror (
    const char * s )
```

6.27.4.7 yylex()

```
int yylex ( )
```

6.27.4.8 yyparse()

```
int yyparse (
    void )
```

6.27.4.9 yyrestart()

```
void yyrestart (
    FILE * )
```

6.27.5 Variable Documentation

6.27.5.1 formulaDescription

```
FormulaTemplate formulaDescription [extern]
```

6.27.5.2 modelDescription

```
set<class AgentTemplate*>* modelDescription [extern]
```

6.27.5.3 yychar

```
int yychar
```

6.27.5.4 yylval

`YYSTYPE` `yylval`

6.27.5.5 yynerrs

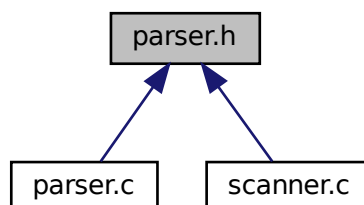
`int` `yynerrs`

6.27.5.6 yytext

`char*` `yytext` [extern]

6.28 parser.h File Reference

This graph shows which files directly or indirectly include this file:



Classes

- union `YYSTYPE`

Macros

- `#define` `YYDEBUG` 0
- `#define` `YYTOKENTYPE`
- `#define` `YYSTYPE_IS_TRIVIAL` 1
- `#define` `YYSTYPE_IS_DECLARED` 1

Typedefs

- typedef enum [yytokentype](#) [yytoken_kind_t](#)
- typedef union [YYSTYPE](#) [YYSTYPE](#)

Enumerations

- enum [yytokentype](#) {
 [YYEMPTY](#) = -2, [YYEOF](#) = 0, [YYerror](#) = 256, [YYUNDEF](#) = 257 ,
 [T_AGENT](#) = 258, [T_INIT](#) = 259, [T_LOCAL](#) = 260, [T_INITIAL](#) = 261 ,
 [T_FORMULA](#) = 262, [T_PERSISTENT](#) = 263, [T_TO](#) = 264, [T_ASSIGN](#) = 265 ,
 [T_OR](#) = 266, [T_AND](#) = 267, [T_EQ](#) = 268, [T_NE](#) = 269 ,
 [T_GT](#) = 270, [T_GE](#) = 271, [T_LT](#) = 272, [T_LE](#) = 273 ,
 [T_NUM](#) = 274, [T_SHARED](#) = 275, [T_FLOAT](#) = 276, [T_IDENT](#) = 277 ,
 [T_KNOW](#) = 278, [T_HARTLEY](#) = 279, [T_PROB](#) = 280, [T_REST](#) = 281 }

Functions

- int [yyparse](#) (void)

Variables

- [YYSTYPE](#) [yylval](#)

6.28.1 Macro Definition Documentation

6.28.1.1 YYDEBUG

```
#define YYDEBUG 0
```

6.28.1.2 YYSTYPE_IS_DECLARED

```
#define YYSTYPE_IS_DECLARED 1
```

6.28.1.3 YYSTYPE_IS_TRIVIAL

```
#define YYSTYPE_IS_TRIVIAL 1
```

6.28.1.4 YYTOKENTYPE

```
#define YYTOKENTYPE
```

6.28.2 Typedef Documentation

6.28.2.1 YYSTYPE

```
typedef union YYSTYPE YYSTYPE
```

6.28.2.2 yytoken_kind_t

```
typedef enum yytokentype yytoken_kind_t
```

6.28.3 Enumeration Type Documentation

6.28.3.1 yytokentype

```
enum yytokentype
```

Enumerator

YYEMPTY	
YYEOF	
YError	
YYUNDEF	
T_AGENT	
T_INIT	
T_LOCAL	
T_INITIAL	
T_FORMULA	
T_PERSISTENT	
T_TO	
T_ASSIGN	
T_OR	
T_AND	
T_EQ	
T_NE	
T_GT	
T_GE	

Enumerator

T_LT	
T_LE	
T_NUM	
T_SHARED	
T_FLOAT	
T_IDENT	
T_KNOW	
T_HARTLEY	
T_PROB	
T_REST	

6.28.4 Function Documentation**6.28.4.1 yyparse()**

```
int yyparse (  
    void )
```

6.28.5 Variable Documentation**6.28.5.1 yylval**

```
YYSTYPE yylval [extern]
```

6.29 README.md File Reference**6.30 scanner.c File Reference**

```
#include <stdio.h>  
#include <string.h>  
#include <errno.h>  
#include <stdlib.h>  
#include <string>  
#include <set>  
#include <vector>  
#include "parser.h"  
#include "expressions.hpp"  
#include "nodes.hpp"
```


- `#define YY_LESS_LINENO(n)`
- `#define YY_LINENO_REWIND_TO(ptr)`
- `#define yyless(n)`
- `#define unput(c) yyunput( c, (yytext_ptr) )`
- `#define YY_STRUCT_YY_BUFFER_STATE`
- `#define YY_BUFFER_NEW 0`
- `#define YY_BUFFER_NORMAL 1`
- `#define YY_BUFFER_EOF_PENDING 2`
- `#define YY_CURRENT_BUFFER`
- `#define YY_CURRENT_BUFFER_LVALUE (yy_buffer_stack)[(yy_buffer_stack_top)]`
- `#define YY_FLUSH_BUFFER yy_flush_buffer( YY_CURRENT_BUFFER )`
- `#define yy_new_buffer yy_create_buffer`
- `#define yy_set_interactive(is_interactive)`
- `#define yy_set_bol(at_bol)`
- `#define YY_AT_BOL() (YY_CURRENT_BUFFER_LVALUE->yy_at_bol)`
- `#define yywrap() (/*CONSTCOND*/1)`
- `#define YY_SKIP_YYWRAP`
- `#define yytext_ptr yytext`
- `#define YY_DO_BEFORE_ACTION`
- `#define YY_NUM_RULES 28`
- `#define YY_END_OF_BUFFER 29`
- `#define REJECT reject_used_but_not_detected`
- `#define yymore() yymore_used_but_not_detected`
- `#define YY_MORE_ADJ 0`
- `#define YY_RESTORE_YY_MORE_OFFSET`
- `#define INITIAL 0`
- `#define YY_EXTRA_TYPE void *`
- `#define YY_READ_BUF_SIZE 8192`
- `#define ECHO do { if (fwrite( yytext, (size_t) yyleng, 1, yyout )) {} } while (0)`
- `#define YY_INPUT(buf, result, max_size)`
- `#define yyterminate() return YY_NULL`
- `#define YY_START_STACK_INCR 25`
- `#define YY_FATAL_ERROR(msg) yy_fatal_error( msg )`
- `#define YY_DECL_IS_OURS 1`
- `#define YY_DECL int yylex (void)`
- `#define YY_USER_ACTION`
- `#define YY_BREAK /*LINTED*/break;`
- `#define YY_RULE_SETUP YY_USER_ACTION`
- `#define YY_EXIT_FAILURE 2`
- `#define yyless(n)`
- `#define YYTABLES_NAME "yytables"`

Typedefs

- `typedef signed char flex_int8_t`
- `typedef short int flex_int16_t`
- `typedef int flex_int32_t`
- `typedef unsigned char flex_uint8_t`
- `typedef unsigned short int flex_uint16_t`
- `typedef unsigned int flex_uint32_t`
- `typedef struct yy_buffer_state * YY_BUFFER_STATE`
- `typedef size_t yy_size_t`
- `typedef flex_uint8_t YY_CHAR`
- `typedef int yy_state_type`

Functions

- void [yyrestart](#) (FILE *input_file)
- void [yy_switch_to_buffer](#) (YY_BUFFER_STATE new_buffer)
- YY_BUFFER_STATE [yy_create_buffer](#) (FILE *file, int size)
- void [yy_delete_buffer](#) (YY_BUFFER_STATE b)
- void [yy_flush_buffer](#) (YY_BUFFER_STATE b)
- void [yypush_buffer_state](#) (YY_BUFFER_STATE new_buffer)
- void [yypop_buffer_state](#) (void)
- YY_BUFFER_STATE [yy_scan_buffer](#) (char *base, [yy_size_t](#) size)
- YY_BUFFER_STATE [yy_scan_string](#) (const char *yy_str)
- YY_BUFFER_STATE [yy_scan_bytes](#) (const char *bytes, int len)
- void * [yyalloc](#) ([yy_size_t](#))
- void * [yyrealloc](#) (void *, [yy_size_t](#))
- void [yyfree](#) (void *)
- int [yylex](#) ()
- int [yylex_destroy](#) (void)
- int [yyget_debug](#) (void)
- void [yyset_debug](#) (int debug_flag)
- YY_EXTRA_TYPE [yyget_extra](#) (void)
- void [yyset_extra](#) (YY_EXTRA_TYPE user_defined)
- FILE * [yyget_in](#) (void)
- void [yyset_in](#) (FILE *_in_str)
- FILE * [yyget_out](#) (void)
- void [yyset_out](#) (FILE *_out_str)
- int [yyget_leng](#) (void)
- char * [yyget_text](#) (void)
- int [yyget_lineno](#) (void)
- void [yyset_lineno](#) (int _line_number)
- if (!(yy_init))

Variables

- int [yyleng](#)
- FILE * [yyin](#) = NULL
- FILE * [yyout](#) = NULL
- int [yylineno](#) = 1
- char * [yytext](#)
- int [yy_flex_debug](#) = 0
- YY_DECL
- char * [yy_cp](#)
- char * [yy_bp](#)
- int [yy_act](#)

6.30.1 Macro Definition Documentation

6.30.1.1 BEGIN

```
#define BEGIN (yy_start) = 1 + 2 *
```


6.30.1.2 ECHO

```
#define ECHO do { if (fwrite( yytext, (size_t) yyleng, 1, yyout )) {} } while (0)
```

6.30.1.3 EOB_ACT_CONTINUE_SCAN

```
#define EOB_ACT_CONTINUE_SCAN 0
```

6.30.1.4 EOB_ACT_END_OF_FILE

```
#define EOB_ACT_END_OF_FILE 1
```

6.30.1.5 EOB_ACT_LAST_MATCH

```
#define EOB_ACT_LAST_MATCH 2
```

6.30.1.6 FLEX_BETA

```
#define FLEX_BETA
```

6.30.1.7 FLEX_SCANNER

```
#define FLEX_SCANNER
```

6.30.1.8 FLEXINT_H

```
#define FLEXINT_H
```

6.30.1.9 INITIAL

```
#define INITIAL 0
```

6.30.1.10 INT16_MAX

```
#define INT16_MAX (32767)
```

6.30.1.11 INT16_MIN

```
#define INT16_MIN (-32767-1)
```

6.30.1.12 INT32_MAX

```
#define INT32_MAX (2147483647)
```

6.30.1.13 INT32_MIN

```
#define INT32_MIN (-2147483647-1)
```

6.30.1.14 INT8_MAX

```
#define INT8_MAX (127)
```

6.30.1.15 INT8_MIN

```
#define INT8_MIN (-128)
```

6.30.1.16 REJECT

```
#define REJECT reject_used_but_not_detected
```

6.30.1.17 SIZE_MAX

```
#define SIZE_MAX (~(size_t)0)
```

6.30.1.18 UINT16_MAX

```
#define UINT16_MAX (65535U)
```

6.30.1.19 UINT32_MAX

```
#define UINT32_MAX (4294967295U)
```

6.30.1.20 UINT8_MAX

```
#define UINT8_MAX (255U)
```

6.30.1.21 unput

```
#define unput(  
    c ) yyunput( c, (yytext_ptr) )
```

6.30.1.22 YY_AT_BOL

```
#define YY_AT_BOL( ) (YY_CURRENT_BUFFER_LVALUE->yy_at_bol)
```

6.30.1.23 YY_BREAK

```
#define YY_BREAK /*LINTED*/break;
```

6.30.1.24 YY_BUF_SIZE

```
#define YY_BUF_SIZE 16384
```

6.30.1.25 YY_BUFFER_EOF_PENDING

```
#define YY_BUFFER_EOF_PENDING 2
```

6.30.1.26 YY_BUFFER_NEW

```
#define YY_BUFFER_NEW 0
```

6.30.1.27 YY_BUFFER_NORMAL

```
#define YY_BUFFER_NORMAL 1
```

6.30.1.28 YY_CURRENT_BUFFER

```
#define YY_CURRENT_BUFFER
```

Value:

```
( (yy_buffer_stack) \
? (yy_buffer_stack)[(yy_buffer_stack_top)] \
: NULL)
```

6.30.1.29 YY_CURRENT_BUFFER_LVALUE

```
#define YY_CURRENT_BUFFER_LVALUE (yy_buffer_stack)[(yy_buffer_stack_top)]
```

6.30.1.30 YY_DECL

```
#define YY_DECL int yylex (void)
```

6.30.1.31 YY_DECL_IS_OURS

```
#define YY_DECL_IS_OURS 1
```

6.30.1.32 YY_DO_BEFORE_ACTION

```
#define YY_DO_BEFORE_ACTION
```

Value:

```
(yytext_ptr) = yy_bp; \
yyleng = (int) (yy_cp - yy_bp); \
(yy_hold_char) = *yy_cp; \
*yy_cp = '\0'; \
(yy_c_buf_p) = yy_cp;
```

6.30.1.33 YY_END_OF_BUFFER

```
#define YY_END_OF_BUFFER 29
```

6.30.1.34 YY_END_OF_BUFFER_CHAR

```
#define YY_END_OF_BUFFER_CHAR 0
```

6.30.1.35 YY_EXIT_FAILURE

```
#define YY_EXIT_FAILURE 2
```

6.30.1.36 YY_EXTRA_TYPE

```
#define YY_EXTRA_TYPE void *
```

6.30.1.37 YY_FATAL_ERROR

```
#define YY_FATAL_ERROR(  
    msg ) yy_fatal_error( msg )
```

6.30.1.38 YY_FLEX_MAJOR_VERSION

```
#define YY_FLEX_MAJOR_VERSION 2
```

6.30.1.39 YY_FLEX_MINOR_VERSION

```
#define YY_FLEX_MINOR_VERSION 6
```

6.30.1.40 YY_FLEX_SUBMINOR_VERSION

```
#define YY_FLEX_SUBMINOR_VERSION 4
```

6.30.1.41 YY_FLUSH_BUFFER

```
#define YY_FLUSH_BUFFER yy_flush_buffer( YY_CURRENT_BUFFER )
```

6.30.1.42 YY_INPUT

```
#define YY_INPUT(
    buf,
    result,
    max_size )
```

Value:

```
if ( YY_CURRENT_BUFFER_LVALUE->yy_is_interactive ) \
{ \
    int c = '*'; \
    int n; \
    for ( n = 0; n < max_size && \
          (c = getc( yyin )) != EOF && c != '\n'; ++n ) \
        buf[n] = (char) c; \
    if ( c == '\n' ) \
        buf[n++] = (char) c; \
    if ( c == EOF && ferror( yyin ) ) \
        YY_FATAL_ERROR( "input in flex scanner failed" ); \
    result = n; \
} \
else \
{ \
    errno=0; \
    while ( (result = (int) fread(buf, 1, (yy_size_t) max_size, yyin)) == 0 && ferror(yyin)) \
    { \
        if( errno != EINTR ) \
        { \
            YY_FATAL_ERROR( "input in flex scanner failed" ); \
            break; \
        } \
        errno=0; \
        clearerr(yyin); \
    } \
} \
\
```

6.30.1.43 YY_INT_ALIGNED

```
#define YY_INT_ALIGNED short int
```

6.30.1.44 YY_LESS_LINENO

```
#define YY_LESS_LINENO(  
    n )
```

6.30.1.45 YY_LINENO_REWIND_TO

```
#define YY_LINENO_REWIND_TO(  
    ptr )
```

6.30.1.46 YY_MORE_ADJ

```
#define YY_MORE_ADJ 0
```

6.30.1.47 yy_new_buffer

```
#define yy_new_buffer yy\_create\_buffer
```

6.30.1.48 YY_NEW_FILE

```
#define YY_NEW_FILE yyrestart( yyin )
```

6.30.1.49 YY_NULL

```
#define YY_NULL 0
```

6.30.1.50 YY_NUM_RULES

```
#define YY_NUM_RULES 28
```

6.30.1.51 YY_READ_BUF_SIZE

```
#define YY_READ_BUF_SIZE 8192
```

6.30.1.52 YY_RESTORE_YY_MORE_OFFSET

```
#define YY_RESTORE_YY_MORE_OFFSET
```

6.30.1.53 YY_RULE_SETUP

```
#define YY_RULE_SETUP  YY_USER_ACTION
```

6.30.1.54 YY_SC_TO_UI

```
#define YY_SC_TO_UI(  
    c ) ((YY_CHAR) (c))
```

6.30.1.55 yy_set_bol

```
#define yy_set_bol(  
    at_bol )
```

Value:

```
{ \
  if ( ! YY_CURRENT_BUFFER ){ \
    yyensure_buffer_stack (); \
    YY_CURRENT_BUFFER_LVALUE = \
      yy_create_buffer( yyin, YY_BUF_SIZE ); \
  } \
  YY_CURRENT_BUFFER_LVALUE->yy_at_bol = at_bol; \
}
```

6.30.1.56 yy_set_interactive

```
#define yy_set_interactive(  
    is_interactive )
```

Value:

```
{ \
  if ( ! YY_CURRENT_BUFFER ){ \
    yyensure_buffer_stack (); \
    YY_CURRENT_BUFFER_LVALUE = \
      yy_create_buffer( yyin, YY_BUF_SIZE ); \
  } \
  YY_CURRENT_BUFFER_LVALUE->yy_is_interactive = is_interactive; \
}
```


6.30.1.57 YY_SKIP_YYWRAP

```
#define YY_SKIP_YYWRAP
```

6.30.1.58 YY_START

```
#define YY_START (((yy_start) - 1) / 2)
```

6.30.1.59 YY_START_STACK_INCR

```
#define YY_START_STACK_INCR 25
```

6.30.1.60 YY_STATE_BUF_SIZE

```
#define YY_STATE_BUF_SIZE ((YY_BUF_SIZE + 2) * sizeof(yy_state_type))
```

6.30.1.61 YY_STATE_EOF

```
#define YY_STATE_EOF(  
    state ) (YY_END_OF_BUFFER + state + 1)
```

6.30.1.62 YY_STRUCT_YY_BUFFER_STATE

```
#define YY_STRUCT_YY_BUFFER_STATE
```

6.30.1.63 YY_TYPEDEF_YY_BUFFER_STATE

```
#define YY_TYPEDEF_YY_BUFFER_STATE
```

6.30.1.64 YY_TYPEDEF_YY_SIZE_T

```
#define YY_TYPEDEF_YY_SIZE_T
```

6.30.1.65 YY_USER_ACTION

```
#define YY_USER_ACTION
```

6.30.1.66 yyconst

```
#define yyconst const
```

6.30.1.67 yyless [1/2]

```
#define yyless(  
    n )
```

Value:

```
do \
{ \
    /* Undo effects of setting up yytext. */ \
    int yyless_macro_arg = (n); \
    YY_LESS_LINENO(yyless_macro_arg); \
    *yy_cp = (yy_hold_char); \
    YY_RESTORE_YY_MORE_OFFSET \
    (yy_c_buf_p) = yy_cp = yy_bp + yyless_macro_arg - YY_MORE_ADJ; \
    YY_DO_BEFORE_ACTION; /* set up yytext again */ \
} \
while ( 0 )
```

6.30.1.68 yyless [2/2]

```
#define yyless(  
    n )
```

Value:

```
do \
{ \
    /* Undo effects of setting up yytext. */ \
    int yyless_macro_arg = (n); \
    YY_LESS_LINENO(yyless_macro_arg); \
    yytext[yy_leng] = (yy_hold_char); \
    (yy_c_buf_p) = yytext + yyless_macro_arg; \
    (yy_hold_char) = *(yy_c_buf_p); \
    *(yy_c_buf_p) = '\0'; \
    yy_leng = yyless_macro_arg; \
} \
while ( 0 )
```

6.30.1.69 yymore

```
#define yymore( ) yymore_used_but_not_detected
```

6.30.1.70 yynoreturn

```
#define yynoreturn
```

6.30.1.71 YYSTATE

```
#define YYSTATE YY_START
```

6.30.1.72 YYTABLES_NAME

```
#define YYTABLES_NAME "yytables"
```

6.30.1.73 yyterminate

```
#define yyterminate( ) return YY_NULL
```

6.30.1.74 yytext_ptr

```
#define yytext_ptr yytext
```

6.30.1.75 yywrap

```
#define yywrap( ) (/*CONSTCOND*/1)
```

6.30.2 Typedef Documentation

6.30.2.1 flex_int16_t

```
typedef short int flex_int16_t
```

6.30.2.2 flex_int32_t

```
typedef int flex_int32_t
```

6.30.2.3 flex_int8_t

```
typedef signed char flex_int8_t
```

6.30.2.4 flex_uint16_t

```
typedef unsigned short int flex_uint16_t
```

6.30.2.5 flex_uint32_t

```
typedef unsigned int flex_uint32_t
```

6.30.2.6 flex_uint8_t

```
typedef unsigned char flex_uint8_t
```

6.30.2.7 YY_BUFFER_STATE

```
typedef struct yy_buffer_state* YY_BUFFER_STATE
```

6.30.2.8 YY_CHAR

```
typedef flex_uint8_t YY_CHAR
```

6.30.2.9 yy_size_t

```
typedef size_t yy_size_t
```

6.30.2.10 yy_state_type

```
typedef int yy_state_type
```

6.30.3 Function Documentation

6.30.3.1 if()

```
if (
    ! yy_init )
```

6.30.3.2 yy_create_buffer()

```
YY_BUFFER_STATE yy_create_buffer (
    FILE * file,
    int size )
```

6.30.3.3 yy_delete_buffer()

```
void yy_delete_buffer (
    YY_BUFFER_STATE b )
```

6.30.3.4 yy_flush_buffer()

```
void yy_flush_buffer (
    YY_BUFFER_STATE b )
```

6.30.3.5 yy_scan_buffer()

```
YY_BUFFER_STATE yy_scan_buffer (
    char * base,
    yy_size_t size )
```

6.30.3.6 yy_scan_bytes()

```
YY_BUFFER_STATE yy_scan_bytes (
    const char * bytes,
    int len )
```

6.30.3.7 yy_scan_string()

```
YY_BUFFER_STATE yy_scan_string (
    const char * yy_str )
```

6.30.3.8 yy_switch_to_buffer()

```
void yy_switch_to_buffer (
    YY_BUFFER_STATE new_buffer )
```

6.30.3.9 yyalloc()

```
void* yyalloc (
    yy_size_t )
```

6.30.3.10 yyfree()

```
void yyfree (
    void * )
```

6.30.3.11 yyget_debug()

```
int yyget_debug (
    void )
```

6.30.3.12 yyget_extra()

```
YY_EXTRA_TYPE yyget_extra (
    void )
```

6.30.3.13 yyget_in()

```
FILE* yyget_in (
    void )
```

6.30.3.14 yyget_leng()

```
int yyget_leng (
    void )
```

6.30.3.15 yyget_lineno()

```
int yyget_lineno (
    void )
```

6.30.3.16 yyget_out()

```
FILE* yyget_out (
    void )
```

6.30.3.17 yyget_text()

```
char* yyget_text (
    void )
```

6.30.3.18 yylex()

```
int yylex ( )
```

6.30.3.19 yylex_destroy()

```
int yylex_destroy (
    void )
```

6.30.3.20 yypop_buffer_state()

```
void yypop_buffer_state (
    void )
```

6.30.3.21 yypush_buffer_state()

```
void yypush_buffer_state (
    YY_BUFFER_STATE new_buffer )
```

6.30.3.22 yyrealloc()

```
void* yyrealloc (
    void * ,
    yy_size_t )
```

6.30.3.23 yyrestart()

```
void yyrestart (
    FILE * input_file )
```

6.30.3.24 yyset_debug()

```
void yyset_debug (
    int debug_flag )
```

6.30.3.25 yyset_extra()

```
void yyset_extra (
    YY_EXTRA_TYPE user_defined )
```


6.30.3.26 yyset_in()

```
void yyset_in (
    FILE * _in_str )
```

6.30.3.27 yyset_lineno()

```
void yyset_lineno (
    int _line_number )
```

6.30.3.28 yyset_out()

```
void yyset_out (
    FILE * _out_str )
```

6.30.4 Variable Documentation

6.30.4.1 yy_act

```
int yy_act
```

6.30.4.2 yy_bp

```
char * yy_bp
```

6.30.4.3 yy_cp

```
char* yy_cp
```

6.30.4.4 YY_DECL

```
YY_DECL
```

Initial value:

```
{  
    yy_state_type yy_current_state
```

The main scanner function which does all the work.

6.30.4.5 yy_flex_debug

```
int yy_flex_debug = 0
```

6.30.4.6 yyin

```
FILE* yyin = NULL
```

6.30.4.7 yyleng

```
int yyleng
```

6.30.4.8 yylineno

```
int yylineno = 1
```

6.30.4.9 yyout

```
FILE * yyout = NULL
```

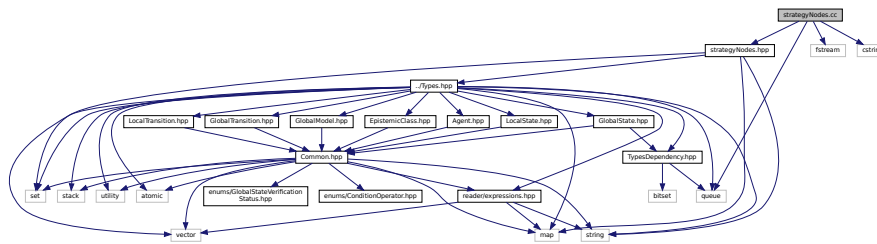
6.30.4.10 yytext

```
char* yytext
```

6.31 scanner.I File Reference

6.32 strategyNodes.cc File Reference

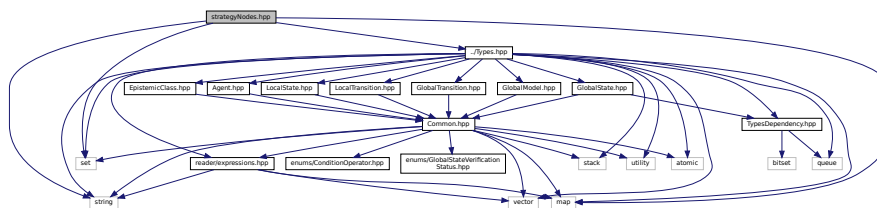
```
#include "strategyNodes.hpp"
#include <queue>
#include <fstream>
#include <cstring>
Include dependency graph for strategyNodes.cc:
```



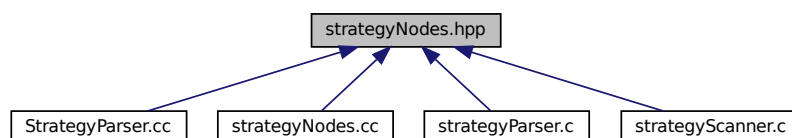
6.33 strategyNodes.hpp File Reference

Strategy parser templates. Class for setting up a new objects from a parser.

```
#include <string>
#include <set>
#include <map>
#include "../Types.hpp"
Include dependency graph for strategyNodes.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

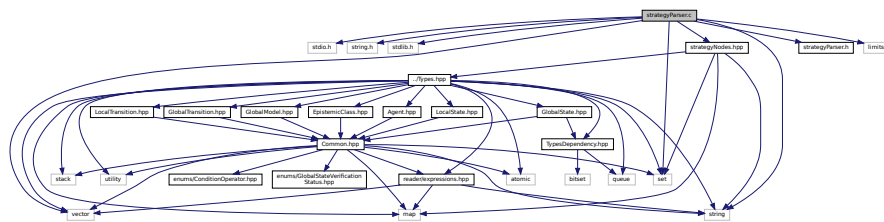
- class [ActionTemplate](#)
Represents a selected action.
- class [StrategyCollectionTemplate](#)

6.33.1 Detailed Description

Strategy parser templates. Class for setting up a new objects from a parser.

6.34 strategyParser.c File Reference

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <string>
#include <set>
#include <vector>
#include "strategyNodes.hpp"
#include "strategyParser.h"
#include <limits.h>
Include dependency graph for strategyParser.c:
```



Classes

- union [yyalloc](#)

Macros

- #define [YYBISON](#) 30802
- #define [YYBISON_VERSION](#) "3.8.2"
- #define [YYSKELETON_NAME](#) "yacc.c"
- #define [YYPURE](#) 0
- #define [YYPUSH](#) 0
- #define [YYPULL](#) 1
- #define [YYSTYPE](#) STRATSTYPE
- #define [yyparse](#) stratparse
- #define [yylex](#) stratlex
- #define [yyerror](#) stratererror
- #define [yydebug](#) stratdebug

- #define `yynerrs` `stratnerrs`
- #define `yylval` `stratival`
- #define `yychar` `stratchar`
- #define `YY_CAST`(Type, Val) ((Type) (Val))
- #define `YY_REINTERPRET_CAST`(Type, Val) ((Type) (Val))
- #define `YY_NULLPTR` ((void*)0)
- #define `YYPTRDIFF_T` `long`
- #define `YYPTRDIFF_MAXIMUM` `LONG_MAX`
- #define `YYSIZE_T` `unsigned`
- #define `YYSIZE_MAXIMUM`
- #define `YYSIZEOF`(X) `YY_CAST (YYPTRDIFF_T, sizeof (X))`
- #define `YY_`(Msgid) `Msgid`
- #define `YY_ATTRIBUTE_PURE`
- #define `YY_ATTRIBUTE_UNUSED`
- #define `YY_USE`(E) ((void) (E))
- #define `YY_INITIAL_VALUE`(Value) `Value`
- #define `YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN`
- #define `YY_IGNORE_MAYBE_UNINITIALIZED_END`
- #define `YY_IGNORE_USELESS_CAST_BEGIN`
- #define `YY_IGNORE_USELESS_CAST_END`
- #define `YY_ASSERT`(E) ((void) (0 && (E)))
- #define `YYSTACK_ALLOC` `YMALLOC`
- #define `YYSTACK_FREE` `YFREE`
- #define `YYSTACK_ALLOC_MAXIMUM` `YYSIZE_MAXIMUM`
- #define `YMALLOC` `malloc`
- #define `YFREE` `free`
- #define `YYSTACK_GAP_MAXIMUM` `(YYSIZEOF (union yyalloc) - 1)`
- #define `YYSTACK_BYTES`(N)
- #define `YYCOPY_NEEDED` `1`
- #define `YYSTACK_RELOCATE`(Stack_alloc, Stack)
- #define `YYCOPY`(Dst, Src, Count)
- #define `YYFINAL` `2`
- #define `YYLAST` `16`
- #define `YYNTOKENS` `18`
- #define `YYNNTS` `3`
- #define `YYNRULES` `5`
- #define `YYNSTATES` `9`
- #define `YYMAXUTOK` `271`
- #define `YYTRANSLATE`(YYX)
- #define `YY_ACCESSING_SYMBOL`(State) `YY_CAST (yy_symbol_kind_t, yystos[State])`
- #define `YYPACT_NINF` `(-15)`
- #define `yypact_value_is_default`(Yyn) ((Yyn) == `YYPACT_NINF`)
- #define `YYTABLE_NINF` `(-1)`
- #define `yytable_value_is_error`(Yyn) `0`
- #define `yyerrok` (`yyerrstatus = 0`)
- #define `yyclearin` (`yychar = STRATEMPTY`)
- #define `YYACCEPT` `goto yyacceptlab`
- #define `YYABORT` `goto yyabortlab`
- #define `YYERROR` `goto yyerrorlab`
- #define `YYNOMEM` `goto yyexhaustedlab`
- #define `YYRECOVERING`() (!`yyerrstatus`)
- #define `YYBACKUP`(Token, Value)
- #define `YYERRCODE` `STRATUNDEF`
- #define `YYDPRINTF`(Args) ((void) 0)
- #define `YY_SYMBOL_PRINT`(Title, Kind, Value, Location)

- `#define YY_STACK_PRINT`(Bottom, Top)
- `#define YY_REDUCE_PRINT`(Rule)
- `#define YYINITDEPTH` 200
- `#define YYMAXDEPTH` 10000
- `#define YYPOPSTACK`(N) (yyvsp -= (N), yyssp -= (N))

Typedefs

- `typedef struct strat_buffer_state * YY_BUFFER_STATE`
- `typedef enum yysymbol_kind_t yysymbol_kind_t`
- `typedef signed char yytype_int8`
- `typedef short yytype_int16`
- `typedef unsigned char yytype_uint8`
- `typedef unsigned short yytype_uint16`
- `typedef yytype_int8 yy_state_t`
- `typedef int yy_state_fast_t`

Enumerations

- `enum yysymbol_kind_t {`
`YYSYMBOL_YEMPTY = -2, YYSYMBOL_YEOF = 0, YYSYMBOL_YError = 1, YYSYMBOL_YUNDEF`
`= 2,`
`YYSYMBOL_T_AGENT = 3, YYSYMBOL_T_INIT = 4, YYSYMBOL_T_LOCAL = 5, YYSYMBOL_T_INITIAL`
`= 6,`
`YYSYMBOL_T_FORMULA = 7, YYSYMBOL_T_PERSISTENT = 8, YYSYMBOL_T_TO = 9,`
`YYSYMBOL_T_ASSIGN = 10,`
`YYSYMBOL_T_OR = 11, YYSYMBOL_T_AND = 12, YYSYMBOL_T_EQ = 13, YYSYMBOL_T_NE = 14,`
`YYSYMBOL_T_GT = 15, YYSYMBOL_T_GE = 16, YYSYMBOL_T_LT = 17, YYSYMBOL_T_LE = 18,`
`YYSYMBOL_T_NUM = 19, YYSYMBOL_T_SHARED = 20, YYSYMBOL_T_FLOAT = 21, YYSYMBOL_T_IDENT`
`= 22,`
`YYSYMBOL_T_KNOW = 23, YYSYMBOL_T_HARTLEY = 24, YYSYMBOL_T_PROB = 25,`
`YYSYMBOL_T_REST = 26,`
`YYSYMBOL_27_ = 27, YYSYMBOL_28_ = 28, YYSYMBOL_29_ = 29, YYSYMBOL_30_ = 30,`
`YYSYMBOL_31_ = 31, YYSYMBOL_32_ = 32, YYSYMBOL_33_ = 33, YYSYMBOL_34_ = 34,`
`YYSYMBOL_35_ = 35, YYSYMBOL_36_ = 36, YYSYMBOL_37_ = 37, YYSYMBOL_38_ = 38,`
`YYSYMBOL_YACCEPT = 39, YYSYMBOL_model = 40, YYSYMBOL_spec = 41, YYSYMBOL_query =`
`42,`
`YYSYMBOL_formula = 43, YYSYMBOL_probability_equality = 44, YYSYMBOL_cond_list = 45,`
`YYSYMBOL_coalition = 46,`
`YYSYMBOL_agent = 47, YYSYMBOL_description = 48, YYSYMBOL_local = 49, YYSYMBOL_persistent`
`= 50,`
`YYSYMBOL_ident_list = 51, YYSYMBOL_initial = 52, YYSYMBOL_assign_list = 53, YYSYMBOL_assignment`
`= 54,`
`YYSYMBOL_transition = 55, YYSYMBOL_shared = 56, YYSYMBOL_assignments = 57, YYSYMBOL_num_exp`
`= 58,`
`YYSYMBOL_num_mul = 59, YYSYMBOL_num_elem = 60, YYSYMBOL_prob_exp = 61, YYSYMBOL_prob_mul`
`= 62,`
`YYSYMBOL_prob_elem = 63, YYSYMBOL_condition = 64, YYSYMBOL_cond = 65, YYSYMBOL_cond_and`
`= 66,`
`YYSYMBOL_cond_elem = 67, YYSYMBOL_YEMPTY = -2, YYSYMBOL_YEOF = 0, YYSYMBOL_YError`
`= 1,`
`YYSYMBOL_YUNDEF = 2, YYSYMBOL_T_TO = 3, YYSYMBOL_T_ASSIGN = 4, YYSYMBOL_T_OR =`
`5,`
`YYSYMBOL_T_AND = 6, YYSYMBOL_T_EQ = 7, YYSYMBOL_T_NE = 8, YYSYMBOL_T_GT = 9,`
`YYSYMBOL_T_GE = 10, YYSYMBOL_T_LT = 11, YYSYMBOL_T_LE = 12, YYSYMBOL_T_NUM = 13,`
`}`

- ```

YYSYMBOL_T_SHARED = 14 , YYSYMBOL_T_FLOAT = 15 , YYSYMBOL_T_IDENT = 16 ,
YYSYMBOL_17_ = 17 ,
YYSYMBOL_YYACCEPT = 18 , YYSYMBOL_strat = 19 , YYSYMBOL_ident_list = 20 }

```
- enum { YYENOMEM = -2 }

## Functions

- int [stratlex](#) ()
- void [stratrestart](#) (FILE \*)
- void [straterror](#) (const char \*)
- YY\_BUFFER\_STATE [strat\\_scan\\_string](#) (const char \*str)
- void \* [malloc](#) (YYSIZE\_T)
- void [free](#) (void \*)
- void [set\\_input\\_string\\_strat](#) (const char \*in)

## Variables

- char \* [strattext](#)
- [StrategyCollectionTemplate](#) [strategyCollectionTemplate](#)
- int [yychar](#)
- YYSTYPE [yylval](#)
- int [yynerrs](#)

### 6.34.1 Macro Definition Documentation

#### 6.34.1.1 YY\_

```

#define YY_(
 Msgid) Msgid

```

#### 6.34.1.2 YY\_ACCESSING\_SYMBOL

```

#define YY_ACCESSING_SYMBOL(
 State) YY_CAST (yysymbol_kind_t, yystos[State])

```

Accessing symbol of state STATE.

#### 6.34.1.3 YY\_ASSERT

```

#define YY_ASSERT(
 E) ((void) (0 && (E)))

```

#### 6.34.1.4 YY\_ATTRIBUTE\_PURE

```
#define YY_ATTRIBUTE_PURE
```

#### 6.34.1.5 YY\_ATTRIBUTE\_UNUSED

```
#define YY_ATTRIBUTE_UNUSED
```

#### 6.34.1.6 YY\_CAST

```
#define YY_CAST(
 Type,
 Val) ((Type) (Val))
```

#### 6.34.1.7 YY\_IGNORE\_MAYBE\_UNINITIALIZED\_BEGIN

```
#define YY_IGNORE_MAYBE_UNINITIALIZED_BEGIN
```

#### 6.34.1.8 YY\_IGNORE\_MAYBE\_UNINITIALIZED\_END

```
#define YY_IGNORE_MAYBE_UNINITIALIZED_END
```

#### 6.34.1.9 YY\_IGNORE\_USELESS\_CAST\_BEGIN

```
#define YY_IGNORE_USELESS_CAST_BEGIN
```

#### 6.34.1.10 YY\_IGNORE\_USELESS\_CAST\_END

```
#define YY_IGNORE_USELESS_CAST_END
```



#### 6.34.1.11 YY\_INITIAL\_VALUE

```
#define YY_INITIAL_VALUE(
 Value) Value
```

#### 6.34.1.12 YY\_NULLPTR

```
#define YY_NULLPTR ((void*)0)
```

#### 6.34.1.13 YY\_REDUCE\_PRINT

```
#define YY_REDUCE_PRINT(
 Rule)
```

#### 6.34.1.14 YY\_REINTERPRET\_CAST

```
#define YY_REINTERPRET_CAST(
 Type,
 Val) ((Type) (Val))
```

#### 6.34.1.15 YY\_STACK\_PRINT

```
#define YY_STACK_PRINT(
 Bottom,
 Top)
```

#### 6.34.1.16 YY\_SYMBOL\_PRINT

```
#define YY_SYMBOL_PRINT(
 Title,
 Kind,
 Value,
 Location)
```

#### 6.34.1.17 YY\_USE

```
#define YY_USE(
 E) ((void) (E))
```

#### 6.34.1.18 YYABORT

```
#define YYABORT goto yyabortlab
```

#### 6.34.1.19 YYACCEPT

```
#define YYACCEPT goto yyacceptlab
```

#### 6.34.1.20 YYBACKUP

```
#define YYBACKUP(
 Token,
 Value)
```

##### Value:

```
do
 if (yychar == STRATEMPTY)
 {
 yychar = (Token);
 yylval = (Value);
 YYPOPSTACK (yylen);
 yystate = *yyssp;
 goto yybackup;
 }
 else
 {
 yyerror (YY_("syntax error: cannot back up")); \
 YYERROR;
 }
while (0)
```

#### 6.34.1.21 YYBISON

```
#define YYBISON 30802
```

#### 6.34.1.22 YYBISON\_VERSION

```
#define YYBISON_VERSION "3.8.2"
```

### 6.34.1.23 yychar

```
#define yychar stratchar
```

### 6.34.1.24 yyclearin

```
#define yyclearin (yychar = STRATEMPTY)
```

### 6.34.1.25 YYCOPY

```
#define YYCOPY(
 Dst,
 Src,
 Count)
```

Value:

```
do
{
 YYPTRDIFF_T yyi;
 for (yyi = 0; yyi < (Count); yyi++)
 (Dst)[yyi] = (Src)[yyi];
}
while (0)
```

```
\\
\\
\\
\\
\\
```

### 6.34.1.26 YYCOPY\_NEEDED

```
#define YYCOPY_NEEDED 1
```

### 6.34.1.27 yydebug

```
#define yydebug stratdebug
```

### 6.34.1.28 YYDPRINTF

```
#define YYDPRINTF(
 Args) ((void) 0)
```

#### 6.34.1.29 YYERRCODE

```
#define YYERRCODE STRATUNDEF
```

#### 6.34.1.30 yyerrok

```
#define yyerrok (yyerrstatus = 0)
```

#### 6.34.1.31 yyerror

```
#define yyerror straterror
```

#### 6.34.1.32 YYERROR

```
#define YYERROR goto yyerrorlab
```

#### 6.34.1.33 YYFINAL

```
#define YYFINAL 2
```

#### 6.34.1.34 YYFREE

```
#define YYFREE free
```

#### 6.34.1.35 YYINITDEPTH

```
#define YYINITDEPTH 200
```

#### 6.34.1.36 YYLAST

```
#define YYLAST 16
```

**6.34.1.37 yylex**

```
int yylex stratlex
```

**6.34.1.38 yylval**

```
#define yylval stratlval
```

**6.34.1.39 YYMALLOC**

```
#define YYMALLOC malloc
```

**6.34.1.40 YYMAXDEPTH**

```
#define YYMAXDEPTH 10000
```

**6.34.1.41 YYMAXUTOK**

```
#define YYMAXUTOK 271
```

**6.34.1.42 yynerrs**

```
#define yynerrs stratnerrs
```

**6.34.1.43 YYNNTS**

```
#define YYNNTS 3
```

**6.34.1.44 YYNOMEM**

```
#define YYNOMEM goto yyexhaustedlab
```

#### 6.34.1.45 YYNRULES

```
#define YYNRULES 5
```

#### 6.34.1.46 YYNSTATES

```
#define YYNSTATES 9
```

#### 6.34.1.47 YYNTOKENS

```
#define YYNTOKENS 18
```

#### 6.34.1.48 YYPACT\_NINF

```
#define YYPACT_NINF (-15)
```

#### 6.34.1.49 yypact\_value\_is\_default

```
#define yypact_value_is_default(
 Yyn) ((Yyn) == YYPACT_NINF)
```

#### 6.34.1.50 yyparse

```
int yyparse(
 void) stratparse
```

#### 6.34.1.51 YYPOPSTACK

```
#define YYPOPSTACK(
 N) (yyvsp -= (N), yyssp -= (N))
```

#### 6.34.1.52 YPTRDIFF\_MAXIMUM

```
#define YPTRDIFF_MAXIMUM LONG_MAX
```

#### 6.34.1.53 YPTRDIFF\_T

```
#define YPTRDIFF_T long
```

#### 6.34.1.54 YYPULL

```
#define YYPULL 1
```

#### 6.34.1.55 YYPURE

```
#define YYPURE 0
```

#### 6.34.1.56 YYPUSH

```
#define YYPUSH 0
```

#### 6.34.1.57 YYRECOVERING

```
#define YYRECOVERING() (!yyerrstatus)
```

#### 6.34.1.58 YYSIZE\_MAXIMUM

```
#define YYSIZE_MAXIMUM
```

#### Value:

```
YY_CAST (YPTRDIFF_T,
 (YPTRDIFF_MAXIMUM < YY_CAST (YYSIZE_T, -1)
 ? YPTRDIFF_MAXIMUM
 : YY_CAST (YYSIZE_T, -1)))
```

#### 6.34.1.59 YYSIZE\_T

```
#define YYSIZE_T unsigned
```

#### 6.34.1.60 YYSIZEOF

```
#define YYSIZEOF(
 X) YY_CAST (YYPTRDIFF_T, sizeof (X))
```

#### 6.34.1.61 YYSKELETON\_NAME

```
#define YYSKELETON_NAME "yacc.c"
```

#### 6.34.1.62 YYSTACK\_ALLOC

```
#define YYSTACK_ALLOC YYMALLOC
```

#### 6.34.1.63 YYSTACK\_ALLOC\_MAXIMUM

```
#define YYSTACK_ALLOC_MAXIMUM YYSIZE_MAXIMUM
```

#### 6.34.1.64 YYSTACK\_BYTES

```
#define YYSTACK_BYTES(
 N)
```

**Value:**

```
(N) * (YYSIZEOF (yy_state_t) + YYSIZEOF (YYSTYPE)) \
+ YYSTACK_GAP_MAXIMUM)
```

#### 6.34.1.65 YYSTACK\_FREE

```
#define YYSTACK_FREE YYFREE
```



**6.34.1.66 YYSTYPE\_GAP\_MAXIMUM**

```
#define YYSTYPE_GAP_MAXIMUM (YYSIZEOF (union yyalloc) - 1)
```

**6.34.1.67 YYSTACK\_RELOCATE**

```
#define YYSTACK_RELOCATE(
 Stack_alloc,
 Stack)
```

**Value:**

```
do
{
 YYPTRDIFF_T yynewbytes;
 YYCOPY (&yyptr->Stack_alloc, Stack, yysize);
 Stack = &yyptr->Stack_alloc;
 yynewbytes = yystacksize * YYSIZEOF (*Stack) + YYSTACK_GAP_MAXIMUM; \
 yyptr += yynewbytes / YYSIZEOF (*yyptr);
}
while (0)
```

**6.34.1.68 YYSTYPE**

```
#define YYSTYPE STRATSTYPE
```

**6.34.1.69 YYTABLE\_NINF**

```
#define YYTABLE_NINF (-1)
```

**6.34.1.70 yytable\_value\_is\_error**

```
#define yytable_value_is_error(
 Yyn) 0
```

**6.34.1.71 YYTRANSLATE**

```
#define YYTRANSLATE(
 YYX)
```

**Value:**

```
(0 <= (YYX) && (YYX) <= YYMAXUTOK \
 ? YY_CAST (yysymbol_kind_t, yytranslate[YYX]) \
 : YYSYMBOL_YYUNDEF)
```

## 6.34.2 Typedef Documentation

### 6.34.2.1 YY\_BUFFER\_STATE

```
typedef struct strat_buffer_state* YY_BUFFER_STATE
```

### 6.34.2.2 yy\_state\_fast\_t

```
typedef int yy_state_fast_t
```

### 6.34.2.3 yy\_state\_t

```
typedef yytype_int8 yy_state_t
```

### 6.34.2.4 yysymbol\_kind\_t

```
typedef enum yysymbol_kind_t yysymbol_kind_t
```

### 6.34.2.5 yytype\_int16

```
typedef short yytype_int16
```

### 6.34.2.6 yytype\_int8

```
typedef signed char yytype_int8
```

### 6.34.2.7 yytype\_uint16

```
typedef unsigned short yytype_uint16
```

### 6.34.2.8 yytype\_uint8

```
typedef unsigned char yytype_uint8
```

## 6.34.3 Enumeration Type Documentation

### 6.34.3.1 anonymous enum

```
anonymous enum
```

## Enumerator

|          |  |
|----------|--|
| YYENOMEM |  |
|----------|--|

## 6.34.3.2 yysymbol\_kind\_t

```
enum yysymbol_kind_t
```

## Enumerator

|                       |  |
|-----------------------|--|
| YYSYMBOL_YEMPTY       |  |
| YYSYMBOL_YEOF         |  |
| YYSYMBOL_YError       |  |
| YYSYMBOL_YUNDEF       |  |
| YYSYMBOL_T_AGENT      |  |
| YYSYMBOL_T_INIT       |  |
| YYSYMBOL_T_LOCAL      |  |
| YYSYMBOL_T_INITIAL    |  |
| YYSYMBOL_T_FORMULA    |  |
| YYSYMBOL_T_PERSISTENT |  |
| YYSYMBOL_T_TO         |  |
| YYSYMBOL_T_ASSIGN     |  |
| YYSYMBOL_T_OR         |  |
| YYSYMBOL_T_AND        |  |
| YYSYMBOL_T_EQ         |  |
| YYSYMBOL_T_NE         |  |
| YYSYMBOL_T_GT         |  |
| YYSYMBOL_T_GE         |  |
| YYSYMBOL_T_LT         |  |
| YYSYMBOL_T_LE         |  |
| YYSYMBOL_T_NUM        |  |
| YYSYMBOL_T_SHARED     |  |
| YYSYMBOL_T_FLOAT      |  |
| YYSYMBOL_T_IDENT      |  |
| YYSYMBOL_T_KNOW       |  |
| YYSYMBOL_T_HARTLEY    |  |
| YYSYMBOL_T_PROB       |  |
| YYSYMBOL_T_REST       |  |
| YYSYMBOL_27_          |  |
| YYSYMBOL_28_          |  |
| YYSYMBOL_29_          |  |
| YYSYMBOL_30_          |  |
| YYSYMBOL_31_          |  |
| YYSYMBOL_32_          |  |
| YYSYMBOL_33_          |  |
| YYSYMBOL_34_          |  |
| YYSYMBOL_35_          |  |
| YYSYMBOL_36_          |  |

## Enumerator

|                               |  |
|-------------------------------|--|
| YYSYMBOL_37_                  |  |
| YYSYMBOL_38_                  |  |
| YYSYMBOL_YYACCEPT             |  |
| YYSYMBOL_model                |  |
| YYSYMBOL_spec                 |  |
| YYSYMBOL_query                |  |
| YYSYMBOL_formula              |  |
| YYSYMBOL_probability_equality |  |
| YYSYMBOL_cond_list            |  |
| YYSYMBOL_coalition            |  |
| YYSYMBOL_agent                |  |
| YYSYMBOL_description          |  |
| YYSYMBOL_local                |  |
| YYSYMBOL_persistent           |  |
| YYSYMBOL_ident_list           |  |
| YYSYMBOL_initial              |  |
| YYSYMBOL_assign_list          |  |
| YYSYMBOL_assignment           |  |
| YYSYMBOL_transition           |  |
| YYSYMBOL_shared               |  |
| YYSYMBOL_assignments          |  |
| YYSYMBOL_num_exp              |  |
| YYSYMBOL_num_mul              |  |
| YYSYMBOL_num_elem             |  |
| YYSYMBOL_prob_exp             |  |
| YYSYMBOL_prob_mul             |  |
| YYSYMBOL_prob_elem            |  |
| YYSYMBOL_condition            |  |
| YYSYMBOL_cond                 |  |
| YYSYMBOL_cond_and             |  |
| YYSYMBOL_cond_elem            |  |
| YYSYMBOL_YYEMPTY              |  |
| YYSYMBOL_YYEOF                |  |
| YYSYMBOL_YYerror              |  |
| YYSYMBOL_YYUNDEF              |  |
| YYSYMBOL_T_TO                 |  |
| YYSYMBOL_T_ASSIGN             |  |
| YYSYMBOL_T_OR                 |  |
| YYSYMBOL_T_AND                |  |
| YYSYMBOL_T_EQ                 |  |
| YYSYMBOL_T_NE                 |  |
| YYSYMBOL_T_GT                 |  |
| YYSYMBOL_T_GE                 |  |
| YYSYMBOL_T_LT                 |  |
| YYSYMBOL_T_LE                 |  |
| YYSYMBOL_T_NUM                |  |
| YYSYMBOL_T_SHARED             |  |
| YYSYMBOL_T_FLOAT              |  |

## Enumerator

|                    |  |
|--------------------|--|
| YYSMBOL_T_IDENT    |  |
| YYSMBOL_17_        |  |
| YYSMBOL_YYACCEPT   |  |
| YYSMBOL_strat      |  |
| YYSMBOL_ident_list |  |

## 6.34.4 Function Documentation

### 6.34.4.1 free()

```
void free (
 void *)
```

### 6.34.4.2 malloc()

```
void* malloc (
 YYSIZE_T)
```

### 6.34.4.3 set\_input\_string\_strat()

```
void set_input_string_strat (
 const char * in)
```

### 6.34.4.4 strat\_scan\_string()

```
YY_BUFFER_STATE strat_scan_string (
 const char * str)
```

### 6.34.4.5 stratererror()

```
void stratererror (
 const char * s)
```

#### 6.34.4.6 stratlex()

```
int stratlex ()
```

#### 6.34.4.7 stratrestart()

```
void stratrestart (
 FILE *)
```

### 6.34.5 Variable Documentation

#### 6.34.5.1 strategyCollectionTemplate

```
StrategyCollectionTemplate strategyCollectionTemplate [extern]
```

#### 6.34.5.2 strattext

```
char* strattext [extern]
```

#### 6.34.5.3 yychar

```
int yychar
```

#### 6.34.5.4 yylval

```
YYSTYPE yylval
```

#### 6.34.5.5 yynerrs

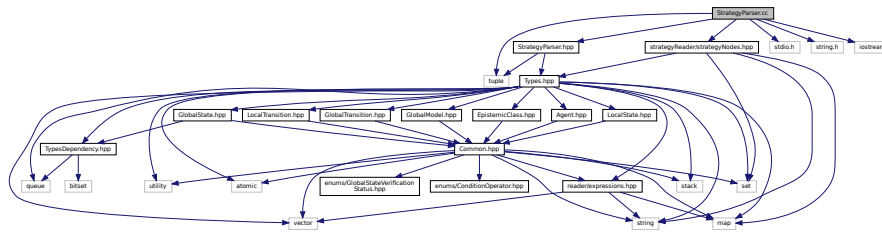
```
int yynerrs
```

## 6.35 StrategyParser.cc File Reference

A strategy file parser. A parser for converting a text file into a strategy.

```
#include "StrategyParser.hpp"
#include "strategyReader/strategyNodes.hpp"
#include <stdio.h>
#include <string.h>
#include <tuple>
#include <iostream>
```

Include dependency graph for StrategyParser.cc:



### Functions

- int [stratparse](#) ()
- void [stratrestart](#) (FILE \*)
- void [set\\_input\\_string\\_strat](#) (const char \*in)

### Variables

- [StrategyCollectionTemplate](#) [strategyCollectionTemplate](#)

#### 6.35.1 Detailed Description

A strategy file parser. A parser for converting a text file into a strategy.

#### 6.35.2 Function Documentation

##### 6.35.2.1 set\_input\_string\_strat()

```
void set_input_string_strat (
 const char * in)
```

### 6.35.2.2 stratparse()

```
int stratparse ()
```

### 6.35.2.3 stratrestart()

```
void stratrestart (
 FILE *)
```

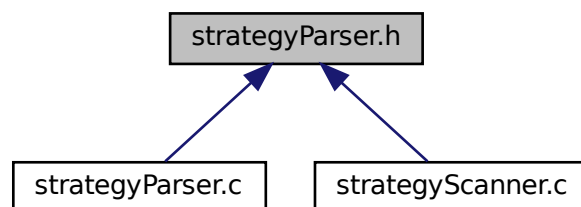
## 6.35.3 Variable Documentation

### 6.35.3.1 strategyCollectionTemplate

[StrategyCollectionTemplate](#) strategyCollectionTemplate

## 6.36 strategyParser.h File Reference

This graph shows which files directly or indirectly include this file:



## Classes

- union [STRATSTYPE](#)

## Macros

- `#define` [STRATDEBUG](#) 0
- `#define` [STRATTOKENTYPE](#)
- `#define` [STRATSTYPE\\_IS\\_TRIVIAL](#) 1
- `#define` [STRATSTYPE\\_IS\\_DECLARED](#) 1



## Typedefs

- typedef enum [strattokentype](#) [strattoken\\_kind\\_t](#)
- typedef union [STRATSTYPE](#) [STRATSTYPE](#)

## Enumerations

- enum [strattokentype](#) {  
    [STRATEMPTY](#) = -2 , [STRATEOF](#) = 0 , [STRATError](#) = 256 , [STRATUNDEF](#) = 257 ,  
    [T\\_TO](#) = 258 , [T\\_ASSIGN](#) = 259 , [T\\_OR](#) = 260 , [T\\_AND](#) = 261 ,  
    [T\\_EQ](#) = 262 , [T\\_NE](#) = 263 , [T\\_GT](#) = 264 , [T\\_GE](#) = 265 ,  
    [T\\_LT](#) = 266 , [T\\_LE](#) = 267 , [T\\_NUM](#) = 268 , [T\\_SHARED](#) = 269 ,  
    [T\\_FLOAT](#) = 270 , [T\\_IDENT](#) = 271 }

## Functions

- int [stratparse](#) (void)

## Variables

- [STRATSTYPE](#) [stratlval](#)

### 6.36.1 Macro Definition Documentation

#### 6.36.1.1 STRATDEBUG

```
#define STRATDEBUG 0
```

#### 6.36.1.2 STRATSTYPE\_IS\_DECLARED

```
#define STRATSTYPE_IS_DECLARED 1
```

#### 6.36.1.3 STRATSTYPE\_IS\_TRIVIAL

```
#define STRATSTYPE_IS_TRIVIAL 1
```

### 6.36.1.4 STRATTOKENTYPE

```
#define STRATTOKENTYPE
```

## 6.36.2 Typedef Documentation

### 6.36.2.1 STRATSTYPE

```
typedef union STRATSTYPE STRATSTYPE
```

### 6.36.2.2 strattoken\_kind\_t

```
typedef enum strattokentype strattoken_kind_t
```

## 6.36.3 Enumeration Type Documentation

### 6.36.3.1 strattokentype

```
enum strattokentype
```

#### Enumerator

|            |  |
|------------|--|
| STRATEMPTY |  |
| STRATEOF   |  |
| STRATError |  |
| STRATUNDEF |  |
| T_TO       |  |
| T_ASSIGN   |  |
| T_OR       |  |
| T_AND      |  |
| T_EQ       |  |
| T_NE       |  |
| T_GT       |  |
| T_GE       |  |
| T_LT       |  |
| T_LE       |  |
| T_NUM      |  |
| T_SHARED   |  |
| T_FLOAT    |  |
| T_IDENT    |  |

## 6.36.4 Function Documentation

### 6.36.4.1 stratparse()

```
int stratparse (
 void)
```

## 6.36.5 Variable Documentation

### 6.36.5.1 stratlval

```
STRATSTYPE stratlval [extern]
```

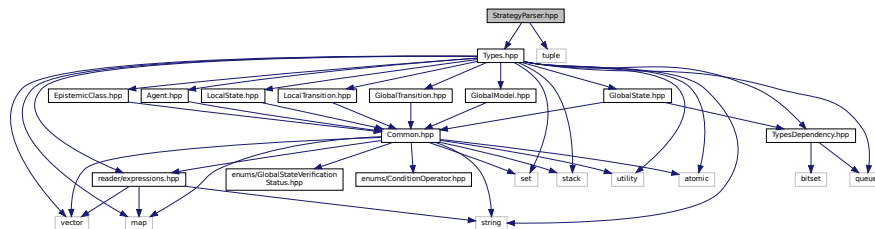
## 6.37 StrategyParser.hpp File Reference

Strategy file parser. A parser for converting a text file into a strategy.

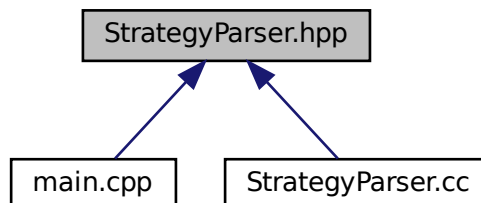
```
#include "Types.hpp"
```

```
#include <tuple>
```

Include dependency graph for StrategyParser.hpp:



This graph shows which files directly or indirectly include this file:





- #define [yypush\\_buffer\\_state](#) stratpush\_buffer\_state
- #define [yypop\\_buffer\\_state](#) stratpop\_buffer\_state
- #define [yyensure\\_buffer\\_stack](#) stratensure\_buffer\_stack
- #define [yy\\_flex\\_debug](#) strat\_flex\_debug
- #define [yyin](#) stratin
- #define [yyleng](#) stratleng
- #define [yylex](#) stratlex
- #define [yylineno](#) stratlineno
- #define [yyout](#) stratout
- #define [yyrestart](#) stratrestart
- #define [yytext](#) strattext
- #define [yywrap](#) stratwrap
- #define [yyalloc](#) stratalloc
- #define [yyrealloc](#) stratrealloc
- #define [yyfree](#) stratfree
- #define [FLEX\\_SCANNER](#)
- #define [YY\\_FLEX\\_MAJOR\\_VERSION](#) 2
- #define [YY\\_FLEX\\_MINOR\\_VERSION](#) 6
- #define [YY\\_FLEX\\_SUBMINOR\\_VERSION](#) 4
- #define [FLEX\\_BETA](#)
- #define [strat\\_create\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [strat\\_delete\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [strat\\_scan\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [strat\\_scan\\_string\\_ALREADY\\_DEFINED](#)
- #define [strat\\_scan\\_bytes\\_ALREADY\\_DEFINED](#)
- #define [strat\\_init\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [strat\\_flush\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [strat\\_load\\_buffer\\_state\\_ALREADY\\_DEFINED](#)
- #define [strat\\_switch\\_to\\_buffer\\_ALREADY\\_DEFINED](#)
- #define [stratpush\\_buffer\\_state\\_ALREADY\\_DEFINED](#)
- #define [stratpop\\_buffer\\_state\\_ALREADY\\_DEFINED](#)
- #define [stratensure\\_buffer\\_stack\\_ALREADY\\_DEFINED](#)
- #define [stratlex\\_ALREADY\\_DEFINED](#)
- #define [stratrestart\\_ALREADY\\_DEFINED](#)
- #define [yylex\\_init](#) stratlex\_init
- #define [yylex\\_init\\_extra](#) stratlex\_init\_extra
- #define [yylex\\_destroy](#) stratlex\_destroy
- #define [yyget\\_debug](#) stratget\_debug
- #define [yyset\\_debug](#) stratset\_debug
- #define [yyget\\_extra](#) stratget\_extra
- #define [yyset\\_extra](#) stratset\_extra
- #define [yyget\\_in](#) stratget\_in
- #define [yyset\\_in](#) stratset\_in
- #define [yyget\\_out](#) stratget\_out
- #define [yyset\\_out](#) stratset\_out
- #define [yyget\\_leng](#) stratget\_leng
- #define [yyget\\_text](#) stratget\_text
- #define [yyget\\_lineno](#) stratget\_lineno
- #define [yyset\\_lineno](#) stratset\_lineno
- #define [stratwrap\\_ALREADY\\_DEFINED](#)
- #define [stratalloc\\_ALREADY\\_DEFINED](#)
- #define [stratrealloc\\_ALREADY\\_DEFINED](#)
- #define [stratfree\\_ALREADY\\_DEFINED](#)
- #define [strattext\\_ALREADY\\_DEFINED](#)
- #define [stratleng\\_ALREADY\\_DEFINED](#)

- `#define stratin_ALREADY_DEFINED`
- `#define stratout_ALREADY_DEFINED`
- `#define strat_flex_debug_ALREADY_DEFINED`
- `#define stratlineno_ALREADY_DEFINED`
- `#define FLEXINT_H`
- `#define INT8_MIN (-128)`
- `#define INT16_MIN (-32767-1)`
- `#define INT32_MIN (-2147483647-1)`
- `#define INT8_MAX (127)`
- `#define INT16_MAX (32767)`
- `#define INT32_MAX (2147483647)`
- `#define UINT8_MAX (255U)`
- `#define UINT16_MAX (65535U)`
- `#define UINT32_MAX (4294967295U)`
- `#define SIZE_MAX (~(size_t)0)`
- `#define yyconst const`
- `#define yynoreturn`
- `#define YY_NULL 0`
- `#define YY_SC_TO_UI(c) ((YY_CHAR) (c))`
- `#define BEGIN (yy_start) = 1 + 2 *`
- `#define YY_START (((yy_start) - 1) / 2)`
- `#define YYSTATE YY_START`
- `#define YY_STATE_EOF(state) (YY_END_OF_BUFFER + state + 1)`
- `#define YY_NEW_FILE yyrestart( yyin )`
- `#define YY_END_OF_BUFFER_CHAR 0`
- `#define YY_BUF_SIZE 16384`
- `#define YY_STATE_BUF_SIZE ((YY_BUF_SIZE + 2) * sizeof(yy_state_type))`
- `#define YY_TYPEDEF_YY_BUFFER_STATE`
- `#define YY_TYPEDEF_YY_SIZE_T`
- `#define EOB_ACT_CONTINUE_SCAN 0`
- `#define EOB_ACT_END_OF_FILE 1`
- `#define EOB_ACT_LAST_MATCH 2`
- `#define YY_LESS_LINENO(n)`
- `#define YY_LINENO_REWIND_TO(ptr)`
- `#define yyless(n)`
- `#define unput(c) yyunput( c, (yytext_ptr) )`
- `#define YY_STRUCT_YY_BUFFER_STATE`
- `#define YY_BUFFER_NEW 0`
- `#define YY_BUFFER_NORMAL 1`
- `#define YY_BUFFER_EOF_PENDING 2`
- `#define YY_CURRENT_BUFFER`
- `#define YY_CURRENT_BUFFER_LVALUE (yy_buffer_stack)[(yy_buffer_stack_top)]`
- `#define YY_FLUSH_BUFFER yy_flush_buffer( YY_CURRENT_BUFFER )`
- `#define yy_new_buffer yy_create_buffer`
- `#define yy_set_interactive(is_interactive)`
- `#define yy_set_bol(at_bol)`
- `#define YY_AT_BOL() (YY_CURRENT_BUFFER_LVALUE->yy_at_bol)`
- `#define stratwrap() (/*CONSTCOND*/1)`
- `#define YY_SKIP_YYWRAP`
- `#define yytext_ptr yytext`
- `#define YY_DO_BEFORE_ACTION`
- `#define YY_NUM_RULES 17`
- `#define YY_END_OF_BUFFER 18`
- `#define REJECT reject_used_but_not_detected`
- `#define yymore() yymore_used_but_not_detected`

- `#define YY_MORE_ADJ 0`
- `#define YY_RESTORE_YY_MORE_OFFSET`
- `#define INITIAL 0`
- `#define YY_EXTRA_TYPE void *`
- `#define YY_READ_BUF_SIZE 8192`
- `#define ECHO do { if (fwrite( yytext, (size_t) yyleng, 1, yyout )) {} } while (0)`
- `#define YY_INPUT(buf, result, max_size)`
- `#define yyterminate() return YY_NULL`
- `#define YY_START_STACK_INCR 25`
- `#define YY_FATAL_ERROR(msg) yy_fatal_error( msg )`
- `#define YY_DECL_IS_OURS 1`
- `#define YY_DECL int yylex (void)`
- `#define YY_USER_ACTION`
- `#define YY_BREAK /*LINTED*/break;`
- `#define YY_RULE_SETUP YY_USER_ACTION`
- `#define YY_EXIT_FAILURE 2`
- `#define yyless(n)`
- `#define YYTABLES_NAME "yytables"`

## Typedefs

- `typedef signed char flex_int8_t`
- `typedef short int flex_int16_t`
- `typedef int flex_int32_t`
- `typedef unsigned char flex_uint8_t`
- `typedef unsigned short int flex_uint16_t`
- `typedef unsigned int flex_uint32_t`
- `typedef struct yy_buffer_state * YY_BUFFER_STATE`
- `typedef size_t yy_size_t`
- `typedef flex_uint8_t YY_CHAR`
- `typedef int yy_state_type`

## Functions

- `void yyrestart (FILE *input_file)`
- `void yy_switch_to_buffer (YY_BUFFER_STATE new_buffer)`
- `YY_BUFFER_STATE yy_create_buffer (FILE *file, int size)`
- `void yy_delete_buffer (YY_BUFFER_STATE b)`
- `void yy_flush_buffer (YY_BUFFER_STATE b)`
- `void yypush_buffer_state (YY_BUFFER_STATE new_buffer)`
- `YY_BUFFER_STATE yy_scan_buffer (char *base, yy_size_t size)`
- `YY_BUFFER_STATE yy_scan_string (const char *yy_str)`
- `YY_BUFFER_STATE yy_scan_bytes (const char *bytes, int len)`
- `void * yyalloc (yy_size_t)`
- `void * yyrealloc (void *, yy_size_t)`
- `void yyfree (void *)`
- `int stratlex ()`
- `void yyset_debug (int debug_flag)`
- `void yyset_extra (YY_EXTRA_TYPE user_defined)`
- `void yyset_in (FILE *_in_str)`
- `void yyset_out (FILE *_out_str)`
- `void yyset_lineno (int _line_number)`
- `if (!(yy_init))`

## Variables

- int `yyleng`
- FILE \* `yyin` = NULL
- FILE \* `yyout` = NULL
- int `yylineno` = 1
- char \* `yytext`
- int `yy_flex_debug` = 0
- `YY_DECL`
- char \* `yy_cp`
- char \* `yy_bp`
- int `yy_act`

### 6.38.1 Macro Definition Documentation

#### 6.38.1.1 BEGIN

```
#define BEGIN (yy_start) = 1 + 2 *
```

#### 6.38.1.2 ECHO

```
#define ECHO do { if (fwrite(yytext, (size_t) yleng, 1, yyout)) {} } while (0)
```

#### 6.38.1.3 EOB\_ACT\_CONTINUE\_SCAN

```
#define EOB_ACT_CONTINUE_SCAN 0
```

#### 6.38.1.4 EOB\_ACT\_END\_OF\_FILE

```
#define EOB_ACT_END_OF_FILE 1
```

#### 6.38.1.5 EOB\_ACT\_LAST\_MATCH

```
#define EOB_ACT_LAST_MATCH 2
```



#### 6.38.1.6 FLEX\_BETA

```
#define FLEX_BETA
```

#### 6.38.1.7 FLEX\_SCANNER

```
#define FLEX_SCANNER
```

#### 6.38.1.8 FLEXINT\_H

```
#define FLEXINT_H
```

#### 6.38.1.9 INITIAL

```
#define INITIAL 0
```

#### 6.38.1.10 INT16\_MAX

```
#define INT16_MAX (32767)
```

#### 6.38.1.11 INT16\_MIN

```
#define INT16_MIN (-32767-1)
```

#### 6.38.1.12 INT32\_MAX

```
#define INT32_MAX (2147483647)
```

#### 6.38.1.13 INT32\_MIN

```
#define INT32_MIN (-2147483647-1)
```

#### 6.38.1.14 INT8\_MAX

```
#define INT8_MAX (127)
```

#### 6.38.1.15 INT8\_MIN

```
#define INT8_MIN (-128)
```

#### 6.38.1.16 REJECT

```
#define REJECT reject_used_but_not_detected
```

#### 6.38.1.17 SIZE\_MAX

```
#define SIZE_MAX (~(size_t)0)
```

#### 6.38.1.18 strat\_create\_buffer\_ALREADY\_DEFINED

```
#define strat_create_buffer_ALREADY_DEFINED
```

#### 6.38.1.19 strat\_delete\_buffer\_ALREADY\_DEFINED

```
#define strat_delete_buffer_ALREADY_DEFINED
```

#### 6.38.1.20 strat\_flex\_debug\_ALREADY\_DEFINED

```
#define strat_flex_debug_ALREADY_DEFINED
```

#### 6.38.1.21 strat\_flush\_buffer\_ALREADY\_DEFINED

```
#define strat_flush_buffer_ALREADY_DEFINED
```

**6.38.1.22 strat\_init\_buffer\_ALREADY\_DEFINED**

```
#define strat_init_buffer_ALREADY_DEFINED
```

**6.38.1.23 strat\_load\_buffer\_state\_ALREADY\_DEFINED**

```
#define strat_load_buffer_state_ALREADY_DEFINED
```

**6.38.1.24 strat\_scan\_buffer\_ALREADY\_DEFINED**

```
#define strat_scan_buffer_ALREADY_DEFINED
```

**6.38.1.25 strat\_scan\_bytes\_ALREADY\_DEFINED**

```
#define strat_scan_bytes_ALREADY_DEFINED
```

**6.38.1.26 strat\_scan\_string\_ALREADY\_DEFINED**

```
#define strat_scan_string_ALREADY_DEFINED
```

**6.38.1.27 strat\_switch\_to\_buffer\_ALREADY\_DEFINED**

```
#define strat_switch_to_buffer_ALREADY_DEFINED
```

**6.38.1.28 stratalloc\_ALREADY\_DEFINED**

```
#define stratalloc_ALREADY_DEFINED
```

**6.38.1.29 stratensure\_buffer\_stack\_ALREADY\_DEFINED**

```
#define stratensure_buffer_stack_ALREADY_DEFINED
```

**6.38.1.30 stratfree\_ALREADY\_DEFINED**

```
#define stratfree_ALREADY_DEFINED
```

**6.38.1.31 stratin\_ALREADY\_DEFINED**

```
#define stratin_ALREADY_DEFINED
```

**6.38.1.32 stratleng\_ALREADY\_DEFINED**

```
#define stratleng_ALREADY_DEFINED
```

**6.38.1.33 stratlex\_ALREADY\_DEFINED**

```
#define stratlex_ALREADY_DEFINED
```

**6.38.1.34 stratlineno\_ALREADY\_DEFINED**

```
#define stratlineno_ALREADY_DEFINED
```

**6.38.1.35 stratout\_ALREADY\_DEFINED**

```
#define stratout_ALREADY_DEFINED
```

**6.38.1.36 stratpop\_buffer\_state\_ALREADY\_DEFINED**

```
#define stratpop_buffer_state_ALREADY_DEFINED
```

**6.38.1.37 stratpush\_buffer\_state\_ALREADY\_DEFINED**

```
#define stratpush_buffer_state_ALREADY_DEFINED
```

**6.38.1.38 stratrealloc\_ALREADY\_DEFINED**

```
#define stratrealloc_ALREADY_DEFINED
```

**6.38.1.39 stratrestart\_ALREADY\_DEFINED**

```
#define stratrestart_ALREADY_DEFINED
```

**6.38.1.40 strattext\_ALREADY\_DEFINED**

```
#define strattext_ALREADY_DEFINED
```

**6.38.1.41 stratwrap**

```
#define stratwrap() (/*CONSTCOND*/1)
```

**6.38.1.42 stratwrap\_ALREADY\_DEFINED**

```
#define stratwrap_ALREADY_DEFINED
```

**6.38.1.43 UINT16\_MAX**

```
#define UINT16_MAX (65535U)
```

**6.38.1.44 UINT32\_MAX**

```
#define UINT32_MAX (4294967295U)
```

**6.38.1.45 UINT8\_MAX**

```
#define UINT8_MAX (255U)
```

**6.38.1.46 unput**

```
#define unput(
 c) yyunput(c, (yytext_ptr))
```

**6.38.1.47 YY\_AT\_BOL**

```
#define YY_AT_BOL() (YY_CURRENT_BUFFER_LVALUE->yy_at_bol)
```

**6.38.1.48 YY\_BREAK**

```
#define YY_BREAK /*LINTED*/break;
```

**6.38.1.49 YY\_BUF\_SIZE**

```
#define YY_BUF_SIZE 16384
```

**6.38.1.50 YY\_BUFFER\_EOF\_PENDING**

```
#define YY_BUFFER_EOF_PENDING 2
```

**6.38.1.51 YY\_BUFFER\_NEW**

```
#define YY_BUFFER_NEW 0
```

**6.38.1.52 YY\_BUFFER\_NORMAL**

```
#define YY_BUFFER_NORMAL 1
```

### 6.38.1.53 yy\_create\_buffer

```
#define yy_create_buffer strat_create_buffer
```

### 6.38.1.54 YY\_CURRENT\_BUFFER

```
#define YY_CURRENT_BUFFER
```

**Value:**

```
(yy_buffer_stack) \
? (yy_buffer_stack)[(yy_buffer_stack_top)] \
: NULL)
```

### 6.38.1.55 YY\_CURRENT\_BUFFER\_LVALUE

```
#define YY_CURRENT_BUFFER_LVALUE (yy_buffer_stack)[(yy_buffer_stack_top)]
```

### 6.38.1.56 YY\_DECL

```
#define YY_DECL int yylex (void)
```

### 6.38.1.57 YY\_DECL\_IS\_OURS

```
#define YY_DECL_IS_OURS 1
```

### 6.38.1.58 yy\_delete\_buffer

```
#define yy_delete_buffer strat_delete_buffer
```

### 6.38.1.59 YY\_DO\_BEFORE\_ACTION

```
#define YY_DO_BEFORE_ACTION
```

**Value:**

```
(yytext_ptr) = yy_bp; \
yyleng = (int) (yy_cp - yy_bp); \
(yy_hold_char) = *yy_cp; \
*yy_cp = '\0'; \
(yy_c_buf_p) = yy_cp;
```

**6.38.1.60 YY\_END\_OF\_BUFFER**

```
#define YY_END_OF_BUFFER 18
```

**6.38.1.61 YY\_END\_OF\_BUFFER\_CHAR**

```
#define YY_END_OF_BUFFER_CHAR 0
```

**6.38.1.62 YY\_EXIT\_FAILURE**

```
#define YY_EXIT_FAILURE 2
```

**6.38.1.63 YY\_EXTRA\_TYPE**

```
#define YY_EXTRA_TYPE void *
```

**6.38.1.64 YY\_FATAL\_ERROR**

```
#define YY_FATAL_ERROR(
 msg) yy_fatal_error(msg)
```

**6.38.1.65 yy\_flex\_debug**

```
int yy_flex_debug strat_flex_debug
```

**6.38.1.66 YY\_FLEX\_MAJOR\_VERSION**

```
#define YY_FLEX_MAJOR_VERSION 2
```



**6.38.1.67 YY\_FLEX\_MINOR\_VERSION**

```
#define YY_FLEX_MINOR_VERSION 6
```

**6.38.1.68 YY\_FLEX\_SUBMINOR\_VERSION**

```
#define YY_FLEX_SUBMINOR_VERSION 4
```

**6.38.1.69 yy\_flush\_buffer**

```
#define yy_flush_buffer strat_flush_buffer
```

**6.38.1.70 YY\_FLUSH\_BUFFER**

```
#define YY_FLUSH_BUFFER yy_flush_buffer(YY_CURRENT_BUFFER)
```

**6.38.1.71 yy\_init\_buffer**

```
#define yy_init_buffer strat_init_buffer
```

**6.38.1.72 YY\_INPUT**

```
#define YY_INPUT(
 buf,
 result,
 max_size)
```

**Value:**

```
if (YY_CURRENT_BUFFER_LVALUE->yy_is_interactive) \
{ \
 int c = '*'; \
 int n; \
 for (n = 0; n < max_size && \
 (c = getc(yyin)) != EOF && c != '\n'; ++n) \
 buf[n] = (char) c; \
 if (c == '\n') \
 buf[n++] = (char) c; \
 if (c == EOF && ferror(yyin)) \
 YY_FATAL_ERROR("input in flex scanner failed"); \
 result = n; \
} \
else \
{ \
 errno=0; \
 while ((result = (int) fread(buf, 1, (yy_size_t) max_size, yyin)) == 0 && ferror(yyin)) \
 { \
 if(errno != EINTR) \
 { \
 YY_FATAL_ERROR("input in flex scanner failed"); \
 break; \
 } \
 errno=0; \
 clearerr(yyin); \
 } \
} \
\
```

**6.38.1.73 YY\_INT\_ALIGNED**

```
#define YY_INT_ALIGNED short int
```

**6.38.1.74 YY\_LESS\_LINENO**

```
#define YY_LESS_LINENO(
 n)
```

**6.38.1.75 YY\_LINENO\_REWIND\_TO**

```
#define YY_LINENO_REWIND_TO(
 ptr)
```

**6.38.1.76 yy\_load\_buffer\_state**

```
static void yy_load_buffer_state strat_load_buffer_state
```

**6.38.1.77 YY\_MORE\_ADJ**

```
#define YY_MORE_ADJ 0
```

**6.38.1.78 yy\_new\_buffer**

```
#define yy_new_buffer yy_create_buffer
```

**6.38.1.79 YY\_NEW\_FILE**

```
#define YY_NEW_FILE yyrestart(yyin)
```

**6.38.1.80 YY\_NULL**

```
#define YY_NULL 0
```

**6.38.1.81 YY\_NUM\_RULES**

```
#define YY_NUM_RULES 17
```

**6.38.1.82 YY\_READ\_BUF\_SIZE**

```
#define YY_READ_BUF_SIZE 8192
```

**6.38.1.83 YY\_RESTORE\_YY\_MORE\_OFFSET**

```
#define YY_RESTORE_YY_MORE_OFFSET
```

**6.38.1.84 YY\_RULE\_SETUP**

```
#define YY_RULE_SETUP YY_USER_ACTION
```

**6.38.1.85 YY\_SC\_TO\_UI**

```
#define YY_SC_TO_UI(
 c) ((YY_CHAR) (c))
```

**6.38.1.86 yy\_scan\_buffer**

```
#define yy_scan_buffer strat_scan_buffer
```

### 6.38.1.87 yy\_scan\_bytes

```
#define yy_scan_bytes strat_scan_bytes
```

### 6.38.1.88 yy\_scan\_string

```
#define yy_scan_string strat_scan_string
```

### 6.38.1.89 yy\_set\_bol

```
#define yy_set_bol(
 at_bol)
```

#### Value:

```
{ \n if (! YY_CURRENT_BUFFER){ \n yyensure_buffer_stack (); \n YY_CURRENT_BUFFER_LVALUE = \n yy_create_buffer(yyin, YY_BUF_SIZE); \n } \n YY_CURRENT_BUFFER_LVALUE->yy_at_bol = at_bol; \n}
```

### 6.38.1.90 yy\_set\_interactive

```
#define yy_set_interactive(
 is_interactive)
```

#### Value:

```
{ \n if (! YY_CURRENT_BUFFER){ \n yyensure_buffer_stack (); \n YY_CURRENT_BUFFER_LVALUE = \n yy_create_buffer(yyin, YY_BUF_SIZE); \n } \n YY_CURRENT_BUFFER_LVALUE->yy_is_interactive = is_interactive; \n}
```

### 6.38.1.91 YY\_SKIP\_YYWRAP

```
#define YY_SKIP_YYWRAP
```

**6.38.1.92 YY\_START**

```
#define YY_START (((yy_start) - 1) / 2)
```

**6.38.1.93 YY\_START\_STACK\_INCR**

```
#define YY_START_STACK_INCR 25
```

**6.38.1.94 YY\_STATE\_BUF\_SIZE**

```
#define YY_STATE_BUF_SIZE ((YY_BUF_SIZE + 2) * sizeof(yy_state_type))
```

**6.38.1.95 YY\_STATE\_EOF**

```
#define YY_STATE_EOF(
 state) (YY_END_OF_BUFFER + state + 1)
```

**6.38.1.96 YY\_STRUCT\_YY\_BUFFER\_STATE**

```
#define YY_STRUCT_YY_BUFFER_STATE
```

**6.38.1.97 yy\_switch\_to\_buffer**

```
#define yy_switch_to_buffer strat_switch_to_buffer
```

**6.38.1.98 YY\_TYPEDEF\_YY\_BUFFER\_STATE**

```
#define YY_TYPEDEF_YY_BUFFER_STATE
```

**6.38.1.99 YY\_TYPEDEF\_YY\_SIZE\_T**

```
#define YY_TYPEDEF_YY_SIZE_T
```

**6.38.1.100 YY\_USER\_ACTION**

```
#define YY_USER_ACTION
```

**6.38.1.101 yyalloc**

```
#define yyalloc stratalloc
```

**6.38.1.102 yyconst**

```
#define yyconst const
```

**6.38.1.103 yyensure\_buffer\_stack**

```
static void yyensure_buffer_stack stratensure_buffer_stack
```

**6.38.1.104 yyfree**

```
#define yyfree stratfree
```

**6.38.1.105 yyget\_debug**

```
int yyget_debug stratget_debug
```

**6.38.1.106 yyget\_extra**

```
YY_EXTRA_TYPE yyget_extra stratget_extra
```

**6.38.1.107 yyget\_in**

```
FILE * yyget_in stratget_in
```

**6.38.1.108 yyget\_leng**

```
int yyget_leng stratget_leng
```

**6.38.1.109 yyget\_lineno**

```
int yyget_lineno stratget_lineno
```

**6.38.1.110 yyget\_out**

```
FILE * yyget_out stratget_out
```

**6.38.1.111 yyget\_text**

```
char * yyget_text stratget_text
```

**6.38.1.112 yyin**

```
FILE * yyin stratin
```

**6.38.1.113 yyleng**

```
int yyleng stratleng
```

**6.38.1.114 yyleless [1/2]**

```
#define yyleless(
 n)
```

**Value:**

```
do \
{ \
 /* Undo effects of setting up yytext. */ \
 int yyless_macro_arg = (n); \
 YY_LESS_LINENO(yyless_macro_arg);\
 *yy_cp = (yy_hold_char); \
 YY_RESTORE_YY_MORE_OFFSET \
 (yy_c_buf_p) = yy_cp = yy_bp + yyless_macro_arg - YY_MORE_ADJ; \
 YY_DO_BEFORE_ACTION; /* set up yytext again */ \
} \
while (0)
```

**6.38.1.115 yyleless [2/2]**

```
#define yyleless(
 n)
```

**Value:**

```
do \
{ \
 /* Undo effects of setting up yytext. */ \
 int yyless_macro_arg = (n); \
 YY_LESS_LINENO(yyless_macro_arg);\
 yytext[yy leng] = (yy_hold_char); \
 (yy_c_buf_p) = yytext + yyless_macro_arg; \
 (yy_hold_char) = *(yy_c_buf_p); \
 *(yy_c_buf_p) = '\0'; \
 yy leng = yyless_macro_arg; \
} \
while (0)
```

**6.38.1.116 yylex**

```
#define yylex(
 void) stratlex
```

**6.38.1.117 yylex\_destroy**

```
int yylex_destroy stratlex_destroy
```

**6.38.1.118 yylex\_init**

```
#define yylex_init stratlex_init
```



**6.38.1.119 yylex\_init\_extra**

```
#define yylex_init_extra stratlex_init_extra
```

**6.38.1.120 yylineno**

```
int yylineno stratlineno
```

**6.38.1.121 yymore**

```
#define yymore() yymore_used_but_not_detected
```

**6.38.1.122 yynoreturn**

```
#define yynoreturn
```

**6.38.1.123 yyout**

```
FILE * yyout stratout
```

**6.38.1.124 yypop\_buffer\_state**

```
void yypop_buffer_state stratpop_buffer_state
```

**6.38.1.125 yypush\_buffer\_state**

```
#define yypush_buffer_state stratpush_buffer_state
```

**6.38.1.126 yyrealloc**

```
#define yyrealloc stratrealloc
```

**6.38.1.127 yyrestart**

```
#define yyrestart stratrestart
```

**6.38.1.128 yyset\_debug**

```
#define yyset_debug stratset_debug
```

**6.38.1.129 yyset\_extra**

```
#define yyset_extra stratset_extra
```

**6.38.1.130 yyset\_in**

```
#define yyset_in stratset_in
```

**6.38.1.131 yyset\_lineno**

```
#define yyset_lineno stratset_lineno
```

**6.38.1.132 yyset\_out**

```
#define yyset_out stratset_out
```

**6.38.1.133 YYSTATE**

```
#define YYSTATE YY_START
```

**6.38.1.134 YYTABLES\_NAME**

```
#define YYTABLES_NAME "yytables"
```

#### 6.38.1.135 yyterminate

```
#define yyterminate() return YY_NULL
```

#### 6.38.1.136 yytext

```
char * yytext strattext
```

#### 6.38.1.137 yytext\_ptr

```
#define yytext_ptr yytext
```

#### 6.38.1.138 yywrap

```
#define yywrap stratwrap
```

### 6.38.2 Typedef Documentation

#### 6.38.2.1 flex\_int16\_t

```
typedef short int flex_int16_t
```

#### 6.38.2.2 flex\_int32\_t

```
typedef int flex_int32_t
```

#### 6.38.2.3 flex\_int8\_t

```
typedef signed char flex_int8_t
```

#### 6.38.2.4 flex\_uint16\_t

```
typedef unsigned short int flex_uint16_t
```

#### 6.38.2.5 flex\_uint32\_t

```
typedef unsigned int flex_uint32_t
```

#### 6.38.2.6 flex\_uint8\_t

```
typedef unsigned char flex_uint8_t
```

#### 6.38.2.7 YY\_BUFFER\_STATE

```
typedef struct yy_buffer_state* YY_BUFFER_STATE
```

#### 6.38.2.8 YY\_CHAR

```
typedef flex_uint8_t YY_CHAR
```

#### 6.38.2.9 yy\_size\_t

```
typedef size_t yy_size_t
```

#### 6.38.2.10 yy\_state\_type

```
typedef int yy_state_type
```

### 6.38.3 Function Documentation

#### 6.38.3.1 if()

```
if (
 ! yy_init)
```

#### 6.38.3.2 stratlex()

```
int stratlex ()
```

#### 6.38.3.3 yy\_create\_buffer()

```
YY_BUFFER_STATE yy_create_buffer (
 FILE * file,
 int size)
```

#### 6.38.3.4 yy\_delete\_buffer()

```
void yy_delete_buffer (
 YY_BUFFER_STATE b)
```

#### 6.38.3.5 yy\_flush\_buffer()

```
void yy_flush_buffer (
 YY_BUFFER_STATE b)
```

#### 6.38.3.6 yy\_scan\_buffer()

```
YY_BUFFER_STATE yy_scan_buffer (
 char * base,
 yy_size_t size)
```

#### 6.38.3.7 yy\_scan\_bytes()

```
YY_BUFFER_STATE yy_scan_bytes (
 const char * bytes,
 int len)
```

#### 6.38.3.8 yy\_scan\_string()

```
YY_BUFFER_STATE yy_scan_string (
 const char * yy_str)
```

#### 6.38.3.9 yy\_switch\_to\_buffer()

```
void yy_switch_to_buffer (
 YY_BUFFER_STATE new_buffer)
```

#### 6.38.3.10 yyalloc()

```
void* yyalloc (
 yy_size_t)
```

#### 6.38.3.11 yyfree()

```
void yyfree (
 void *)
```

#### 6.38.3.12 yypush\_buffer\_state()

```
void yypush_buffer_state (
 YY_BUFFER_STATE new_buffer)
```

#### 6.38.3.13 yyrealloc()

```
void* yyrealloc (
 void * ,
 yy_size_t)
```

#### 6.38.3.14 yyrestart()

```
void yyrestart (
 FILE * input_file)
```

### 6.38.3.15 yyset\_debug()

```
void yyset_debug (
 int debug_flag)
```

### 6.38.3.16 yyset\_extra()

```
void yyset_extra (
 YY_EXTRA_TYPE user_defined)
```

### 6.38.3.17 yyset\_in()

```
void yyset_in (
 FILE * _in_str)
```

### 6.38.3.18 yyset\_lineno()

```
void yyset_lineno (
 int _line_number)
```

### 6.38.3.19 yyset\_out()

```
void yyset_out (
 FILE * _out_str)
```

## 6.38.4 Variable Documentation

### 6.38.4.1 yy\_act

```
int yy_act
```

#### 6.38.4.2 yy\_bp

```
char * yy_bp
```

#### 6.38.4.3 yy\_cp

```
char* yy_cp
```

#### 6.38.4.4 YY\_DECL

```
YY_DECL
```

##### Initial value:

```
{
 yy_state_type yy_current_state
```

The main scanner function which does all the work.

#### 6.38.4.5 yy\_flex\_debug

```
int yy_flex_debug = 0
```

#### 6.38.4.6 yyin

```
FILE* yyin = NULL
```

#### 6.38.4.7 yyleng

```
int yyleng
```

#### 6.38.4.8 yylineno

```
int yylineno = 1
```



#### 6.38.4.9 yyout

```
FILE * yyout = NULL
```

#### 6.38.4.10 yytext

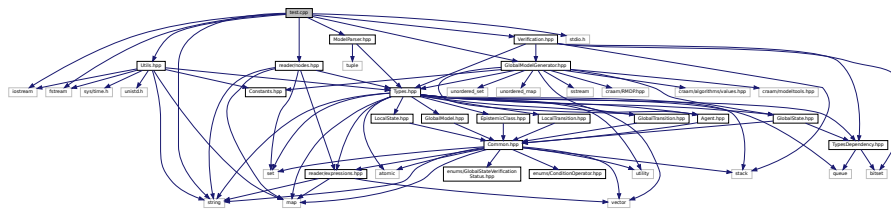
```
char* yytext
```

### 6.39 strategyScanner.l File Reference

## 6.40 test.cpp File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include <stdio.h>
#include "ModelParser.hpp"
#include "reader/nodes.hpp"
#include "GlobalModelGenerator.hpp"
#include "Verification.hpp"
#include "Utils.hpp"
```

Include dependency graph for test.cpp:



## Functions

- `int main (int argc, char *argv[ ])`

## Variables

- Cfg config

### 6.40.1 Function Documentation



## Classes

- struct [Var](#)  
*Represents a variable in the model, containing name, initial value and persistence.*
- struct [Condition](#)  
*Represents a condition for [LocalTransition](#).*
- struct [FormulaTemplate](#)  
*Contains a template for coalition of [Agent](#) as string from the formula.*
- struct [Formula](#)  
*Contains the processed verification formula.*
- struct [LocalModels](#)  
*Represents a single local model, contains all agents and variables.*
- struct [Result](#)

## Enumerations

- enum [ProbabilitySign](#) {  
  [NONE](#) , [EQ](#) , [NE](#) , [GT](#) ,  
  [GE](#) , [LT](#) , [LE](#) }  
*Probability inequality sign used for the probabilistic verification.*
- enum [VerificationFormulaMode](#) { [NOT\\_SET](#) = 0 , [F](#) = 1 , [G](#) = 2 }

### 6.41.1 Detailed Description

Custom data structures. Data structures and classes containing model data.

### 6.41.2 Enumeration Type Documentation

#### 6.41.2.1 ProbabilitySign

enum [ProbabilitySign](#)

Probability inequality sign used for the probabilistic verification.

##### Enumerator

|      |                          |
|------|--------------------------|
| NONE | No probability detected. |
| EQ   | Equal (==)               |
| NE   | Not equal (!=)           |
| GT   | Greater than (>)         |
| GE   | Greater equal (>=)       |
| LT   | Less than (<)            |
| LE   | Less equal (<=)          |



- *A comparator for two bitsets containing values for the global states.*
- struct [ProbabilityTrueFalse](#)
  - *Contains probability of a state being in a true or false state.*
- class [ProbabilityEntry](#)
  - *Class for handling the probability operations during probability verification.*
- struct [StateVerificationInfo](#)
  - *Handles all single state verification information during probabilistic verification.*
- struct [Action](#)
  - *Single action template for probabilistic verification.*
- class [StrategyCollection](#)
  - *Handles currently selected strategy when the strategy is set.*

## Macros

- `#define STRATEGY_BITS 64`

## Enumerations

- enum [VerifResult](#) { [NOT\\_VERIFIED](#) = 0 , [TRUE](#) = 1 , [FALSE](#) = 2 }
- enum [StrategyEntryType](#) { [NOT\\_MODIFIED](#) , [ADDED](#) }
  - *[StrategyEntry](#) entry type.*
- enum [HistoryEntryType](#) { [DECISION](#) , [STATE\\_STATUS](#) , [CONTEXT](#) , [MARK\\_DECISION\\_AS\\_INVALID](#) , [UNCONTROLLED\\_DECISION](#) , [PROBABILITY](#) , [ANSWER\\_PROBABILITY](#) }
  - *[HistoryEntry](#) entry type.*
- enum [ProbabilityCalculationType](#) { [SUM\\_PROBABILITY](#) , [MIN\\_PROBABILITY](#) , [NONE\\_PROBABILITY](#) }
  - *Contains the type of the probability calculation in a given node.*

### 6.42.1 Detailed Description

Custom data structures, using other custom data structures. Data structures and classes containing model data that are using other custom data structures.

### 6.42.2 Macro Definition Documentation

#### 6.42.2.1 STRATEGY\_BITS

```
#define STRATEGY_BITS 64
```

### 6.42.3 Enumeration Type Documentation

#### 6.42.3.1 HistoryEntryType

```
enum HistoryEntryType
```

[HistoryEntry](#) entry type.

## Enumerator

|                          |                                                                          |
|--------------------------|--------------------------------------------------------------------------|
| DECISION                 | Made the decision to go to a state using a transition.                   |
| STATE_STATUS             | Changed verification status.                                             |
| CONTEXT                  | Recursion has gone deeper.                                               |
| MARK_DECISION_AS_INVALID | Marking a transition as invalid.                                         |
| UNCONTROLLED_DECISION    | One uncontrolled choice from a set, from which all of them has to be OK. |
| PROBABILITY              | There's a change in probability.                                         |
| ANSWER_PROBABILITY       | There's a change in the answer probability.                              |

**6.42.3.2 ProbabilityCalculationType**

enum `ProbabilityCalculationType`

Contains the type of the probability calculation in a given node.

## Enumerator

|                  |  |
|------------------|--|
| SUM_PROBABILITY  |  |
| MIN_PROBABILITY  |  |
| NONE_PROBABILITY |  |

**6.42.3.3 StrategyEntryType**

enum `StrategyEntryType`

`StrategyEntry` entry type.

## Enumerator

|              |                                    |
|--------------|------------------------------------|
| NOT_MODIFIED | Entry didn't have to get modified. |
| ADDED        | Entry got added to the map.        |

**6.42.3.4 VerifResult**

enum `VerifResult`

## Enumerator

|              |  |
|--------------|--|
| NOT_VERIFIED |  |
| TRUE         |  |
| FALSE        |  |



### 6.43.1 Detailed Description

Utility functions. A collection of utility functions to use in the project.

### 6.43.2 Function Documentation

#### 6.43.2.1 agentToString()

```
string agentToString (
 Agent * agt)
```

Converts pointer to an Agent into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

##### Parameters

|            |                                                             |
|------------|-------------------------------------------------------------|
| <i>agt</i> | Pointer to an <a href="#">Agent</a> to parse into a string. |
|------------|-------------------------------------------------------------|

##### Returns

String containing all of [Agent](#) data.

#### 6.43.2.2 envToString()

```
string envToString (
 map< string, int > env)
```

Converts a map of string and int to a string.

##### Parameters

|            |                                    |
|------------|------------------------------------|
| <i>env</i> | Map to be converted into a string. |
|------------|------------------------------------|

##### Returns

Returns string " (first\_name, second\_name, ..., last\_name=int\_value)"

#### 6.43.2.3 getContextModel()

```
map<LocalState*, vector<GlobalState*> > getContextModel (
 Formula * formula,
```



```
LocalModels * localModels,
Agent * agt)
```

#### 6.43.2.4 getLocalStatesSCC()

```
vector<set<LocalState*> > getLocalStatesSCC (
 Agent * agt)
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

##### Parameters

|            |                                                |
|------------|------------------------------------------------|
| <i>agt</i> | - an agent whose local graph will be inspected |
|------------|------------------------------------------------|

##### Returns

localStates partition in a form of the vector, where each set corresponds to a SCC

#### 6.43.2.5 getMemCap()

```
unsigned long getMemCap ()
```

#### 6.43.2.6 loadConfigFromArgs()

```
void loadConfigFromArgs (
 int argc,
 char ** argv)
```

#### 6.43.2.7 loadConfigFromFile()

```
void loadConfigFromFile (
 string filename)
```

#### 6.43.2.8 localModelsToString()

```
string localModelsToString (
 LocalModels * lm)
```

Converts pointer to the [LocalModels](#) into a string containing all [Agent](#) instances from the model, initial values of the variables and names of the persistent values.

## Parameters

|           |                                                    |
|-----------|----------------------------------------------------|
| <i>lm</i> | Pointer to the local model to parse into a string. |
|-----------|----------------------------------------------------|

## Returns

String containing all of [LocalModels](#) data.

**6.43.2.9 outputGlobalModel()**

```
void outputGlobalModel (
 GlobalModel * globalModel)
```

Prints the whole [GlobalModel](#) into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

## Parameters

|                    |                                                                     |
|--------------------|---------------------------------------------------------------------|
| <i>globalModel</i> | Pointer to a <a href="#">GlobalModel</a> to print into the console. |
|--------------------|---------------------------------------------------------------------|

**6.43.2.10 tarjanVisit()**

```
void tarjanVisit (
 LocalState * v,
 map< int, int > * dindex,
 map< int, int > * lowlink,
 stack< LocalState * > * stack,
 map< int, bool > * onstack,
 int * depth,
 vector< set< LocalState * >> * comp)
```

Utility function for SCC-computatation.

## Parameters

|                |                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <i>v</i>       | - current vertex                                                                                                                       |
| <i>dindex</i>  | - vertex.index (alt. vertex.num)                                                                                                       |
| <i>lowlink</i> | - vertex.lowlink (by df. lowest dindex in the same scc reachable from vertex using tree edges followed by at most one back/cross edge) |
| <i>stack</i>   | - holds candidates for SCC                                                                                                             |
| <i>onstack</i> | - used as condition for back/cross-edge case                                                                                           |
| <i>depth</i>   | - next available (discovery) index                                                                                                     |
| <i>comp</i>    | - result of SCC partitioning                                                                                                           |

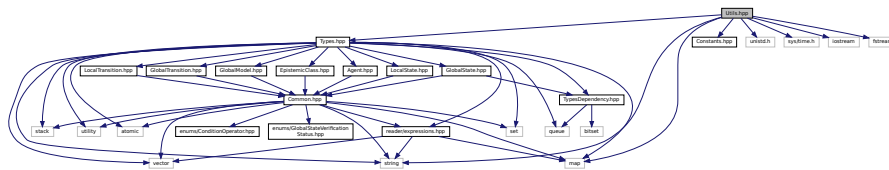
### 6.43.3 Variable Documentation

#### 6.43.3.1 config

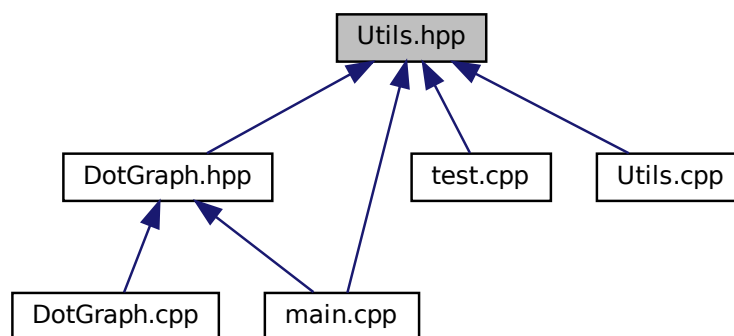
`Cfg config [extern]`

## 6.44 Utils.hpp File Reference

```
#include "Types.hpp"
#include "Constants.hpp"
#include <map>
#include <string>
#include <unistd.h>
#include <sys/time.h>
#include <iostream>
#include <fstream>
Include dependency graph for Utils.hpp:
```



This graph shows which files directly or indirectly include this file:



## Functions

- string `envToString` (map< string, int > env)  
Converts a map of string and int to a string.
- string `agentToString` (Agent \*agt)  
Converts pointer to an Agent into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.
- string `localModelsToString` (LocalModels \*lm)  
Converts pointer to the LocalModels into a string containing all Agent instances from the model, initial values of the variables and names of the persistent values.
- void `outputGlobalModel` (GlobalModel \*globalModel)  
Prints the whole GlobalModel into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.
- unsigned long `getMemCap` ()
- vector< set< LocalState \* > > `getLocalStatesSCC` (Agent \*agt)  
a quick implementation of a Tarjan SCC algorithm (based on DFS)
- map< LocalState \*, vector< GlobalState \* > > `getContextModel` (Formula \*formula, LocalModels \*localModels, Agent \*agt)
- void `loadConfigFromFile` (string filename="config.txt")
- void `loadConfigFromArgs` (int argc, char \*\*argv)

### 6.44.1 Function Documentation

#### 6.44.1.1 agentToString()

```
string agentToString (
 Agent * agt)
```

Converts pointer to an Agent into a string containing name of the agent, its initial state, transitions with their local and global names, shared count and conditions.

##### Parameters

|                  |                                             |
|------------------|---------------------------------------------|
| <code>agt</code> | Pointer to an Agent to parse into a string. |
|------------------|---------------------------------------------|

##### Returns

String containing all of Agent data.

#### 6.44.1.2 envToString()

```
string envToString (
 map< string, int > env)
```

Converts a map of string and int to a string.

**Parameters**

|            |                                    |
|------------|------------------------------------|
| <i>env</i> | Map to be converted into a string. |
|------------|------------------------------------|

**Returns**

Returns string " (first\_name, second\_name, ..., last\_name=int\_value)"

**6.44.1.3 getContextModel()**

```
map<LocalState*, vector<GlobalState*> > getContextModel (
 Formula * formula,
 LocalModels * localModels,
 Agent * agt)
```

**6.44.1.4 getLocalStatesSCC()**

```
vector<set<LocalState*> > getLocalStatesSCC (
 Agent * agt)
```

a quick implementation of a Tarjan SCC algorithm (based on DFS)

**Parameters**

|            |                                                |
|------------|------------------------------------------------|
| <i>agt</i> | - an agent whose local graph will be inspected |
|------------|------------------------------------------------|

**Returns**

localStates partition in a form of the vector, where each set correponds to a SCC

**6.44.1.5 getMemCap()**

```
unsigned long getMemCap ()
```

**6.44.1.6 loadConfigFromArgs()**

```
void loadConfigFromArgs (
 int argc,
 char ** argv)
```

#### 6.44.1.7 loadConfigFromFile()

```
void loadConfigFromFile (
 string filename = "config.txt")
```

#### 6.44.1.8 localModelsToString()

```
string localModelsToString (
 LocalModels * lm)
```

Converts pointer to the [LocalModels](#) into a string containing all [Agent](#) instances from the model, initial values of the variables and names of the persistent values.

##### Parameters

|           |                                                    |
|-----------|----------------------------------------------------|
| <i>lm</i> | Pointer to the local model to parse into a string. |
|-----------|----------------------------------------------------|

##### Returns

String containing all of [LocalModels](#) data.

#### 6.44.1.9 outputGlobalModel()

```
void outputGlobalModel (
 GlobalModel * globalModel)
```

Prints the whole [GlobalModel](#) into the console. Contains global states with hashes, local states, variables inside those states, global variables, global transitions, local transitions and epistemic classes of agents.

##### Parameters

|                    |                                                                     |
|--------------------|---------------------------------------------------------------------|
| <i>globalModel</i> | Pointer to a <a href="#">GlobalModel</a> to print into the console. |
|--------------------|---------------------------------------------------------------------|

## 6.45 Verification.cpp File Reference

Class for verification of the formula on a model. Class for verification of the specified formula on a specified model.

```
#include "Verification.hpp"
#include <bits/stdc++.h>
```



## Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>prefix</i> | A prefix string to append to the front of the entry.                            |
| <i>h</i>      | A pointer to the <a href="#">HistoryEntry</a> struct which will be printed out. |

**6.45.3.2 dbgVerifStatus()**

```
void dbgVerifStatus (
 string prefix,
 GlobalState * gs,
 GlobalStateVerificationStatus st,
 string reason)
```

Print a debug message of a verification status to the console.

## Parameters

|               |                                                                                       |
|---------------|---------------------------------------------------------------------------------------|
| <i>prefix</i> | A prefix string to append to the front of every entry.                                |
| <i>gs</i>     | Pointer to a <a href="#">GlobalState</a> .                                            |
| <i>st</i>     | Enum with a verification status of a global state.                                    |
| <i>reason</i> | String with a reason why the function was called, e.g. "entered state", "all passed". |

**6.45.3.3 verStatusToStr()**

```
string verStatusToStr (
 GlobalStateVerificationStatus status)
```

Converts global verification status into a string.

## Parameters

|               |                             |
|---------------|-----------------------------|
| <i>status</i> | Enum value to be converted. |
|---------------|-----------------------------|

## Returns

[Verification](#) status converted into a string.

**6.45.4 Variable Documentation****6.45.4.1 config**

```
Cfg config [extern]
```





## Functions

- string `verStatusToStr` (`GlobalStateVerificationStatus` status)  
*Converts global verification status into a string.*

### 6.46.1 Macro Definition Documentation

#### 6.46.1.1 STRATEGY\_BITS

```
#define STRATEGY_BITS 64
```

### 6.46.2 Enumeration Type Documentation

#### 6.46.2.1 TraversalMode

```
enum TraversalMode
```

Current model traversal mode.

Enumerator

|         |                                                     |
|---------|-----------------------------------------------------|
| NORMAL  | Normal model traversal.                             |
| REVERT  | Backtracking through recursion with state rollback. |
| RESTORE | Backtracking through recursion.                     |

### 6.46.3 Function Documentation

#### 6.46.3.1 verStatusToStr()

```
string verStatusToStr (
 GlobalStateVerificationStatus status)
```

Converts global verification status into a string.

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>status</i> | Enum value to be converted. |
|---------------|-----------------------------|

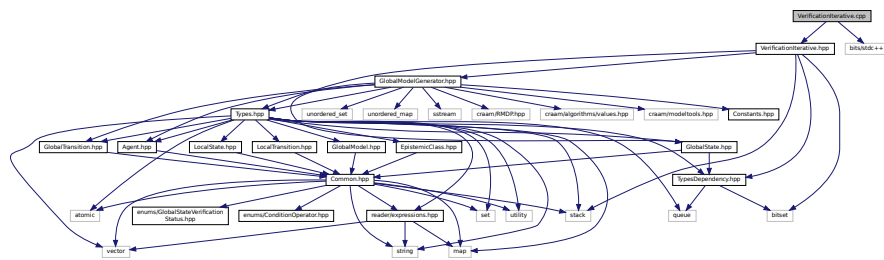
## Returns

[Verification](#) status converted into a string.

## 6.47 VerificationIterative.cpp File Reference

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

```
#include "VerificationIterative.hpp"
#include <bits/stdc++.h>
Include dependency graph for VerificationIterative.cpp:
```



### Macros

- `#define` [DEPTH\\_PREFIX](#) `string(depth * 4, '')`

### Functions

- `string` [verifStatusToStr](#) ([GlobalStateVerificationStatus](#) status)  
*Converts global verification status into a string.*

### Variables

- [Cfg config](#)

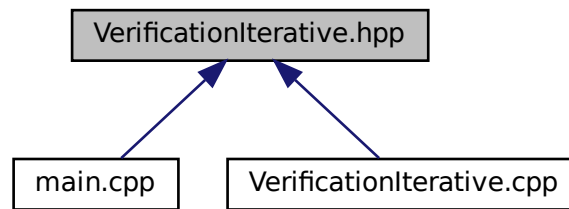
#### 6.47.1 Detailed Description

Class for iteratively generated verification of the formula on a model. Class for iteratively generated verification of the specified formula on a specified model.

#### 6.47.2 Macro Definition Documentation



This graph shows which files directly or indirectly include this file:



## Classes

- struct [DecisionEntry](#)
- class [VerificationIterative](#)

*A class that verifies if the model fulfills the formula. Also can do some operations on decision history.*

## Enumerations

- enum [GraphTraversalMode](#) { [NORMAL\\_TRAVERSAL](#) , [RESTORE\\_STATES](#) }

*Current model traversal mode.*

## Functions

- string [verifStatusToStr](#) ([GlobalStateVerificationStatus](#) status)

*Converts global verification status into a string.*

### 6.48.1 Enumeration Type Documentation

#### 6.48.1.1 GraphTraversalMode

enum [GraphTraversalMode](#)

Current model traversal mode.

Enumerator

|                                  |                           |
|----------------------------------|---------------------------|
| <a href="#">NORMAL_TRAVERSAL</a> | Normal model traversal.   |
| <a href="#">RESTORE_STATES</a>   | Reconstructing recursion. |

## 6.48.2 Function Documentation

### 6.48.2.1 `verifStatusToStr()`

```
string verfStatusToStr (
 GlobalStateVerificationStatus status)
```

Converts global verification status into a string.

#### Parameters

|               |                             |
|---------------|-----------------------------|
| <i>status</i> | Enum value to be converted. |
|---------------|-----------------------------|

#### Returns

[Verification](#) status converted into a string.

# Index

- ~GlobalModelGenerator
  - GlobalModelGenerator, [97](#)
- ~HistoryDbg
  - HistoryDbg, [118](#)
- ~ModelParser
  - ModelParser, [132](#)
- ~ProbabilityEntry
  - ProbabilityEntry, [134](#)
- ~StrategyCollection
  - StrategyCollection, [157](#)
- ~StrategyParser
  - StrategyParser, [162](#)
- ~Verification
  - Verification, [172](#)
- ~VerificationIterative
  - VerificationIterative, [187](#)
- Action, [13](#)
  - actionName, [14](#)
  - hash, [14](#)
  - states, [14](#)
- actionCounts
  - GlobalModelGenerator, [106](#)
- actionName
  - Action, [14](#)
  - ActionTemplate, [15](#)
  - StrategyEntry, [161](#)
- ActionTemplate, [14](#)
  - actionName, [15](#)
  - ActionTemplate, [15](#)
  - hash, [15](#)
  - states, [16](#)
- activeProbabilityEntryType
  - DecisionEntry, [33](#)
- add\_epsilon\_transitions
  - Cfg, [28](#)
- addAction
  - StrategyCollection, [158](#)
  - StrategyCollectionTemplate, [159](#)
- ADDED
  - TypesDependency.hpp, [330](#)
- addEdge
  - DotGraph, [38](#)
- addedStates
  - GlobalModelGenerator, [106](#)
- addEntry
  - HistoryDbg, [118](#)
- addHistoryAnswerProbability
  - VerificationIterative, [188](#)
- addHistoryContext
  - Verification, [172](#)
  - VerificationIterative, [188](#)
- addHistoryDecision
  - Verification, [172](#)
  - VerificationIterative, [188](#)
- addHistoryMarkDecisionAsInvalid
  - Verification, [173](#)
  - VerificationIterative, [189](#)
- addHistoryProbability
  - VerificationIterative, [189](#)
- addHistoryStateStatus
  - Verification, [173](#)
  - VerificationIterative, [189](#)
- addHistoryUncontrolledDecision
  - Verification, [173](#)
- addInitial
  - AgentTemplate, [20](#)
- addLocal
  - AgentTemplate, [21](#)
- addNode
  - DotGraph, [38](#)
- addPersistent
  - AgentTemplate, [21](#)
- addStrategy
  - StrategyCollection, [158](#)
- addTransition
  - AgentTemplate, [22](#)
- Agent, [16](#)
  - Agent, [17](#)
  - id, [18](#)
  - includesState, [17](#)
  - initState, [18](#)
  - localStates, [18](#)
  - localTransitions, [18](#)
  - name, [18](#)
  - vars, [19](#)
- agent
  - LocalState, [126](#)
  - LocalTransition, [129](#)
  - Var, [168](#)
  - YYSTYPE, [202](#)
- Agent.cpp, [205](#)
- Agent.hpp, [205](#)
- AGENT\_TEMPLATE
  - DotGraph.hpp, [210](#)
- agentIndex
  - GlobalModelGenerator, [106](#)
- agents
  - GlobalModel, [93](#)

- LocalModels, 124
- AgentTemplate, 19
  - addInitial, 20
  - addLocal, 21
  - addPersistent, 21
  - addTransition, 22
  - AgentTemplate, 20
  - DotGraph, 24
  - generateAgent, 22
  - setIdent, 23
  - setInitState, 24
- agentToString
  - Utils.cpp, 332
  - Utils.hpp, 336
- ANSWER\_PROBABILITY
  - TypesDependency.hpp, 330
- areGlobalStatesInTheSameEpistemicClass
  - Verification, 174
  - VerificationIterative, 189
- assign
  - Assignment, 26
  - YYSTYPE, 202
- Assignment, 25
  - assign, 26
  - Assignment, 26
  - ident, 26
  - value, 26
- assignments
  - TransitionTemplate, 166
- assignSet
  - YYSTYPE, 202
- BEGIN
  - scanner.c, 252
  - strategyScanner.c, 300
- candidateStateDepths
  - GlobalModelGenerator, 106
- Cfg, 27
  - add\_epsilon\_transitions, 28
  - counterexample, 28
  - dotdir, 28
  - fixpoint, 28
  - fname, 29
  - formula, 29
  - formula\_from\_parameter, 29
  - model\_id, 29
  - natural\_strategy, 29
  - output\_dot\_files, 29
  - output\_global\_model, 29
  - output\_local\_models, 30
  - probability, 30
  - reduce, 30
  - reduce\_all, 30
  - reduce\_args, 30
  - strategy\_file\_path, 30
  - stv\_mode, 30
  - verify\_strategy, 31
- changeMinProbabilityFalse
  - ProbabilityEntry, 134
- changeMinProbabilityTrue
  - ProbabilityEntry, 135
- changeSumProbabilityFalse
  - ProbabilityEntry, 135
- changeSumProbabilityTrue
  - ProbabilityEntry, 135
- changeVerificationMode
  - ProbabilityEntry, 135
- checkIfProbabilistic
  - VerificationIterative, 190
- checkLocalStates
  - GlobalModelGenerator, 98
- checkUncontrolledSet
  - Verification, 174
- choiceIndices
  - GlobalModelGenerator, 106
- clear
  - StrategyCollection, 158
- cloneEntry
  - HistoryDbg, 118
- coalition
  - Formula, 90
  - FormulaTemplate, 92
- coalitionTransitions
  - GlobalModelGenerator, 106
- Common.hpp, 206
- compare
  - LocalState, 125
- comparedValue
  - Condition, 32
- computeEpistemicClassHash
  - GlobalModelGenerator, 98
- computeGlobalStateHash
  - GlobalModelGenerator, 98
- Condition, 31
  - comparedValue, 32
  - conditionOperator, 32
  - var, 32
- condition
  - TransitionTemplate, 166
- ConditionOperator
  - ConditionOperator.hpp, 208
- conditionOperator
  - Condition, 32
- ConditionOperator.hpp, 207
  - ConditionOperator, 208
  - Equals, 208
  - NotEquals, 208
- conditions
  - LocalTransition, 129
- condList
  - YYSTYPE, 203
- config
  - GlobalModelGenerator.cpp, 215
  - GlobalState.hpp, 218
  - main.cpp, 223
  - test.cpp, 326



- Utils.cpp, 335
- Verification.cpp, 340
- VerificationIterative.cpp, 344
- Constants.hpp, 208
  - DEBUG\_ON, 208
  - VERBOSE, 208
- CONTEXT
  - TypesDependency.hpp, 330
- controlled
  - StateVerificationInfo, 153
- controlledProbabilisticTransitionsLeftToProcess
  - StateVerificationInfo, 153
- controlledTransitionsLeftToProcess
  - StateVerificationInfo, 153
- correctModel
  - GlobalModelGenerator, 106
- counterexample
  - Cfg, 28
- createProbabilityStrategy
  - GlobalModelGenerator, 98
- currentStrategy
  - GlobalModelGenerator, 107
- dbgHistEnt
  - Verification.cpp, 339
- dbgVerifStatus
  - Verification.cpp, 340
- DEBUG\_ON
  - Constants.hpp, 208
- DECISION
  - TypesDependency.hpp, 330
- decision
  - DecisionEntry, 34
  - HistoryEntry, 121
- DecisionEntry, 32
  - activeProbabilityEntryType, 33
  - decision, 34
  - depth, 34
  - globalState, 34
  - globalTransitionControlled, 34
  - newStatus, 34
  - previousProbabilityAnswerValues, 34
  - previousProbabilityEntryType, 35
  - prevStatus, 35
  - stateProbabilityPrevious, 35
  - strategy, 35
  - toString, 33
  - type, 35
- depth
  - DecisionEntry, 34
  - HistoryEntry, 121
  - StateVerificationInfo, 153
- DEPTH\_PREFIX
  - Verification.cpp, 339
  - VerificationIterative.cpp, 343
- dissolveLoopedProbabilityTransition
  - VerificationIterative, 190
- dotdir
  - Cfg, 28
- DotGraph, 36
  - addEdge, 38
  - addNode, 38
  - AgentTemplate, 24
  - DotGraph, 37
  - edges, 39
  - graphBase, 39
  - graphName, 39
  - nodes, 39
  - saveToFile, 38
  - styleString, 39
- DotGraph.cpp, 209
- DotGraph.hpp, 209
  - AGENT\_TEMPLATE, 210
  - DotGraphBase, 210
  - GLOBAL\_MODEL, 210
  - LOCAL\_MODEL, 210
- DotGraphBase
  - DotGraph.hpp, 210
- ECHO
  - scanner.c, 252
  - strategyScanner.c, 300
- edges
  - DotGraph, 39
- endState
  - TransitionTemplate, 166
- entries
  - HistoryDbg, 119
- Environment
  - expressions.hpp, 214
- environment
  - LocalState, 126
- envToString
  - Utils.cpp, 332
  - Utils.hpp, 336
- EOB\_ACT\_CONTINUE\_SCAN
  - scanner.c, 253
  - strategyScanner.c, 300
- EOB\_ACT\_END\_OF\_FILE
  - scanner.c, 253
  - strategyScanner.c, 300
- EOB\_ACT\_LAST\_MATCH
  - scanner.c, 253
  - strategyScanner.c, 300
- EpistemicClass, 40
  - fixedCoalitionTransition, 41
  - globalStates, 41
  - hash, 41
- EpistemicClass.hpp, 211
- epistemicClasses
  - GlobalModel, 93
  - GlobalState, 111
- epistemicClassesAllAgents
  - GlobalState, 111
- epistemicClassesKnowledge
  - GlobalModel, 94
- epistemicGlobalStates
  - LocalState, 126

- EQ
  - Types.hpp, 327
- Equals
  - ConditionOperator.hpp, 208
- equivalentGlobalTransitions
  - Verification, 175
  - VerificationIterative, 191
- eval
  - ExprAdd, 43
  - ExprAnd, 47
  - ExprConst, 49
  - ExprDiv, 52
  - ExprEq, 54
  - ExprGe, 57
  - ExprGt, 59
  - ExprHart, 62
  - ExprIdent, 64, 65
  - ExprKnow, 67
  - ExprLe, 70
  - ExprLt, 72
  - ExprMul, 75
  - ExprNe, 77
  - ExprNode, 79
  - ExprNot, 81
  - ExprOr, 84
  - ExprRem, 86
  - ExprSub, 89
  - ProbAdd, 139
  - ProbConst, 141
  - ProbDiv, 144
  - ProbMul, 146
  - ProbNode, 148
  - ProbSub, 150
- expandAllStates
  - GlobalModelGenerator, 99
- expandAndReduceAllStates
  - GlobalModelGenerator, 99
- expandState
  - GlobalModelGenerator, 99
- expandStateAndReturn
  - GlobalModelGenerator, 99
- expr
  - STRATSTYPE, 163
  - YYSTYPE, 203
- ExprAdd, 42
  - eval, 43
- ExprAdd, 43
- ExprAnd, 45
  - eval, 47
- ExprAnd, 46
- ExprConst, 47
  - eval, 49
- ExprConst, 49
- ExprDiv, 50
  - eval, 52
- ExprDiv, 51
- ExprEq, 52
  - eval, 54
- ExprEq, 54
- expressions.cc, 211
- expressions.hpp, 212
- Environment, 214
- ExprGe, 55
  - eval, 57
- ExprGe, 56
- ExprGt, 57
  - eval, 59
- ExprGt, 59
- ExprHart, 60
  - eval, 62
- ExprHart, 61
- ExprIdent, 63
  - eval, 64, 65
- ExprIdent, 64
- ExprKnow, 65
  - eval, 67
- ExprKnow, 67
- ExprLe, 68
  - eval, 70
- ExprLe, 69
- ExprLt, 70
  - eval, 72
- ExprLt, 72
- ExprMul, 73
  - eval, 75
- ExprMul, 74
- ExprNe, 75
  - eval, 77
- ExprNe, 77
- ExprNode, 78
  - eval, 79
- ExprNot, 79
  - eval, 81
- ExprNot, 81
- ExprOr, 82
  - eval, 84
- ExprOr, 83
- ExprRem, 84
  - eval, 86
- ExprRem, 86
- ExprSub, 87
  - eval, 89
- ExprSub, 88
- F
  - Types.hpp, 328
- FALSE
  - TypesDependency.hpp, 330
- findGlobalStateInEpistemicClass
  - GlobalModelGenerator, 100
- findLoopedProbabilityTransitions
  - VerificationIterative, 191
- findOrCreateEpistemicClass
  - GlobalModelGenerator, 100
- findOrCreateEpistemicClassForKnowledge
  - GlobalModelGenerator, 101
- fixedCoalitionTransition

- EpistemicClass, 41
- fixpoint
  - Cfg, 28
- fixpointVerify
  - Verification, 175
- FLEX\_BETA
  - scanner.c, 253
  - strategyScanner.c, 300
- flex\_int16\_t
  - scanner.c, 263
  - strategyScanner.c, 319
- flex\_int32\_t
  - scanner.c, 264
  - strategyScanner.c, 319
- flex\_int8\_t
  - scanner.c, 264
  - strategyScanner.c, 319
- FLEX\_SCANNER
  - scanner.c, 253
  - strategyScanner.c, 301
- flex\_uint16\_t
  - scanner.c, 264
  - strategyScanner.c, 319
- flex\_uint32\_t
  - scanner.c, 264
  - strategyScanner.c, 320
- flex\_uint8\_t
  - scanner.c, 264
  - strategyScanner.c, 320
- FLEXINT\_H
  - scanner.c, 253
  - strategyScanner.c, 301
- floatno
  - STRATSTYPE, 164
  - YYSTYPE, 203
- fname
  - Cfg, 29
- Formula, 89
  - coalition, 90
  - isCTL, 90
  - isF, 90
  - p, 91
  - probability, 91
  - probabilitySign, 91
- formula
  - Cfg, 29
  - FormulaTemplate, 92
  - GlobalModel, 94
  - GlobalModelGenerator, 107
- formula\_from\_parameter
  - Cfg, 29
- formulaDescription
  - main.cpp, 223
  - ModelParser.cc, 225
  - parser.c, 245
- FormulaTemplate, 91
  - coalition, 92
  - formula, 92
  - isF, 92
  - probability, 92
  - probabilitySign, 92
- free
  - parser.c, 244
  - strategyParser.c, 289
- from
  - GlobalTransition, 115
  - LocalTransition, 129
- fromState
  - StateVerificationInfo, 154
- G
  - Types.hpp, 328
- GE
  - Types.hpp, 327
- generateAgent
  - AgentTemplate, 22
- generateGlobalTransitions
  - GlobalModelGenerator, 101
- generateInitState
  - GlobalModelGenerator, 101
- generateNextMDP
  - GlobalModelGenerator, 102
- generateStateFromLocalStates
  - GlobalModelGenerator, 102
- generator
  - Verification, 183
  - VerificationIterative, 195
- getActionNameFromStateInStrategy
  - GlobalModelGenerator, 102
- getAgentInstanceByName
  - GlobalModelGenerator, 103
- getCoalitionActionSignature
  - GlobalModelGenerator, 103
- getCoalitionIdentifier
  - GlobalModelGenerator, 103
- getContextModel
  - Utils.cpp, 332
  - Utils.hpp, 337
- getCurrentGlobalModel
  - GlobalModelGenerator, 103
- getEpistemicClassForGlobalState
  - Verification, 175
  - VerificationIterative, 191
- getFormula
  - GlobalModelGenerator, 104
- getFormulaCorectness
  - GlobalModelGenerator, 104
- getFormulaSize
  - GlobalModelGenerator, 104
- getGlobalStateEnvironment
  - GlobalState, 110
- getLocalStatesSCC
  - Utils.cpp, 333
  - Utils.hpp, 337
- getMemCap
  - Utils.cpp, 333
  - Utils.hpp, 337

- getMinProbability
  - ProbabilityEntry, 135
- getNaturalStrategy
  - Verification, 175
  - VerificationIterative, 192
- getNextPath
  - GlobalModelGenerator, 104
- getProbability
  - GlobalTransition, 115
- getReducedStrategy
  - Verification, 176
  - VerificationIterative, 192
- getStrategy
  - StrategyCollection, 158
- getStrategyComplexity
  - Verification, 176
  - VerificationIterative, 192
- getSumProbability
  - ProbabilityEntry, 135
- getVerificationMode
  - ProbabilityEntry, 135
- GLOBAL\_MODEL
  - DotGraph.hpp, 210
- GlobalModel, 93
  - agents, 93
  - epistemicClasses, 93
  - epistemicClassesKnowledge, 94
  - formula, 94
  - globalStates, 94
  - initState, 94
- globalModel
  - GlobalModelGenerator, 107
- GlobalModel.hpp, 214
- globalModelCandidates
  - GlobalModelGenerator, 107
- GlobalModelGenerator, 95
  - ~GlobalModelGenerator, 97
  - actionCounts, 106
  - addedStates, 106
  - agentIndex, 106
  - candidateStateDepths, 106
  - checkLocalStates, 98
  - choiceIndices, 106
  - coalitionTransitions, 106
  - computeEpistemicClassHash, 98
  - computeGlobalStateHash, 98
  - correctModel, 106
  - createProbabilityStrategy, 98
  - currentStrategy, 107
  - expandAllStates, 99
  - expandAndReduceAllStates, 99
  - expandState, 99
  - expandStateAndReturn, 99
  - findGlobalStateInEpistemicClass, 100
  - findOrCreateEpistemicClass, 100
  - findOrCreateEpistemicClassForKnowledge, 101
  - formula, 107
  - generateGlobalTransitions, 101
  - generateInitState, 101
  - generateNextMDP, 102
  - generateStateFromLocalStates, 102
  - getActionNameFromStateInStrategy, 102
  - getAgentInstanceByName, 103
  - getCoalitionActionSignature, 103
  - getCoalitionIdentifier, 103
  - getCurrentGlobalModel, 103
  - getFormula, 104
  - getFormulaCorectness, 104
  - getFormulaSize, 104
  - getNextPath, 104
  - globalModel, 107
  - globalModelCandidates, 107
  - GlobalModelGenerator, 97
  - initModel, 105
  - initStrategy, 105
  - localModels, 107
  - markFormulaAsIncorrect, 105
  - opponentsTransitions, 107
  - stateDepths, 108
  - stateHashMapCache, 108
  - strategiesExhausted, 108
  - strategyCollection, 108
  - strategyGenerationInit, 108
- GlobalModelGenerator.cpp, 215
  - config, 215
- GlobalModelGenerator.hpp, 216
- GlobalModelGenerator::GlobalStateTupleHash, 113
  - operator(), 113
- GlobalState, 109
  - epistemicClasses, 111
  - epistemicClassesAllAgents, 111
  - getGlobalStateEnvironment, 110
  - GlobalState, 110
  - globalTransitions, 111
  - goodState, 111
  - hash, 111
  - isExpanded, 111
  - localStatesProjection, 111
  - preimage, 112
  - probability, 112
  - probabilityLoop, 112
  - probabilityResult, 112
  - stateVerifResult, 112
  - toString, 110
  - verificationStatus, 112
- globalState
  - DecisionEntry, 34
  - HistoryEntry, 121
  - StateVerificationInfo, 154
- GlobalState.cpp, 217
- GlobalState.hpp, 217
  - config, 218
- globalStates
  - EpistemicClass, 41
  - GlobalModel, 94
- globalStateToValueBits

- Verification, 176
- VerificationIterative, 192
- GlobalStateVerificationStatus
  - GlobalStateVerificationStatus.hpp, 218
- GlobalStateVerificationStatus.hpp, 218
- GlobalStateVerificationStatus, 218
- PENDING, 219
- UNVERIFIED, 219
- VERIFIED\_ERR, 219
- VERIFIED\_OK, 219
- GlobalTransition, 114
  - from, 115
  - getProbability, 115
  - GlobalTransition, 115
  - id, 115
  - isInvalidDecision, 116
  - joinLocalTransitionNames, 115
  - localTransitions, 116
  - next\_id, 116
  - to, 116
- GlobalTransition.cpp, 219
- GlobalTransition.hpp, 219
- globalTransitionControlled
  - DecisionEntry, 34
  - HistoryEntry, 121
- globalTransitions
  - GlobalState, 111
- globalValues
  - StrategyEntry, 161
- goodState
  - GlobalState, 111
- gotResponseFromOtherState
  - StateVerificationInfo, 154
- gotThroughPreselectedTransition
  - StateVerificationInfo, 154
- graphBase
  - DotGraph, 39
- graphName
  - DotGraph, 39
- GraphTraversalMode
  - VerificationIterative.hpp, 345
- GT
  - Types.hpp, 327
- hasControlledProbabilistic
  - StateVerificationInfo, 154
- hash
  - Action, 14
  - ActionTemplate, 15
  - EpistemicClass, 41
  - GlobalState, 111
- hasUncontrolledProbabilistic
  - StateVerificationInfo, 154
- hasValidControlledTransition
  - StateVerificationInfo, 154
- hasValidUncontrolledTransition
  - StateVerificationInfo, 154
- history
  - VerificationIterative, 195
- HistoryDbg, 117
  - ~HistoryDbg, 118
  - addEntry, 118
  - cloneEntry, 118
  - entries, 119
  - HistoryDbg, 118
  - markEntry, 119
  - print, 119
- historyDecisionsERR
  - Verification, 177
- historyEnd
  - Verification, 183
- HistoryEntry, 120
  - decision, 121
  - depth, 121
  - globalState, 121
  - globalTransitionControlled, 121
  - newStatus, 122
  - next, 122
  - prev, 122
  - prevStatus, 122
  - strategy, 122
  - toString, 121
  - type, 122
- HistoryEntryType
  - TypesDependency.hpp, 329
- historyStart
  - Verification, 183
- historyToRestore
  - Verification, 183
- id
  - Agent, 18
  - GlobalTransition, 115
  - LocalState, 126
  - LocalTransition, 130
- ident
  - Assignment, 26
  - STRATSTYPE, 164
  - YYSTYPE, 203
- identList
  - STRATSTYPE, 164
- identSet
  - YYSTYPE, 203
- if
  - scanner.c, 265
  - strategyScanner.c, 320
- includesState
  - Agent, 17
- increaseProbability
  - Verification, 177
- INITIAL
  - scanner.c, 253
  - strategyScanner.c, 301
- initialValue
  - Var, 168
- initModel
  - GlobalModelGenerator, 105
- initState

- Agent, 18
- GlobalModel, 94
- initStrategy
  - GlobalModelGenerator, 105
- INT16\_MAX
  - scanner.c, 253
  - strategyScanner.c, 301
- INT16\_MIN
  - scanner.c, 254
  - strategyScanner.c, 301
- INT32\_MAX
  - scanner.c, 254
  - strategyScanner.c, 301
- INT32\_MIN
  - scanner.c, 254
  - strategyScanner.c, 301
- INT8\_MAX
  - scanner.c, 254
  - strategyScanner.c, 301
- INT8\_MIN
  - scanner.c, 254
  - strategyScanner.c, 302
- isAgentInCoalition
  - Verification, 177
  - VerificationIterative, 193
- isControlledByCoalition
  - StateVerificationInfo, 155
- isCTL
  - Formula, 90
- isExpanded
  - GlobalState, 111
- isF
  - Formula, 90
  - FormulaTemplate, 92
- isGlobalTransitionControlledByCoalition
  - Verification, 177
  - VerificationIterative, 193
- isInvalidDecision
  - GlobalTransition, 116
- isShared
  - LocalTransition, 130
- joinLocalTransitionNames
  - GlobalTransition, 115
- LE
  - Types.hpp, 327
- loadConfigFromArgs
  - Utils.cpp, 333
  - Utils.hpp, 337
- loadConfigFromFile
  - Utils.cpp, 333
  - Utils.hpp, 337
- LOCAL\_MODEL
  - DotGraph.hpp, 210
- LocalModels, 123
  - agents, 124
- localModels
  - GlobalModelGenerator, 107
- localModelsToString
  - Utils.cpp, 333
  - Utils.hpp, 338
- localName
  - LocalTransition, 130
- LocalState, 124
  - agent, 126
  - compare, 125
  - environment, 126
  - epistemicGlobalStates, 126
  - id, 126
  - localTransitions, 126
  - name, 127
  - toString, 125
- LocalState.cpp, 220
- LocalState.hpp, 221
- localStates
  - Agent, 18
- localStatesProjection
  - GlobalState, 111
- LocalStateTemplate, 127
  - name, 128
  - transitions, 128
- LocalTransition, 128
  - agent, 129
  - conditions, 129
  - from, 129
  - id, 130
  - isShared, 130
  - localName, 130
  - name, 130
  - probability, 130
  - sharedCount, 130
  - to, 131
- LocalTransition.hpp, 221
- localTransitions
  - Agent, 18
  - GlobalTransition, 116
  - LocalState, 126
- lowerProbability
  - Verification, 178
- LT
  - Types.hpp, 327
- main
  - main.cpp, 223
  - test.cpp, 325
- main.cpp, 222
  - config, 223
  - formulaDescription, 223
  - main, 223
  - modelDescription, 223
- main2
  - parser.c, 244
- malloc
  - parser.c, 244
  - strategyParser.c, 289
- MARK\_DECISION\_AS\_INVALID
  - TypesDependency.hpp, 330

- markEntry
  - HistoryDbg, 119
- markFormulaAsIncorrect
  - GlobalModelGenerator, 105
- matchName
  - TransitionTemplate, 166
- maxFixpointVerify
  - Verification, 178
- MIN\_PROBABILITY
  - TypesDependency.hpp, 330
- minFixpointVerify
  - Verification, 178
- mode
  - Verification, 184
  - VerificationIterative, 196
- model
  - YYSTYPE, 203
- model\_id
  - Cfg, 29
- modelDescription
  - main.cpp, 223
  - ModelParser.cc, 225
  - parser.c, 245
- ModelParser, 131
  - ~ModelParser, 132
  - ModelParser, 132
  - parse, 132
  - parseAndOverwriteFormula, 132
- ModelParser.cc, 224
  - formulaDescription, 225
  - modelDescription, 225
  - set\_input\_string, 224
  - yyparse, 224
  - yyrestart, 225
- ModelParser.hpp, 225
- name
  - Agent, 18
  - LocalState, 127
  - LocalStateTemplate, 128
  - LocalTransition, 130
  - Var, 168
- natural\_strategy
  - Cfg, 29
- naturalStrategy
  - Verification, 184
  - VerificationIterative, 196
- NE
  - Types.hpp, 327
- newHistoryMarkDecisionAsInvalid
  - Verification, 178
- newStatus
  - DecisionEntry, 34
  - HistoryEntry, 122
- next
  - HistoryEntry, 122
- next\_id
  - GlobalTransition, 116
- nodes
  - DotGraph, 39
  - nodes.cc, 226
  - nodes.hpp, 227
  - NONE
    - Types.hpp, 327
  - NONE\_PROBABILITY
    - TypesDependency.hpp, 330
  - NORMAL
    - Verification.hpp, 342
  - NORMAL\_TRAVERSAL
    - VerificationIterative.hpp, 345
  - NOT\_MODIFIED
    - TypesDependency.hpp, 330
  - NOT\_SET
    - Types.hpp, 328
  - NOT\_VERIFIED
    - TypesDependency.hpp, 330
  - NotEquals
    - ConditionOperator.hpp, 208
- operator()
  - GlobalModelGenerator::GlobalStateTupleHash, 113
  - StrategyBitsComparator, 156
- opponentsTransitions
  - GlobalModelGenerator, 107
- output\_dot\_files
  - Cfg, 29
- output\_global\_model
  - Cfg, 29
- output\_local\_models
  - Cfg, 30
- outputGlobalModel
  - Utils.cpp, 334
  - Utils.hpp, 338
- p
  - Formula, 91
- parse
  - ModelParser, 132
  - StrategyParser, 162
- parseAndOverwriteFormula
  - ModelParser, 132
- parser.c, 228
  - formulaDescription, 245
  - free, 244
  - main2, 244
  - malloc, 244
  - modelDescription, 245
  - set\_input\_string, 244
  - YY\_, 231
  - YY\_ACCESSING\_SYMBOL, 231
  - YY\_ASSERT, 231
  - YY\_ATTRIBUTE\_PURE, 231
  - YY\_ATTRIBUTE\_UNUSED, 231
  - YY\_BUFFER\_STATE, 240
  - YY\_CAST, 231
  - YY\_IGNORE\_MAYBE\_UNINITIALIZED\_BEGIN, 232

- YY\_IGNORE\_MAYBE\_UNINITIALIZED\_END, 232
- YY\_IGNORE\_USELESS\_CAST\_BEGIN, 232
- YY\_IGNORE\_USELESS\_CAST\_END, 232
- YY\_INITIAL\_VALUE, 232
- YY\_NULLPTR, 232
- YY\_REDUCE\_PRINT, 232
- YY\_REINTERPRET\_CAST, 233
- yy\_scan\_string, 244
- YY\_STACK\_PRINT, 233
- yy\_state\_fast\_t, 241
- yy\_state\_t, 241
- YY\_SYMBOL\_PRINT, 233
- YY\_USE, 233
- YYABORT, 233
- YYACCEPT, 233
- YYBACKUP, 234
- YYBISON, 234
- YYBISON\_VERSION, 234
- yychar, 245
- yyclearin, 234
- YYCOPY, 234
- YYCOPY\_NEEDED, 235
- YYDPRINTF, 235
- YYENOMEM, 242
- YYERRCODE, 235
- yyerrok, 235
- YYERROR, 235
- yyerror, 244
- YYFINAL, 235
- YYFREE, 236
- YYINITDEPTH, 236
- YYLAST, 236
- yylex, 245
- yylval, 245
- YYMALLOC, 236
- YYMAXDEPTH, 236
- YYMAXUTOK, 236
- yynerres, 246
- YYNNTS, 236
- YYNOMEM, 236
- YYNRULES, 237
- YYNSTATES, 237
- YYNTOKENS, 237
- YYPACT\_NINF, 237
- yypact\_value\_is\_default, 237
- yyparse, 245
- YYPPOPSTACK, 237
- YYPTRDIFF\_MAXIMUM, 237
- YYPTRDIFF\_T, 238
- YYPULL, 238
- YYPURE, 238
- YYPUSH, 238
- YYRECOVERING, 238
- yyrestart, 245
- YYSIZE\_MAXIMUM, 238
- YYSIZE\_T, 238
- YYSIZEOF, 239
- YYSKELETON\_NAME, 239
- YYSTACK\_ALLOC, 239
- YYSTACK\_ALLOC\_MAXIMUM, 239
- YYSTACK\_BYTES, 239
- YYSTACK\_FREE, 239
- YYSTACK\_GAP\_MAXIMUM, 239
- YYSTACK\_RELOCATE, 240
- YYSYMBOL\_17\_, 244
- YYSYMBOL\_27\_, 242
- YYSYMBOL\_28\_, 242
- YYSYMBOL\_29\_, 242
- YYSYMBOL\_30\_, 242
- YYSYMBOL\_31\_, 242
- YYSYMBOL\_32\_, 242
- YYSYMBOL\_33\_, 242
- YYSYMBOL\_34\_, 242
- YYSYMBOL\_35\_, 242
- YYSYMBOL\_36\_, 242
- YYSYMBOL\_37\_, 243
- YYSYMBOL\_38\_, 243
- YYSYMBOL\_agent, 243
- YYSYMBOL\_assign\_list, 243
- YYSYMBOL\_assignment, 243
- YYSYMBOL\_assignments, 243
- YYSYMBOL\_coalition, 243
- YYSYMBOL\_cond, 243
- YYSYMBOL\_cond\_and, 243
- YYSYMBOL\_cond\_elem, 243
- YYSYMBOL\_cond\_list, 243
- YYSYMBOL\_condition, 243
- YYSYMBOL\_description, 243
- YYSYMBOL\_formula, 243
- YYSYMBOL\_ident\_list, 243, 244
- YYSYMBOL\_initial, 243
- yysymbol\_kind\_t, 241, 242
- YYSYMBOL\_local, 243
- YYSYMBOL\_model, 243
- YYSYMBOL\_num\_elem, 243
- YYSYMBOL\_num\_exp, 243
- YYSYMBOL\_num\_mul, 243
- YYSYMBOL\_persistent, 243
- YYSYMBOL\_prob\_elem, 243
- YYSYMBOL\_prob\_exp, 243
- YYSYMBOL\_prob\_mul, 243
- YYSYMBOL\_probability\_equality, 243
- YYSYMBOL\_query, 243
- YYSYMBOL\_shared, 243
- YYSYMBOL\_spec, 243
- YYSYMBOL\_strat, 244
- YYSYMBOL\_T\_AGENT, 242
- YYSYMBOL\_T\_AND, 242, 243
- YYSYMBOL\_T\_ASSIGN, 242, 243
- YYSYMBOL\_T\_EQ, 242, 243
- YYSYMBOL\_T\_FLOAT, 242, 243
- YYSYMBOL\_T\_FORMULA, 242
- YYSYMBOL\_T\_GE, 242, 243
- YYSYMBOL\_T\_GT, 242, 243
- YYSYMBOL\_T\_HARTLEY, 242
- YYSYMBOL\_T\_IDENT, 242, 244



- YYSYMBOL\_T\_INIT, [242](#)
- YYSYMBOL\_T\_INITIAL, [242](#)
- YYSYMBOL\_T\_KNOW, [242](#)
- YYSYMBOL\_T\_LE, [242](#), [243](#)
- YYSYMBOL\_T\_LOCAL, [242](#)
- YYSYMBOL\_T\_LT, [242](#), [243](#)
- YYSYMBOL\_T\_NE, [242](#), [243](#)
- YYSYMBOL\_T\_NUM, [242](#), [243](#)
- YYSYMBOL\_T\_OR, [242](#), [243](#)
- YYSYMBOL\_T\_PERSISTENT, [242](#)
- YYSYMBOL\_T\_PROB, [242](#)
- YYSYMBOL\_T\_REST, [242](#)
- YYSYMBOL\_T\_SHARED, [242](#), [243](#)
- YYSYMBOL\_T\_TO, [242](#), [243](#)
- YYSYMBOL\_transition, [243](#)
- YYSYMBOL\_YYACCEPT, [243](#), [244](#)
- YYSYMBOL\_YYEMPTY, [242](#), [243](#)
- YYSYMBOL\_YYEOF, [242](#), [243](#)
- YYSYMBOL\_YYerror, [242](#), [243](#)
- YYSYMBOL\_YYUNDEF, [242](#), [243](#)
- YYTABLE\_NINF, [240](#)
- yytable\_value\_is\_error, [240](#)
- yytext, [246](#)
- YYTRANSLATE, [240](#)
- yytype\_int16, [241](#)
- yytype\_int8, [241](#)
- yytype\_uint16, [241](#)
- yytype\_uint8, [241](#)
- parser.h, [246](#)
  - T\_AGENT, [248](#)
  - T\_AND, [248](#)
  - T\_ASSIGN, [248](#)
  - T\_EQ, [248](#)
  - T\_FLOAT, [249](#)
  - T\_FORMULA, [248](#)
  - T\_GE, [248](#)
  - T\_GT, [248](#)
  - T\_HARTLEY, [249](#)
  - T\_IDENT, [249](#)
  - T\_INIT, [248](#)
  - T\_INITIAL, [248](#)
  - T\_KNOW, [249](#)
  - T\_LE, [249](#)
  - T\_LOCAL, [248](#)
  - T\_LT, [249](#)
  - T\_NE, [248](#)
  - T\_NUM, [249](#)
  - T\_OR, [248](#)
  - T\_PERSISTENT, [248](#)
  - T\_PROB, [249](#)
  - T\_REST, [249](#)
  - T\_SHARED, [249](#)
  - T\_TO, [248](#)
  - YYDEBUG, [247](#)
  - YYEMPTY, [248](#)
  - YYEOF, [248](#)
  - YYerror, [248](#)
  - yylval, [249](#)
  - yyvsparse, [249](#)
  - YYSTYPE, [248](#)
  - YYSTYPE\_IS\_DECLARED, [247](#)
  - YYSTYPE\_IS\_TRIVIAL, [247](#)
  - yytoken\_kind\_t, [248](#)
  - YYTOKENTYPE, [247](#)
  - yytokentype, [248](#)
  - YYUNDEF, [248](#)
- patternName
  - TransitionTemplate, [166](#)
- PENDING
  - GlobalStateVerificationStatus.hpp, [219](#)
- persistent
  - Var, [168](#)
- preimage
  - GlobalState, [112](#)
- prev
  - HistoryEntry, [122](#)
- previousProbabilityAnswerValues
  - DecisionEntry, [34](#)
- previousProbabilityEntryType
  - DecisionEntry, [35](#)
- prevStatus
  - DecisionEntry, [35](#)
  - HistoryEntry, [122](#)
- print
  - HistoryDbg, [119](#)
- printCurrentHistory
  - Verification, [179](#)
- prob
  - YYSTYPE, [203](#)
- PROBABILITY
  - TypesDependency.hpp, [330](#)
- probability
  - Cfg, [30](#)
  - Formula, [91](#)
  - FormulaTemplate, [92](#)
  - GlobalState, [112](#)
  - LocalTransition, [130](#)
  - TransitionTemplate, [167](#)
- ProbabilityCalculationType
  - TypesDependency.hpp, [330](#)
- ProbabilityEntry, [133](#)
  - ~ProbabilityEntry, [134](#)
  - changeMinProbabilityFalse, [134](#)
  - changeMinProbabilityTrue, [135](#)
  - changeSumProbabilityFalse, [135](#)
  - changeSumProbabilityTrue, [135](#)
  - changeVerificationMode, [135](#)
  - getMinProbability, [135](#)
  - getSumProbability, [135](#)
  - getVerificationMode, [135](#)
  - ProbabilityEntry, [134](#)
  - returnProbability, [136](#)
  - setMinProbability, [136](#)
  - setSumProbability, [136](#)
- probabilityFalse
  - ProbabilityTrueFalse, [137](#)

- probabilityLoop
  - GlobalState, [112](#)
- probabilityResult
  - GlobalState, [112](#)
  - Result, [152](#)
- ProbabilitySign
  - Types.hpp, [327](#)
- probabilitySign
  - Formula, [91](#)
  - FormulaTemplate, [92](#)
- probabilityTrue
  - ProbabilityTrueFalse, [137](#)
- ProbabilityTrueFalse, [136](#)
  - probabilityFalse, [137](#)
  - probabilityTrue, [137](#)
- ProbAdd, [137](#)
  - eval, [139](#)
  - ProbAdd, [138](#)
- ProbConst, [139](#)
  - eval, [141](#)
  - ProbConst, [141](#)
- ProbDiv, [142](#)
  - eval, [144](#)
  - ProbDiv, [143](#)
- ProbMul, [144](#)
  - eval, [146](#)
  - ProbMul, [146](#)
- ProbNode, [147](#)
  - eval, [148](#)
- ProbSub, [149](#)
  - eval, [150](#)
  - ProbSub, [150](#)
- processed
  - StateVerificationInfo, [155](#)
- README.md, [249](#)
- reduce
  - Cfg, [30](#)
- reduce\_all
  - Cfg, [30](#)
- reduce\_args
  - Cfg, [30](#)
- reduceStrategy
  - Verification, [179](#)
  - VerificationIterative, [193](#)
- reductionComplexityBefore
  - Verification, [184](#)
  - VerificationIterative, [196](#)
- REJECT
  - scanner.c, [254](#)
  - strategyScanner.c, [302](#)
- RESTORE
  - Verification.hpp, [342](#)
- RESTORE\_STATES
  - VerificationIterative.hpp, [345](#)
- restoreHistory
  - Verification, [180](#)
- Result, [151](#)
  - probabilityResult, [152](#)
  - verificationResult, [152](#)
- returnProbability
  - ProbabilityEntry, [136](#)
- REVERT
  - Verification.hpp, [342](#)
- revertLastDecision
  - Verification, [180](#)
- revertToGlobalState
  - Verification, [184](#)
  - VerificationIterative, [196](#)
- revertToLastDecision
  - VerificationIterative, [194](#)
- saveToFile
  - DotGraph, [38](#)
- scanner.c, [249](#)
  - BEGIN, [252](#)
  - ECHO, [252](#)
  - EOB\_ACT\_CONTINUE\_SCAN, [253](#)
  - EOB\_ACT\_END\_OF\_FILE, [253](#)
  - EOB\_ACT\_LAST\_MATCH, [253](#)
  - FLEX\_BETA, [253](#)
  - flex\_int16\_t, [263](#)
  - flex\_int32\_t, [264](#)
  - flex\_int8\_t, [264](#)
  - FLEX\_SCANNER, [253](#)
  - flex\_uint16\_t, [264](#)
  - flex\_uint32\_t, [264](#)
  - flex\_uint8\_t, [264](#)
  - FLEXINT\_H, [253](#)
  - if, [265](#)
  - INITIAL, [253](#)
  - INT16\_MAX, [253](#)
  - INT16\_MIN, [254](#)
  - INT32\_MAX, [254](#)
  - INT32\_MIN, [254](#)
  - INT8\_MAX, [254](#)
  - INT8\_MIN, [254](#)
  - REJECT, [254](#)
  - SIZE\_MAX, [254](#)
  - UINT16\_MAX, [254](#)
  - UINT32\_MAX, [255](#)
  - UINT8\_MAX, [255](#)
  - unput, [255](#)
  - yy\_act, [269](#)
  - YY\_AT\_BOL, [255](#)
  - yy\_bp, [269](#)
  - YY\_BREAK, [255](#)
  - YY\_BUF\_SIZE, [255](#)
  - YY\_BUFFER\_EOF\_PENDING, [255](#)
  - YY\_BUFFER\_NEW, [256](#)
  - YY\_BUFFER\_NORMAL, [256](#)
  - YY\_BUFFER\_STATE, [264](#)
  - YY\_CHAR, [264](#)
  - yy\_cp, [269](#)
  - yy\_create\_buffer, [265](#)
  - YY\_CURRENT\_BUFFER, [256](#)
  - YY\_CURRENT\_BUFFER\_LVALUE, [256](#)
  - YY\_DECL, [256](#), [269](#)

YY\_DECL\_IS\_OURS, [256](#)  
yy\_delete\_buffer, [265](#)  
YY\_DO\_BEFORE\_ACTION, [256](#)  
YY\_END\_OF\_BUFFER, [257](#)  
YY\_END\_OF\_BUFFER\_CHAR, [257](#)  
YY\_EXIT\_FAILURE, [257](#)  
YY\_EXTRA\_TYPE, [257](#)  
YY\_FATAL\_ERROR, [257](#)  
yy\_flex\_debug, [270](#)  
YY\_FLEX\_MAJOR\_VERSION, [257](#)  
YY\_FLEX\_MINOR\_VERSION, [257](#)  
YY\_FLEX\_SUBMINOR\_VERSION, [258](#)  
YY\_FLUSH\_BUFFER, [258](#)  
yy\_flush\_buffer, [265](#)  
YY\_INPUT, [258](#)  
YY\_INT\_ALIGNED, [258](#)  
YY\_LESS\_LINENO, [258](#)  
YY\_LINENO\_REWIND\_TO, [259](#)  
YY\_MORE\_ADJ, [259](#)  
yy\_new\_buffer, [259](#)  
YY\_NEW\_FILE, [259](#)  
YY\_NULL, [259](#)  
YY\_NUM\_RULES, [259](#)  
YY\_READ\_BUF\_SIZE, [259](#)  
YY\_RESTORE\_YY\_MORE\_OFFSET, [260](#)  
YY\_RULE\_SETUP, [260](#)  
YY\_SC\_TO\_UI, [260](#)  
yy\_scan\_buffer, [265](#)  
yy\_scan\_bytes, [266](#)  
yy\_scan\_string, [266](#)  
yy\_set\_bol, [260](#)  
yy\_set\_interactive, [260](#)  
yy\_size\_t, [264](#)  
YY\_SKIP\_YYWRAP, [260](#)  
YY\_START, [261](#)  
YY\_START\_STACK\_INCR, [261](#)  
YY\_STATE\_BUF\_SIZE, [261](#)  
YY\_STATE\_EOF, [261](#)  
yy\_state\_type, [265](#)  
YY\_STRUCT\_YY\_BUFFER\_STATE, [261](#)  
yy\_switch\_to\_buffer, [266](#)  
YY\_TYPEDEF\_YY\_BUFFER\_STATE, [261](#)  
YY\_TYPEDEF\_YY\_SIZE\_T, [261](#)  
YY\_USER\_ACTION, [262](#)  
yyalloc, [266](#)  
yyconst, [262](#)  
yyfree, [266](#)  
yyget\_debug, [266](#)  
yyget\_extra, [266](#)  
yyget\_in, [267](#)  
yyget\_leng, [267](#)  
yyget\_lineno, [267](#)  
yyget\_out, [267](#)  
yyget\_text, [267](#)  
yyin, [270](#)  
yyleng, [270](#)  
yyless, [262](#)  
yylex, [267](#)  
yylex\_destroy, [267](#)  
yylineno, [270](#)  
yymore, [262](#)  
yynoreturn, [263](#)  
yyout, [270](#)  
yypop\_buffer\_state, [268](#)  
yypush\_buffer\_state, [268](#)  
yyrealloc, [268](#)  
yyrestart, [268](#)  
yyset\_debug, [268](#)  
yyset\_extra, [268](#)  
yyset\_in, [268](#)  
yyset\_lineno, [269](#)  
yyset\_out, [269](#)  
YYSTATE, [263](#)  
YYTABLES\_NAME, [263](#)  
yyterminate, [263](#)  
yytext, [270](#)  
yytext\_ptr, [263](#)  
yywrap, [263](#)  
scanner.l, [271](#)  
selectedStrategy  
    StrategyCollectionTemplate, [160](#)  
set\_input\_string  
    ModelParser.cc, [224](#)  
    parser.c, [244](#)  
set\_input\_string\_strat  
    strategyParser.c, [289](#)  
    StrategyParser.cc, [291](#)  
setIdent  
    AgentTemplate, [23](#)  
setInitState  
    AgentTemplate, [24](#)  
setMinProbability  
    ProbabilityEntry, [136](#)  
setSumProbability  
    ProbabilityEntry, [136](#)  
shared  
    TransitionTemplate, [167](#)  
sharedCount  
    LocalTransition, [130](#)  
SIZE\_MAX  
    scanner.c, [254](#)  
    strategyScanner.c, [302](#)  
startState  
    TransitionTemplate, [167](#)  
STATE\_STATUS  
    TypesDependency.hpp, [330](#)  
stateDepths  
    GlobalModelGenerator, [108](#)  
stateHashMapCache  
    GlobalModelGenerator, [108](#)  
stateProbabilityPrevious  
    DecisionEntry, [35](#)  
states  
    Action, [14](#)  
    ActionTemplate, [16](#)  
statesToProcess

- VerificationIterative, 196
- StateVerificationInfo, 152
  - controlled, 153
  - controlledProbabilisticTransitionsLeftToProcess, 153
  - controlledTransitionsLeftToProcess, 153
  - depth, 153
  - fromState, 154
  - globalState, 154
  - gotResponseFromOtherState, 154
  - gotThroughPreselectedTransition, 154
  - hasControlledProbabilistic, 154
  - hasUncontrolledProbabilistic, 154
  - hasValidControlledTransition, 154
  - hasValidUncontrolledTransition, 154
  - isControlledByCoalition, 155
  - processed, 155
  - uncontrolled, 155
  - uncontrolledProbabilisticTransitionsLeftToProcess, 155
  - uncontrolledTransitionsLeftToProcess, 155
  - verifResult, 155
- stateVerifResult
  - GlobalState, 112
- strat\_create\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 302
- strat\_delete\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 302
- strat\_flex\_debug\_ALREADY\_DEFINED
  - strategyScanner.c, 302
- strat\_flush\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 302
- strat\_init\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 302
- strat\_load\_buffer\_state\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- strat\_scan\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- strat\_scan\_bytes\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- strat\_scan\_string
  - strategyParser.c, 289
- strat\_scan\_string\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- strat\_switch\_to\_buffer\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- stratalloc\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- STRATDEBUG
  - strategyParser.h, 293
- strategiesExhausted
  - GlobalModelGenerator, 108
- strategy
  - DecisionEntry, 35
  - HistoryEntry, 122
- STRATEGY\_BITS
  - TypesDependency.hpp, 329
  - Verification.hpp, 342
- strategy\_file\_path
  - Cfg, 30
- StrategyBitsComparator, 156
  - operator(), 156
- StrategyCollection, 157
  - ~StrategyCollection, 157
  - addAction, 158
  - addStrategy, 158
  - clear, 158
  - getStrategy, 158
  - StrategyCollection, 157, 158
- strategyCollection
  - GlobalModelGenerator, 108
- StrategyCollectionTemplate, 159
  - addAction, 159
  - selectedStrategy, 160
  - StrategyCollectionTemplate, 159
- strategyCollectionTemplate
  - strategyParser.c, 290
  - StrategyParser.cc, 292
- StrategyEntry, 160
  - actionName, 161
  - globalValues, 161
  - type, 161
- StrategyEntryType
  - TypesDependency.hpp, 330
- strategyGenerationInit
  - GlobalModelGenerator, 108
- strategyNodes.cc, 271
- strategyNodes.hpp, 271
- StrategyParser, 161
  - ~StrategyParser, 162
  - parse, 162
  - StrategyParser, 162
- strategyParser.c, 272
  - free, 289
  - malloc, 289
  - set\_input\_string\_strat, 289
  - strat\_scan\_string, 289
  - strategyCollectionTemplate, 290
  - straterror, 289
  - stratlex, 289
  - stratrestart, 290
  - strattext, 290
  - YY\_, 275
  - YY\_ACCESSING\_SYMBOL, 275
  - YY\_ASSERT, 275
  - YY\_ATTRIBUTE\_PURE, 275
  - YY\_ATTRIBUTE\_UNUSED, 276
  - YY\_BUFFER\_STATE, 286
  - YY\_CAST, 276
  - YY\_IGNORE\_MAYBE\_UNINITIALIZED\_BEGIN, 276
  - YY\_IGNORE\_MAYBE\_UNINITIALIZED\_END, 276
  - YY\_IGNORE\_USELESS\_CAST\_BEGIN, 276
  - YY\_IGNORE\_USELESS\_CAST\_END, 276
  - YY\_INITIAL\_VALUE, 276
  - YY\_NULLPTR, 277

YY\_REDUCE\_PRINT, [277](#)  
YY\_REINTERPRET\_CAST, [277](#)  
YY\_STACK\_PRINT, [277](#)  
yy\_state\_fast\_t, [286](#)  
yy\_state\_t, [286](#)  
YY\_SYMBOL\_PRINT, [277](#)  
YY\_USE, [277](#)  
YYABORT, [278](#)  
YYACCEPT, [278](#)  
YYBACKUP, [278](#)  
YYBISON, [278](#)  
YYBISON\_VERSION, [278](#)  
yychar, [278](#), [290](#)  
yyclearin, [279](#)  
YYCOPY, [279](#)  
YYCOPY\_NEEDED, [279](#)  
yydebug, [279](#)  
YYDPRINTF, [279](#)  
YYENOMEM, [287](#)  
YYERRCODE, [279](#)  
yyerrok, [280](#)  
YYERROR, [280](#)  
yyerror, [280](#)  
YYFINAL, [280](#)  
YYFREE, [280](#)  
YYINITDEPTH, [280](#)  
YYLAST, [280](#)  
yylex, [280](#)  
yyval, [281](#), [290](#)  
YYMALLOC, [281](#)  
YYMAXDEPTH, [281](#)  
YYMAXUTOK, [281](#)  
yynerrs, [281](#), [290](#)  
YYNNTS, [281](#)  
YYNOMEM, [281](#)  
YYNRULES, [281](#)  
YYNSTATES, [282](#)  
YYNTOKENS, [282](#)  
YYPACT\_NINF, [282](#)  
yypact\_value\_is\_default, [282](#)  
yyparse, [282](#)  
YYPOPSTACK, [282](#)  
YYPTRDIFF\_MAXIMUM, [282](#)  
YYPTRDIFF\_T, [283](#)  
YYPULL, [283](#)  
YYPURE, [283](#)  
YYPUSH, [283](#)  
YYRECOVERING, [283](#)  
YYSIZE\_MAXIMUM, [283](#)  
YYSIZE\_T, [283](#)  
YYSIZEOF, [284](#)  
YYSKELETON\_NAME, [284](#)  
YYSTACK\_ALLOC, [284](#)  
YYSTACK\_ALLOC\_MAXIMUM, [284](#)  
YYSTACK\_BYTES, [284](#)  
YYSTACK\_FREE, [284](#)  
YYSTACK\_GAP\_MAXIMUM, [284](#)  
YYSTACK\_RELOCATE, [285](#)  
YYSTYPE, [285](#)  
YYSYMBOL\_17\_, [289](#)  
YYSYMBOL\_27\_, [287](#)  
YYSYMBOL\_28\_, [287](#)  
YYSYMBOL\_29\_, [287](#)  
YYSYMBOL\_30\_, [287](#)  
YYSYMBOL\_31\_, [287](#)  
YYSYMBOL\_32\_, [287](#)  
YYSYMBOL\_33\_, [287](#)  
YYSYMBOL\_34\_, [287](#)  
YYSYMBOL\_35\_, [287](#)  
YYSYMBOL\_36\_, [287](#)  
YYSYMBOL\_37\_, [288](#)  
YYSYMBOL\_38\_, [288](#)  
YYSYMBOL\_agent, [288](#)  
YYSYMBOL\_assign\_list, [288](#)  
YYSYMBOL\_assignment, [288](#)  
YYSYMBOL\_assignments, [288](#)  
YYSYMBOL\_coalition, [288](#)  
YYSYMBOL\_cond, [288](#)  
YYSYMBOL\_cond\_and, [288](#)  
YYSYMBOL\_cond\_elem, [288](#)  
YYSYMBOL\_cond\_list, [288](#)  
YYSYMBOL\_condition, [288](#)  
YYSYMBOL\_description, [288](#)  
YYSYMBOL\_formula, [288](#)  
YYSYMBOL\_ident\_list, [288](#), [289](#)  
YYSYMBOL\_initial, [288](#)  
yysymbol\_kind\_t, [286](#), [287](#)  
YYSYMBOL\_local, [288](#)  
YYSYMBOL\_model, [288](#)  
YYSYMBOL\_num\_elem, [288](#)  
YYSYMBOL\_num\_exp, [288](#)  
YYSYMBOL\_num\_mul, [288](#)  
YYSYMBOL\_persistent, [288](#)  
YYSYMBOL\_prob\_elem, [288](#)  
YYSYMBOL\_prob\_exp, [288](#)  
YYSYMBOL\_prob\_mul, [288](#)  
YYSYMBOL\_probability\_equality, [288](#)  
YYSYMBOL\_query, [288](#)  
YYSYMBOL\_shared, [288](#)  
YYSYMBOL\_spec, [288](#)  
YYSYMBOL\_strat, [289](#)  
YYSYMBOL\_T\_AGENT, [287](#)  
YYSYMBOL\_T\_AND, [287](#), [288](#)  
YYSYMBOL\_T\_ASSIGN, [287](#), [288](#)  
YYSYMBOL\_T\_EQ, [287](#), [288](#)  
YYSYMBOL\_T\_FLOAT, [287](#), [288](#)  
YYSYMBOL\_T\_FORMULA, [287](#)  
YYSYMBOL\_T\_GE, [287](#), [288](#)  
YYSYMBOL\_T\_GT, [287](#), [288](#)  
YYSYMBOL\_T\_HARTLEY, [287](#)  
YYSYMBOL\_T\_IDENT, [287](#), [289](#)  
YYSYMBOL\_T\_INIT, [287](#)  
YYSYMBOL\_T\_INITIAL, [287](#)  
YYSYMBOL\_T\_KNOW, [287](#)  
YYSYMBOL\_T\_LE, [287](#), [288](#)  
YYSYMBOL\_T\_LOCAL, [287](#)

- YYSYMBOL\_T\_LT, [287](#), [288](#)
- YYSYMBOL\_T\_NE, [287](#), [288](#)
- YYSYMBOL\_T\_NUM, [287](#), [288](#)
- YYSYMBOL\_T\_OR, [287](#), [288](#)
- YYSYMBOL\_T\_PERSISTENT, [287](#)
- YYSYMBOL\_T\_PROB, [287](#)
- YYSYMBOL\_T\_REST, [287](#)
- YYSYMBOL\_T\_SHARED, [287](#), [288](#)
- YYSYMBOL\_T\_TO, [287](#), [288](#)
- YYSYMBOL\_transition, [288](#)
- YYSYMBOL\_YYACCEPT, [288](#), [289](#)
- YYSYMBOL\_YYEMPTY, [287](#), [288](#)
- YYSYMBOL\_YYEOF, [287](#), [288](#)
- YYSYMBOL\_YYerror, [287](#), [288](#)
- YYSYMBOL\_YYUNDEF, [287](#), [288](#)
- YYTABLE\_NINF, [285](#)
- yytable\_value\_is\_error, [285](#)
- YYTRANSLATE, [285](#)
- yytype\_int16, [286](#)
- yytype\_int8, [286](#)
- yytype\_uint16, [286](#)
- yytype\_uint8, [286](#)
- StrategyParser.cc, [291](#)
  - set\_input\_string\_strat, [291](#)
  - strategyCollectionTemplate, [292](#)
  - stratparse, [291](#)
  - stratrestart, [292](#)
- strategyParser.h, [292](#)
  - STRATDEBUG, [293](#)
  - STRATEMPTY, [294](#)
  - STRATEOF, [294](#)
  - STRATerror, [294](#)
  - stratival, [295](#)
  - stratparse, [295](#)
  - STRATSTYPE, [294](#)
  - STRATSTYPE\_IS\_DECLARED, [293](#)
  - STRATSTYPE\_IS\_TRIVIAL, [293](#)
  - strattoken\_kind\_t, [294](#)
  - STRATTOKENTYPE, [293](#)
  - strattokentype, [294](#)
  - STRATUNDEF, [294](#)
  - T\_AND, [294](#)
  - T\_ASSIGN, [294](#)
  - T\_EQ, [294](#)
  - T\_FLOAT, [294](#)
  - T\_GE, [294](#)
  - T\_GT, [294](#)
  - T\_IDENT, [294](#)
  - T\_LE, [294](#)
  - T\_LT, [294](#)
  - T\_NE, [294](#)
  - T\_NUM, [294](#)
  - T\_OR, [294](#)
  - T\_SHARED, [294](#)
  - T\_TO, [294](#)
- StrategyParser.hpp, [295](#)
- strategyScanner.c, [296](#)
  - BEGIN, [300](#)
  - ECHO, [300](#)
  - EOB\_ACT\_CONTINUE\_SCAN, [300](#)
  - EOB\_ACT\_END\_OF\_FILE, [300](#)
  - EOB\_ACT\_LAST\_MATCH, [300](#)
  - FLEX\_BETA, [300](#)
  - flex\_int16\_t, [319](#)
  - flex\_int32\_t, [319](#)
  - flex\_int8\_t, [319](#)
  - FLEX\_SCANNER, [301](#)
  - flex\_uint16\_t, [319](#)
  - flex\_uint32\_t, [320](#)
  - flex\_uint8\_t, [320](#)
  - FLEXINT\_H, [301](#)
  - if, [320](#)
  - INITIAL, [301](#)
  - INT16\_MAX, [301](#)
  - INT16\_MIN, [301](#)
  - INT32\_MAX, [301](#)
  - INT32\_MIN, [301](#)
  - INT8\_MAX, [301](#)
  - INT8\_MIN, [302](#)
  - REJECT, [302](#)
  - SIZE\_MAX, [302](#)
  - strat\_create\_buffer\_ALREADY\_DEFINED, [302](#)
  - strat\_delete\_buffer\_ALREADY\_DEFINED, [302](#)
  - strat\_flex\_debug\_ALREADY\_DEFINED, [302](#)
  - strat\_flush\_buffer\_ALREADY\_DEFINED, [302](#)
  - strat\_init\_buffer\_ALREADY\_DEFINED, [302](#)
  - strat\_load\_buffer\_state\_ALREADY\_DEFINED, [303](#)
  - strat\_scan\_buffer\_ALREADY\_DEFINED, [303](#)
  - strat\_scan\_bytes\_ALREADY\_DEFINED, [303](#)
  - strat\_scan\_string\_ALREADY\_DEFINED, [303](#)
  - strat\_switch\_to\_buffer\_ALREADY\_DEFINED, [303](#)
  - stratalloc\_ALREADY\_DEFINED, [303](#)
  - stratensure\_buffer\_stack\_ALREADY\_DEFINED, [303](#)
  - stratfree\_ALREADY\_DEFINED, [303](#)
  - stratin\_ALREADY\_DEFINED, [304](#)
  - stratleng\_ALREADY\_DEFINED, [304](#)
  - stratlex, [321](#)
  - stratlex\_ALREADY\_DEFINED, [304](#)
  - stratlineno\_ALREADY\_DEFINED, [304](#)
  - stratout\_ALREADY\_DEFINED, [304](#)
  - stratpop\_buffer\_state\_ALREADY\_DEFINED, [304](#)
  - stratpush\_buffer\_state\_ALREADY\_DEFINED, [304](#)
  - stratrealloc\_ALREADY\_DEFINED, [304](#)
  - stratrestart\_ALREADY\_DEFINED, [305](#)
  - strattext\_ALREADY\_DEFINED, [305](#)
  - stratwrap, [305](#)
  - stratwrap\_ALREADY\_DEFINED, [305](#)
  - UINT16\_MAX, [305](#)
  - UINT32\_MAX, [305](#)
  - UINT8\_MAX, [305](#)
  - unput, [305](#)
  - yy\_act, [323](#)
  - YY\_AT\_BOL, [306](#)
  - yy\_bp, [323](#)
  - YY\_BREAK, [306](#)

- YY\_BUF\_SIZE, 306
- YY\_BUFFER\_EOF\_PENDING, 306
- YY\_BUFFER\_NEW, 306
- YY\_BUFFER\_NORMAL, 306
- YY\_BUFFER\_STATE, 320
- YY\_CHAR, 320
- yy\_cp, 324
- yy\_create\_buffer, 306, 321
- YY\_CURRENT\_BUFFER, 307
- YY\_CURRENT\_BUFFER\_LVALUE, 307
- YY\_DECL, 307, 324
- YY\_DECL\_IS\_OURS, 307
- yy\_delete\_buffer, 307, 321
- YY\_DO\_BEFORE\_ACTION, 307
- YY\_END\_OF\_BUFFER, 307
- YY\_END\_OF\_BUFFER\_CHAR, 308
- YY\_EXIT\_FAILURE, 308
- YY\_EXTRA\_TYPE, 308
- YY\_FATAL\_ERROR, 308
- yy\_flex\_debug, 308, 324
- YY\_FLEX\_MAJOR\_VERSION, 308
- YY\_FLEX\_MINOR\_VERSION, 308
- YY\_FLEX\_SUBMINOR\_VERSION, 309
- YY\_FLUSH\_BUFFER, 309
- yy\_flush\_buffer, 309, 321
- yy\_init\_buffer, 309
- YY\_INPUT, 309
- YY\_INT\_ALIGNED, 309
- YY\_LESS\_LINENO, 310
- YY\_LINENO\_REWIND\_TO, 310
- yy\_load\_buffer\_state, 310
- YY\_MORE\_ADJ, 310
- yy\_new\_buffer, 310
- YY\_NEW\_FILE, 310
- YY\_NULL, 310
- YY\_NUM\_RULES, 311
- YY\_READ\_BUF\_SIZE, 311
- YY\_RESTORE\_YY\_MORE\_OFFSET, 311
- YY\_RULE\_SETUP, 311
- YY\_SC\_TO\_UI, 311
- yy\_scan\_buffer, 311, 321
- yy\_scan\_bytes, 311, 321
- yy\_scan\_string, 312, 321
- yy\_set\_bol, 312
- yy\_set\_interactive, 312
- yy\_size\_t, 320
- YY\_SKIP\_YYWRAP, 312
- YY\_START, 312
- YY\_START\_STACK\_INCR, 313
- YY\_STATE\_BUF\_SIZE, 313
- YY\_STATE\_EOF, 313
- yy\_state\_type, 320
- YY\_STRUCT\_YY\_BUFFER\_STATE, 313
- yy\_switch\_to\_buffer, 313, 322
- YY\_TYPEDEF\_YY\_BUFFER\_STATE, 313
- YY\_TYPEDEF\_YY\_SIZE\_T, 313
- YY\_USER\_ACTION, 314
- yyalloc, 314, 322
- yyconst, 314
- yyensure\_buffer\_stack, 314
- yyfree, 314, 322
- yyget\_debug, 314
- yyget\_extra, 314
- yyget\_in, 314
- yyget\_leng, 315
- yyget\_lineno, 315
- yyget\_out, 315
- yyget\_text, 315
- yyin, 315, 324
- yylen, 315, 324
- yyless, 315, 316
- yylex, 316
- yylex\_destroy, 316
- yylex\_init, 316
- yylex\_init\_extra, 316
- yylineno, 317, 324
- yymore, 317
- yynoreturn, 317
- yyout, 317, 324
- yypop\_buffer\_state, 317
- yypush\_buffer\_state, 317, 322
- yyrealloc, 317, 322
- yyrestart, 317, 322
- yyset\_debug, 318, 322
- yyset\_extra, 318, 323
- yyset\_in, 318, 323
- yyset\_lineno, 318, 323
- yyset\_out, 318, 323
- YYSTATE, 318
- YYTABLES\_NAME, 318
- yyterminate, 318
- yytext, 319, 325
- yytext\_ptr, 319
- yywrap, 319
- strategyScanner.l, 325
- strategyVariableLimit
  - Verification, 184
  - VerificationIterative, 196
- STRATEMPTY
  - strategyParser.h, 294
- stratensure\_buffer\_stack\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- STRATEOF
  - strategyParser.h, 294
- STRATerror
  - strategyParser.h, 294
- straterror
  - strategyParser.c, 289
- stratfree\_ALREADY\_DEFINED
  - strategyScanner.c, 303
- stratin\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratleng\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratlex
  - strategyParser.c, 289



- strategyScanner.c, 321
- stratlex\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratlineno\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratlval
  - strategyParser.h, 295
- stratout\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratparse
  - StrategyParser.cc, 291
  - strategyParser.h, 295
- stratpop\_buffer\_state\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratpush\_buffer\_state\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratrealloc\_ALREADY\_DEFINED
  - strategyScanner.c, 304
- stratrestart
  - strategyParser.c, 290
  - StrategyParser.cc, 292
- stratrestart\_ALREADY\_DEFINED
  - strategyScanner.c, 305
- STRATSTYPE, 163
  - expr, 163
  - floatno, 164
  - ident, 164
  - identList, 164
  - strategyParser.h, 294
  - val, 164
- STRATSTYPE\_IS\_DECLARED
  - strategyParser.h, 293
- STRATSTYPE\_IS\_TRIVIAL
  - strategyParser.h, 293
- strattext
  - strategyParser.c, 290
- strattext\_ALREADY\_DEFINED
  - strategyScanner.c, 305
- strattoken\_kind\_t
  - strategyParser.h, 294
- STRATTOKENTYPE
  - strategyParser.h, 293
- strattokentype
  - strategyParser.h, 294
- STRATUNDEF
  - strategyParser.h, 294
- stratwrap
  - strategyScanner.c, 305
- stratwrap\_ALREADY\_DEFINED
  - strategyScanner.c, 305
- stv\_mode
  - Cfg, 30
- styleString
  - DotGraph, 39
- SUM\_PROBABILITY
  - TypesDependency.hpp, 330
- T\_AGENT
  - parser.h, 248
- T\_AND
  - parser.h, 248
  - strategyParser.h, 294
- T\_ASSIGN
  - parser.h, 248
  - strategyParser.h, 294
- T\_EQ
  - parser.h, 248
  - strategyParser.h, 294
- T\_FLOAT
  - parser.h, 249
  - strategyParser.h, 294
- T\_FORMULA
  - parser.h, 248
- T\_GE
  - parser.h, 248
  - strategyParser.h, 294
- T\_GT
  - parser.h, 248
  - strategyParser.h, 294
- T\_HARTLEY
  - parser.h, 249
- T\_IDENT
  - parser.h, 249
  - strategyParser.h, 294
- T\_INIT
  - parser.h, 248
- T\_INITIAL
  - parser.h, 248
- T\_KNOW
  - parser.h, 249
- T\_LE
  - parser.h, 249
  - strategyParser.h, 294
- T\_LOCAL
  - parser.h, 248
- T\_LT
  - parser.h, 249
  - strategyParser.h, 294
- T\_NE
  - parser.h, 248
  - strategyParser.h, 294
- T\_NUM
  - parser.h, 249
  - strategyParser.h, 294
- T\_OR
  - parser.h, 248
  - strategyParser.h, 294
- T\_PERSISTENT
  - parser.h, 248
- T\_PROB
  - parser.h, 249
- T\_REST
  - parser.h, 249
- T\_SHARED
  - parser.h, 249
  - strategyParser.h, 294
- T\_TO



- parser.h, 248
  - strategyParser.h, 294
- tarjanVisit
  - Utils.cpp, 334
- test.cpp, 325
  - config, 326
  - main, 325
- testForAndFixBadAgents
  - VerificationIterative, 194
- to
  - GlobalTransition, 116
  - LocalTransition, 131
- toString
  - DecisionEntry, 33
  - GlobalState, 110
  - HistoryEntry, 121
  - LocalState, 125
- trans
  - YYSTYPE, 203
- transitions
  - LocalStateTemplate, 128
- TransitionTemplate, 164
  - assignments, 166
  - condition, 166
  - endState, 166
  - matchName, 166
  - patternName, 166
  - probability, 167
  - shared, 167
  - startState, 167
  - TransitionTemplate, 165
- TraversalMode
  - Verification.hpp, 342
- TRUE
  - TypesDependency.hpp, 330
- type
  - DecisionEntry, 35
  - HistoryEntry, 122
  - StrategyEntry, 161
- Types.hpp, 326
  - EQ, 327
  - F, 328
  - G, 328
  - GE, 327
  - GT, 327
  - LE, 327
  - LT, 327
  - NE, 327
  - NONE, 327
  - NOT\_SET, 328
  - ProbabilitySign, 327
  - VerificationFormulaMode, 328
- TypesDependency.hpp, 328
  - ADDED, 330
  - ANSWER\_PROBABILITY, 330
  - CONTEXT, 330
  - DECISION, 330
  - FALSE, 330
  - HistoryEntryType, 329
  - MARK\_DECISION\_AS\_INVALID, 330
  - MIN\_PROBABILITY, 330
  - NONE\_PROBABILITY, 330
  - NOT\_MODIFIED, 330
  - NOT\_VERIFIED, 330
  - PROBABILITY, 330
  - ProbabilityCalculationType, 330
  - STATE\_STATUS, 330
  - STRATEGY\_BITS, 329
  - StrategyEntryType, 330
  - SUM\_PROBABILITY, 330
  - TRUE, 330
  - UNCONTROLLED\_DECISION, 330
  - VerifResult, 330
- UINT16\_MAX
  - scanner.c, 254
  - strategyScanner.c, 305
- UINT32\_MAX
  - scanner.c, 255
  - strategyScanner.c, 305
- UINT8\_MAX
  - scanner.c, 255
  - strategyScanner.c, 305
- uncontrolled
  - StateVerificationInfo, 155
- UNCONTROLLED\_DECISION
  - TypesDependency.hpp, 330
- uncontrolledProbabilisticTransitionsLeftToProcess
  - StateVerificationInfo, 155
- uncontrolledTransitionsLeftToProcess
  - StateVerificationInfo, 155
- undoHistoryUntil
  - Verification, 180
- undoLastHistoryEntry
  - Verification, 181
  - VerificationIterative, 194
- unput
  - scanner.c, 255
  - strategyScanner.c, 305
- UNVERIFIED
  - GlobalStateVerificationStatus.hpp, 219
- Utils.cpp, 331
  - agentToString, 332
  - config, 335
  - envToString, 332
  - getContextModel, 332
  - getLocalStatesSCC, 333
  - getMemCap, 333
  - loadConfigFromArgs, 333
  - loadConfigFromFile, 333
  - localModelsToString, 333
  - outputGlobalModel, 334
  - tarjanVisit, 334
- Utils.hpp, 335
  - agentToString, 336
  - envToString, 336
  - getContextModel, 337

- getLocalStatesSCC, 337
- getMemCap, 337
- loadConfigFromArgs, 337
- loadConfigFromFile, 337
- localModelsToString, 338
- outputGlobalModel, 338
- val
  - STATSTYPE, 164
  - YYSTYPE, 204
- value
  - Assignment, 26
- Var, 167
  - agent, 168
  - initialValue, 168
  - name, 168
  - persistent, 168
- var
  - Condition, 32
- variableNames
  - Verification, 184
  - VerificationIterative, 197
- vars
  - Agent, 19
- VERBOSE
  - Constants.hpp, 208
- Verification, 169
  - ~Verification, 172
  - addHistoryContext, 172
  - addHistoryDecision, 172
  - addHistoryMarkDecisionAsInvalid, 173
  - addHistoryStateStatus, 173
  - addHistoryUncontrolledDecision, 173
  - areGlobalStatesInTheSameEpistemicClass, 174
  - checkUncontrolledSet, 174
  - equivalentGlobalTransitions, 175
  - fixpointVerify, 175
  - generator, 183
  - getEpistemicClassForGlobalState, 175
  - getNaturalStrategy, 175
  - getReducedStrategy, 176
  - getStrategyComplexity, 176
  - globalStateToValueBits, 176
  - historyDecisionsERR, 177
  - historyEnd, 183
  - historyStart, 183
  - historyToRestore, 183
  - increaseProbability, 177
  - isAgentInCoalition, 177
  - isGlobalTransitionControlledByCoalition, 177
  - lowerProbability, 178
  - maxFixpointVerify, 178
  - minFixpointVerify, 178
  - mode, 184
  - naturalStrategy, 184
  - newHistoryMarkDecisionAsInvalid, 178
  - printCurrentHistory, 179
  - reduceStrategy, 179
  - reductionComplexityBefore, 184
  - restoreHistory, 180
  - revertLastDecision, 180
  - revertToGlobalState, 184
  - strategyVariableLimit, 184
  - undoHistoryUntil, 180
  - undoLastHistoryEntry, 181
  - variableNames, 184
  - Verification, 171
  - verify, 181
  - verifyGlobalState, 181
  - verifyLocalStates, 182
  - verifyMDP, 182
  - verifyStrategy, 182
  - verifyTransitionSets, 182
- Verification.cpp, 338
  - config, 340
  - dbgHistEnt, 339
  - dbgVerifStatus, 340
  - DEPTH\_PREFIX, 339
  - verStatusToStr, 340
- Verification.hpp, 341
  - NORMAL, 342
  - RESTORE, 342
  - REVERT, 342
  - STRATEGY\_BITS, 342
  - TraversalMode, 342
  - verStatusToStr, 342
- VerificationFormulaMode
  - Types.hpp, 328
- VerificationIterative, 185
  - ~VerificationIterative, 187
  - addHistoryAnswerProbability, 188
  - addHistoryContext, 188
  - addHistoryDecision, 188
  - addHistoryMarkDecisionAsInvalid, 189
  - addHistoryProbability, 189
  - addHistoryStateStatus, 189
  - areGlobalStatesInTheSameEpistemicClass, 189
  - checkIfProbabilistic, 190
  - dissolveLoopedProbabilityTransition, 190
  - equivalentGlobalTransitions, 191
  - findLoopedProbabilityTransitions, 191
  - generator, 195
  - getEpistemicClassForGlobalState, 191
  - getNaturalStrategy, 192
  - getReducedStrategy, 192
  - getStrategyComplexity, 192
  - globalStateToValueBits, 192
  - history, 195
  - isAgentInCoalition, 193
  - isGlobalTransitionControlledByCoalition, 193
  - mode, 196
  - naturalStrategy, 196
  - reduceStrategy, 193
  - reductionComplexityBefore, 196
  - revertToGlobalState, 196
  - revertToLastDecision, 194
  - statesToProcess, 196

- strategyVariableLimit, 196
- testForAndFixBadAgents, 194
- undoLastHistoryEntry, 194
- variableNames, 197
- VerificationIterative, 187
- verify, 195
- verifyLocalStates, 195
- VerificationIterative.cpp, 343
  - config, 344
  - DEPTH\_PREFIX, 343
  - verifStatusToStr, 344
- VerificationIterative.hpp, 344
  - GraphTraversalMode, 345
  - NORMAL\_TRAVERSAL, 345
  - RESTORE\_STATES, 345
  - verifStatusToStr, 346
- verificationResult
  - Result, 152
- verificationStatus
  - GlobalState, 112
- VERIFIED\_ERR
  - GlobalStateVerificationStatus.hpp, 219
- VERIFIED\_OK
  - GlobalStateVerificationStatus.hpp, 219
- VerifResult
  - TypesDependency.hpp, 330
- verifResult
  - StateVerificationInfo, 155
- verifStatusToStr
  - VerificationIterative.cpp, 344
  - VerificationIterative.hpp, 346
- verify
  - Verification, 181
  - VerificationIterative, 195
- verify\_strategy
  - Cfg, 31
- verifyGlobalState
  - Verification, 181
- verifyLocalStates
  - Verification, 182
  - VerificationIterative, 195
- verifyMDP
  - Verification, 182
- verifyStrategy
  - Verification, 182
- verifyTransitionSets
  - Verification, 182
- verStatusToStr
  - Verification.cpp, 340
  - Verification.hpp, 342
- YY\_
  - parser.c, 231
  - strategyParser.c, 275
- YY\_ACCESSING\_SYMBOL
  - parser.c, 231
  - strategyParser.c, 275
- yy\_act
  - scanner.c, 269
- strategyScanner.c, 323
- YY\_ASSERT
  - parser.c, 231
  - strategyParser.c, 275
- YY\_AT\_BOL
  - scanner.c, 255
  - strategyScanner.c, 306
- yy\_at\_bol
  - yy\_buffer\_state, 198
- YY\_ATTRIBUTE\_PURE
  - parser.c, 231
  - strategyParser.c, 275
- YY\_ATTRIBUTE\_UNUSED
  - parser.c, 231
  - strategyParser.c, 276
- yy\_bp
  - scanner.c, 269
  - strategyScanner.c, 323
- YY\_BREAK
  - scanner.c, 255
  - strategyScanner.c, 306
- yy\_bs\_column
  - yy\_buffer\_state, 198
- yy\_bs\_lineno
  - yy\_buffer\_state, 198
- yy\_buf\_pos
  - yy\_buffer\_state, 198
- YY\_BUF\_SIZE
  - scanner.c, 255
  - strategyScanner.c, 306
- yy\_buf\_size
  - yy\_buffer\_state, 198
- YY\_BUFFER\_EOF\_PENDING
  - scanner.c, 255
  - strategyScanner.c, 306
- YY\_BUFFER\_NEW
  - scanner.c, 256
  - strategyScanner.c, 306
- YY\_BUFFER\_NORMAL
  - scanner.c, 256
  - strategyScanner.c, 306
- YY\_BUFFER\_STATE
  - parser.c, 240
  - scanner.c, 264
  - strategyParser.c, 286
  - strategyScanner.c, 320
- yy\_buffer\_state, 197
  - yy\_at\_bol, 198
  - yy\_bs\_column, 198
  - yy\_bs\_lineno, 198
  - yy\_buf\_pos, 198
  - yy\_buf\_size, 198
  - yy\_buffer\_status, 198
  - yy\_ch\_buf, 199
  - yy\_fill\_buffer, 199
  - yy\_input\_file, 199
  - yy\_is\_interactive, 199
  - yy\_is\_our\_buffer, 199

- yy\_n\_chars, [199](#)
- yy\_buffer\_status
  - yy\_buffer\_state, [198](#)
- YY\_CAST
  - parser.c, [231](#)
  - strategyParser.c, [276](#)
- yy\_ch\_buf
  - yy\_buffer\_state, [199](#)
- YY\_CHAR
  - scanner.c, [264](#)
  - strategyScanner.c, [320](#)
- yy\_cp
  - scanner.c, [269](#)
  - strategyScanner.c, [324](#)
- yy\_create\_buffer
  - scanner.c, [265](#)
  - strategyScanner.c, [306](#), [321](#)
- YY\_CURRENT\_BUFFER
  - scanner.c, [256](#)
  - strategyScanner.c, [307](#)
- YY\_CURRENT\_BUFFER\_LVALUE
  - scanner.c, [256](#)
  - strategyScanner.c, [307](#)
- YY\_DECL
  - scanner.c, [256](#), [269](#)
  - strategyScanner.c, [307](#), [324](#)
- YY\_DECL\_IS\_OURS
  - scanner.c, [256](#)
  - strategyScanner.c, [307](#)
- yy\_delete\_buffer
  - scanner.c, [265](#)
  - strategyScanner.c, [307](#), [321](#)
- YY\_DO\_BEFORE\_ACTION
  - scanner.c, [256](#)
  - strategyScanner.c, [307](#)
- YY\_END\_OF\_BUFFER
  - scanner.c, [257](#)
  - strategyScanner.c, [307](#)
- YY\_END\_OF\_BUFFER\_CHAR
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- YY\_EXIT\_FAILURE
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- YY\_EXTRA\_TYPE
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- YY\_FATAL\_ERROR
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- yy\_fill\_buffer
  - yy\_buffer\_state, [199](#)
- yy\_flex\_debug
  - scanner.c, [270](#)
  - strategyScanner.c, [308](#), [324](#)
- YY\_FLEX\_MAJOR\_VERSION
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- YY\_FLEX\_MINOR\_VERSION
  - scanner.c, [257](#)
  - strategyScanner.c, [308](#)
- YY\_FLEX\_SUBMINOR\_VERSION
  - scanner.c, [258](#)
  - strategyScanner.c, [309](#)
- YY\_FLUSH\_BUFFER
  - scanner.c, [258](#)
  - strategyScanner.c, [309](#)
- yy\_flush\_buffer
  - scanner.c, [265](#)
  - strategyScanner.c, [309](#), [321](#)
- YY\_IGNORE\_MAYBE\_UNINITIALIZED\_BEGIN
  - parser.c, [232](#)
  - strategyParser.c, [276](#)
- YY\_IGNORE\_MAYBE\_UNINITIALIZED\_END
  - parser.c, [232](#)
  - strategyParser.c, [276](#)
- YY\_IGNORE\_USELESS\_CAST\_BEGIN
  - parser.c, [232](#)
  - strategyParser.c, [276](#)
- YY\_IGNORE\_USELESS\_CAST\_END
  - parser.c, [232](#)
  - strategyParser.c, [276](#)
- yy\_init\_buffer
  - strategyScanner.c, [309](#)
- YY\_INITIAL\_VALUE
  - parser.c, [232](#)
  - strategyParser.c, [276](#)
- YY\_INPUT
  - scanner.c, [258](#)
  - strategyScanner.c, [309](#)
- yy\_input\_file
  - yy\_buffer\_state, [199](#)
- YY\_INT\_ALIGNED
  - scanner.c, [258](#)
  - strategyScanner.c, [309](#)
- yy\_is\_interactive
  - yy\_buffer\_state, [199](#)
- yy\_is\_our\_buffer
  - yy\_buffer\_state, [199](#)
- YY\_LESS\_LINENO
  - scanner.c, [258](#)
  - strategyScanner.c, [310](#)
- YY\_LINENO\_REWIND\_TO
  - scanner.c, [259](#)
  - strategyScanner.c, [310](#)
- yy\_load\_buffer\_state
  - strategyScanner.c, [310](#)
- YY\_MORE\_ADJ
  - scanner.c, [259](#)
  - strategyScanner.c, [310](#)
- yy\_n\_chars
  - yy\_buffer\_state, [199](#)
- yy\_new\_buffer
  - scanner.c, [259](#)
  - strategyScanner.c, [310](#)
- YY\_NEW\_FILE

- scanner.c, 259
- strategyScanner.c, 310
- YY\_NULL
  - scanner.c, 259
  - strategyScanner.c, 310
- YY\_NULLPTR
  - parser.c, 232
  - strategyParser.c, 277
- YY\_NUM\_RULES
  - scanner.c, 259
  - strategyScanner.c, 311
- yy\_nxt
  - yy\_trans\_info, 200
- YY\_READ\_BUF\_SIZE
  - scanner.c, 259
  - strategyScanner.c, 311
- YY\_REDUCE\_PRINT
  - parser.c, 232
  - strategyParser.c, 277
- YY\_REINTERPRET\_CAST
  - parser.c, 233
  - strategyParser.c, 277
- YY\_RESTORE\_YY\_MORE\_OFFSET
  - scanner.c, 260
  - strategyScanner.c, 311
- YY\_RULE\_SETUP
  - scanner.c, 260
  - strategyScanner.c, 311
- YY\_SC\_TO\_UI
  - scanner.c, 260
  - strategyScanner.c, 311
- yy\_scan\_buffer
  - scanner.c, 265
  - strategyScanner.c, 311, 321
- yy\_scan\_bytes
  - scanner.c, 266
  - strategyScanner.c, 311, 321
- yy\_scan\_string
  - parser.c, 244
  - scanner.c, 266
  - strategyScanner.c, 312, 321
- yy\_set\_bol
  - scanner.c, 260
  - strategyScanner.c, 312
- yy\_set\_interactive
  - scanner.c, 260
  - strategyScanner.c, 312
- yy\_size\_t
  - scanner.c, 264
  - strategyScanner.c, 320
- YY\_SKIP\_YYWRAP
  - scanner.c, 260
  - strategyScanner.c, 312
- YY\_STACK\_PRINT
  - parser.c, 233
  - strategyParser.c, 277
- YY\_START
  - scanner.c, 261
- strategyScanner.c, 312
- YY\_START\_STACK\_INCR
  - scanner.c, 261
  - strategyScanner.c, 313
- YY\_STATE\_BUF\_SIZE
  - scanner.c, 261
  - strategyScanner.c, 313
- YY\_STATE\_EOF
  - scanner.c, 261
  - strategyScanner.c, 313
- yy\_state\_fast\_t
  - parser.c, 241
  - strategyParser.c, 286
- yy\_state\_t
  - parser.c, 241
  - strategyParser.c, 286
- yy\_state\_type
  - scanner.c, 265
  - strategyScanner.c, 320
- YY\_STRUCT\_YY\_BUFFER\_STATE
  - scanner.c, 261
  - strategyScanner.c, 313
- yy\_switch\_to\_buffer
  - scanner.c, 266
  - strategyScanner.c, 313, 322
- YY\_SYMBOL\_PRINT
  - parser.c, 233
  - strategyParser.c, 277
- yy\_trans\_info, 200
  - yy\_nxt, 200
  - yy\_verify, 200
- YY\_TYPEDEF\_YY\_BUFFER\_STATE
  - scanner.c, 261
  - strategyScanner.c, 313
- YY\_TYPEDEF\_YY\_SIZE\_T
  - scanner.c, 261
  - strategyScanner.c, 313
- YY\_USE
  - parser.c, 233
  - strategyParser.c, 277
- YY\_USER\_ACTION
  - scanner.c, 262
  - strategyScanner.c, 314
- yy\_verify
  - yy\_trans\_info, 200
- YYABORT
  - parser.c, 233
  - strategyParser.c, 278
- YYACCEPT
  - parser.c, 233
  - strategyParser.c, 278
- yyalloc, 201
  - scanner.c, 266
  - strategyScanner.c, 314, 322
- yyss\_alloc, 201
- yyvs\_alloc, 201
- YYBACKUP
  - parser.c, 234

- strategyParser.c, 278
- YYBISON
  - parser.c, 234
  - strategyParser.c, 278
- YYBISON\_VERSION
  - parser.c, 234
  - strategyParser.c, 278
- yychar
  - parser.c, 245
  - strategyParser.c, 278, 290
- yyclearin
  - parser.c, 234
  - strategyParser.c, 279
- yyconst
  - scanner.c, 262
  - strategyScanner.c, 314
- YYCOPY
  - parser.c, 234
  - strategyParser.c, 279
- YYCOPY\_NEEDED
  - parser.c, 235
  - strategyParser.c, 279
- YYDEBUG
  - parser.h, 247
- yydebug
  - strategyParser.c, 279
- YYDPRINTF
  - parser.c, 235
  - strategyParser.c, 279
- YYEMPTY
  - parser.h, 248
- YYENOMEM
  - parser.c, 242
  - strategyParser.c, 287
- yyensure\_buffer\_stack
  - strategyScanner.c, 314
- YYEOF
  - parser.h, 248
- YYERRCODE
  - parser.c, 235
  - strategyParser.c, 279
- yyerrok
  - parser.c, 235
  - strategyParser.c, 280
- YYERROR
  - parser.c, 235
  - strategyParser.c, 280
- YYerror
  - parser.h, 248
- yyerror
  - parser.c, 244
  - strategyParser.c, 280
- YYFINAL
  - parser.c, 235
  - strategyParser.c, 280
- YYFREE
  - parser.c, 236
  - strategyParser.c, 280
- yyfree
  - scanner.c, 266
  - strategyScanner.c, 314, 322
- yyget\_debug
  - scanner.c, 266
  - strategyScanner.c, 314
- yyget\_extra
  - scanner.c, 266
  - strategyScanner.c, 314
- yyget\_in
  - scanner.c, 267
  - strategyScanner.c, 314
- yyget\_leng
  - scanner.c, 267
  - strategyScanner.c, 315
- yyget\_lineno
  - scanner.c, 267
  - strategyScanner.c, 315
- yyget\_out
  - scanner.c, 267
  - strategyScanner.c, 315
- yyget\_text
  - scanner.c, 267
  - strategyScanner.c, 315
- yyin
  - scanner.c, 270
  - strategyScanner.c, 315, 324
- YYINITDEPTH
  - parser.c, 236
  - strategyParser.c, 280
- YYLAST
  - parser.c, 236
  - strategyParser.c, 280
- yyleng
  - scanner.c, 270
  - strategyScanner.c, 315, 324
- yyless
  - scanner.c, 262
  - strategyScanner.c, 315, 316
- yylex
  - parser.c, 245
  - scanner.c, 267
  - strategyParser.c, 280
  - strategyScanner.c, 316
- yylex\_destroy
  - scanner.c, 267
  - strategyScanner.c, 316
- yylex\_init
  - strategyScanner.c, 316
- yylex\_init\_extra
  - strategyScanner.c, 316
- yylineno
  - scanner.c, 270
  - strategyScanner.c, 317, 324
- yyval
  - parser.c, 245
  - parser.h, 249
  - strategyParser.c, 281, 290

- YYMALLOC
  - parser.c, [236](#)
  - strategyParser.c, [281](#)
- YYMAXDEPTH
  - parser.c, [236](#)
  - strategyParser.c, [281](#)
- YYMAXUTOK
  - parser.c, [236](#)
  - strategyParser.c, [281](#)
- yymore
  - scanner.c, [262](#)
  - strategyScanner.c, [317](#)
- yynerrs
  - parser.c, [246](#)
  - strategyParser.c, [281](#), [290](#)
- YYNNTS
  - parser.c, [236](#)
  - strategyParser.c, [281](#)
- YYNOMEM
  - parser.c, [236](#)
  - strategyParser.c, [281](#)
- yynoreturn
  - scanner.c, [263](#)
  - strategyScanner.c, [317](#)
- YYNRULES
  - parser.c, [237](#)
  - strategyParser.c, [281](#)
- YYNSTATES
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- YYNTOKENS
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- yyout
  - scanner.c, [270](#)
  - strategyScanner.c, [317](#), [324](#)
- YYPACT\_NINF
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- yypact\_value\_is\_default
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- yyparse
  - ModelParser.cc, [224](#)
  - parser.c, [245](#)
  - parser.h, [249](#)
  - strategyParser.c, [282](#)
- yypop\_buffer\_state
  - scanner.c, [268](#)
  - strategyScanner.c, [317](#)
- YYPOPSTACK
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- YYPTRDIFF\_MAXIMUM
  - parser.c, [237](#)
  - strategyParser.c, [282](#)
- YYPTRDIFF\_T
  - parser.c, [238](#)
- strategyParser.c, [283](#)
- YYPULL
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- YYPURE
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- YYPUSH
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- yypush\_buffer\_state
  - scanner.c, [268](#)
  - strategyScanner.c, [317](#), [322](#)
- yyrealloc
  - scanner.c, [268](#)
  - strategyScanner.c, [317](#), [322](#)
- YYRECOVERING
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- yyrestart
  - ModelParser.cc, [225](#)
  - parser.c, [245](#)
  - scanner.c, [268](#)
  - strategyScanner.c, [317](#), [322](#)
- yyset\_debug
  - scanner.c, [268](#)
  - strategyScanner.c, [318](#), [322](#)
- yyset\_extra
  - scanner.c, [268](#)
  - strategyScanner.c, [318](#), [323](#)
- yyset\_in
  - scanner.c, [268](#)
  - strategyScanner.c, [318](#), [323](#)
- yyset\_lineno
  - scanner.c, [269](#)
  - strategyScanner.c, [318](#), [323](#)
- yyset\_out
  - scanner.c, [269](#)
  - strategyScanner.c, [318](#), [323](#)
- YYSIZE\_MAXIMUM
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- YYSIZE\_T
  - parser.c, [238](#)
  - strategyParser.c, [283](#)
- YYSIZEOF
  - parser.c, [239](#)
  - strategyParser.c, [284](#)
- YYSKELETON\_NAME
  - parser.c, [239](#)
  - strategyParser.c, [284](#)
- yyss\_alloc
  - yyalloc, [201](#)
- YYSTACK\_ALLOC
  - parser.c, [239](#)
  - strategyParser.c, [284](#)
- YYSTACK\_ALLOC\_MAXIMUM
  - parser.c, [239](#)

- strategyParser.c, 284
- YYSTACK\_BYTES
  - parser.c, 239
  - strategyParser.c, 284
- YYSTACK\_FREE
  - parser.c, 239
  - strategyParser.c, 284
- YYSTACK\_GAP\_MAXIMUM
  - parser.c, 239
  - strategyParser.c, 284
- YYSTACK\_RELOCATE
  - parser.c, 240
  - strategyParser.c, 285
- YYSTATE
  - scanner.c, 263
  - strategyScanner.c, 318
- YYSTYPE, 202
  - agent, 202
  - assign, 202
  - assignSet, 202
  - condList, 203
  - expr, 203
  - floatno, 203
  - ident, 203
  - identSet, 203
  - model, 203
  - parser.h, 248
  - prob, 203
  - strategyParser.c, 285
  - trans, 203
  - val, 204
- YYSTYPE\_IS\_DECLARED
  - parser.h, 247
- YYSTYPE\_IS\_TRIVIAL
  - parser.h, 247
- YYSYMBOL\_17\_
  - parser.c, 244
  - strategyParser.c, 289
- YYSYMBOL\_27\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_28\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_29\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_30\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_31\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_32\_
  - parser.c, 242
  - strategyParser.c, 287
- YYSYMBOL\_33\_
  - parser.c, 242
- strategyParser.c, 287
  - YYSYMBOL\_34\_
    - parser.c, 242
    - strategyParser.c, 287
  - YYSYMBOL\_35\_
    - parser.c, 242
    - strategyParser.c, 287
  - YYSYMBOL\_36\_
    - parser.c, 242
    - strategyParser.c, 287
  - YYSYMBOL\_37\_
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_38\_
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_agent
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_assign\_list
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_assignment
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_assignments
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_coalition
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_cond
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_cond\_and
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_cond\_elem
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_cond\_list
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_condition
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_description
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_formula
    - parser.c, 243
    - strategyParser.c, 288
  - YYSYMBOL\_ident\_list
    - parser.c, 243, 244
    - strategyParser.c, 288, 289
  - YYSYMBOL\_initial
    - parser.c, 243
    - strategyParser.c, 288



yy symbol\_kind\_t  
    parser.c, 241, 242  
    strategyParser.c, 286, 287

YYSYMBOL\_local  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_model  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_num\_elem  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_num\_exp  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_num\_mul  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_persistent  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_prob\_elem  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_prob\_exp  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_prob\_mul  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_probability\_equality  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_query  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_shared  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_spec  
    parser.c, 243  
    strategyParser.c, 288

YYSYMBOL\_strat  
    parser.c, 244  
    strategyParser.c, 289

YYSYMBOL\_T\_AGENT  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_AND  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_ASSIGN  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_EQ  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_FLOAT  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_FORMULA  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_GE  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_GT  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_HARTLEY  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_IDENT  
    parser.c, 242, 244  
    strategyParser.c, 287, 289

YYSYMBOL\_T\_INIT  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_INITIAL  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_KNOW  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_LE  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_LOCAL  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_LT  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_NE  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_NUM  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_OR  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_PERSISTENT  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_PROB  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_REST  
    parser.c, 242  
    strategyParser.c, 287

YYSYMBOL\_T\_SHARED  
    parser.c, 242, 243  
    strategyParser.c, 287, 288

YYSYMBOL\_T\_TO  
    parser.c, 242, 243

- strategyParser.c, [287](#), [288](#)
- YYSYMBOL\_transition
  - parser.c, [243](#)
  - strategyParser.c, [288](#)
- YYSYMBOL YYACCEPT
  - parser.c, [243](#), [244](#)
  - strategyParser.c, [288](#), [289](#)
- YYSYMBOL YYEMPTY
  - parser.c, [242](#), [243](#)
  - strategyParser.c, [287](#), [288](#)
- YYSYMBOL YYEOF
  - parser.c, [242](#), [243](#)
  - strategyParser.c, [287](#), [288](#)
- YYSYMBOL YYerror
  - parser.c, [242](#), [243](#)
  - strategyParser.c, [287](#), [288](#)
- YYSYMBOL YYUNDEF
  - parser.c, [242](#), [243](#)
  - strategyParser.c, [287](#), [288](#)
- YYTABLE\_NINF
  - parser.c, [240](#)
  - strategyParser.c, [285](#)
- yytable\_value\_is\_error
  - parser.c, [240](#)
  - strategyParser.c, [285](#)
- YYTABLES\_NAME
  - scanner.c, [263](#)
  - strategyScanner.c, [318](#)
- yyterminate
  - scanner.c, [263](#)
  - strategyScanner.c, [318](#)
- yytext
  - parser.c, [246](#)
  - scanner.c, [270](#)
  - strategyScanner.c, [319](#), [325](#)
- yytext\_ptr
  - scanner.c, [263](#)
  - strategyScanner.c, [319](#)
- yytoken\_kind\_t
  - parser.h, [248](#)
- YYTOKENTYPE
  - parser.h, [247](#)
- yytokentype
  - parser.h, [248](#)
- YYTRANSLATE
  - parser.c, [240](#)
  - strategyParser.c, [285](#)
- yytype\_int16
  - parser.c, [241](#)
  - strategyParser.c, [286](#)
- yytype\_int8
  - parser.c, [241](#)
  - strategyParser.c, [286](#)
- yytype\_uint16
  - parser.c, [241](#)
  - strategyParser.c, [286](#)
- yytype\_uint8
  - parser.c, [241](#)
- strategyParser.c, [286](#)
- YYUNDEF
  - parser.h, [248](#)
- yyvs\_alloc
  - yyalloc, [201](#)
- yywrap
  - scanner.c, [263](#)
  - strategyScanner.c, [319](#)